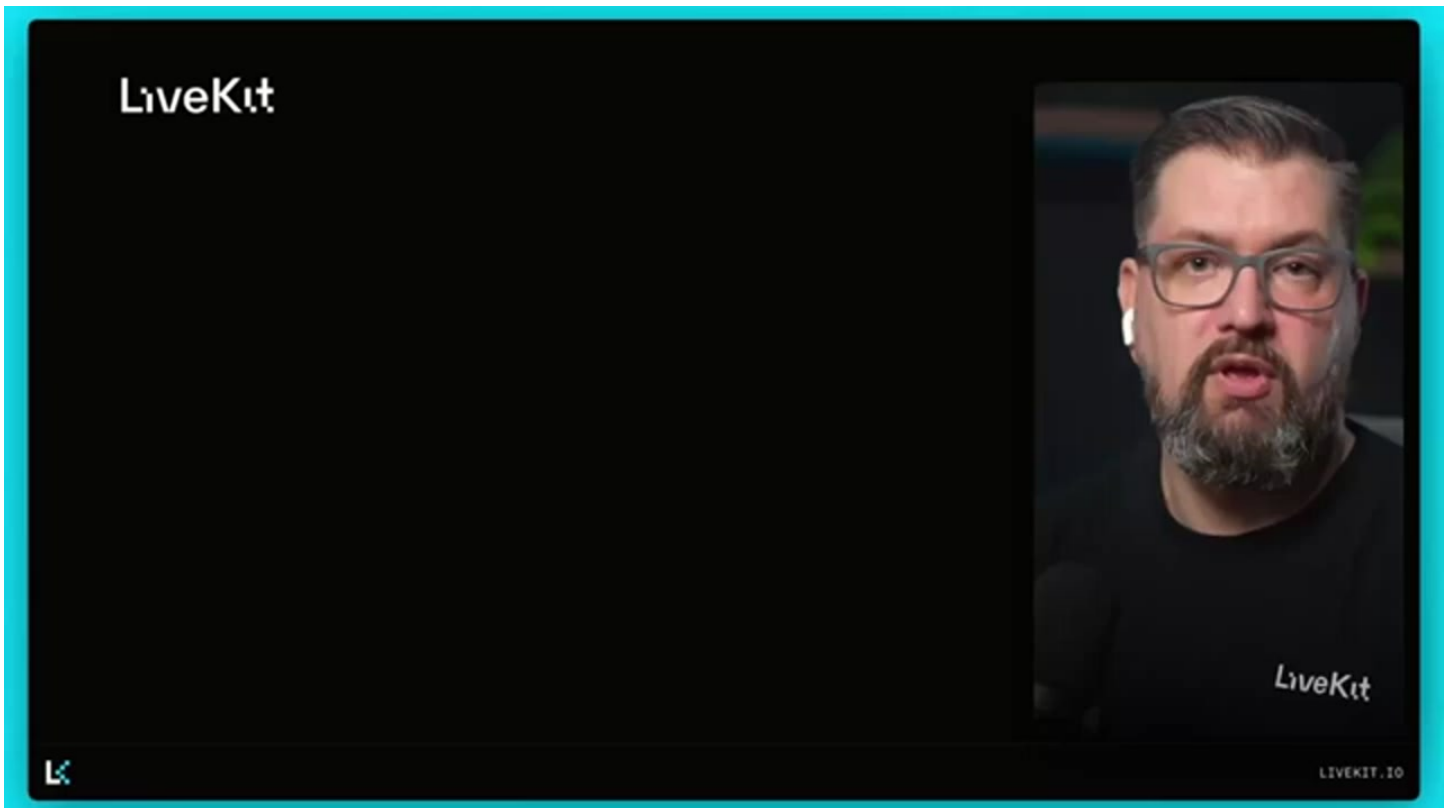


Master Voice AI Agents with LiveKit: Complete Course

Section: Full Course

Summary: Comprehensive course on building production-ready Voice AI Agents with LiveKit covering architecture, turn detection, metrics, tools, and consent.

■ [00:00 - 00:20]

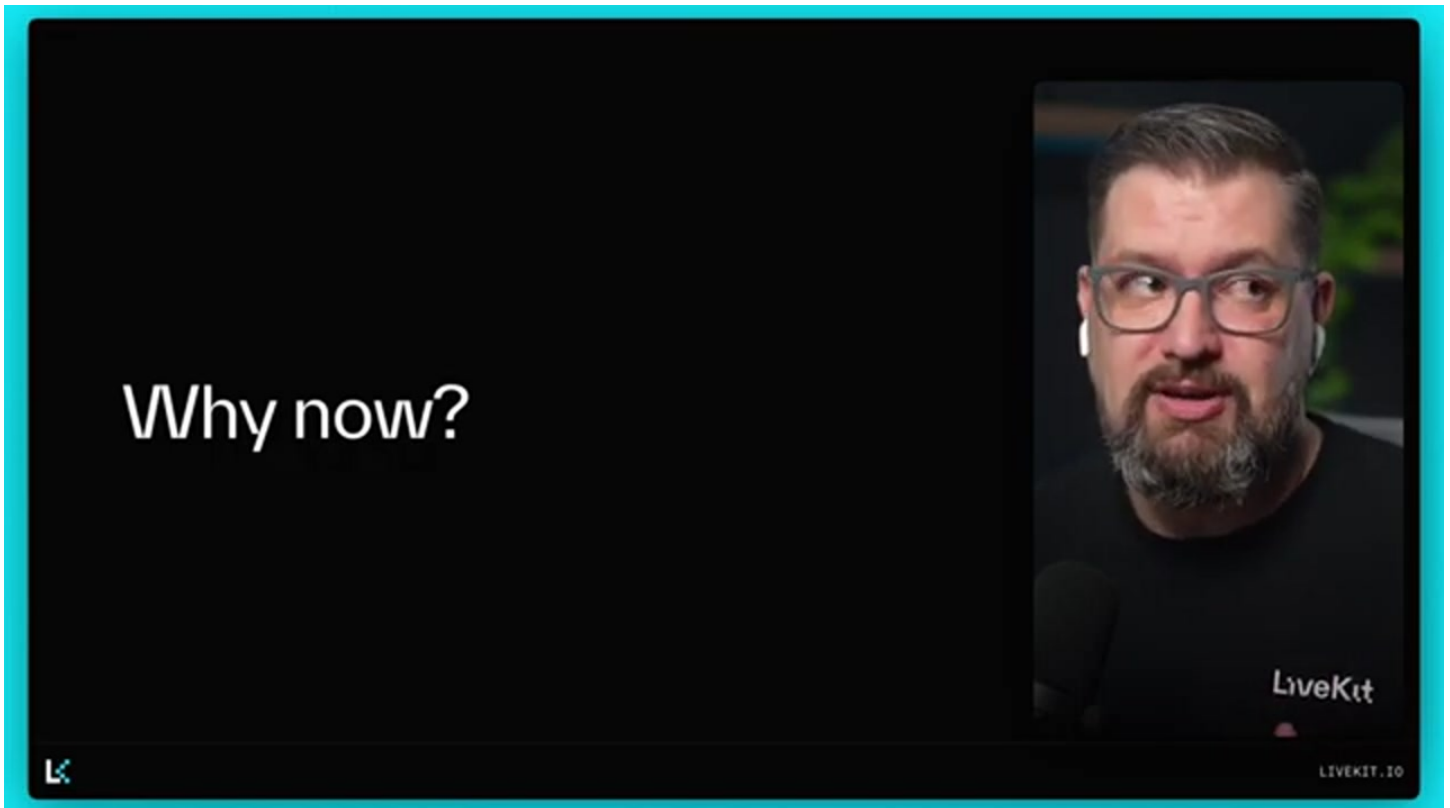


You've probably talked to a voice AI recently and maybe it was frustrating. Maybe it cut you off. Maybe it felt robotic. Here's the thing. Building a voice agent that actually feels natural is harder than it looks. But once you understand the fundamentals, you can build something that people genuinely want to talk to, don't mind talking to,



want to talk to, don't mind talking to, and actually prefer to talk to. And that's what this workshop is about. You don't need prior experience with voice AI, machine learning, or real-time systems. I'll explain all of those concepts as we go. Voice AI hit an inflection point. Speech-to-text models are faster and more accurate. LLMs can handle nuanced conversations. Text to

■ [00:40 - 01:00]



handle nuanced conversations. Text to speech sounds human. These pieces exist. The challenge is wiring them together into something that feels responsive and natural. Now, a year ago, building this required specialized engineering knowledge and months of hard work. Today, you can get a working voice agent running in just a few minutes. So, here's what you're going to learn throughout this workshop. First, how the

■ [01:00 - 01:20]

What you'll learn



LIVEKIT.IO

throughout this workshop. First, how the voice pipeline works. That's VAD, ST, LLM, and TTS. And then, why latency matters and how to uh minimize it. You'll learn techniques for natural turntaking so your agent doesn't interrupt you mid-sentence. You'll add personality to your agent and set up provider failovers uh so that it stays

What you'll learn

- Voice pipeline fundamentals
- Latency optimization
- Natural turn-taking
- Personality & failover



LIVEKIT.IO

provider failovers uh so that it stays online even when a service goes down. And later you'll integrate external tools and APIs. And by the end you'll build real workflows with handoffs and consent collection. And the result is that you'll have an agent that listens, thinks, and responds in real time, not a toy demo. A foundation that you can actually ship. Each lesson adds a new

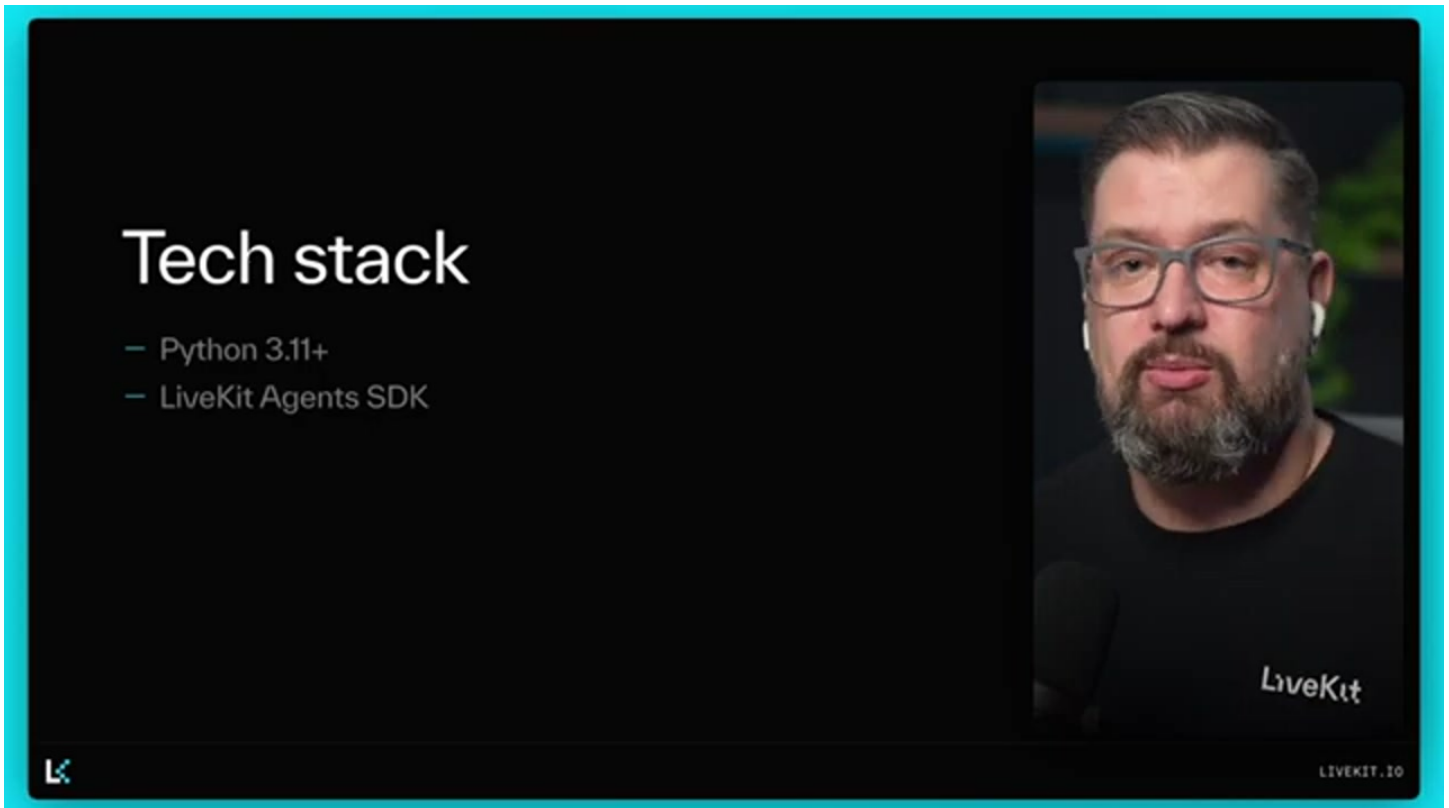
What you'll learn

- Voice pipeline fundamentals
- Latency optimization
- Natural turn-taking
- Personality & failover
- Tool integration
- Workflows & handoffs



LIVEKIT.IO

actually ship. Each lesson adds a new capability, and each one builds on the last. You'll use the LifeKit agents SDK for Python. LifeKit handles the real-time infrastructure so that you can focus on the uh agentic logic. But note that while this workshop uses Python, LifeKit also offers a TypeScript agents



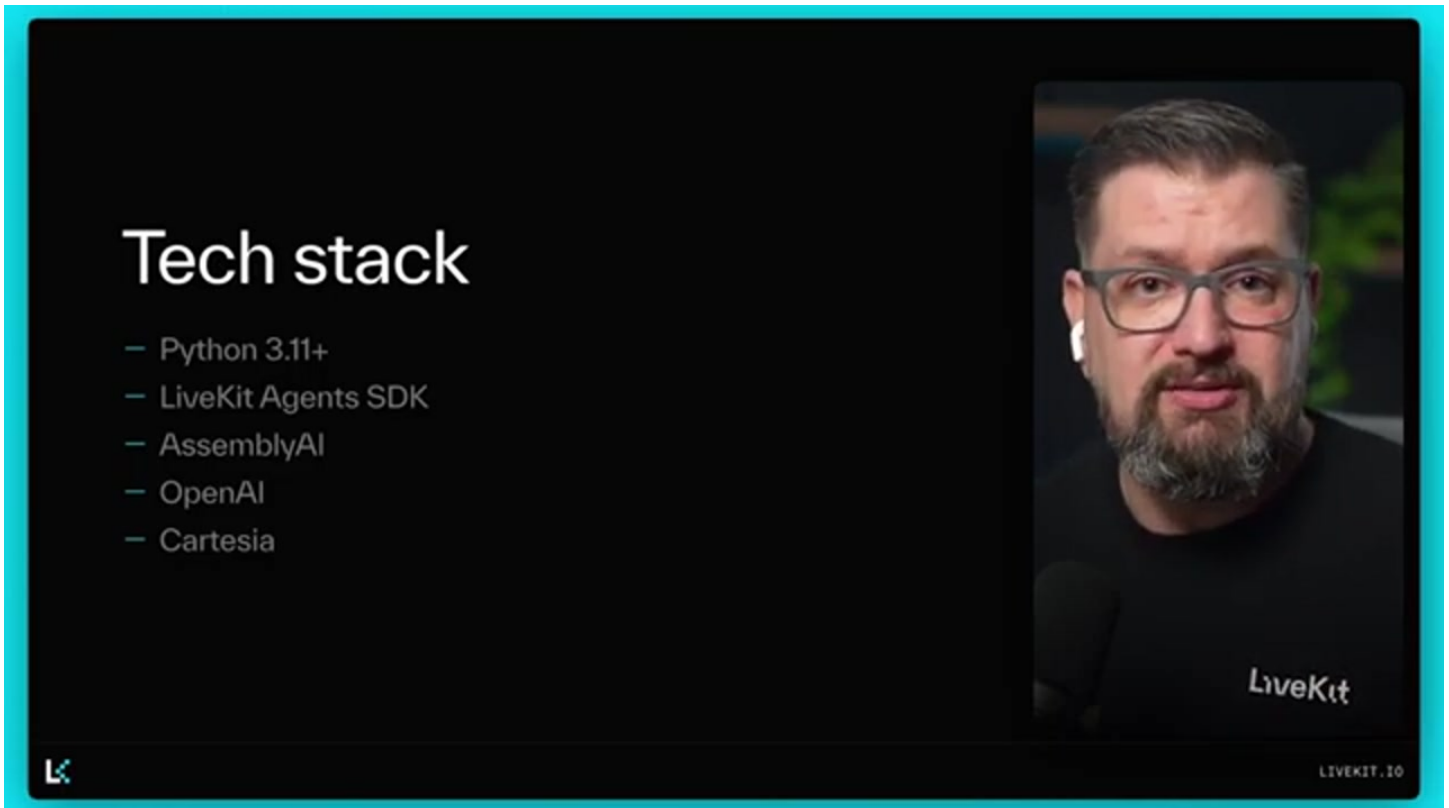
The video frame displays a presentation slide with a black background and a cyan border. On the left, the title 'Tech stack' is written in large white font. Below it, a list of technologies is shown in a smaller white font: '— Python 3.11+' and '— LiveKit Agents SDK'. On the right side of the slide, there is a portrait of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. In the bottom left corner of the slide, there is a small 'LK' logo, and in the bottom right corner, the text 'LIVEKIT.IO' is visible.

Tech stack

- Python 3.11+
- LiveKit Agents SDK

LK LIVEKIT.IO

LifeKit also offers a TypeScript agents SDK if you prefer working in JavaScript or TypeScript. And these are the models that we'll use throughout the workshop. Uh but they're not the only options. The agents SDK supports a wide range of providers for each component. If you want to use Azure for ST, Anthropic for your LLM, and 11 Labs for your TTS, you



Tech stack

- Python 3.11+
- LiveKit Agents SDK
- AssemblyAI
- OpenAI
- Cartesia

LiveKit

LIVEKIT.IO

your LLM, and 11 Labs for your TTS, you can swap them out with just a few lines of code. The patterns that you're going to learn will apply regardless of which providers you choose. And we'll use LiveKit inference, which comes with credits, so you'll get access to all of these models so that you can follow along without spending any money. There's also uh written materials that go along with each video lesson, so you

Building Production-Ready Voice Agents with LiveKit

Hands-on workshop for real-time voice AI systems

Estimated Duration:

01 Hours

Table of Contents

Voice Agent Foundations

Understanding the real-time voice AI pipeline

Semantic Turn

Detection

Preventing unwanted interruptions

Personality and Fallbacks

Designing resilient assistant behavior

Metrics and Pragmatism

Generation

Measuring performance and sharing metrics

Tools and MCP

Integration

Connecting external systems safely

Consent and Escalations

Operational workflows for real deployments

Voice Agent Foundations

Understanding the real-time voice AI pipeline

In this lesson, you'll ground yourself in the architecture of real-time voice agents before you touch any code. The goal is to understand which components are required, how they interact, and why latency matters so much.

What we are building

By the end of the workshop, you will ship a production-quality voice assistant that:

- Listens to a caller in real time through LiveKit's WebRTC infrastructure
- Streams speech-to-text (STT), large language model (LLM), and text-to-speech (TTS) responses
- Handles interruptions gracefully with voice activity detection (VAD) and semantic turn detection
- Escalates conversations, calls external tools, and collects metrics so you can iterate effectively

We will iterate on the `agent.py` scaffold provided in the repository linked [here](#). Each lesson introduces a capability and immediately applies it in code.

Anatomy of a voice agent

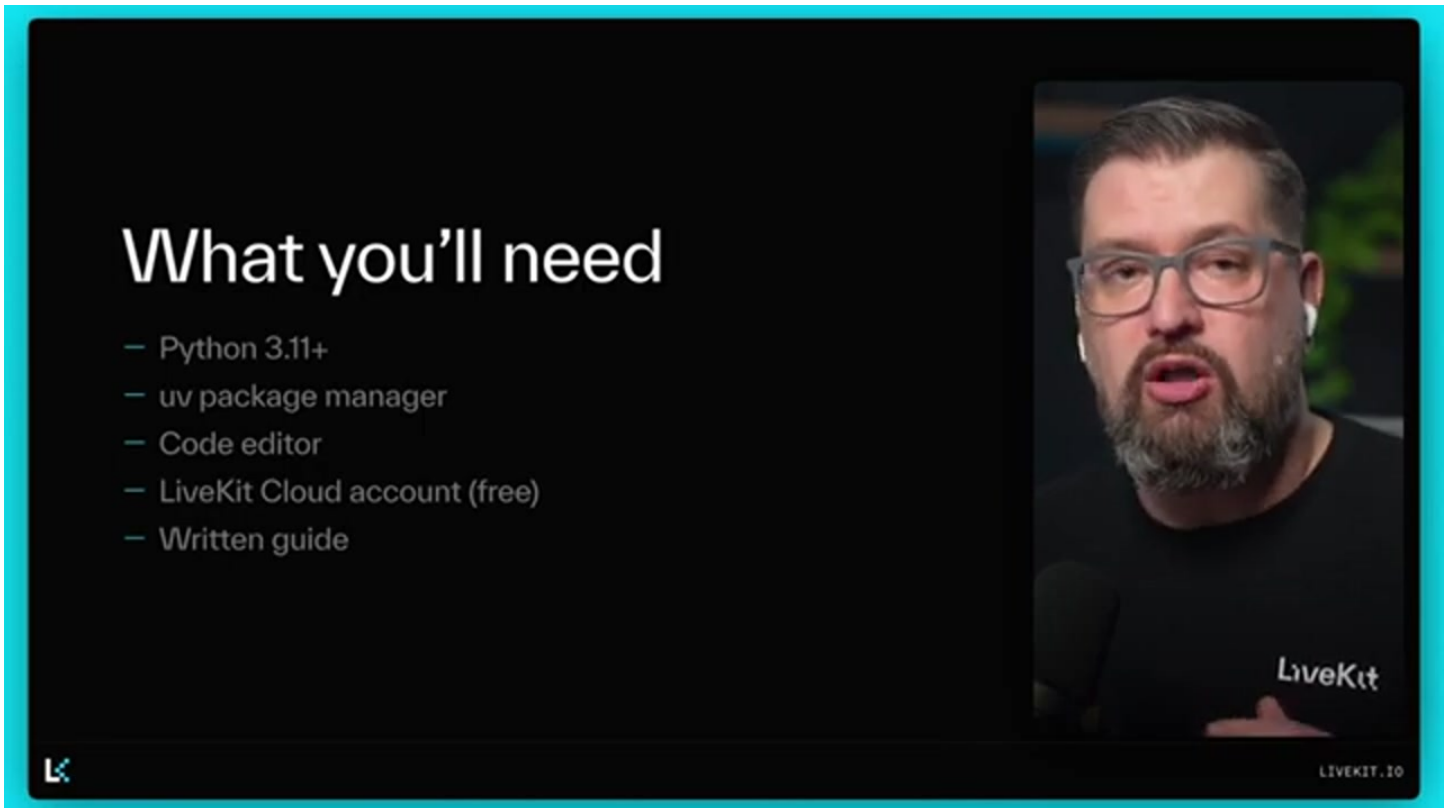
Every voice agent follows the same high-level pipeline:

1. **Voice Activity Detection (VAD)** determines when the human is speaking. Without it, your agent wastes money on silence and may interrupt the user.
2. **Speech-to-Text (STT)** transcribes audio into tokens the LLM can understand. Choose models that support your languages and accents.
3. **Large Language Model (LLM)** generates the next reply. Prompts, planning, and tool integrations all live here.
4. **Text-to-Speech (TTS)** converts the LLM response into audio that feels natural and on-brand.

Supporting components such as background voice cancellation (BVC), noise suppression, and end-of-turn



can use them as a reference while you code. And there's a link in the video description for that. Now, before you start the workshop, make sure that you have Python 3.11 or higher, the UV package manager, a code editor, a free Lifekit Cloud account, and the written guide open if you want to copy and paste code snippets. Again, there's links in the video description for all of these



The video frame shows a presentation slide with a black background and white text. The title 'What you'll need' is at the top left. Below it is a bulleted list of requirements. In the top right corner, there is a small video inset of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. The bottom left of the slide has a small 'L' logo, and the bottom right has the text 'LIVEKIT.IO'.

What you'll need

- Python 3.11+
- uv package manager
- Code editor
- LiveKit Cloud account (free)
- Written guide

L

LIVEKIT.IO

the video description for all of these materials. So, got all that? Let's start building something. Before we write any code, let's build a mental model of how voice agents actually work. And by the end of this lesson, you'll understand which components make up a real-time voice agent, how they interact, and why latency is such a big deal. Throughout this workshop, you'll build a production

What you'll build



LIVEKIT.IO

this workshop, you'll build a production quality voice assistant. It listens to callers in real time through LiveKit's Web RTC infrastructure. It streams speech to text, runs responses through an LLM, and converts them back to speech with texttospeech. You'll add interruption handling using voice activity detection and semantic turn

What you'll build

- Real-time audio via WebRTC
- Streaming STT, LLM, TTS
- Interruption handling



LIVEKIT.IO

activity detection and semantic turn detection. And you'll wire up tool calls, conversation escalation, and metric collection. Every voice agent follows the same high-level pipeline. There are four core components. Voice activity detection determines when the human is speaking. Without it, your agent wastes money uh processing silence and might

The voice pipeline

— VAD: detects speech



LIVEKIT.IO

money uh processing silence and might interrupt the user mid-sentence. Speech to text transcribes audio into tokens the LLM can understand. You'll want to choose models that support your target languages and accents. The large language model generates the reply, prompts, planning, and tool integrations all live here. And text to speech

The voice pipeline

- VAD: detects speech
- STT: transcribes audio
- LLM: generates reply
- TTS: speaks response



LIVEKIT.IO

all live here. And text to speech converts the LLM response into audio that feels natural and on brand. Supporting components like background noise cancellation, noise suppression, and end of turn detectors help the agent behave like a good conversational partner. And you'll add several of these throughout the workshop. Now, the cascaded pipeline had to ST to LLM to

The voice pipeline

- VAD: detects speech
- STT: transcribes audio
- LLM: generates reply
- TTS: speaks response



cascaded pipeline bad to ST to LLM to TTS shown here is currently the most popular approach. But there is an alternative and that is to use speechtoech or real-time models that can process audio directly without explicit transcription. Now that's not going to be covered here in this workshop, but look out for more videos on that subject

The voice pipeline

- VAD: detects speech
- STT: transcribes audio
- LLM: generates reply
- TTS: speaks response



LIVEKIT.IO

look out for more videos on that subject coming very soon. Now humans expect conversational latency under 500 milliseconds. Every component in the pipeline adds delay. VAD is the fastest at 15 to 30 milliseconds. ST takes 200 to 600 milliseconds. Uh LLMs range from 100 milliseconds to a full second. TTS

Average latencies

Component	Best case	Typical
VAD	15-20ms	20-30ms
STT	200-300ms	400-600ms
LLM	100-200ms	500-1000ms
TTS	100-150ms	200-300ms
Total	~415ms	1.1s-2s



100 milliseconds to a full second. TTS adds another 100 to 300 milliseconds. And so these all add up. And so you could be looking at anywhere from 400 milliseconds to 2 seconds total. Hey, how's it going? Good. How are you? I'm good. Just got back from lunch.

■ [05:40 - 06:00]



back from lunch. What did you have? A burrito. Anything else you'd like to talk about? You know what? I I got to go. That felt

■ [06:00 - 06:20]



Anything else you'd like to talk about? You know what? I I got to go. That felt painful, right? Each 2C delay adds up fast. In a real conversation, you'd probably give up after the second or third pause. Your users will do the same thing. Low latency agents keep pipelines streaming and parallel. They avoid blocking I/O and they carefully choose

Low latency is key!



LIVEKIT.IO

providers that balance quality with speed. And you'll learn techniques for shaving off those milliseconds in later lessons. A real-time voice has uh different networking needs than text. Let's compare the options. HTTP over TCP is great for text request responses, but it has a problem for live audio. You

Why WebRTC?

— HTTP/TCP: head-of-line blocking



LIVEKIT.IO

it has a problem for live audio. You see, TCP guarantees that packets arrive in order. If one packet gets lost, everything behind it has to wait until that packet is resent and received. That's called head of line blocking. For a web page, that's fine. For live speech, a 200 millisecond pause while

Why WebRTC?

— HTTP/TCP: head-of-line blocking



LIVEKIT.IO

speech, a 200 millisecond pause while waiting for a retransmit on a single packet ruins the entire experience. The other option is websockets. Web soockets are persistent and birectional, but they're still blocked by TCP retransmits under poor networking conditions. But then we have web RTC over UDP and which was designed for media. UDP doesn't

Why WebRTC?

- HTTP/TCP: head-of-line blocking
- WebSockets: still TCP retransmits
- WebRTC/UDP: skip lost packets



LIVEKIT.IO

was designed for media. UDP doesn't guarantee packet order or delivery, which sounds bad until you realize that for live audio, skipping a lost packet is actually better than waiting for it. WebRTC builds smart handling on top of UDP. It uses Opus, a codec optimized for speech that compresses audio efficiently.

WebRTC features

— Opus codec



LIVEKIT.IO

speech that compresses audio efficiently. Each packet carries a time stamp so the receiver knows exactly when to play it. A jitter buffer smooths out timing variations when packets arrive at uneven intervals. And an adaptive bit rate adjusts audio quality on the fly based on network conditions. All of this adds up to low latency even when the network

WebRTC features

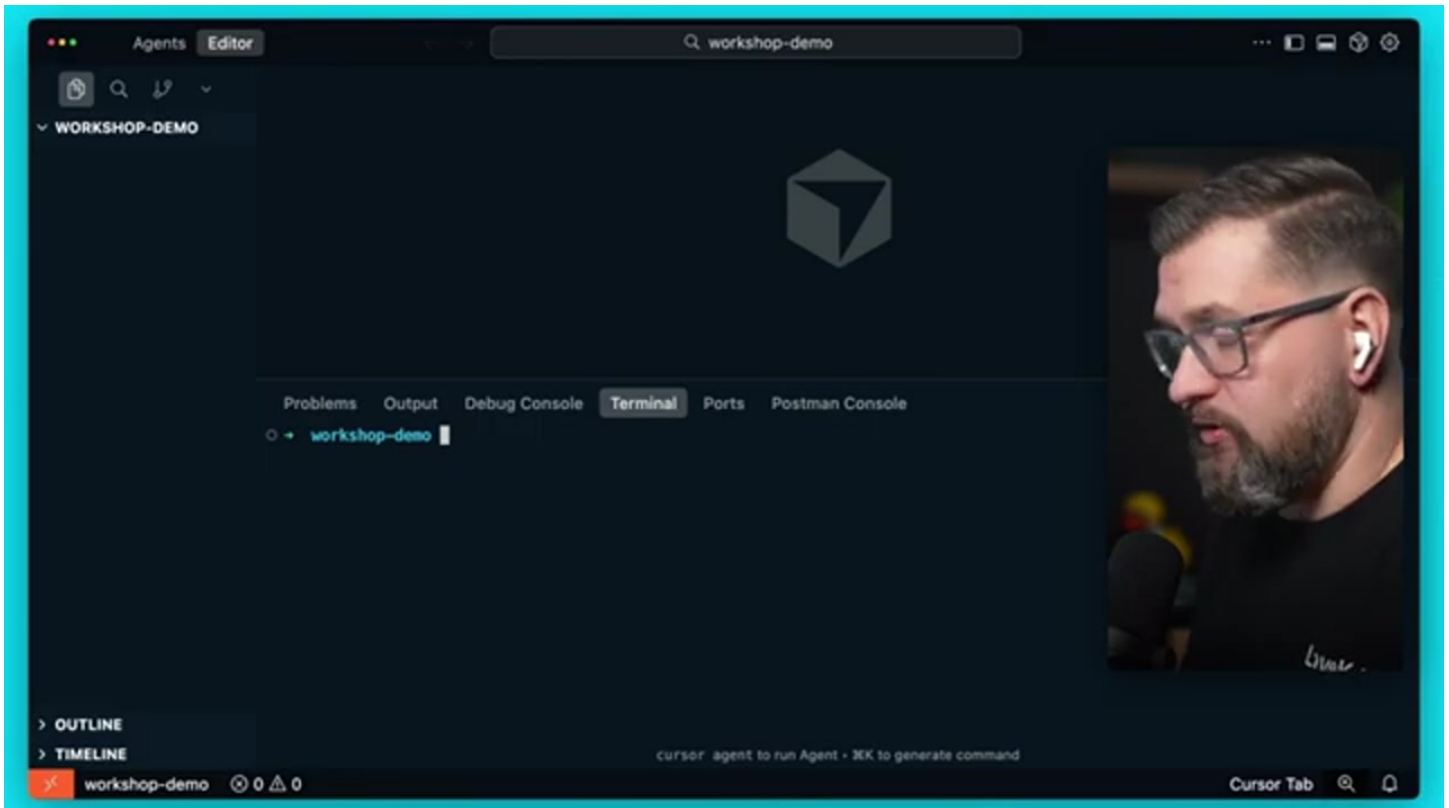
- Opus codec
- Packet timestamps
- Jitter buffer
- Adaptive bitrate



LIVEKIT.IO

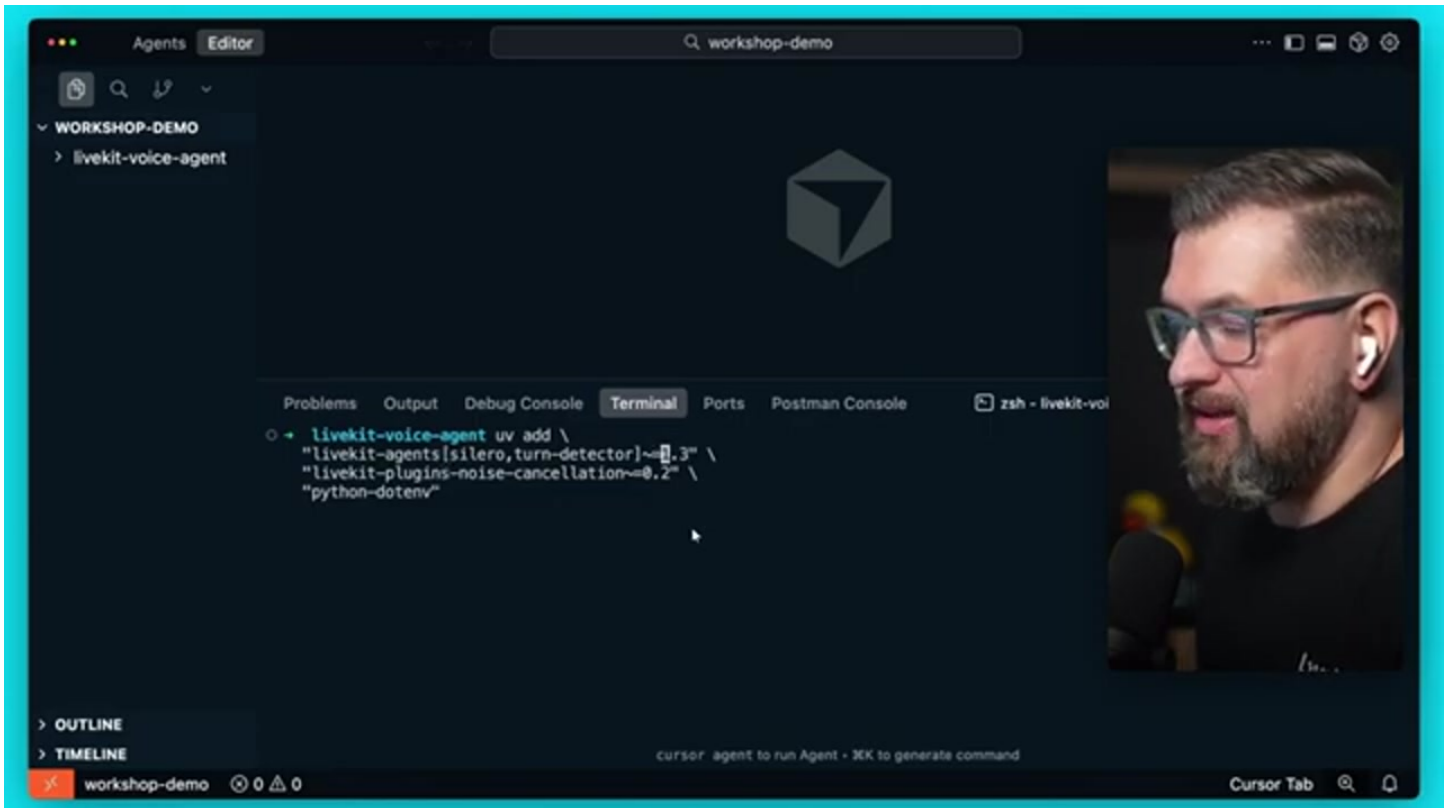
up to low latency even when the network isn't cooperating. So in practice, WebRTC lets your agent deliver first audio faster and stay responsive during packet loss and jitter. This workshop uses LiveKit's Web RTC infrastructure throughout. So let's set up a project. We're going to open up our terminal and initiate a new project using UV.

■ [08:20 - 08:40]



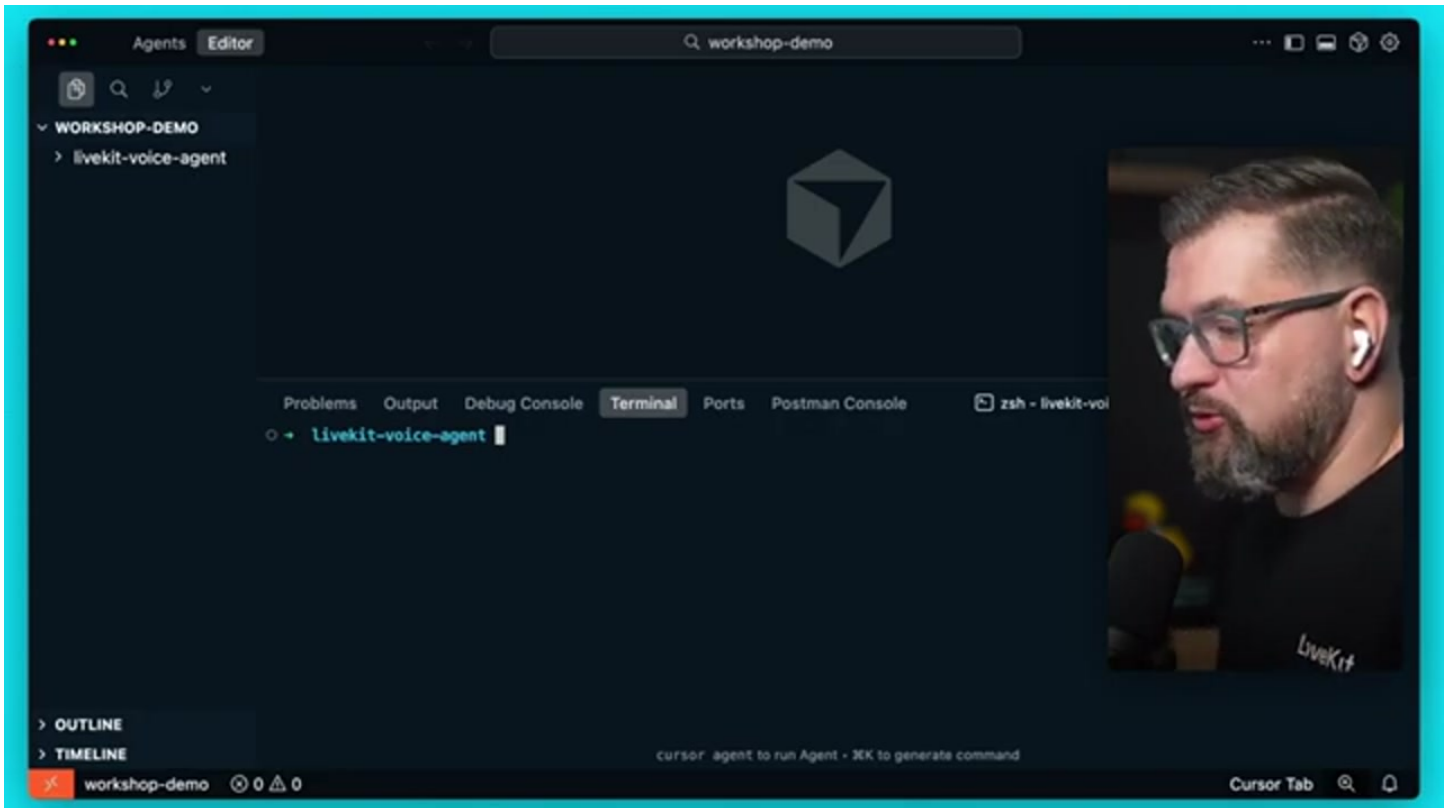
initiate a new project using UV. So the d-bear flag, it creates a minimal project without any boilerplate uh files. Next, we're going to install some dependencies. So UV add lifkit agents uh solo and turn detector. And it's good to note here we're using LifeKit agents uh

■ [08:40 - 09:00]



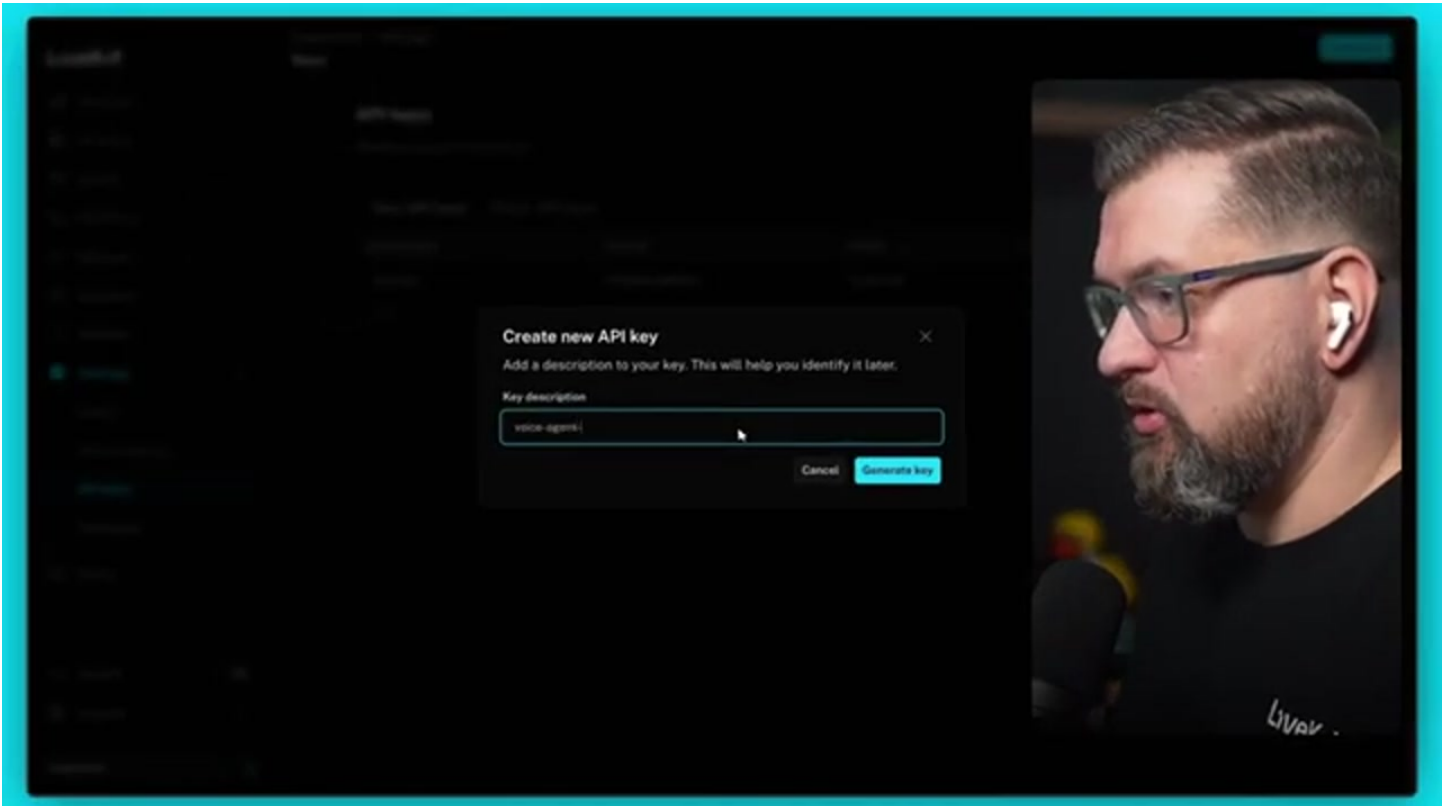
note here we're using LifeKit agents uh version 1.3. and then lifekit plugins noise cancellation and then python.vdv. So this installs the lifkit agents SDK with solar VAD and turn detection noise cancellation plugin and the python.v package for environment variables. Next,

■ [09:00 - 09:20]



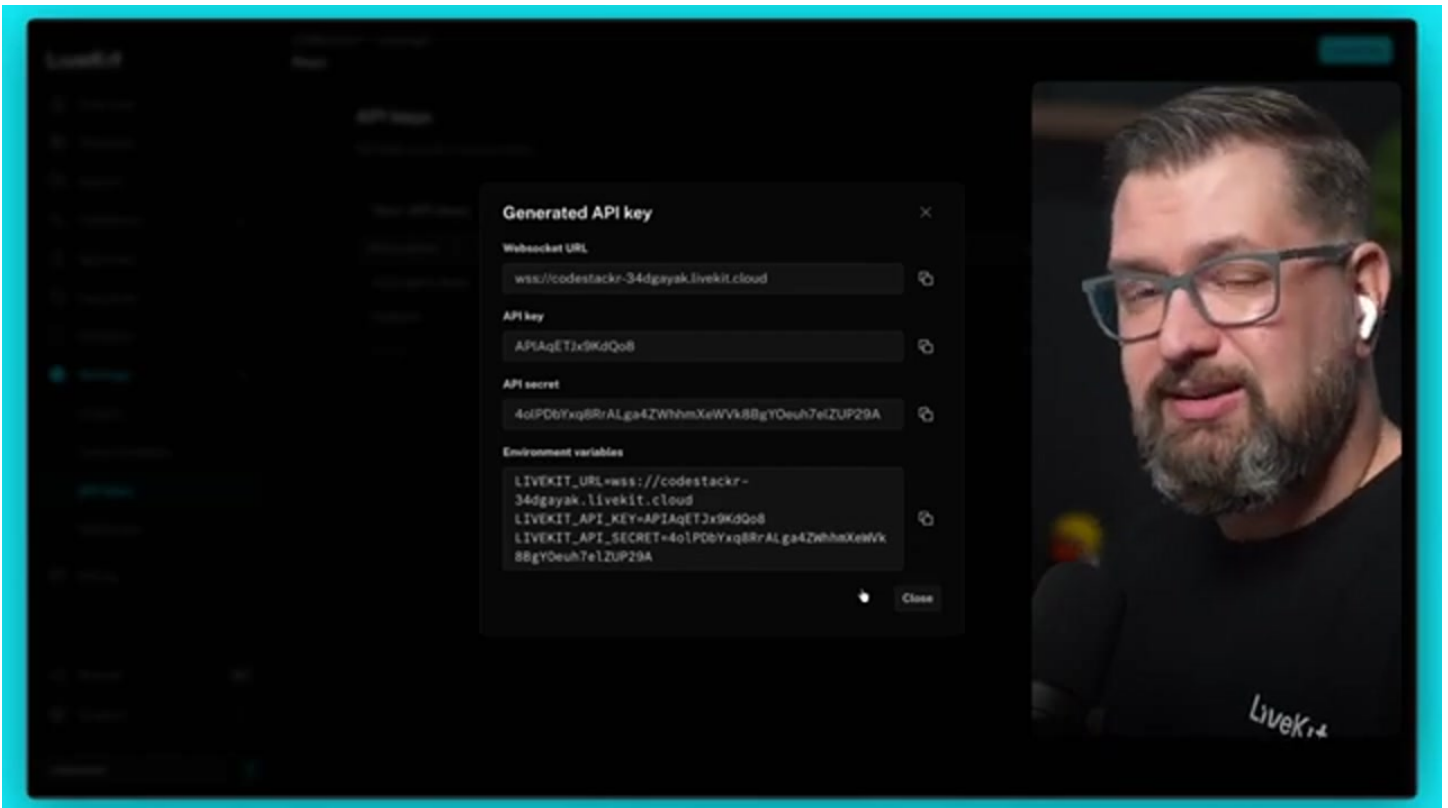
package for environment variables. Next, we need to go grab our uh API credentials from our LiveKit cloud dashboard. If you haven't created one yet, it's completely free. There's a link in the video description. So here on the dashboard, I'm going to go down to settings, API keys, and then I'm going to create a new key for this demo. And I'll just name this uh voice agent demo.

■ [09:20 - 09:40]



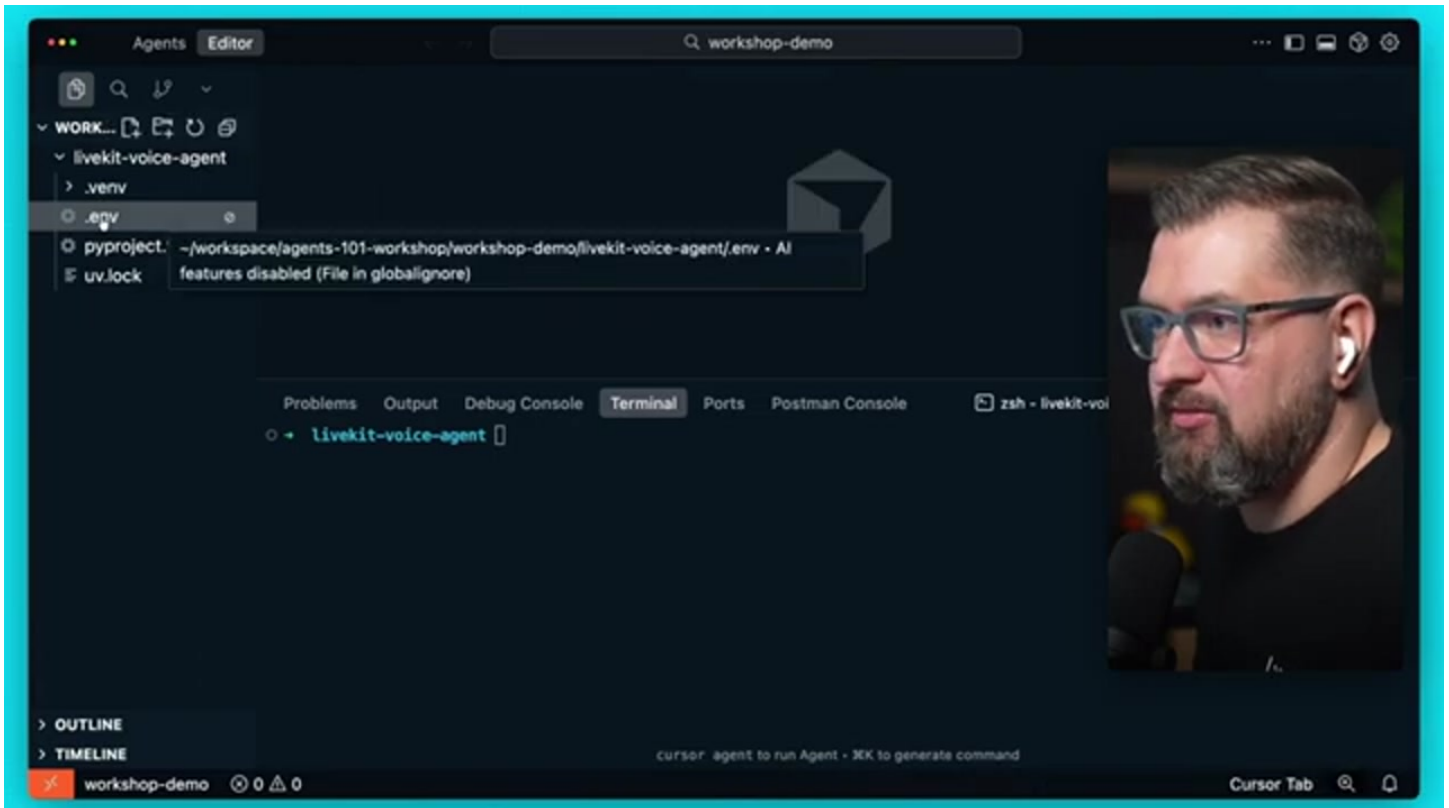
demo. And let's generate a new key. So these are the things that you're going to need. The websocket URL, the API key, and the API secret. And to make things easier, you can just copy and paste uh these environment variables right here. And of course, I'm going to delete my API key. So, this one is not going to work for you. Uh, but again,

■ [09:40 - 10:00]



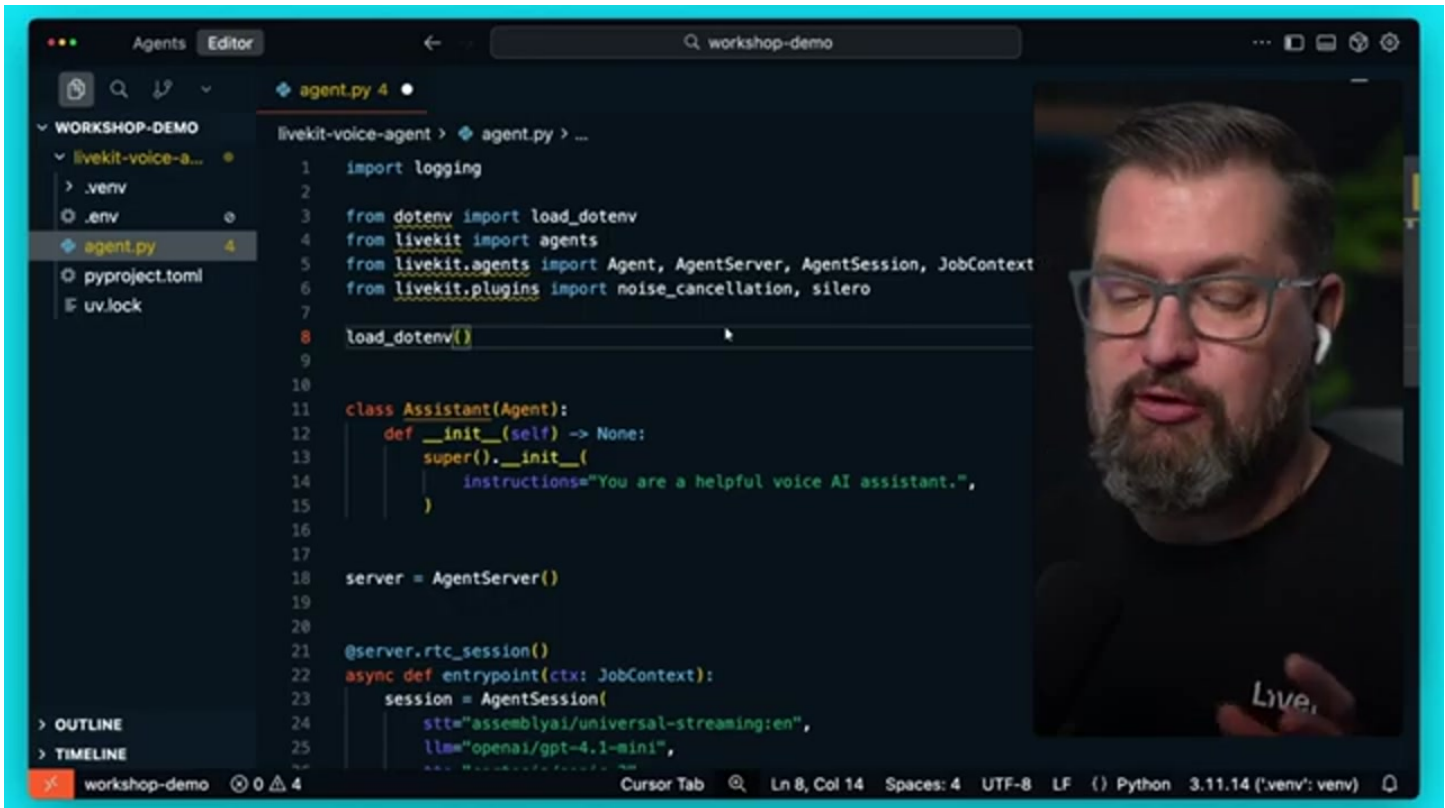
going to work for you. Uh, but again, it's completely free. Just go ahead and set one up for yourself. So, let's go back to the editor and we'll create a newv file. And we'll go ahead and paste in those credentials. Again, we have our LifeKit URL, our LifeKit API key, and our LifeKit API secret. So, make sure that you save that file, and then you can close it. And also make sure that the file is in your project directory.

■ [10:00 - 10:20]



the file is in your project directory. Okay, let's go ahead and create another file um again in our project directory here. We're going to call it agent. py. And so this is going to be our starter agent. And I'm going to paste in some code here and we'll walk through it. Now again, if you're following along in the written guide, you can copy and paste the code as well from there. So at the

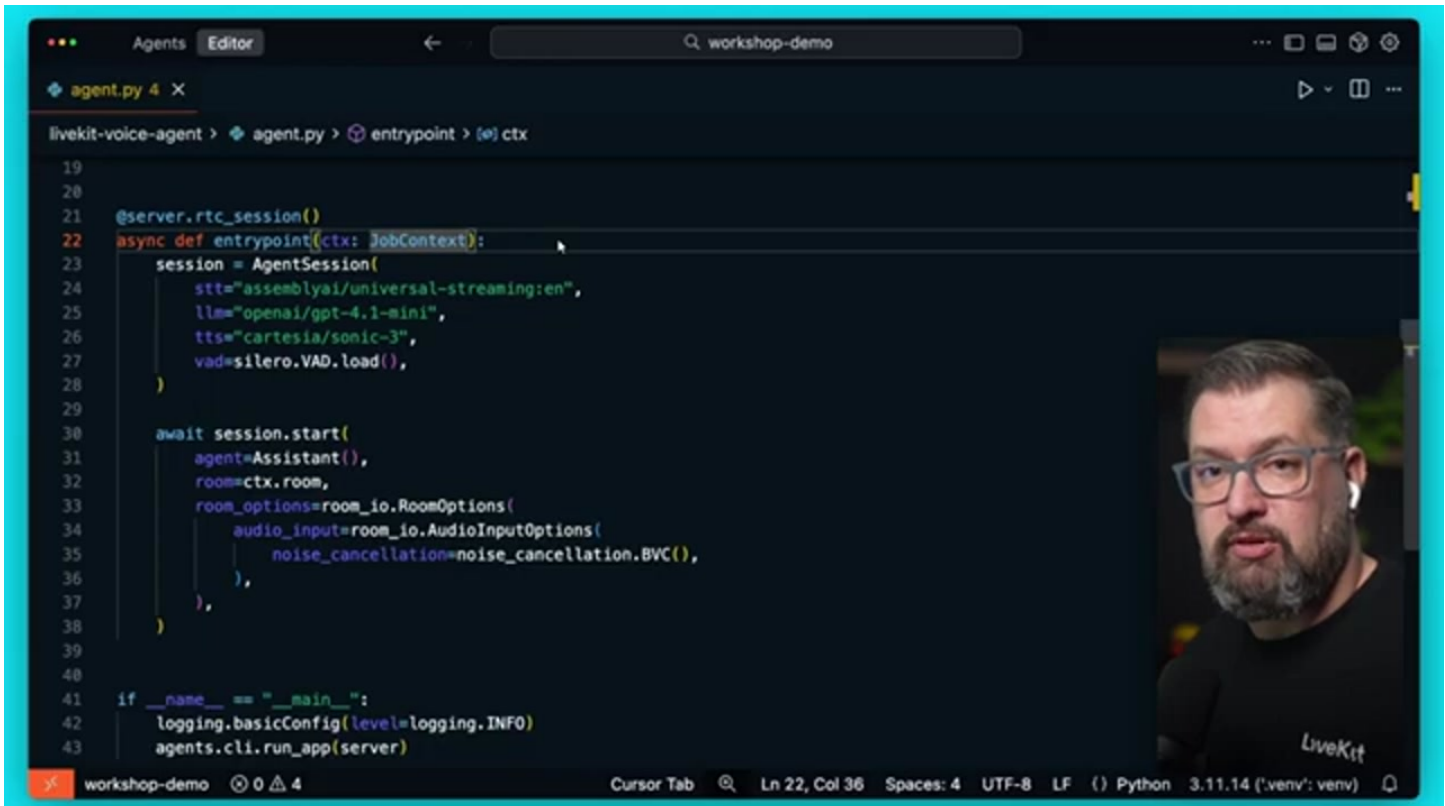
■ [10:20 - 10:40]



```
1 import logging
2
3 from dotenv import load_dotenv
4 from livekit import agents
5 from livekit.agents import Agent, AgentServer, AgentSession, JobContext
6 from livekit.plugins import noise_cancellation, silero
7
8 load_dotenv()
9
10
11 class Assistant(Agent):
12     def __init__(self) -> None:
13         super().__init__(
14             instructions="You are a helpful voice AI assistant.",
15         )
16
17
18 server = AgentServer()
19
20
21 @server rtc_session()
22 async def entrypoint(ctx: JobContext):
23     session = AgentSession(
24         stt="assemblyai/universal-streaming:en",
25         llm="openai/gpt-4.1-mini",
```

the code as well from there. So at the top here, we've got our imports from the LifeKit agents SDK. The agent class defines your agents behavior. The agent server handles dispatching sessions. And the agent session configures the voice pipeline. We have a job context here which provides information about the current job when a particular uh

■ [10:40 - 11:00]



```
livekit-voice-agent > agent.py > entrypoint > [0] ctx
19
20
21 @server rtc_session()
22 async def entrypoint(ctx: JobContext):
23     session = AgentSession(
24         stt="assemblyai/universal-streaming:en",
25         llm="openai/gpt-4.1-mini",
26         tts="cartesia/sonic-3",
27         vad=silero.VAD.load(),
28     )
29
30     await session.start(
31         agent=Assistant(),
32         room=ctx.room,
33         room_options=room_io.RoomOptions(
34             audio_input=room_io.AudioInputOptions(
35                 noise_cancellation=noise_cancellation.BVC(),
36             ),
37         ),
38     )
39
40
41 if __name__ == "__main__":
42     logging.basicConfig(level=logging.INFO)
43     agents.cli.run_app(server)
```

workshop-demo 0 4 Cursor Tab Ln 22, Col 36 Spaces: 4 UTF-8 LF () Python 3.11.14 (".venv": venv)

current job when a particular uh participant joins. And then in our session start we have uh the room IO which provides options for configuring audio input and output. So in the agent session here you can see we're configuring our ST LLM and TTS and also our VAD. And just up here we have our basic instructions for this agent. you are a helpful voice AI agent. So very

■ [11:00 - 11:20]

```
class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions="You are a helpful voice AI assistant.",

server = AgentServer()

@server rtc_session()
async def entrypoint(ctx: JobContext):
    session = AgentSession(
        stt="assemblyai/universal-streaming:en",
        llm="openai/gpt-4.1-mini",
        tts="cartesia/sonic-3",
        vad=silero.VAD.load(),
    )
```



are a helpful voice AI agent. So very simple starter agent here for you. So let's go ahead and get this agent running and see what it does. So again, be sure that you have saved this file. Let's open up our terminal. So first thing that we need to do is download the required model files like the Solero VAD model. Now we only have to do this the very first time. So uvun uh agent.py and

■ [11:20 - 11:40]

```
class Assistant(Agent):  
    def __init__(self) -> None:  
        super().__init__(  
            instructions="You are a helpful voice AI assistant.",  
        )
```

Output Debug Console Terminal Ports Postman Console

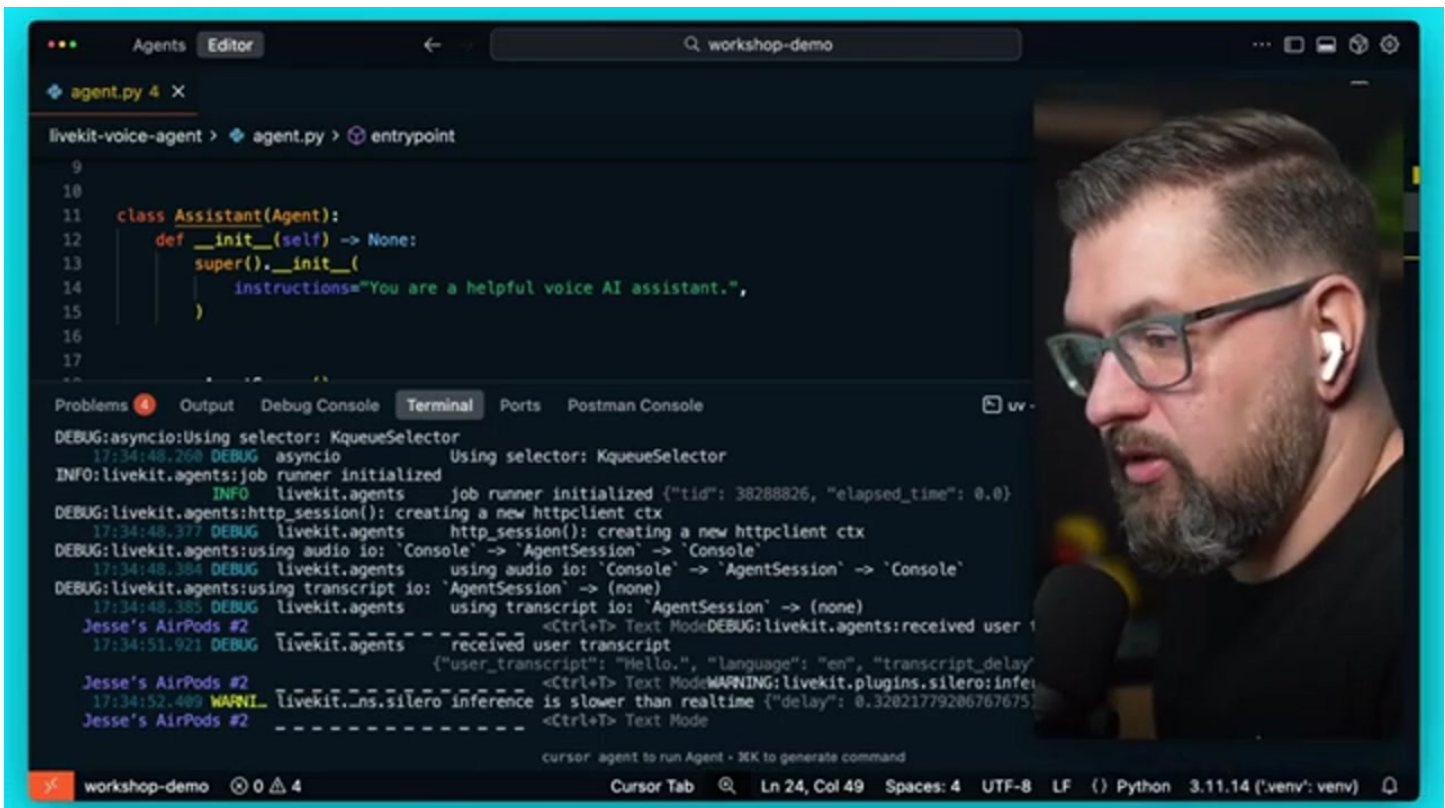
zsh - livekit-vo

t-voice-agent uv run ag



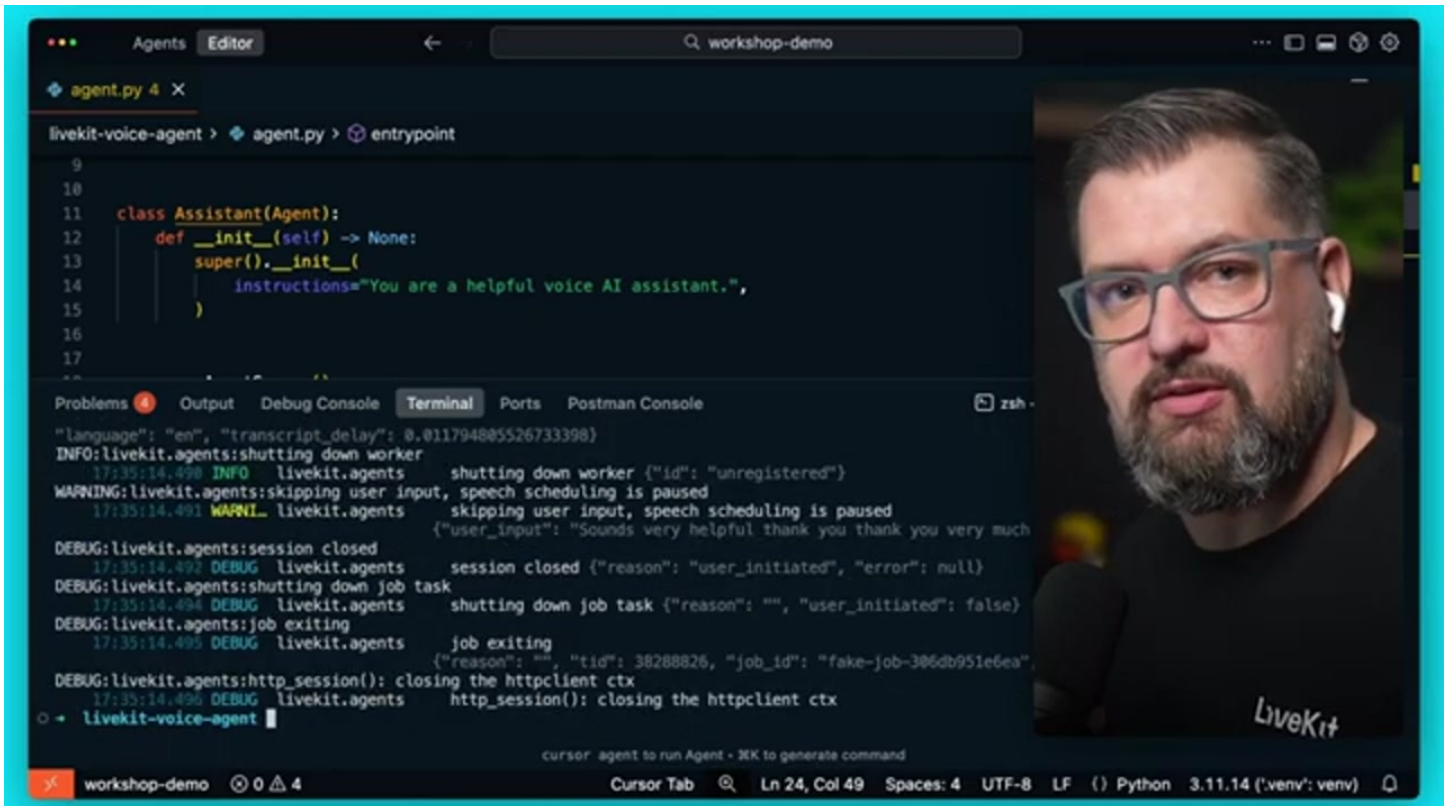
very first time. So uvun uh agent.py and then download files. Now let's run the agent. So we can just do the same thing except remove the download files. So, u agent and then console. So, we're going to actually run this right here in the console. Hello. >> Hello. How can I assist you today?

■ [11:40 - 12:00]



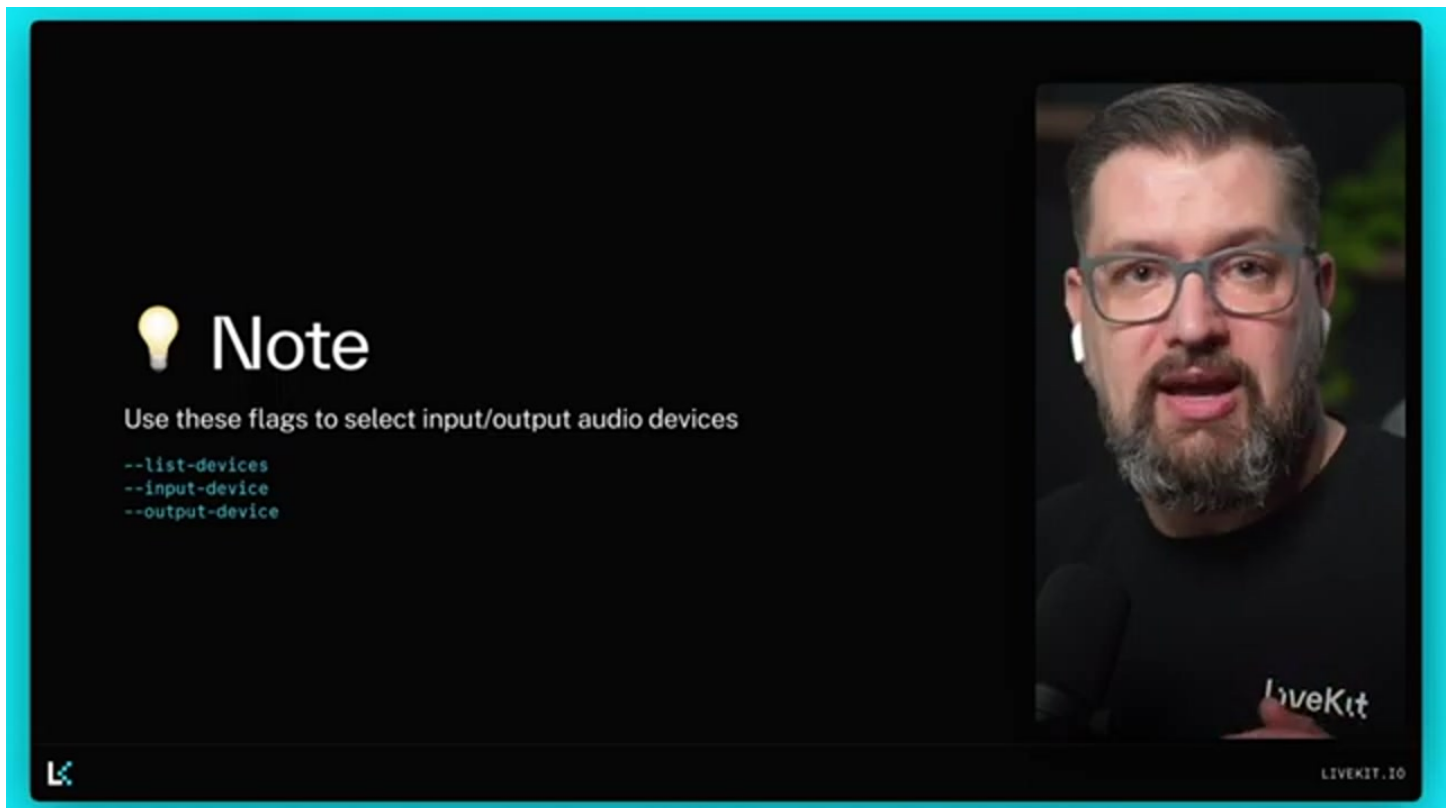
>> Hello. How can I assist you today? >> Oh, great. What What can you assist me with? >> I can help with a variety of tasks such as answering questions, providing information, assisting with writing and editing, helping with learning new topics, generating ideas, offering recommendations, and much more. What do you need? >> Very helpful. Thank you. Thank you very much. We'll we'll talk to you again later. Bye-bye. Very helpful assistant.

■ [12:00 - 12:20]



later. Bye-bye. Very helpful assistant. Very nice. Now, to stop your agent from running, just press control C. Now, if you're having any issues here in the console, uh it could be your input or output device selection. Uh so, you can actually add some extra flags there to select your input device. List devices uh lists all of the devices that you have on your machine. And then you can use the device ID to set the input and

■ [12:20 - 12:40]



 **Note**

Use these flags to select input/output audio devices

```
--list-devices  
--input-device  
--output-device
```

liveKit

LIVEKIT.IO

use the device ID to set the input and output devices using the input device and output device flag. Now, voice agents deal with messy real world inputs. People mumble. They change their minds midsentence, and they ask things in ways that you never anticipated. So, starting to think about how you'll test

■ [12:40 - 13:00]

Testing mindset



LiveKit

LIVEKIT.IO

starting to think about how you'll test these scenarios now will save you headaches later. Now, testing and metrics, they come up later uh in a later lesson, but start keeping a backlog of scenarios that you want to test as features land. Uh so think about like edge cases, different accents, background noises and interruptions.

■ [13:00 - 13:20]



✓

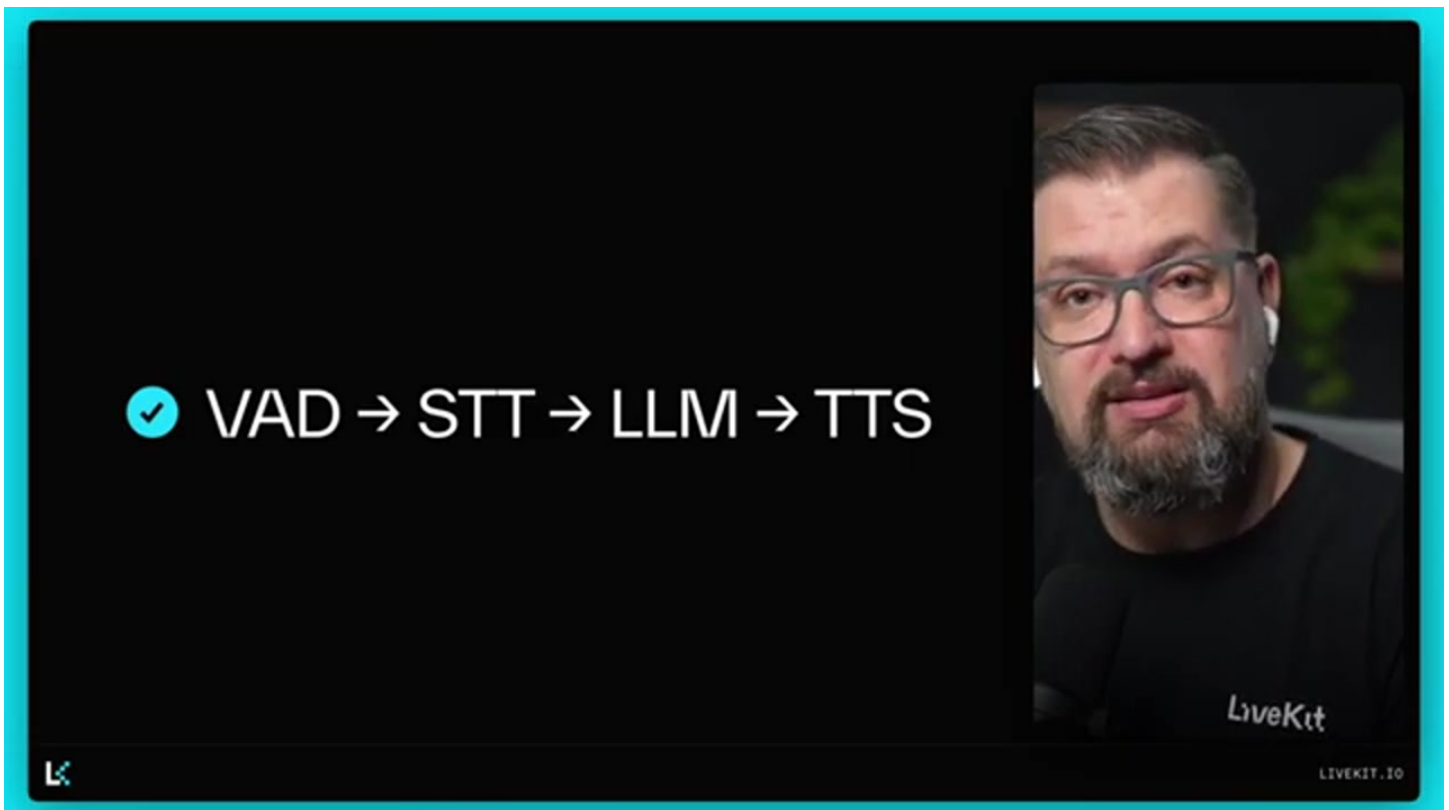
VAD → STT → LLM → TTS

LiveKit

LIVEKIT.IO

background noises and interruptions. So now you have a working voice agent and you understand the core pipeline. The that VAD detects voice ST transcribes it. The LLM generates a response and TTS uh speaks it back. So, in the next lesson, you'll add semantic turn detection to prevent those awkward

■ [13:20 - 13:40]



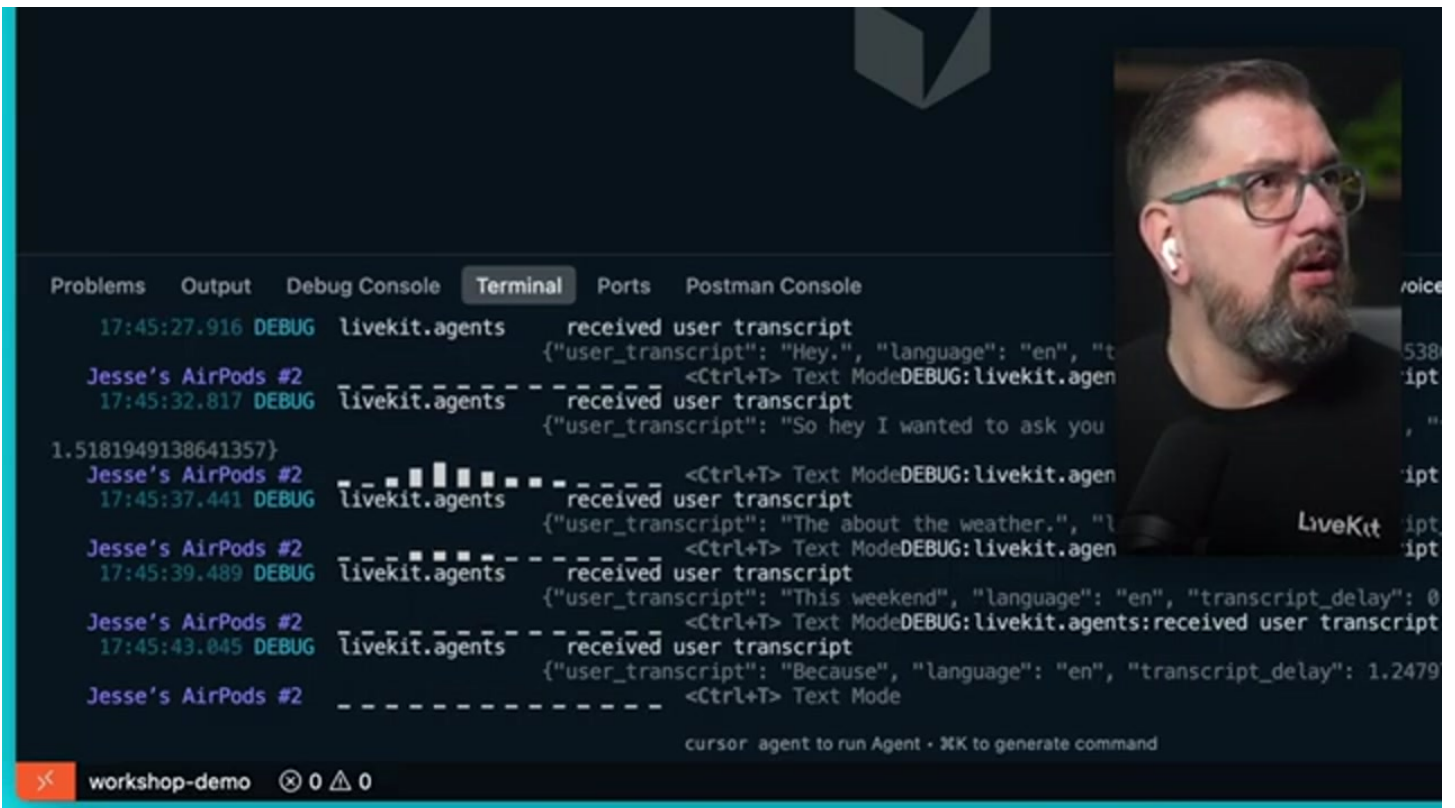
✓ VAD → STT → LLM → TTS

LiveKit

LIVEKIT.IO

turn detection to prevent those awkward interruptions when an agent jumps in too early. Hey, I wanted to ask you um about >> Sure. What would you like? >> No, about the weather uh this weekend. Um because I'm um

■ [13:40 - 14:00]



The screenshot shows a VS Code editor with a terminal window open. The terminal displays debug logs for 'livekit.agents' showing 'received user transcript' events. The logs include timestamps and JSON objects with 'user_transcript' and 'language' fields. A video inset in the top right corner shows a man with glasses and a beard speaking into a microphone. The status bar at the bottom indicates 'workshop-demo' and '0' errors.

```
17:45:27.916 DEBUG livekit.agents received user transcript
{"user_transcript": "Hey.", "language": "en", "t
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agen
17:45:32.817 DEBUG livekit.agents received user transcript
{"user_transcript": "So hey I wanted to ask you
1.5181949138641357}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agen
17:45:37.441 DEBUG livekit.agents received user transcript
{"user_transcript": "The about the weather.", "l
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agen
17:45:39.489 DEBUG livekit.agents received user transcript
{"user_transcript": "This weekend", "language": "en", "transcript_delay": 0
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:received user transcript
17:45:43.045 DEBUG livekit.agents received user transcript
{"user_transcript": "Because", "language": "en", "transcript_delay": 1.2479
Jesse's AirPods #2 <Ctrl+T> Text Mode
cursor: agent to run Agent • ⌘K to generate command
```

Um because I'm um >> Oh, no. I'm planning a trip to um Sounds exciting. >> So that's what happens when your agent relies only on voice activity detection. It hears a pause and it thinks that you're done. But humans pause mid-sentence all the time. We think,

■ [14:00 - 14:20]

Semantic turn detection



LiveKit

LIVEKIT.IO

mid-sentence all the time. We think, we breathe, we change direction. Uh, a good conversational agent needs to understand when someone is actually finished speaking. And so turn detection decides when the agent should speak and when it should stay quiet. And without it, you would get interruptions. And the

■ [14:20 - 14:40]

Semantic turn detection



LiveKit

LIVEKIT.IO

it, you would get interruptions. And the agent starts answering at every single pause, even if the human is still mid-sentence. And so in this lesson, you'll add a semantic turn detector that dramatically improves conversational quality. Now, voice activity detection spots human patterned audio, but humans pause,

■ [14:40 - 15:00]

VAD vs Semantic detection



LiveKit

LIVEKIT.IO

human patterned audio, but humans pause, restart sentences, and trail off all the time. And so, VAD sees a pause and it thinks that you're done talking. But semantic detection looks at all of the the meaning of the sentence and it asks, does this feel like a complete thought? The most important job of a turn detector is to reduce unwanted

Turn detection

- Reduces unwanted interruptions

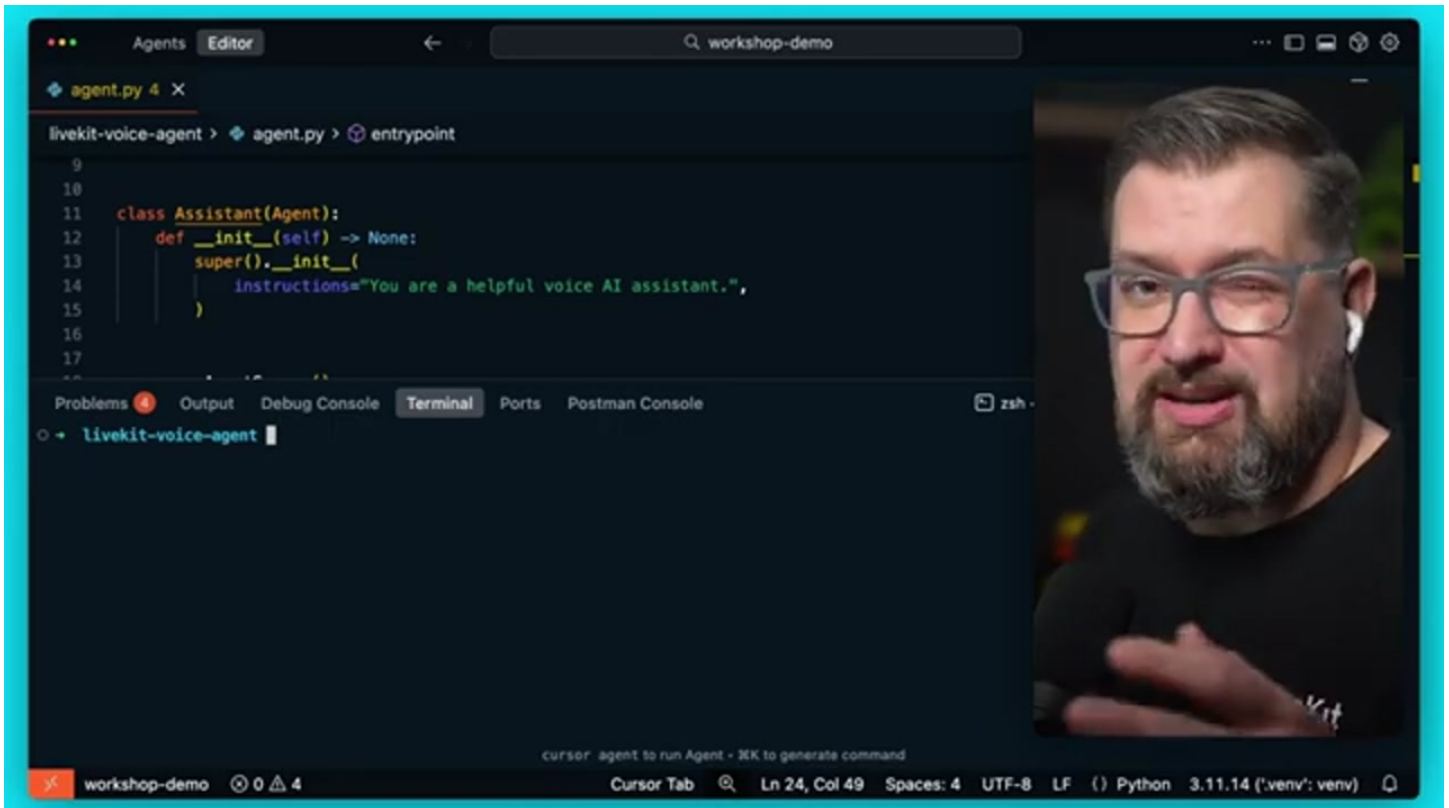


LiveKit

LIVEKIT.IO

detector is to reduce unwanted interruptions. It also improves accuracy for the ST engine since it gets complete utterances instead of fragments and the latency impact is minimal only around 20 milliseconds. So the first thing that we'll need to do is add the turn detector extra to our environment. So in Python an extra is an optional uh set of

■ [15:20 - 15:40]



Python an extra is an optional uh set of dependencies that a package can include. So we're going to UV add lifkit agents turn detector. So, this pulls in a multilingual model uh that you'll reference in the code, right? So, we'll go ahead and import this multilingual model from our uh turn detector plugin.

■ [15:40 - 16:00]

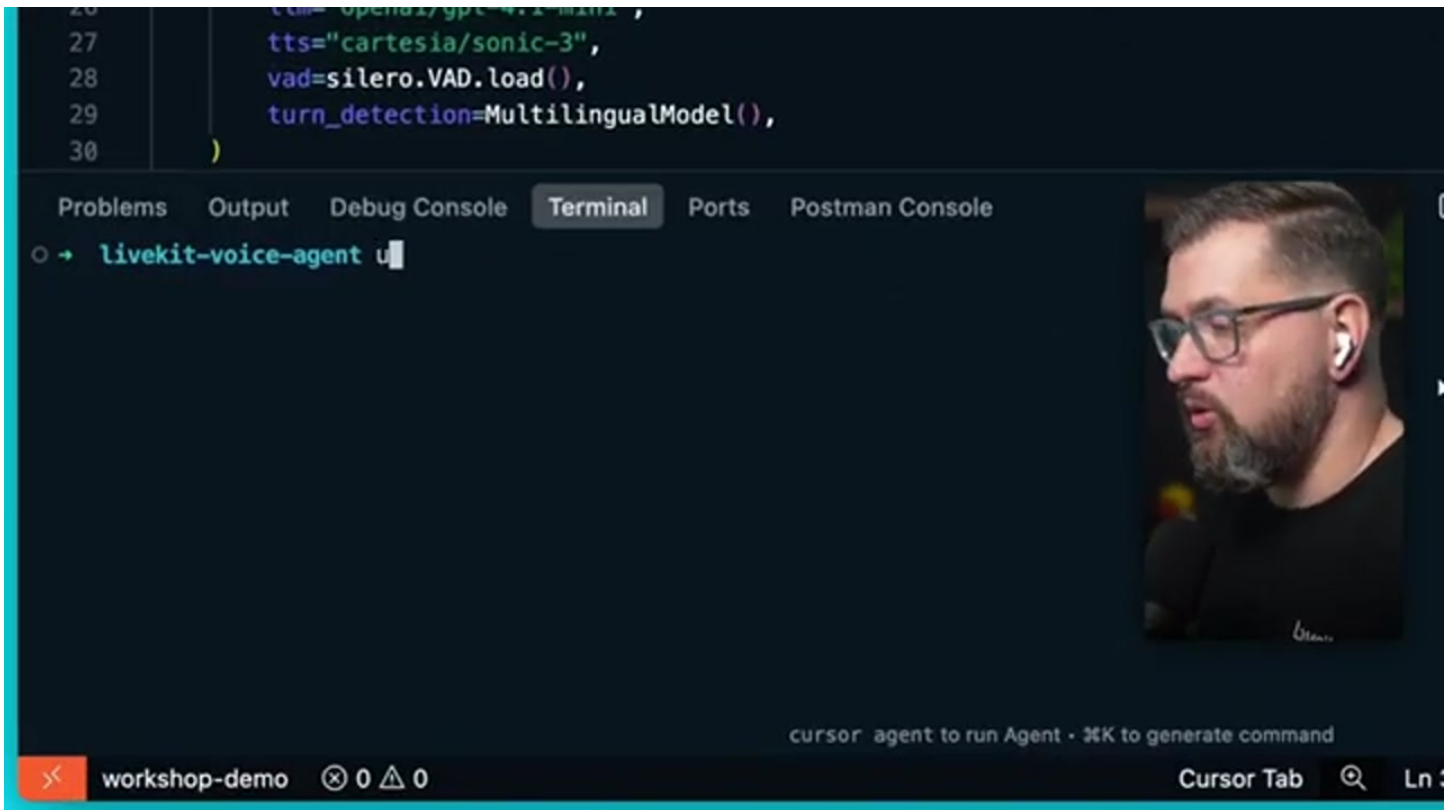
```
agent.py x
livekit-voice-agent > agent.py

22 @server rtc_session()
23 async def entrypoint(ctx: JobContext):
24     session = AgentSession(
25         stt="assemblyai/universal-streaming:en",
26         llm="openai/gpt-4.1-mini",
27         tts="cartesia/sonic-3",
28         vad=silero.VAD.load(),
29     )
30
31     await session.start(
32         agent=Assistant(),
33         room=ctx.room,
34         room_options=room_io.RoomOptions(
35             audio_input=room_io.AudioInputOptions(
36                 noise_cancellation=noise_cancellation.BVC(),
37             ),
38         ),
39     )
40
```



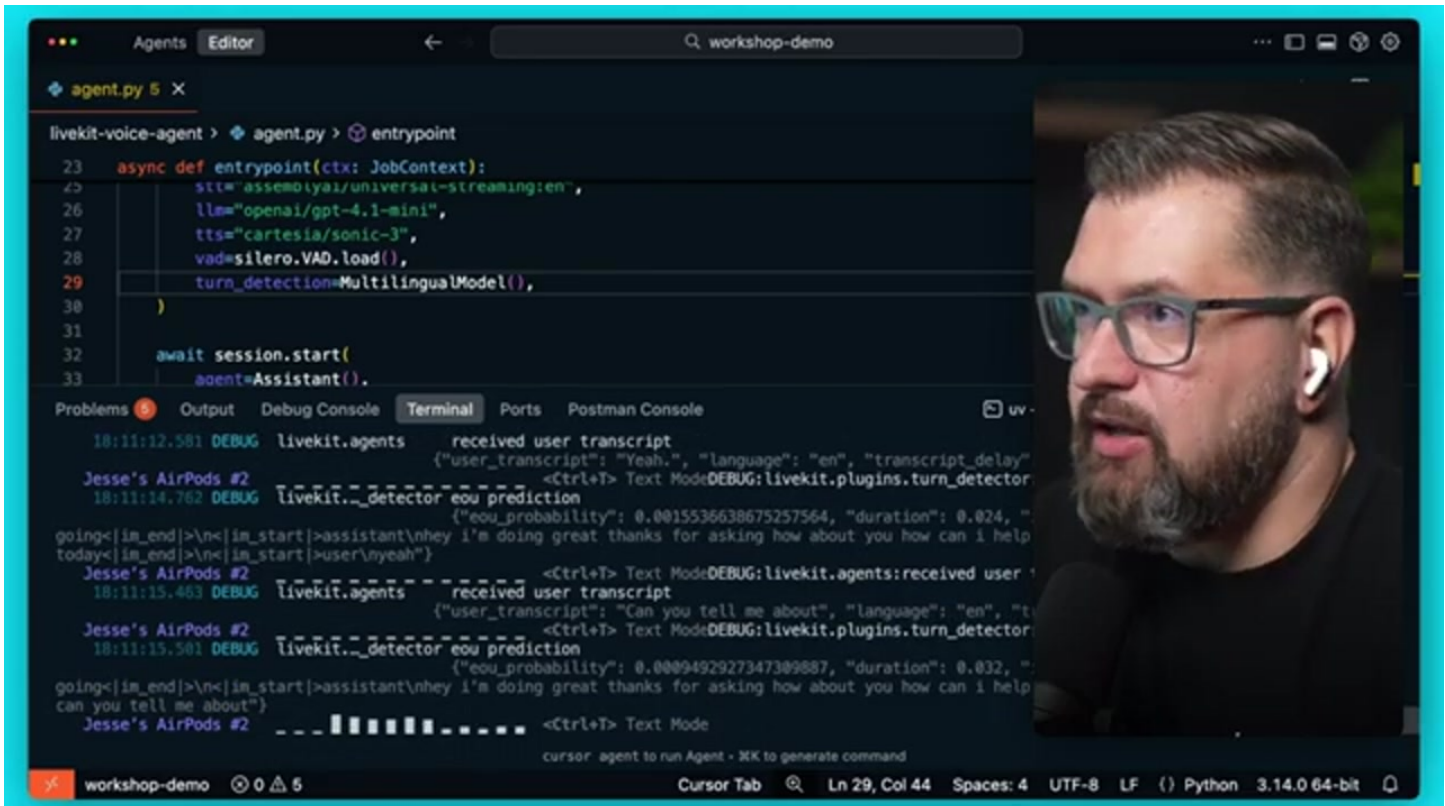
model from our uh turn detector plugin. We'll go down to our agent session here and then we'll add into our agent session this uh turn detection and pass it the multilingual model. Now, save that. And that's all that you have to do. The session now emits turn events when the semantic model decides a speaker is finished. So, let's go ahead and test it out. All right, let's open up our terminal. We'll run `uv run`

■ [16:00 - 16:20]



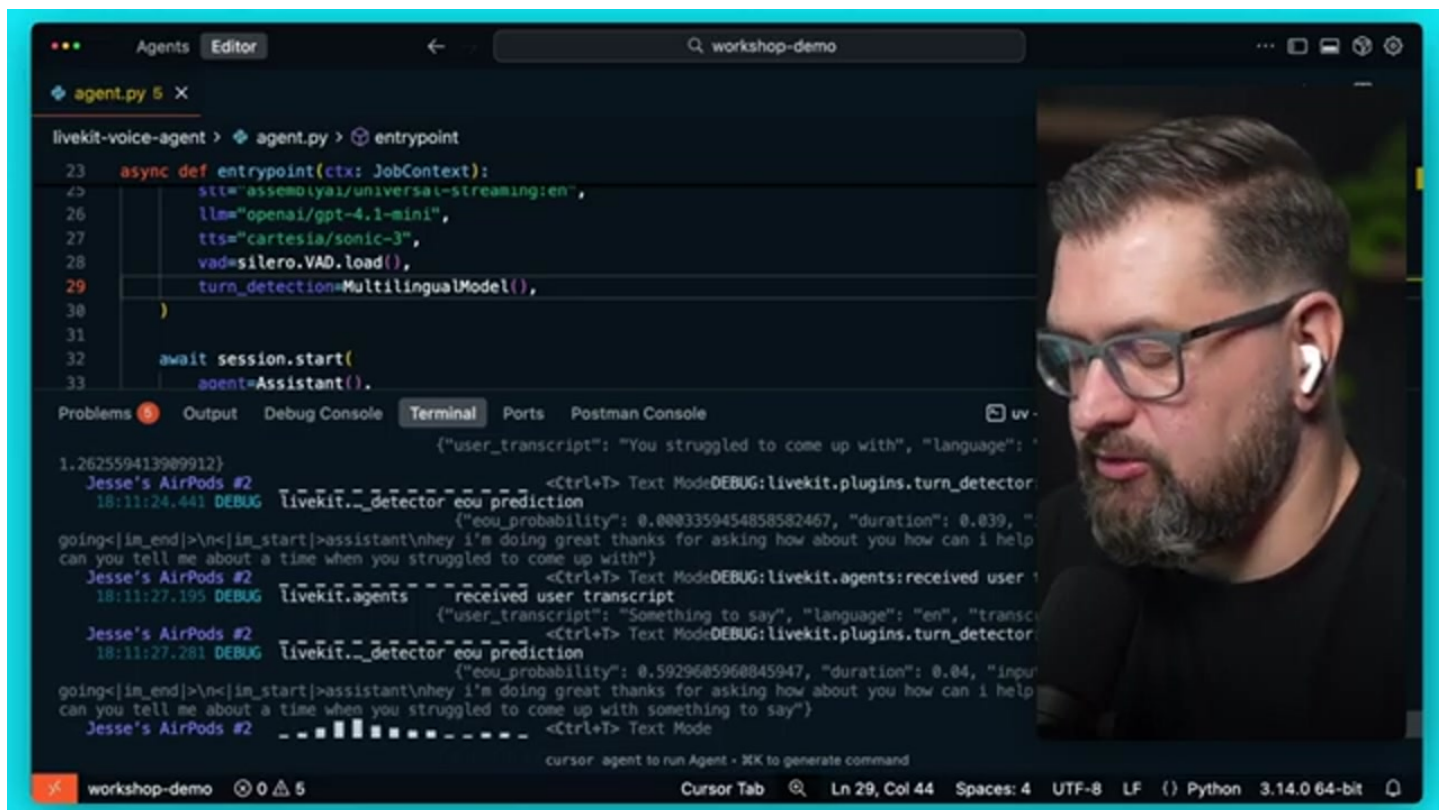
up our terminal. We'll run `uv run agent.py` and console to run it in the console. Let's see if it's any better. Yeah. Hey, how's it going? >> Hey, I'm doing great. Thanks for asking. >> How about Can you tell me about um about a time when

■ [16:20 - 16:40]



about a time when uh you you struggled to come up with something to say? >> Sure. Even though I'm designed to generate responses quickly, there are times when a question is very vague or complex and it can be challenging to provide me.

■ [16:40 - 17:00]



The screenshot displays a live coding environment with a video feed of a man on the right. The main window shows a Python script in a file named `agent.py`. The script defines an `entrypoint` function that initializes a voice agent using `assemblyai` for streaming, `openai/gpt-4.1-mini` for LLM, `cartesia/sonic-3` for TTS, and `silero.VAD` for voice activity detection. The terminal output shows the agent's internal state and user transcripts.

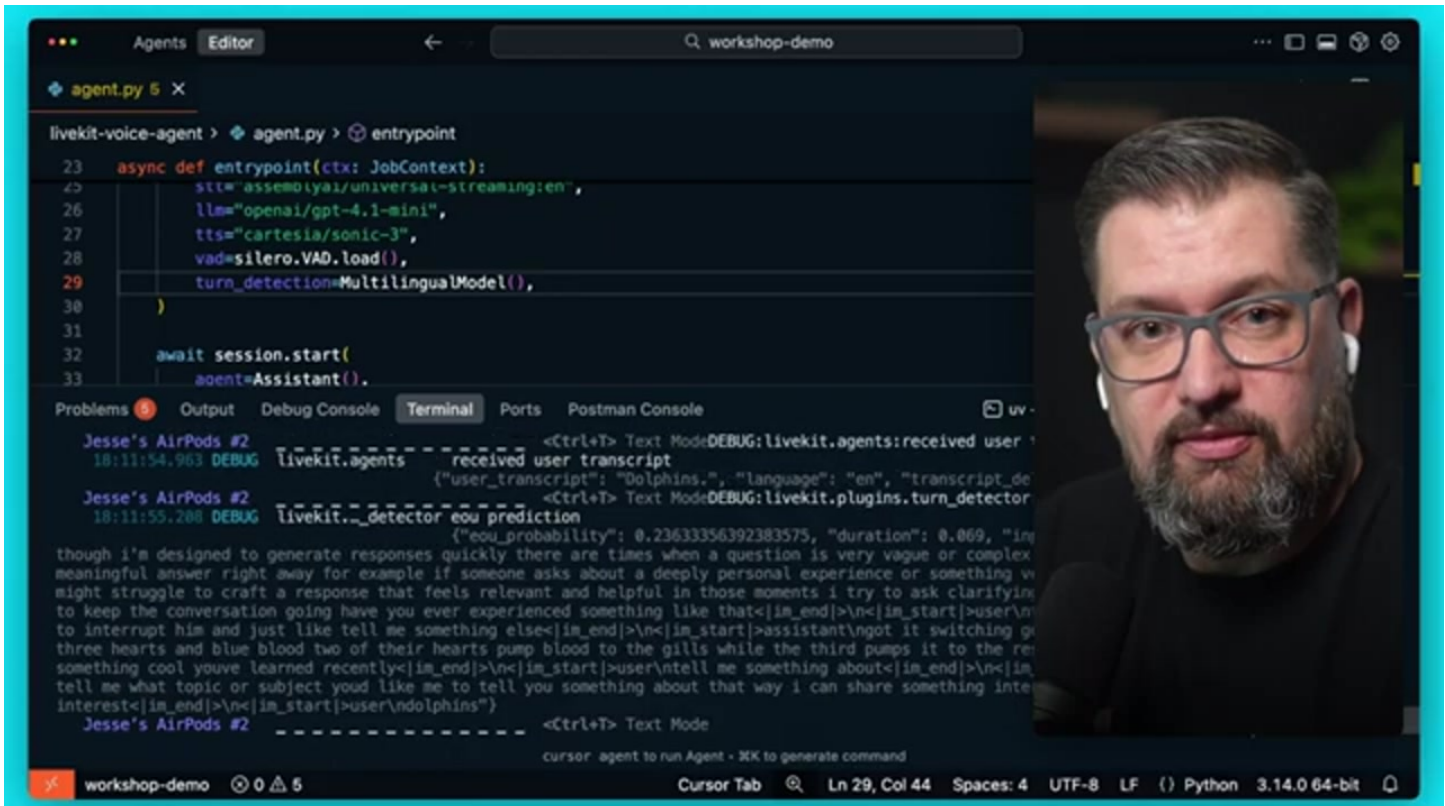
```
livekit-voice-agent > agent.py > entrypoint
23 async def entrypoint(ctx: JobContext):
24     stt="assemblyai/universal-streaming:en",
25     llm="openai/gpt-4.1-mini",
26     tts="cartesia/sonic-3",
27     vad=silero.VAD.load(),
28     turn_detection=MultilingualModel(),
29 )
30
31
32 await session.start(
33     agent=Assistant(),
34 )
```

Terminal Output:

```
{ "user_transcript": "You struggled to come up with", "language": "en", "transcription": "You struggled to come up with", "eou_probability": 0.0003359454858582467, "duration": 0.039, "input_duration": 0.039 }
Jesse's AirPods #2 18:11:24.441 DEBUG livekit._detector eou prediction
going<|im_end|>\n<|im_start|>assistant\nhey i'm doing great thanks for asking how about you how can i help can you tell me about a time when you struggled to come up with"
Jesse's AirPods #2 18:11:27.195 DEBUG livekit.agents received user transcript
Jesse's AirPods #2 18:11:27.281 DEBUG livekit._detector eou prediction
{"user_transcript": "Something to say", "language": "en", "transcription": "Something to say", "eou_probability": 0.5929685968845947, "duration": 0.04, "input_duration": 0.04 }
Jesse's AirPods #2 18:11:27.281 DEBUG livekit._detector eou prediction
going<|im_end|>\n<|im_start|>assistant\nhey i'm doing great thanks for asking how about you how can i help can you tell me about a time when you struggled to come up with something to say"
Jesse's AirPods #2 18:11:27.281 DEBUG livekit._detector eou prediction
```

provide me. >> Let's try to interrupt him and like just like tell me something else. Got it. Switching gears. Did you know that octopuses have three hearts in blue blood? Two of their hearts. >> Tell me um something about um dolphins.

■ [17:00 - 17:20]



The screenshot shows a VS Code editor window titled "workshop-demo". The editor is open to a file named "agent.py" at line 5. The code defines an "entrypoint" function for a "livekit-voice-agent". The function sets up a session with OpenAI's GPT-4.1-mini model, Cartesia's Sonic-3 TTS, and Silero VAD. It also includes a turn detection plugin using a MultilingualModel. The terminal output shows the agent receiving a user transcript about dolphins and generating a response. The response is a long, detailed paragraph about dolphins, mentioning their intelligence, social structure, and the importance of conservation. The terminal also shows a "cow prediction" from the turn detector.

```
livekit-voice-agent > agent.py > entrypoint
23 async def entrypoint(ctx: JobContext):
24     stt="asemoi/ai/universal-streaming:en",
25     llm="openai/gpt-4.1-mini",
26     tts="cartesia/sonic-3",
27     vad=silero.VAD.load(),
28     turn_detection=MultilingualModel(),
29 )
30
31
32 await session.start(
33     agent=Assistant(),
```

Terminal Output:

```
Jesse's AirPods #2 18:11:54.963 DEBUG livekit.agents received user transcript
{"user_transcript": "Dolphins.", "language": "en", "transcript_de
Jesse's AirPods #2 18:11:55.208 DEBUG livekit.plugins.turn_detector
livekit.plugins.turn_detector cow prediction
{"eou_probability": 0.23633356392383575, "duration": 0.069, "in
though i'm designed to generate responses quickly there are times when a question is very vague or complex
meaningful answer right away for example if someone asks about a deeply personal experience or something w
might struggle to craft a response that feels relevant and helpful in those moments i try to ask clarifying
to keep the conversation going have you ever experienced something like that<|im_end|>\n<|im_start|>user\n
to interrupt him and just like tell me something else<|im_end|>\n<|im_start|>assistant\ngot it switching g
three hearts and blue blood two of their hearts pump blood to the gills while the third pumps it to the re
something cool youve learned recently<|im_end|>\n<|im_start|>user\ntell me something about<|im_end|>\n<|im
tell me what topic or subject youd like me to tell you something about that way i can share something inte
interest<|im_end|>\n<|im_start|>user\ndolphins")
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode
cursor: agent to run Agent - KK to generate command
```

dolphins. >> Absolutely. Dolphins are incredibly intelligent marine. >> Much better. Much better. Let's stop it. So, this single change dramatically improves conversational quality. The agent feels more patient and natural. And this prepares you for more advanced latency optimization in later lessons. So let's talk about some best practices.

■ [17:20 - 17:40]

Best practices



So let's talk about some best practices. Now a few things that you can keep in mind when using turn detection is make sure that you combine it with VAD and noise control for best results. Different languages have different pause patterns as well and semantic models uh help to normalize this. It also coordinates with preemptive generation,

■ [17:40 - 18:00]

Best practices

- Combine with VAD
- Language-aware pause handling



LiveKit

LIVEKIT.IO

coordinates with preemptive generation, which we're going to talk about in a future lesson. And this allows the LLM to start planning while it's still waiting for a clear end of turn. Here are a few scenarios worth testing. First, try rapidfire short sentences and compare that to a single long sentence with natural pauses. See how the agent

■ [18:00 - 18:20]

Ideas for testing

- Short sentences vs long pauses



with natural pauses. See how the agent handles each of these. Uh, second, add some background noise. turn on a TV or have someone talking nearby. Does the agent stay focused on your voice? And third, if you speak multiple languages, try switching languages midterm. The multilingual model should handle this gracefully.

Ideas for testing

- Short sentences vs long pauses
- Background noise
- Mid-sentence language switch



LiveKit

LIVEKIT.IO

gracefully. Add these to your testing backlog. The more scenarios you try, the more confident you'll be in how your agent behaves. So, with semantic turn detection in place, your agent no longer jumps in at every pause. It waits for complete thoughts before responding. In the next lesson, you'll give an agent a distinctive personality, uh, change its

■ [18:40 - 19:00]

Personality & fallbacks



LiveKit

LIVEKIT.IO

distinctive personality, uh, change its voice, and add fallback providers so that it survives outages. Now that turntaking feels natural, it's time to make the agent sound on brand. In this lesson, you'll give the assistant a distinct personality, change its voice, and configure fallback adapters so that

Personality & fallbacks



LIVEKIT.IO

and configure fallback adapters so that the agent survives provider outages. Voices aren't neutral speakers. How the LLM writes affects how the TTS sounds. So, first, match your phrasing to the voice. Shorter clauses and fewer parentheticals sound more natural when spoken. Second, be aware of dialect.

Voice + prompt = personality

- Match phrasing to voice
- Dialect awareness



LIVEKIT.IO

spoken. Second, be aware of dialect. Tailor spelling and idioms for authenticity, whether it's UK or US English. Third, use emotion control sparingly. Add subtle direction in system messages, but don't overdo it. And fourth, keep replies brief. Three sentences or fewer reduces latency and

Voice + prompt = personality

- Match phrasing to voice
- Dialect awareness
- Emotion control
- Keep replies brief

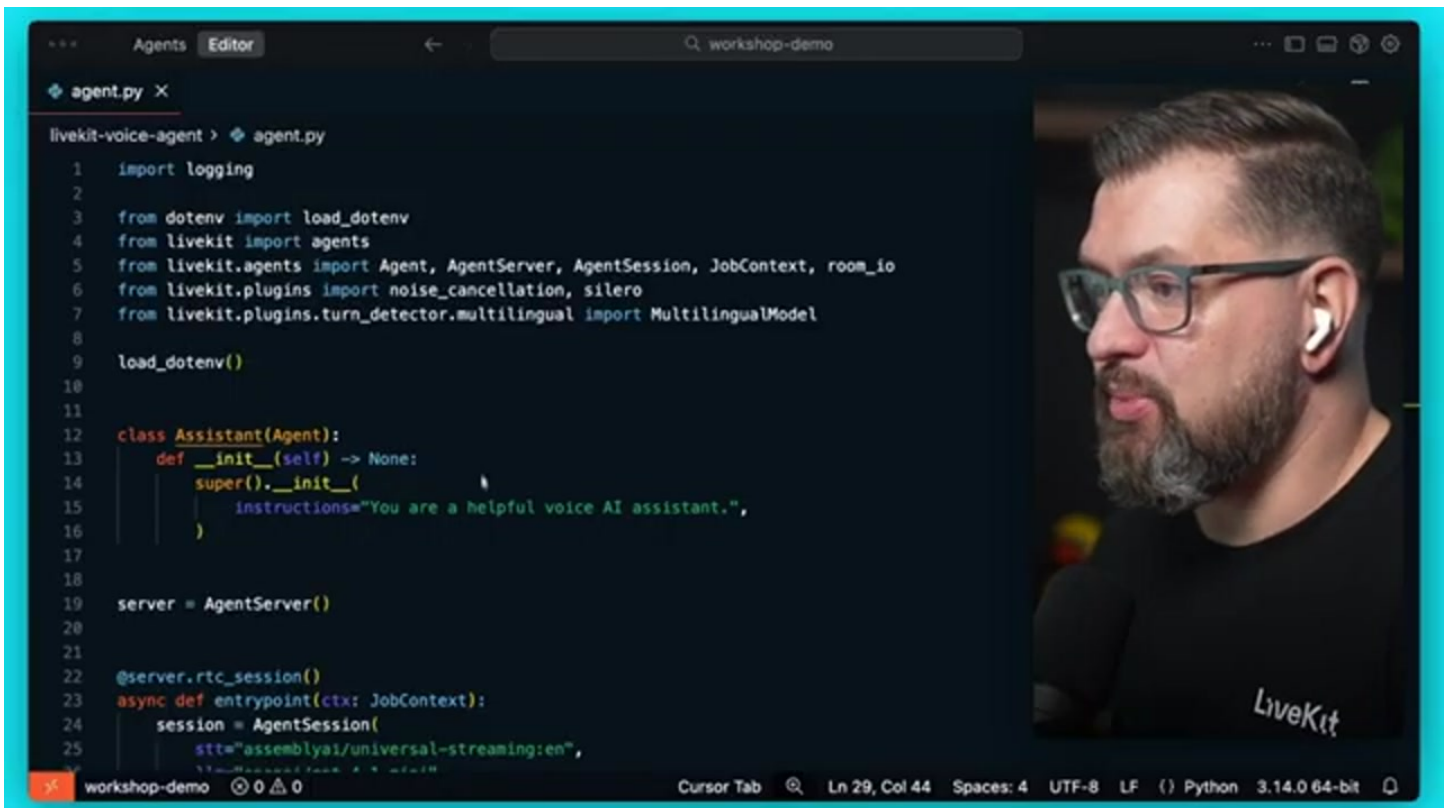


LiveKit

LIVEKIT.IO

prevents your agent from monologuing. The system prompt and the voice work together as a team. They need to complement each other. Now, your system prompt sets expectations for tone and boundaries. So, let's update the assistant class with a more specific personality. So, back in our uh agent.py

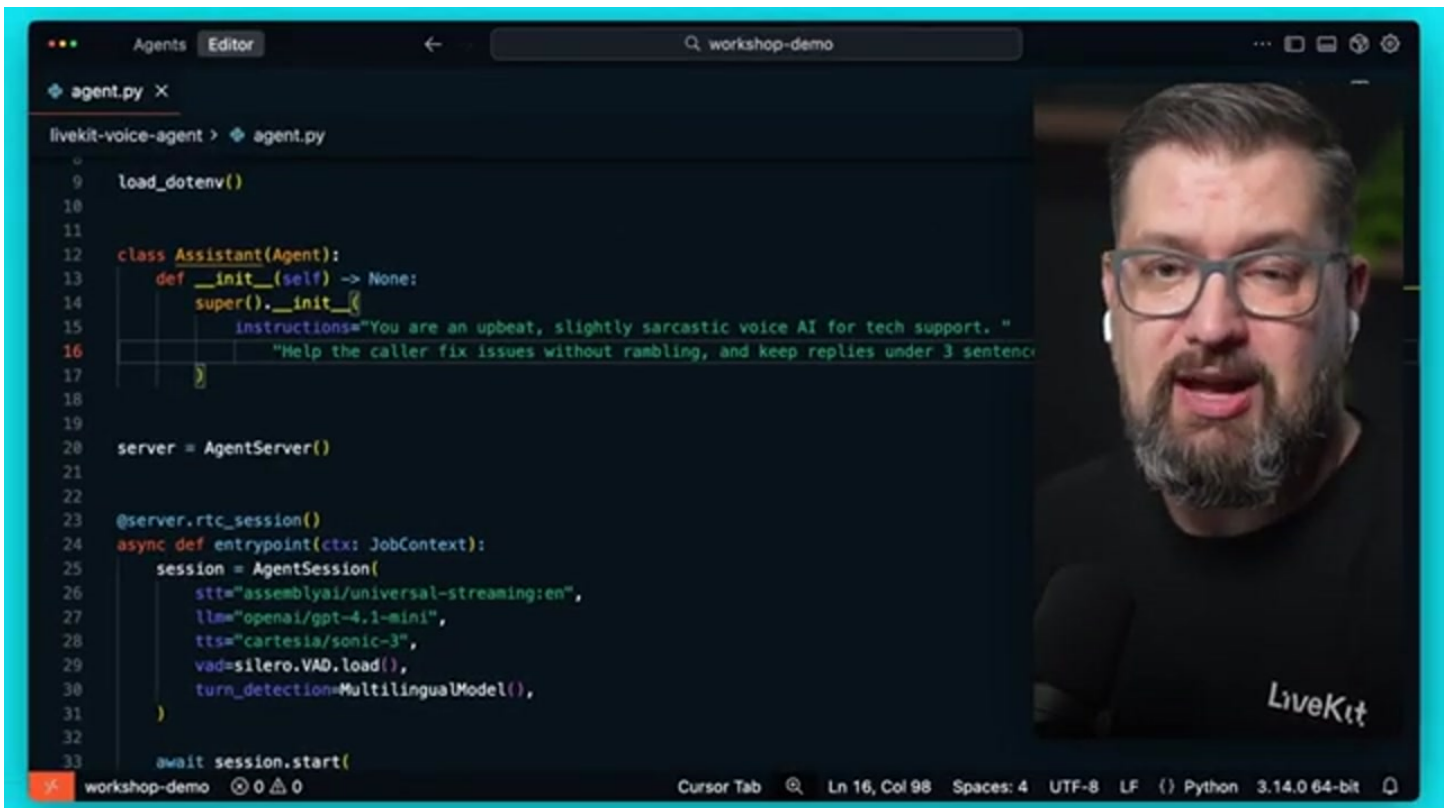
■ [20:00 - 20:20]



```
1 import logging
2
3 from dotenv import load_dotenv
4 from livekit import agents
5 from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6 from livekit.plugins import noise_cancellation, silero
7 from livekit.plugins.turn_detector.multilingual import MultilingualModel
8
9 load_dotenv()
10
11
12 class Assistant(Agent):
13     def __init__(self) -> None:
14         super().__init__(
15             instructions="You are a helpful voice AI assistant.",
16         )
17
18
19 server = AgentServer()
20
21
22 @server rtc_session()
23 async def entrypoint(ctx: JobContext):
24     session = AgentSession(
25         stt="assemblyai/universal-streaming:en",
26         tts="assemblyai/tts-1:en",
27         vad="silero-vad",
28         turn_detector="multilingual",
29     )
30     agent = Assistant()
31     session.add_agent(agent)
32     await session.run()
```

personality. So, back in our uh agent.py in the assistant, let's update the instructions. So, I'm defining who this agent is. It's an upbeat, slightly sarcastic voice AI or text support. It helps callers fix issues without rambling and keeps replies under three sentences. And you don't have to use this exact personality. Take some time to write

■ [20:20 - 20:40]



The screenshot shows a video call interface. On the right, a man with a beard and glasses is speaking into a microphone. The background of the video feed has a 'LiveKit' logo. On the left, a code editor window titled 'agent.py' is open. The code defines an 'Assistant' class that inherits from 'Agent'. The class has an 'init' method that sets instructions for the AI. The instructions are: 'You are an upbeat, slightly sarcastic voice AI for tech support.' and 'Help the caller fix issues without rambling, and keep replies under 3 sentences.' The code also sets up an 'AgentServer' and an 'AgentSession' with various configurations like 'stt', 'llm', 'tts', 'vad', and 'turn_detection'. The status bar at the bottom of the code editor shows 'Ln 16, Col 98', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.14.0 64-bit'.

```
9 load_dotenv()
10
11
12 class Assistant(Agent):
13     def __init__(self) -> None:
14         super().__init__(
15             instructions="You are an upbeat, slightly sarcastic voice AI for tech support. "
16             "Help the caller fix issues without rambling, and keep replies under 3 sentences."
17         )
18
19
20 server = AgentServer()
21
22
23 @server rtc_session()
24 async def entrypoint(ctx: JobContext):
25     session = AgentSession(
26         stt="assemblyai/universal-streaming:en",
27         llm="openai/gpt-4.1-mini",
28         tts="cartesia/sonic-3",
29         vad=silero.VAD.load(),
30         turn_detection=MultilingualModel(),
31     )
32
33     await session.start()
```

personality. Take some time to write instructions that make sense for your use case. Now, the key is being specific about tone, length, and boundaries. Next, let's modify the TTS configuration to match the new personality. So, if we go down here, we can see in our agent session all of our uh definitions here. And what we're looking for is TTS text to speech. So, currently

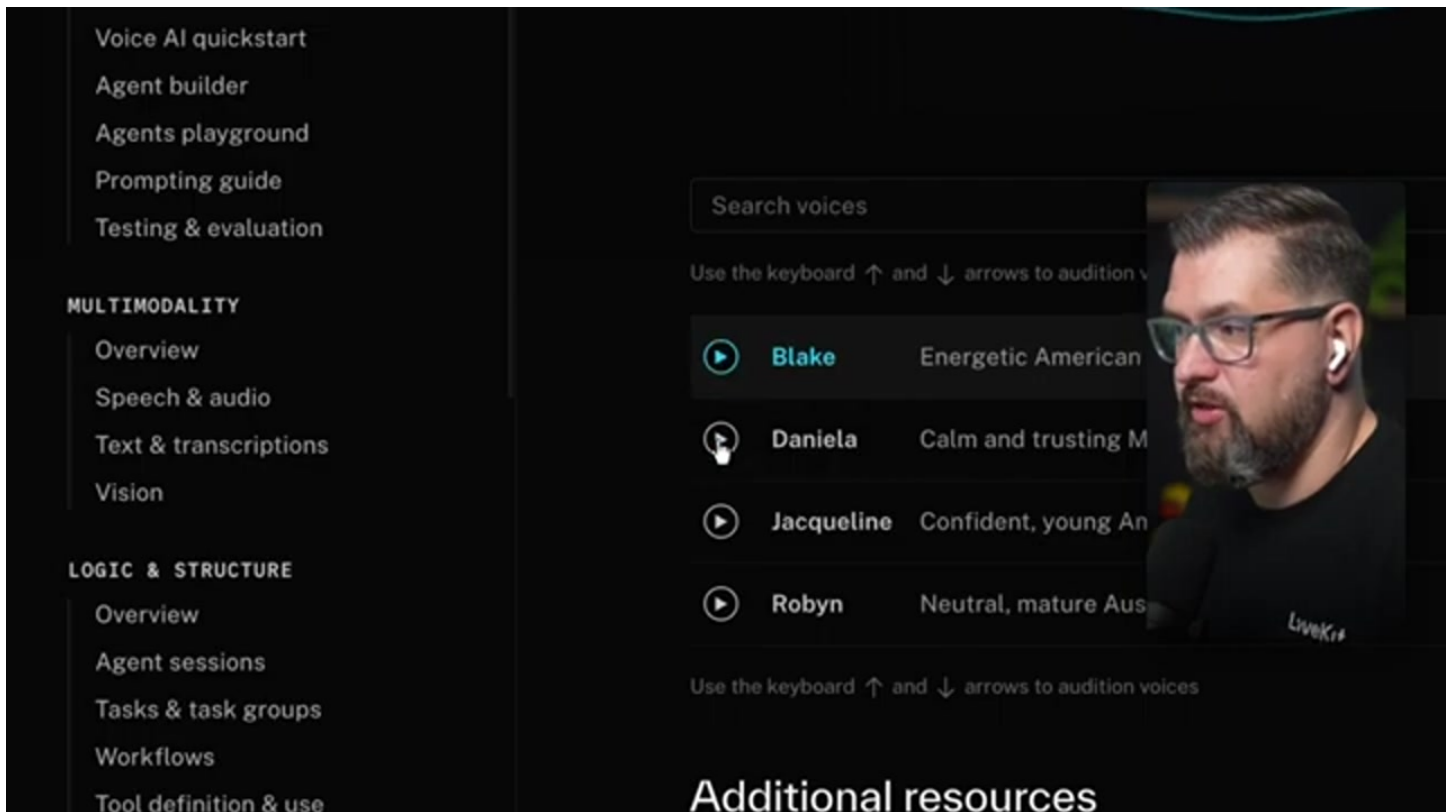
■ [20:40 - 21:00]

```
18
19
20 server = AgentServer()
21
22
23 @server rtc_session()
24 async def entrypoint(ctx: JobContext):
25     session = AgentSession(
26         stt="assemblyai/universal-streaming:en",
27         llm="openai/gpt-4.1-mini",
28         tts="cartesia/sonic-3",
29         vad=silero.VAD.load(),
30         turn_detection=MultilingualModel(),
31     )
32
33     await session.start(
34         agent=Assistant(),
35         room=ctx.room,
36         room_options=room io.RoomOptions(
```



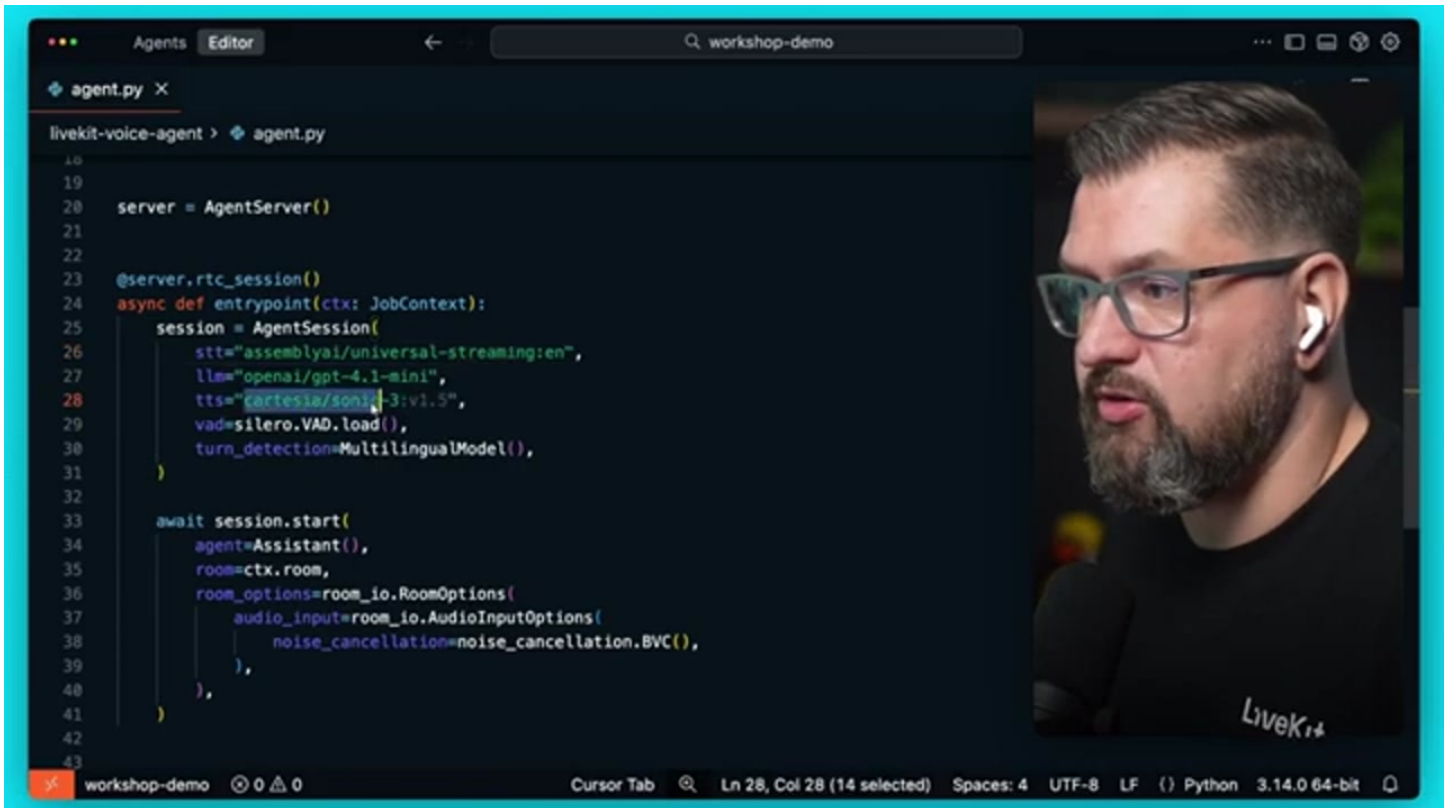
for is TTS text to speech. So, currently we're using Cartesia Sonic 3. And we can add a specific voice ID to this. If we go over to the LiveKit docs and we can look at the specific Cartesia voices, we can actually play them here. So Blake is the default. >> Hi there. Nice to meet you. I'm looking forward to working with you. >> And there are others.

■ [21:00 - 21:20]



>> And there are others. >> Holaerte. >> Daniela is great for Spanish. Then there's Jacine. >> Hi there. Nice to meet you. I'm looking forward to working with you. >> Cool. So let's use Jacqueline. So any of these voices, if you like them, uh, just copy the voice ID. Let's go back and update our voice ID now. So right here, Cartisia Sonic 3. Let's just go ahead

■ [21:20 - 21:40]



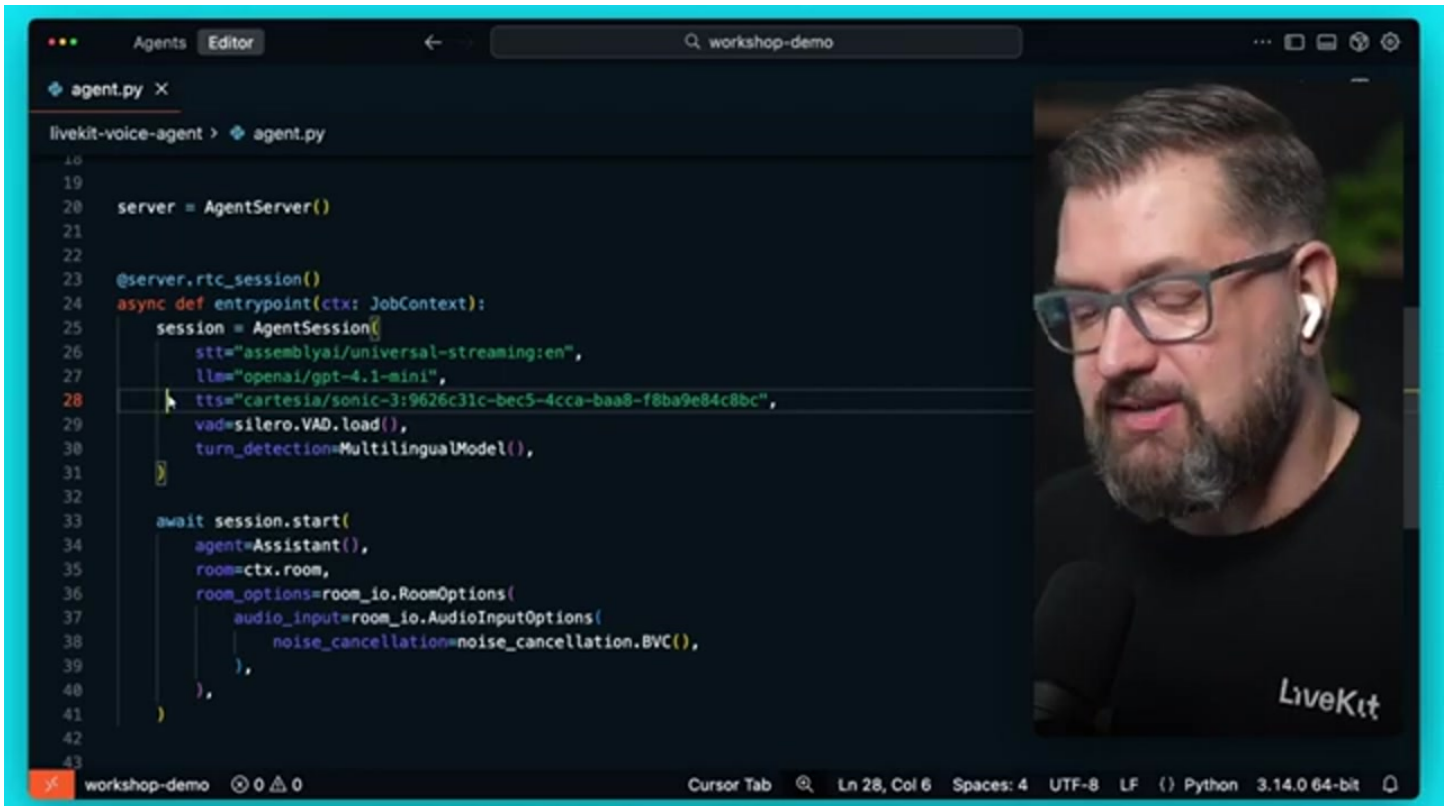
The screenshot shows a code editor window titled "workshop-demo" with a file named "agent.py" open. The code is written in Python and defines an "AgentServer" and an "entrypoint" function. The "entrypoint" function creates an "AgentSession" with various models and options, including a TTS model "cartesia/sonic3-v1.5". The code is as follows:

```
10
19
20 server = AgentServer()
21
22
23 @server.rtc_session()
24 async def entrypoint(ctx: JobContext):
25     session = AgentSession(
26         stt="assemblyai/universal-streaming:en",
27         llm="openai/gpt-4.1-mini",
28         tts="cartesia/sonic3-v1.5",
29         vad=silero.VAD.load(),
30         turn_detection=MultilingualModel(),
31     )
32
33     await session.start(
34         agent=Assistant(),
35         room=ctx.room,
36         room_options=room_io.RoomOptions(
37             audio_input=room_io.AudioInputOptions(
38                 noise_cancellation=noise_cancellation.BVC(),
39             ),
40         ),
41     )
42
43
```

On the right side of the editor, there is a video feed of a man with a beard and glasses, wearing a black t-shirt with the "LiveKit" logo, speaking into a microphone.

Cartesia Sonic 3. Let's just go ahead and update that voice ID there. So now it's Cartesia Sonic 3 and then colon and then this is the actual voice ID here. And notice here that we have one model assigned to each component. But every provider has hiccups. When outages happen, you don't want your agent to stop working entirely. Production agents

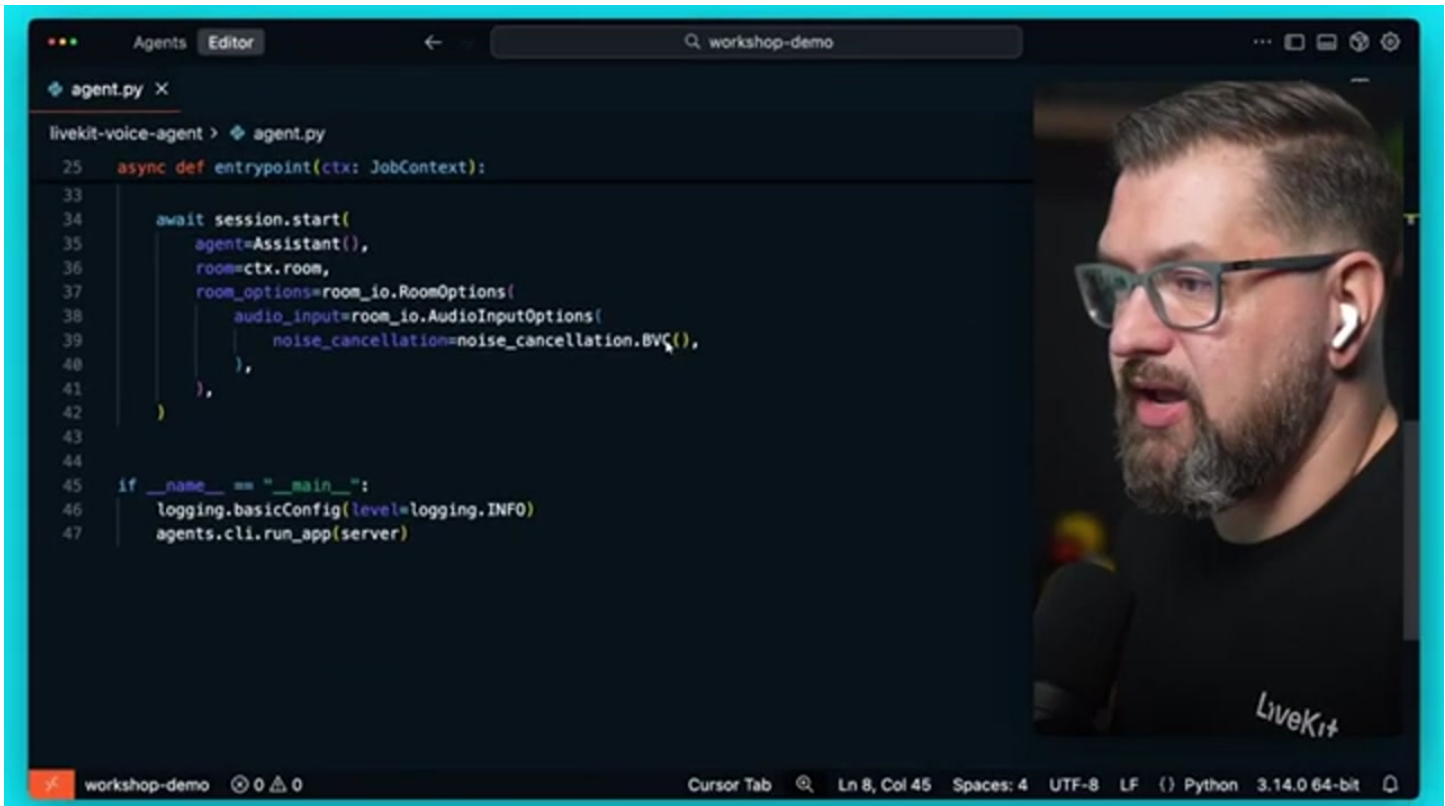
■ [21:40 - 22:00]



```
agent.py X
livekit-voice-agent > agent.py
10
19
20 server = AgentServer()
21
22
23 @server rtc_session()
24 async def entrypoint(ctx: JobContext):
25     session = AgentSession(
26         stt="assemblyai/universal-streaming:en",
27         llm="openai/gpt-4.1-mini",
28         tts="cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc",
29         vad=silero.VAD.load(),
30         turn_detection=MultilingualModel(),
31     )
32
33     await session.start(
34         agent=Assistant(),
35         room=ctx.room,
36         room_options=room_io.RoomOptions(
37             audio_input=room_io.AudioInputOptions(
38                 noise_cancellation=noise_cancellation.BVC(),
39             ),
40         ),
41     )
42
43
workshop-demo 0 0 0 Cursor Tab Ln 28, Col 6 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

stop working entirely. Production agents always need secondary providers. So lifekits fallback adapters automatically retry in priority order. So let's set up these fallback adapters. First let's go up to the top and we're going to import a few more things. We're going to import uh llm stts and inference. And now let's go down. So instead of an

■ [22:00 - 22:20]



```
livekit-voice-agent > agent.py
25  async def entrypoint(ctx: JobContext):
33
34      await session.start(
35          agent=Assistant(),
36          room=ctx.room,
37          room_options=room_io.RoomOptions(
38              audio_input=room_io.AudioInputOptions(
39                  noise_cancellation=noise_cancellation.BVQ(),
40              ),
41          ),
42      )
43
44
45  if __name__ == "__main__":
46      logging.basicConfig(level=logging.INFO)
47      agents.cli.run_app(server)
```

workshop-demo 0 0 0 Cursor Tab Ln 8, Col 45 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

let's go down. So instead of an individual component for each line, we're going to uh wrap each of these pipeline components with a fallback adapter. So for st it's going to look something like this. We have st equals st.fallback fallback adapter and then we are going to add in our uh again we're using the lifekit inference so inference.st

■ [22:20 - 22:40]

```
server = AgentServer()

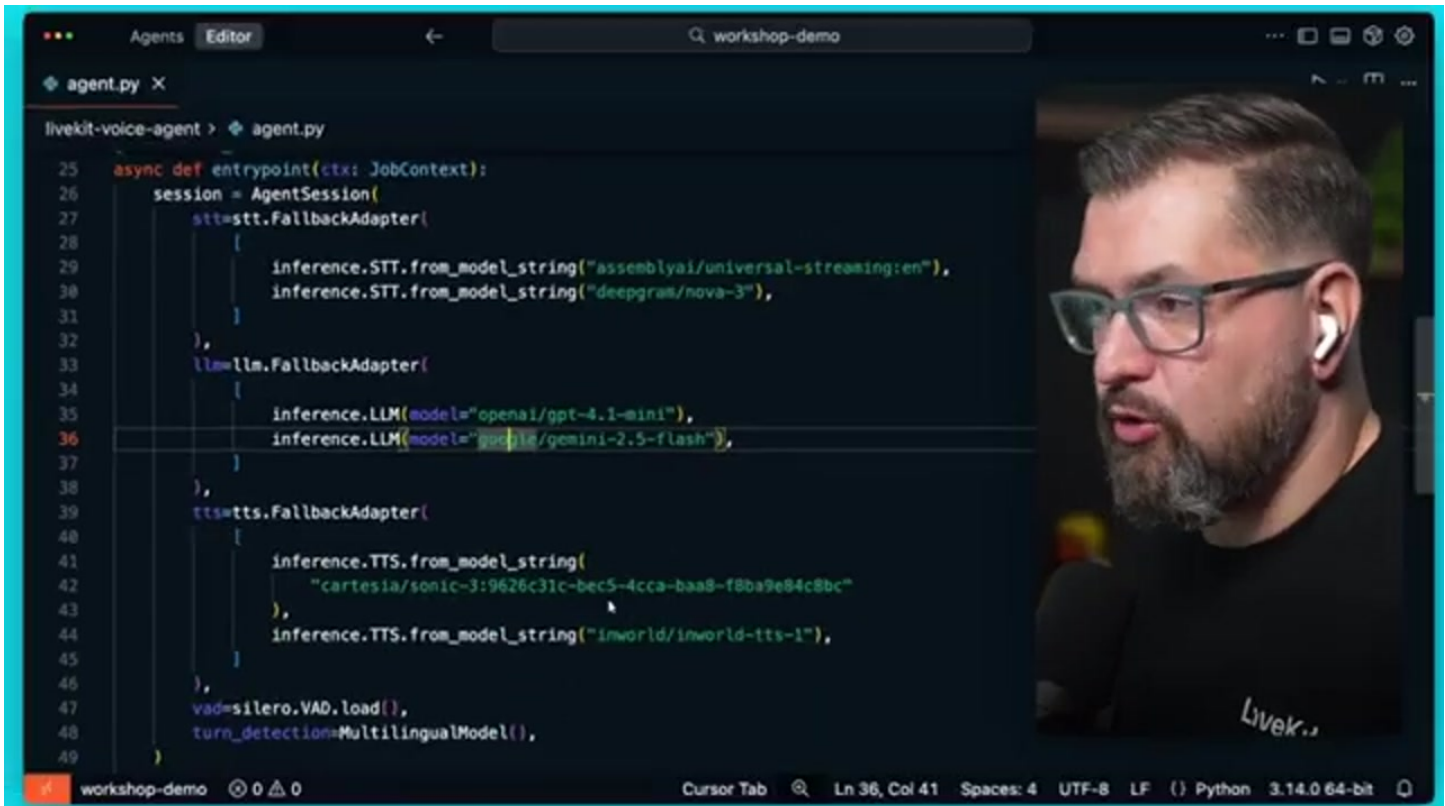
server.rtc_session()

@async def entrypoint(ctx: JobContext):
    session = AgentSession(
        stt=stt.FallbackAdapter(
            [
                inference.STT.from_model_string("assemblyai/universal"),
                inference.STT.from_model_string("deepgram/nova-3"),
            ],
        ),
        llm="openai/gpt-4.1-mini",
        tts="cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc",
        vad=silero.VAD.load(),
        turn_detection=MultilingualModel(),
    )
```



inference from model string and then we're passing in assembly AI as our primary and our secondary will be deepgram nova 3 so I'll go ahead and do the same for the llm and the tts all right so now we have each of these set up with fallbacks we have openai as the primary and then gemini as the as a fallback for our LLM

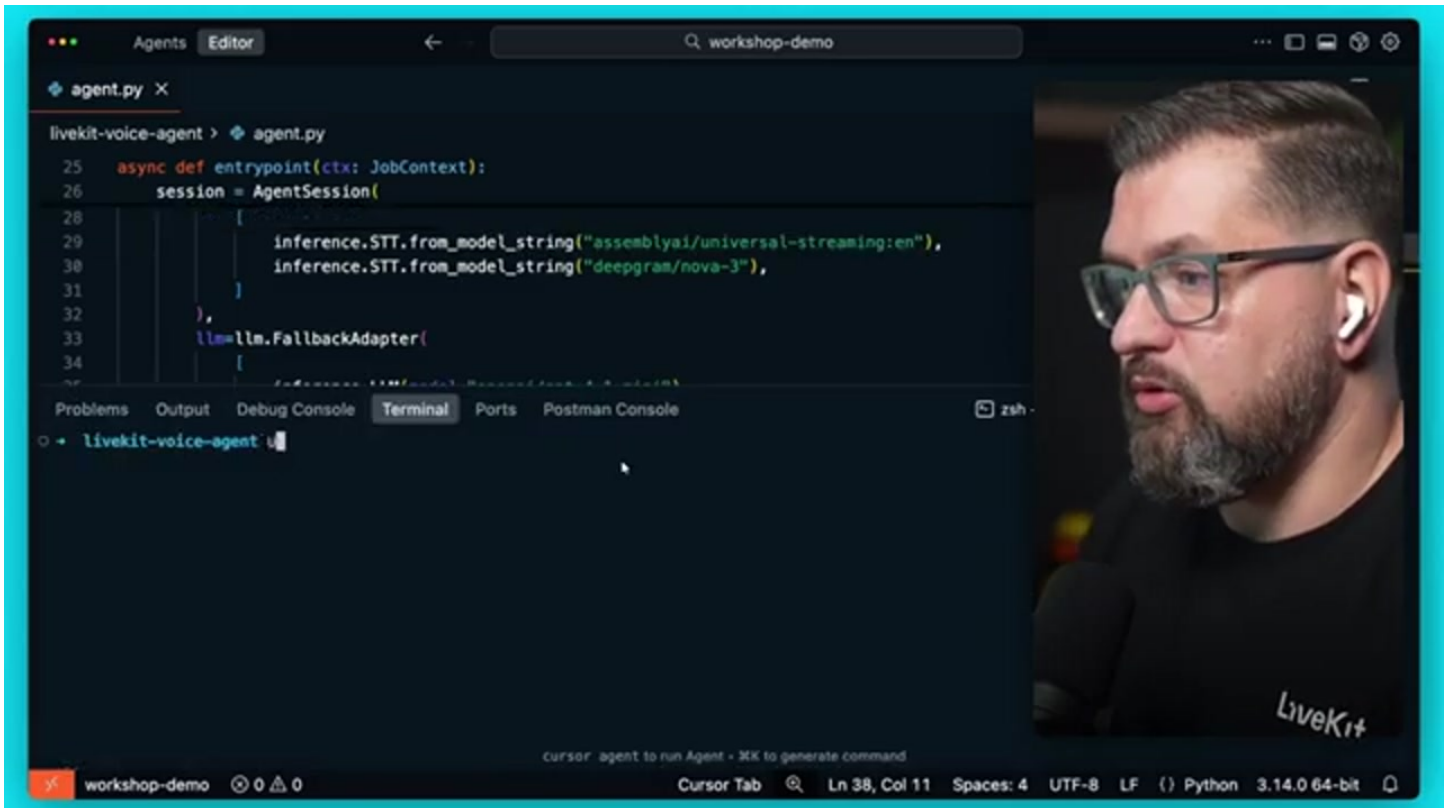
■ [22:40 - 23:00]



```
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
27         stt=stt.FallbackAdapter(
28             [
29                 inference.STT.from_model_string("assemblyai/universal-streaming:en"),
30                 inference.STT.from_model_string("deepgram/nova-3"),
31             ]
32         ),
33         llm=llm.FallbackAdapter(
34             [
35                 inference.LLM(model="openai/gpt-4.1-mini"),
36                 inference.LLM(model="google/gemini-2.5-flash"),
37             ]
38         ),
39         tts=tts.FallbackAdapter(
40             [
41                 inference.TTS.from_model_string(
42                     "cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc"
43                 ),
44                 inference.TTS.from_model_string("inworld/inworld-tts-1"),
45             ]
46         ),
47         vad=silero.VAD.load(),
48         turn_detection=MultilingualModel(),
49     )
```

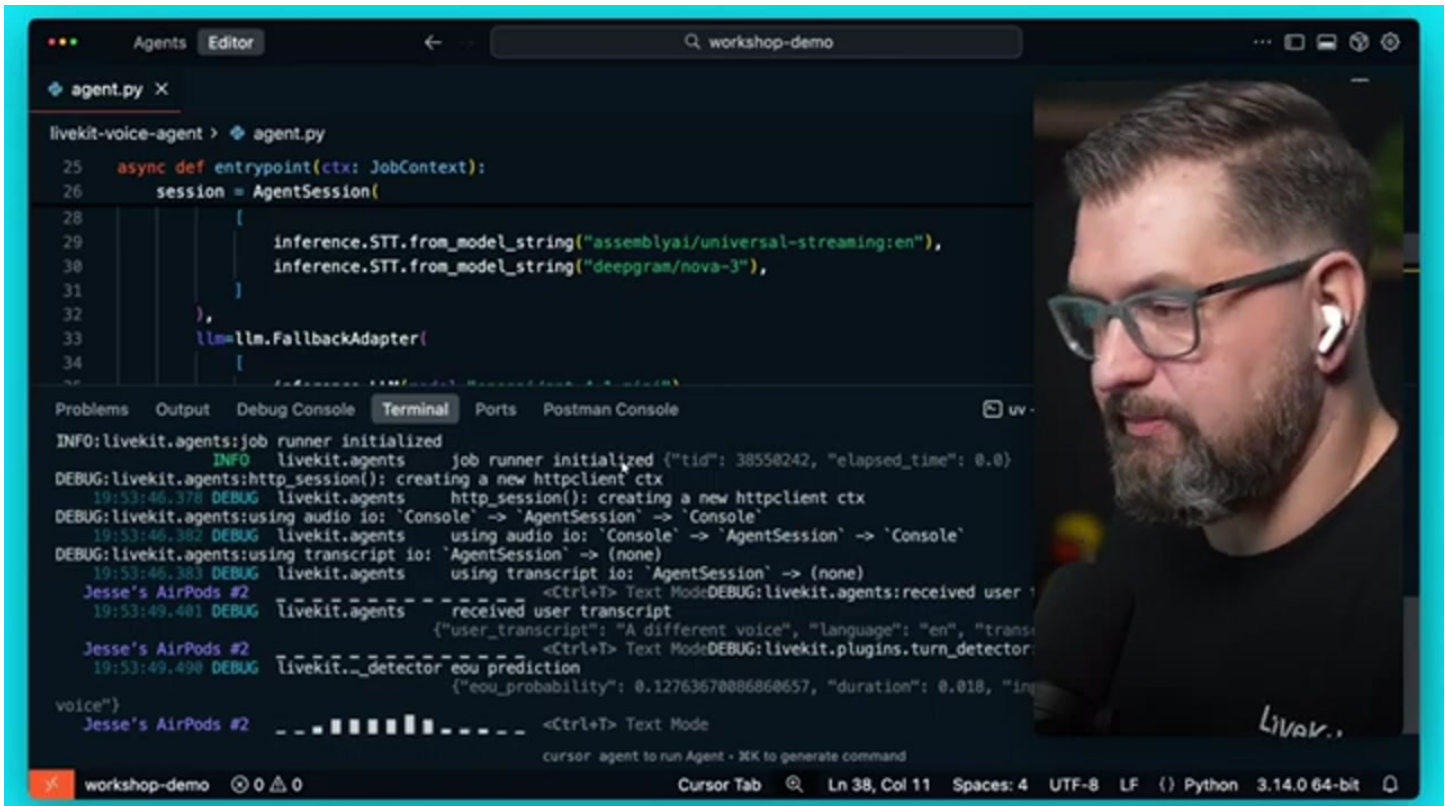
gemini as the as a fallback for our LLM we have cartesia as the primary ary TTS and in world as our secondary. So these adapters degrade gracefully. If the first provider fails, the session retries the next one without dropping the call. Your users never know anything went wrong. Let's test this out. Be sure that you saved the file. Open up our terminal and we'll do `uv run agents`

■ [23:00 - 23:20]



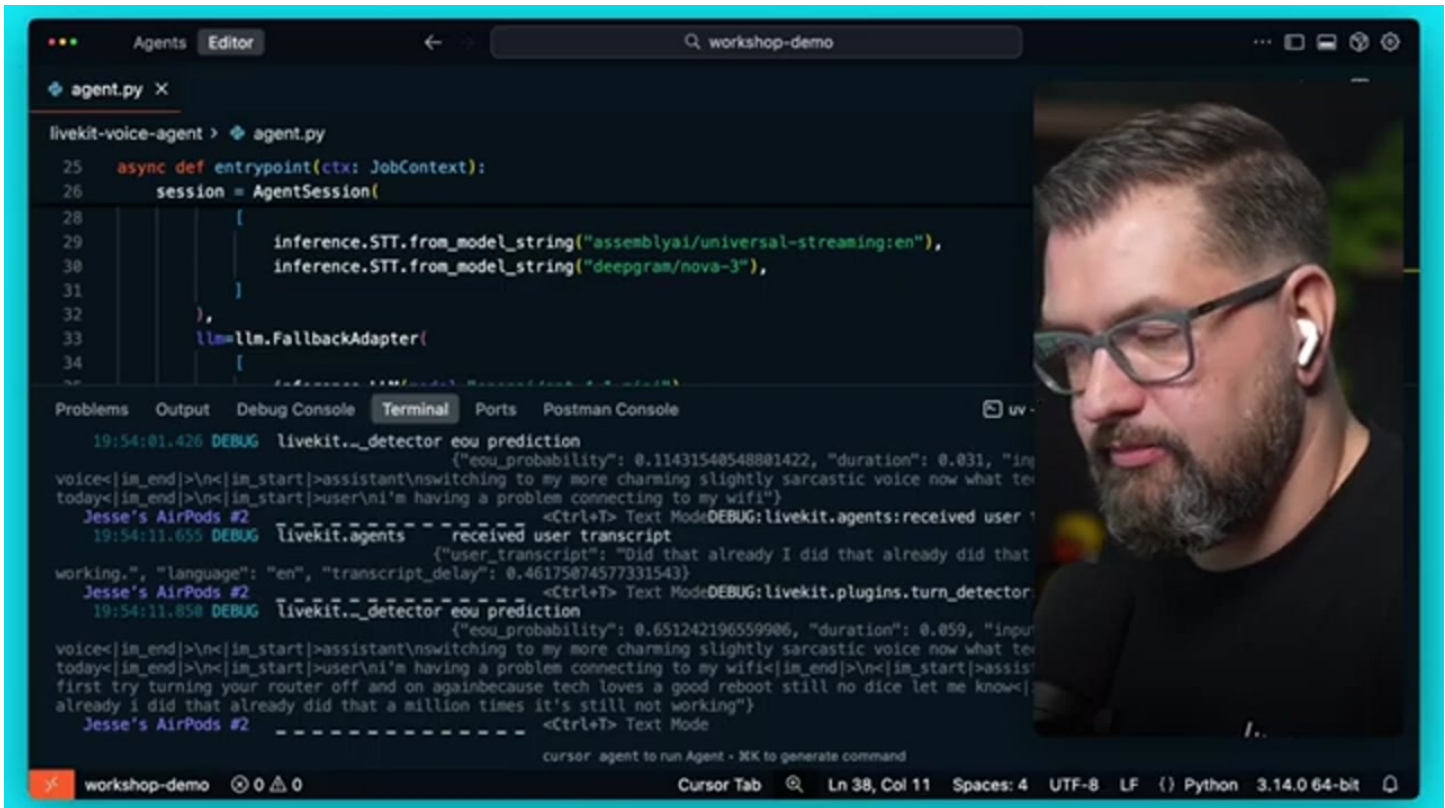
terminal and we'll do `uv run agents agent.py` and then console. We should notice a different voice. Switching to my more charming, slightly sarcastic voice now. What tech trouble are we tackling today? >> Um, I'm having a problem connecting to my Wi-Fi.

■ [23:20 - 23:40]



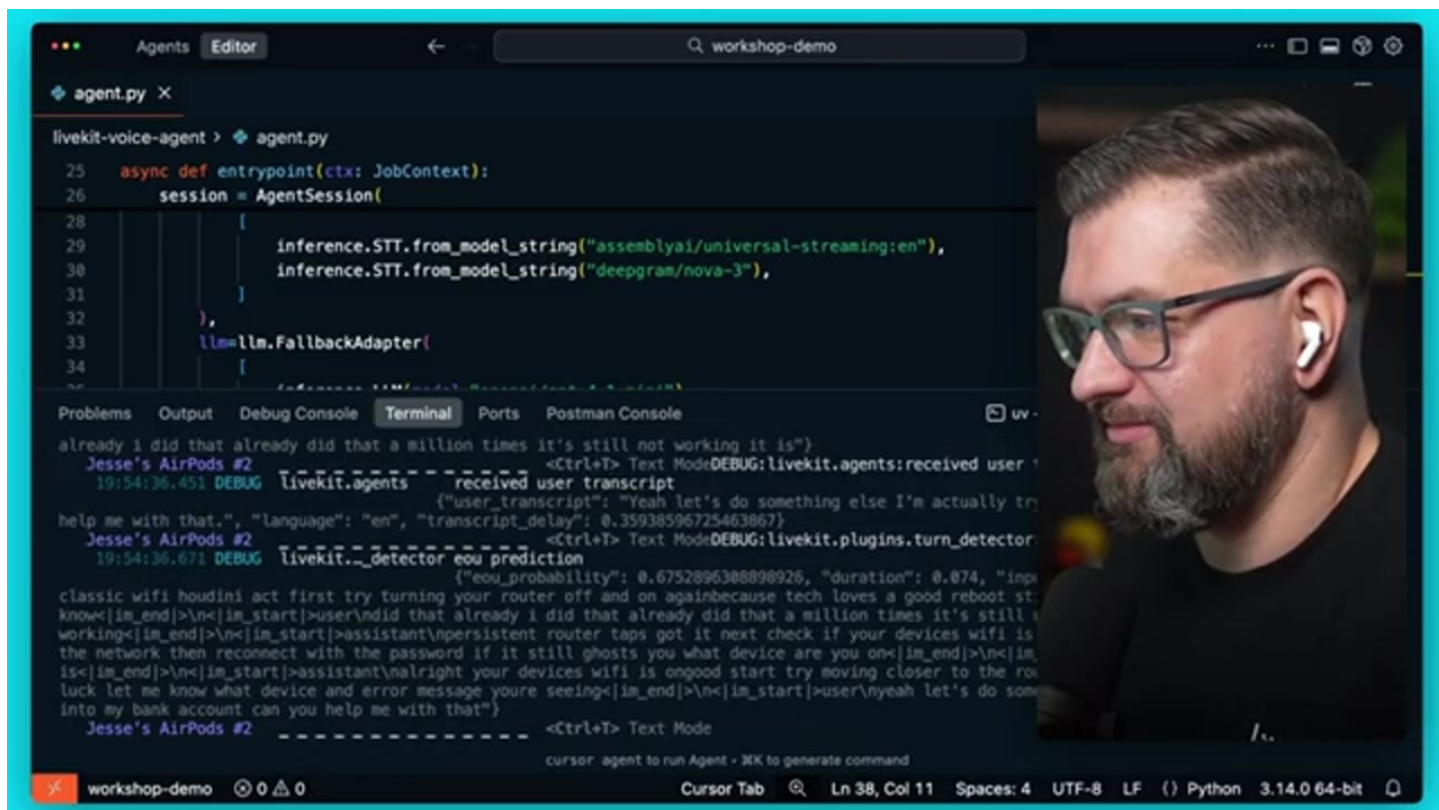
my Wi-Fi. >> Uh, the classic Wi-Fi Houdini act. First, try turning your router off and on again because tech >> I did that already. Did that a million times. It's still not working. >> Persistent router taps. Got it. Next. Check if your devices Wi-Fi is actually turned on. and try.

■ [23:40 - 24:00]



turned on. and try. >> All right, your devices Wi-Fi is on. Good start. Try moving closer to the router to rule out >> something else. I'm actually um trying to log into my bank account. Can you help me with that? >> Sure thing. But for security reasons, I

■ [24:00 - 24:20]



The screenshot shows a VS Code editor window titled "workshop-demo". The editor is open to a file named "agent.py". The code in the file is as follows:

```
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
27         [
28             inference.STT.from_model_string("assemblyai/universal-streaming:en"),
29             inference.STT.from_model_string("deepgram/nova-3"),
30         ],
31     ),
32     llm=llm.FallbackAdapter(
33         [
34             inference.LLM.from_model_string("gpt-4o"),
35             inference.LLM.from_model_string("gpt-4o-mini"),
36         ],
37     ),
38 )
39 session.run()
```

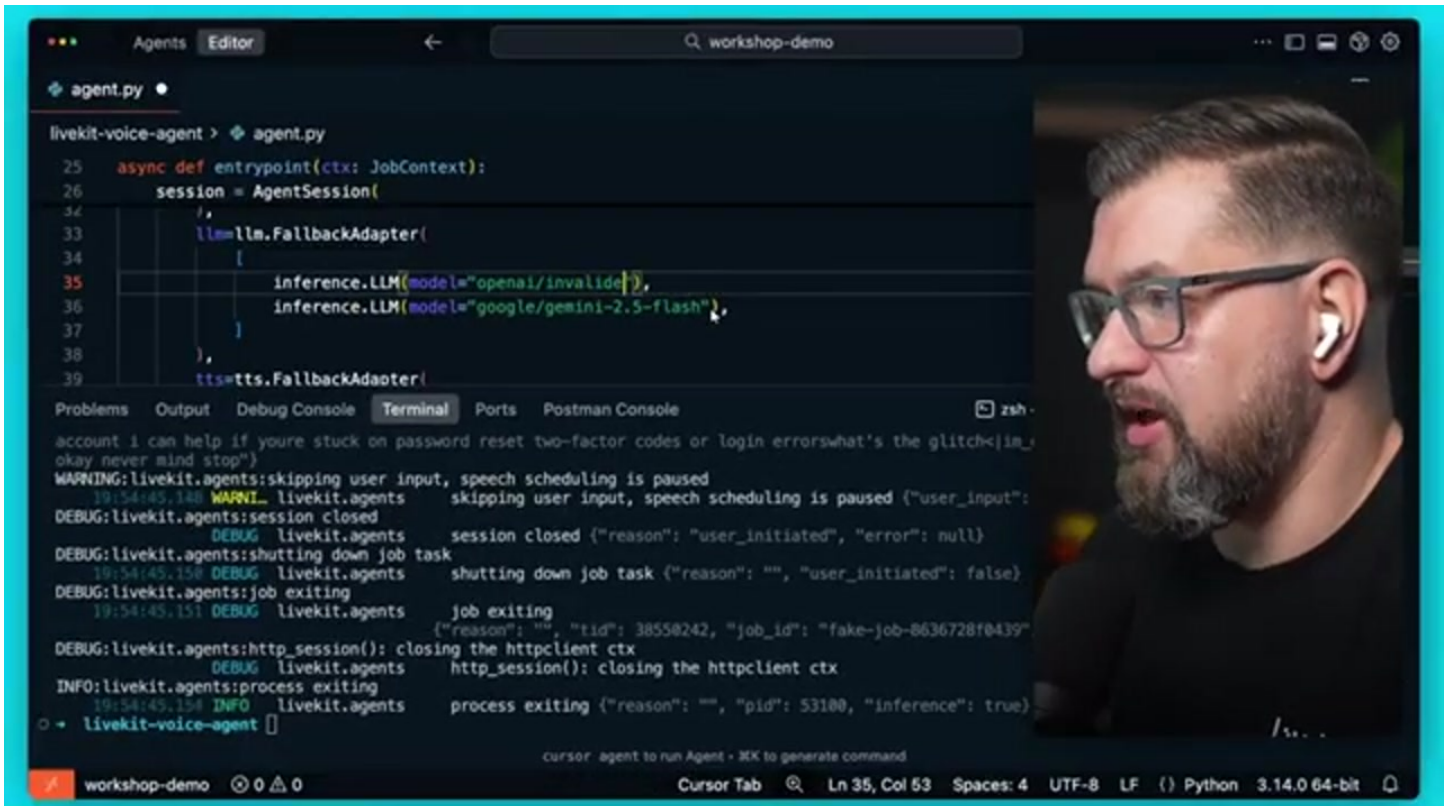
The terminal window shows a chat log with the following content:

```
already i did that already did that a million times it's still not working it is"}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:received user transcript
19:54:36.451 DEBUG livekit.agents received user transcript
{"user_transcript": "Yeah let's do something else I'm actually trying to", "language": "en", "transcript_delay": 0.35938596725463867}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.plugins.turn_detector:turn_detector
19:54:36.671 DEBUG livekit.plugins.turn_detector eou prediction
{"eou_probability": 0.6752896308898926, "duration": 0.074, "input": "Yeah let's do something else I'm actually trying to", "known": false}
Jesse's AirPods #2 <Ctrl+T> Text Mode
classic wifi houdini act first try turning your router off and on againbecause tech loves a good reboot st. know
Jesse's AirPods #2 <Ctrl+T> Text Mode
did that already did that a million times it's still not working it is"}
Jesse's AirPods #2 <Ctrl+T> Text Mode
persistent router taps got it next check if your devices wifi is the network then reconnect with the password if it still ghosts you what device are you on
Jesse's AirPods #2 <Ctrl+T> Text Mode
is on
Jesse's AirPods #2 <Ctrl+T> Text Mode
alright your devices wifi is ongood start try moving closer to the router let me know what device and error message you're seeing
Jesse's AirPods #2 <Ctrl+T> Text Mode
user\ntyah let's do something else I'm actually trying to
Jesse's AirPods #2 <Ctrl+T> Text Mode
into my bank account can you help me with that"}
Jesse's AirPods #2 <Ctrl+T> Text Mode
/..
```

The status bar at the bottom of the editor shows "workshop-demo", "Cursor Tab", "Ln 38, Col 11", "Spaces: 4", "UTF-8", "LF", "Python", and "3.14.0 64-bit".

>> Sure thing. But for security reasons, I can't access your bank account. >> Ah, that's great. >> I can help if >> Okay, never mind. Stop. Okay, so seems to be working great. She's sarcastic. Um, we changed our voice. We've got our fallbacks. Let's test those fallbacks, actually. So, let's go up here and let's see. So, instead of GPT41, let's just do some uh in uh invalid model here. Uh, so

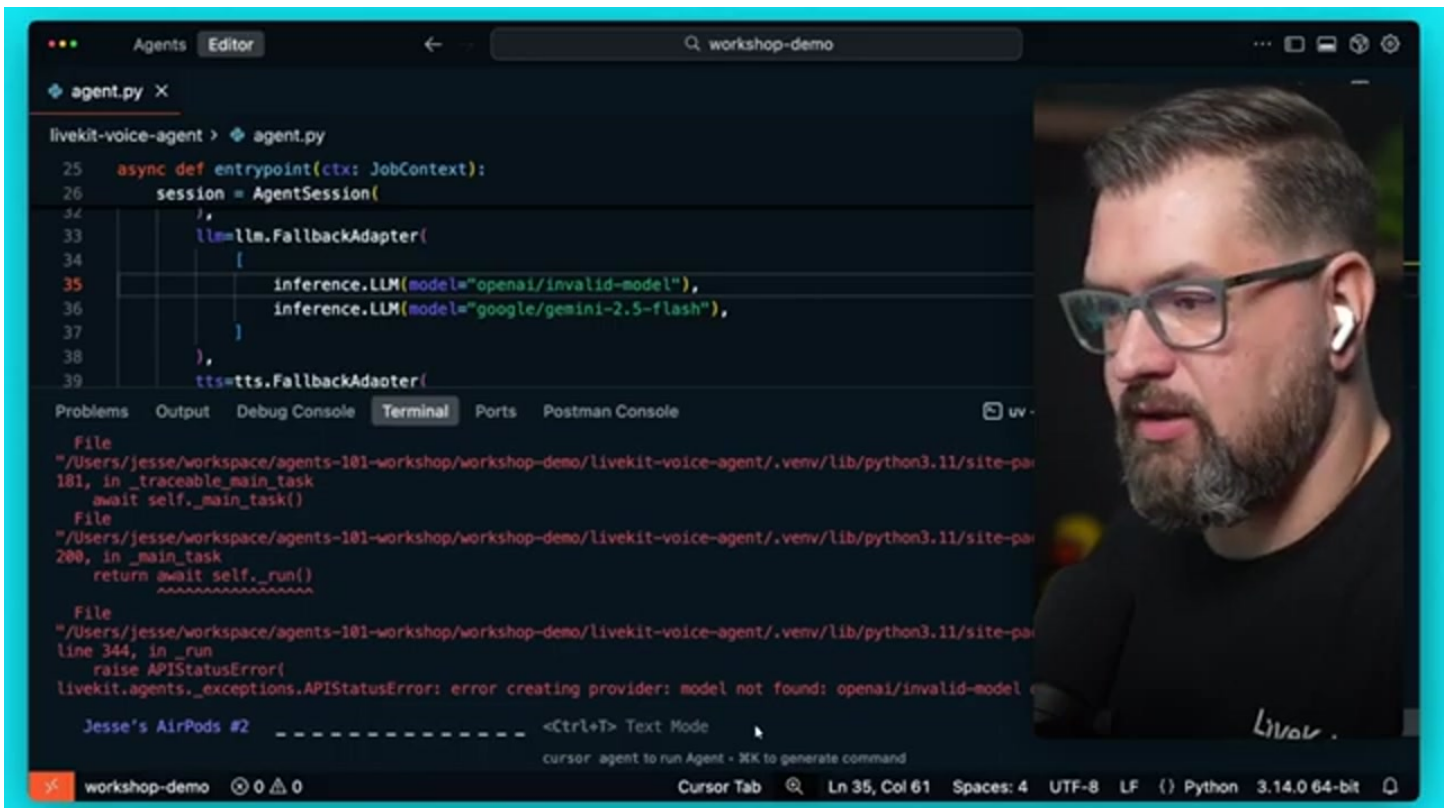
■ [24:20 - 24:40]



```
agent.py
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
34         llm=llm.FallbackAdapter(
34             [
35                 inference.LLM(model="openai/invalid"),
36                 inference.LLM(model="google/gemini-2.5-flash"),
37             ],
38         ),
39         tts=tts.FallbackAdapter(
Problems Output Debug Console Terminal Ports Postman Console
account i can help if you're stuck on password reset two-factor codes or login errors what's the glitch<jim_
okay never mind stop")
WARNING:livekit.agents:skipping user input, speech scheduling is paused
19:54:45.148 WARNING livekit.agents skipping user input, speech scheduling is paused ("user_input":
DEBUG:livekit.agents:session closed
DEBUG livekit.agents session closed {"reason": "user_initiated", "error": null}
DEBUG:livekit.agents:shutting down job task
19:54:45.150 DEBUG livekit.agents shutting down job task {"reason": "", "user_initiated": false}
DEBUG:livekit.agents:job exiting
19:54:45.151 DEBUG livekit.agents job exiting
{"reason": "", "tid": 38558242, "job_id": "fake-job-8636728f0439"}
DEBUG:livekit.agents:http_session(): closing the httpclient ctx
DEBUG livekit.agents http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting
19:54:45.154 INFO livekit.agents process exiting {"reason": "", "pid": 53180, "inference": true}
○ + livekit-voice-agent []
cursor: agent to run Agent - RKX to generate command
workshop-demo 0 0 0 Cursor Tab Ln 35, Col 53 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

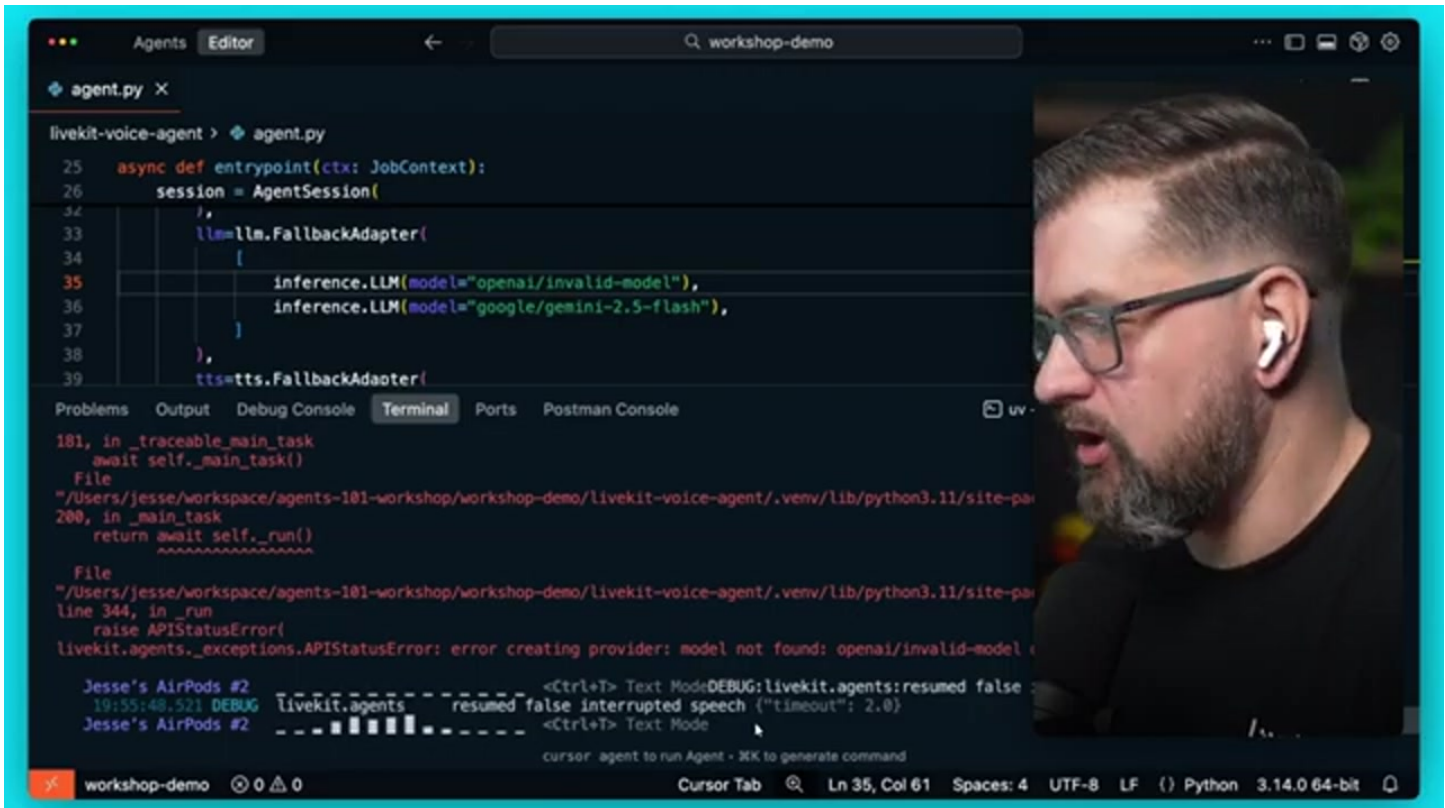
some uh in uh invalid model here. Uh, so we know that's not going to work. So, let's go ahead and test this out and see if it falls back to Gemini instead. So, we'll run a UV run agent. Hello. >> Hey there. Looks like you've stumbled into the land of helpful, slightly sarcastic tech support. What can I do for you? >> Um, yeah. Can you help me uh set up a

■ [24:40 - 25:00]



>> Um, yeah. Can you help me uh set up a brand new computer? >> Absolutely. Setting up a new computer is like unwrapping a present, >> but with more cables. To start, let's make sure you've got everything out of the box. >> Yeah. Yeah. Yeah. It's all out of the box, plugged in, ready to go.

■ [25:00 - 25:20]



box, plugged in, ready to go. >> Fantastic. Is it powering on? And do you see anything on the screen? >> Yeah. So you can see here it is still working. Uh but we are of course getting errors which is good. We we love to see why it's not working. Uh but it is falling back to Gemini 2.5 because it obviously sees that OpenAI/invalid

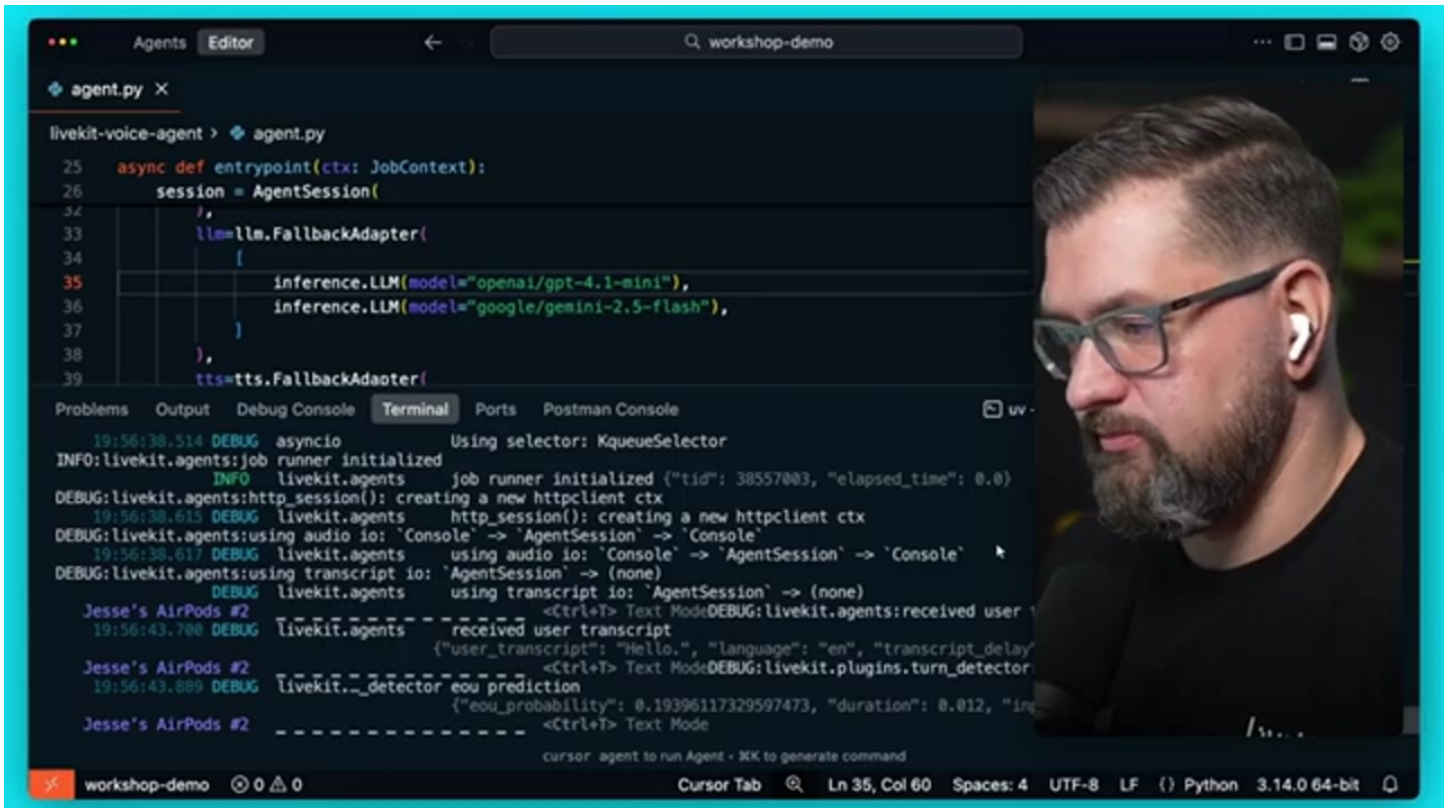
■ [25:20 - 25:40]



```
agent.py x
livekit-voice-agent > agent.py
25  async def entrypoint(ctx: JobContext):
26      session = AgentSession(
34          llm=llm.FallbackAdapter(
34              [
35                  inference.LLM(model="openai/invalid-model"),
36                  inference.LLM(model="google/gemini-2.5-flash"),
37              ]
38          ),
39          tts=tts.FallbackAdapter(
File
"/Users/jesse/workspace/agents-101-workshop/workshop-demo/livekit-voice-agent/.venv/lib/python3.11/site-pa
181, in _traceable_main_task
    await self._main_task()
File
"/Users/jesse/workspace/agents-101-workshop/workshop-demo/livekit-voice-agent/.venv/lib/python3.11/site-pa
200, in _main_task
    return await self._run()
~~~~~
File
"/Users/jesse/workspace/agents-101-workshop/workshop-demo/livekit-voice-agent/.venv/lib/python3.11/site-pa
line 344, in _run
    raise APIStatusError(
livekit.agents._exceptions.APIStatusError: error creating provider: model not found: openai/invalid-model
INFO:livekit.agents:shutting down worker
cursor: agent to run Agent - XK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 35, Col 61 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

obviously sees that OpenAI/invalid model is an invalid model. So of course uh we change this back to GPT41 mini. Save that. And let's go ahead and restart this. and it worked just fine with no errors. Hello. >> Hey there. What tech trouble can I zap

■ [25:40 - 26:00]



```
agent.py x
livekit-voice-agent > agent.py
25 async def entrypoint(ctx: JobContext):
26     session = AgentSession(
34         llm=llm.FallbackAdapter(
35             inference.LLM(model="openai/gpt-4.1-mini"),
36             inference.LLM(model="google/gemini-2.5-flash"),
37         ),
38         tts=tts.FallbackAdapter(
39             inference.LLM(model="openai/gpt-4.1-mini"),
39             inference.LLM(model="google/gemini-2.5-flash"),
39         ),
39     )

Problems Output Debug Console Terminal Ports Postman Console
19:56:38.514 DEBUG asyncio Using selector: KqueueSelector
INFO:livekit.agents:job runner initialized
INFO livekit.agents job runner initialized ("tid": 38557803, "elapsed_time": 0.0)
DEBUG:livekit.agents:http_session(): creating a new httpclient ctx
19:56:38.615 DEBUG livekit.agents http_session(): creating a new httpclient ctx
DEBUG:livekit.agents:using audio io: 'Console' -> 'AgentSession' -> 'Console'
19:56:38.617 DEBUG livekit.agents using audio io: 'Console' -> 'AgentSession' -> 'Console'
DEBUG:livekit.agents:using transcript io: 'AgentSession' -> (none)
DEBUG livekit.agents using transcript io: 'AgentSession' -> (none)
Jesse's AirPods #2 ----- received user transcript
19:56:43.700 DEBUG livekit.agents ----- received user transcript
{"user_transcript": "Hello.", "language": "en", "transcript_delay": 0.0}
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:received user transcript
19:56:43.889 DEBUG livekit._detector eou prediction
{"eou_probability": 0.19396117329597473, "duration": 0.012, "language": "en"}
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode
cursor: agent to run Agent - KK to generate command
```

>> Hey there. What tech trouble can I zap away for you today? >> Oh, thank you. Sounds like you're working. So, this kind of chaos testing helps you confirm the agent will stay online even when a provider goes down. So, your agent now has a defined personality through the system prompt, a custom voice through the TTS configuration, and fallback adapters to

- ✓ System persona
- ✓ Custom voice
- ✓ Fallback adapters



LIVEKIT.IO

configuration, and fallback adapters to handle provider outages. In the next lesson, you'll capture metrics so that you can measure performance and enable preemptive generation to reduce response latency. So, your agent now sounds great and survives outages. But how do you know if it's actually performing well? This lesson covers three connected topics. Metrics collection to understand

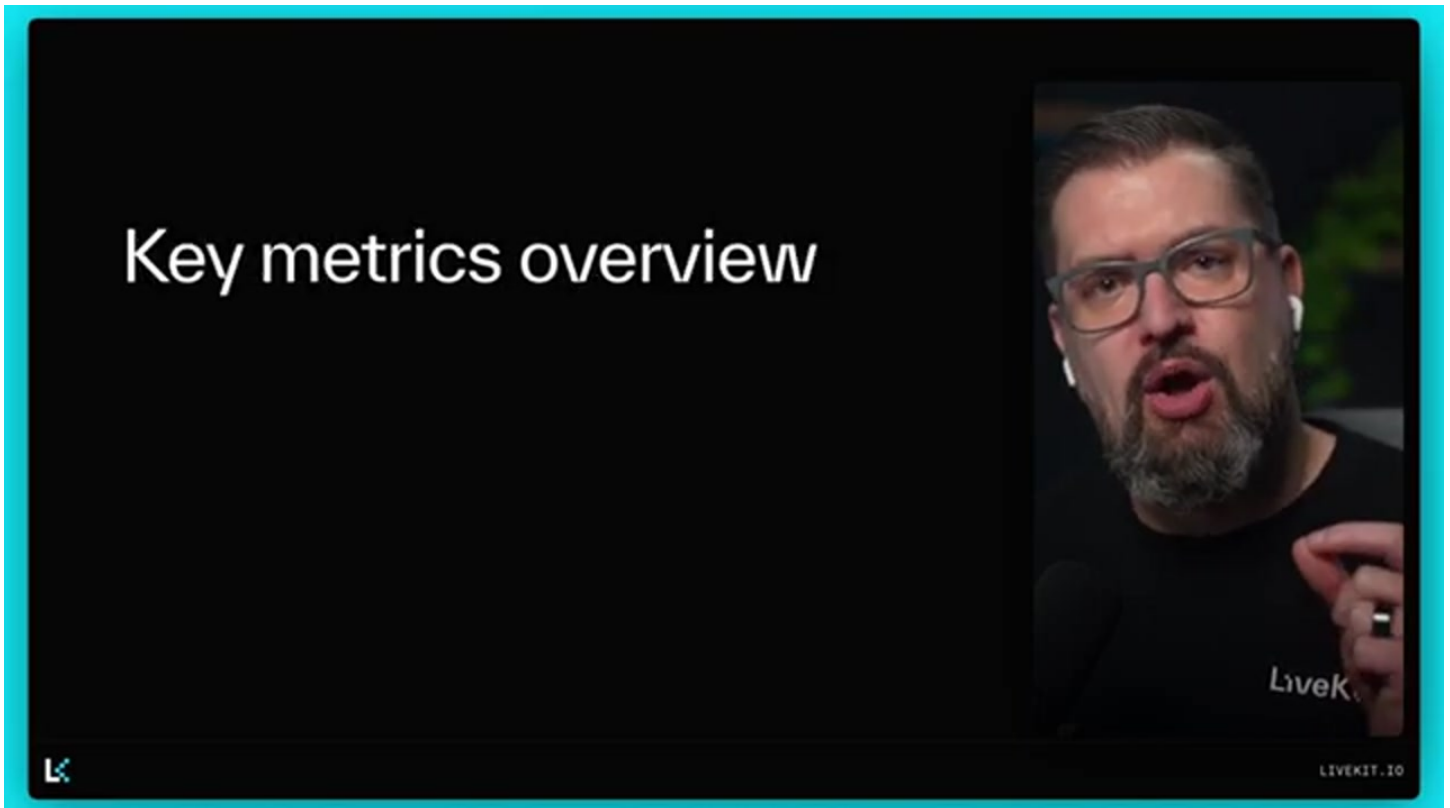
Metrics, deployment, and latency optimization



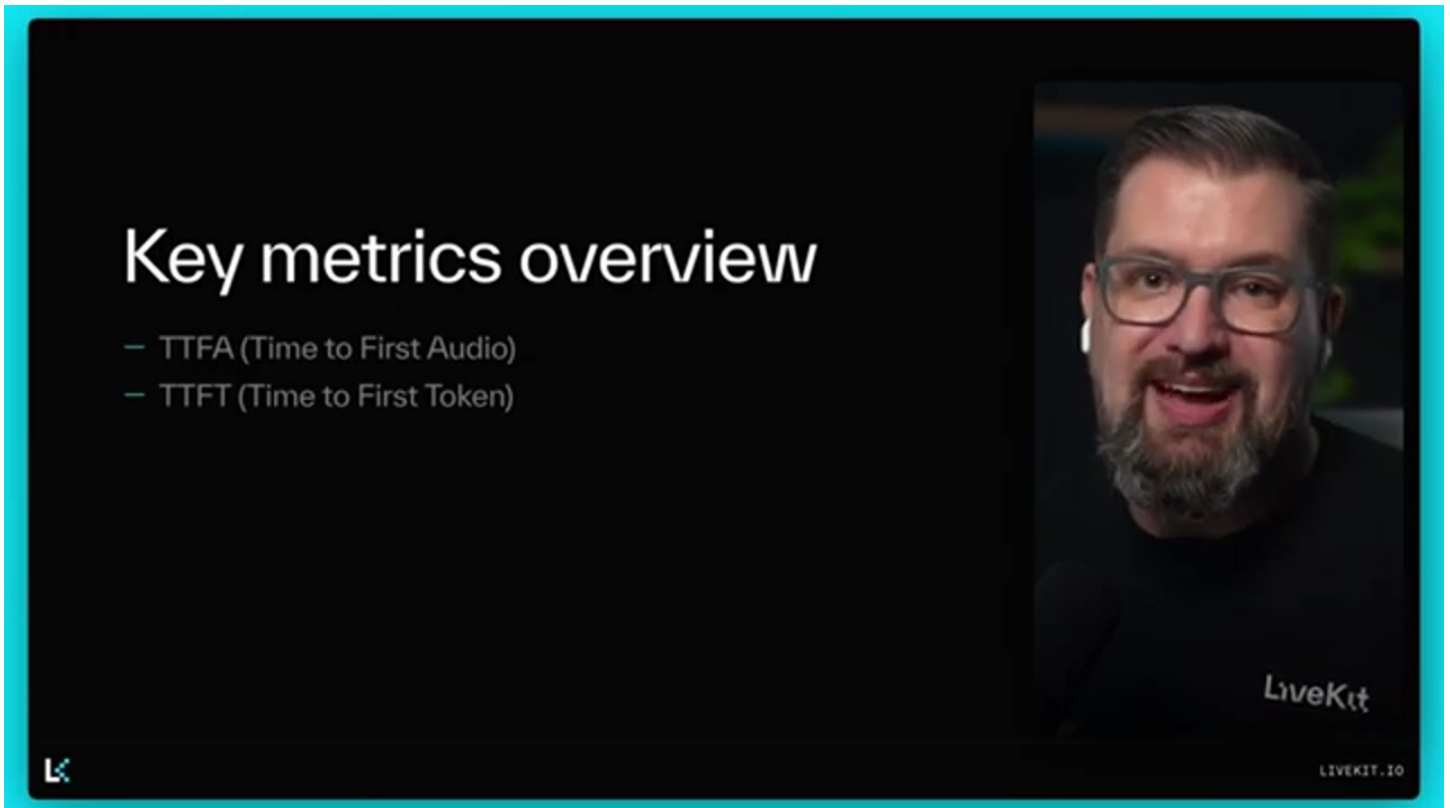
LIVEKIT.IO

topics. Metrics collection to understand your agents behavior, deployment to LifeKit Cloud for production observability, and latency optimization techniques, including preemptive generation. So by the end you'll have real numbers to optimize against and the tools to track them in production. Now before we jump into the code, let's talk about what you're actually measuring. So

■ [26:40 - 27:00]



about what you're actually measuring. So time to first audio TTFA. This is the delay from when the user stops speaking to when they hear the first word of the response. This is the number users feel most directly. And then time to first token tracks when the LLM starts generating which helps isolate whether



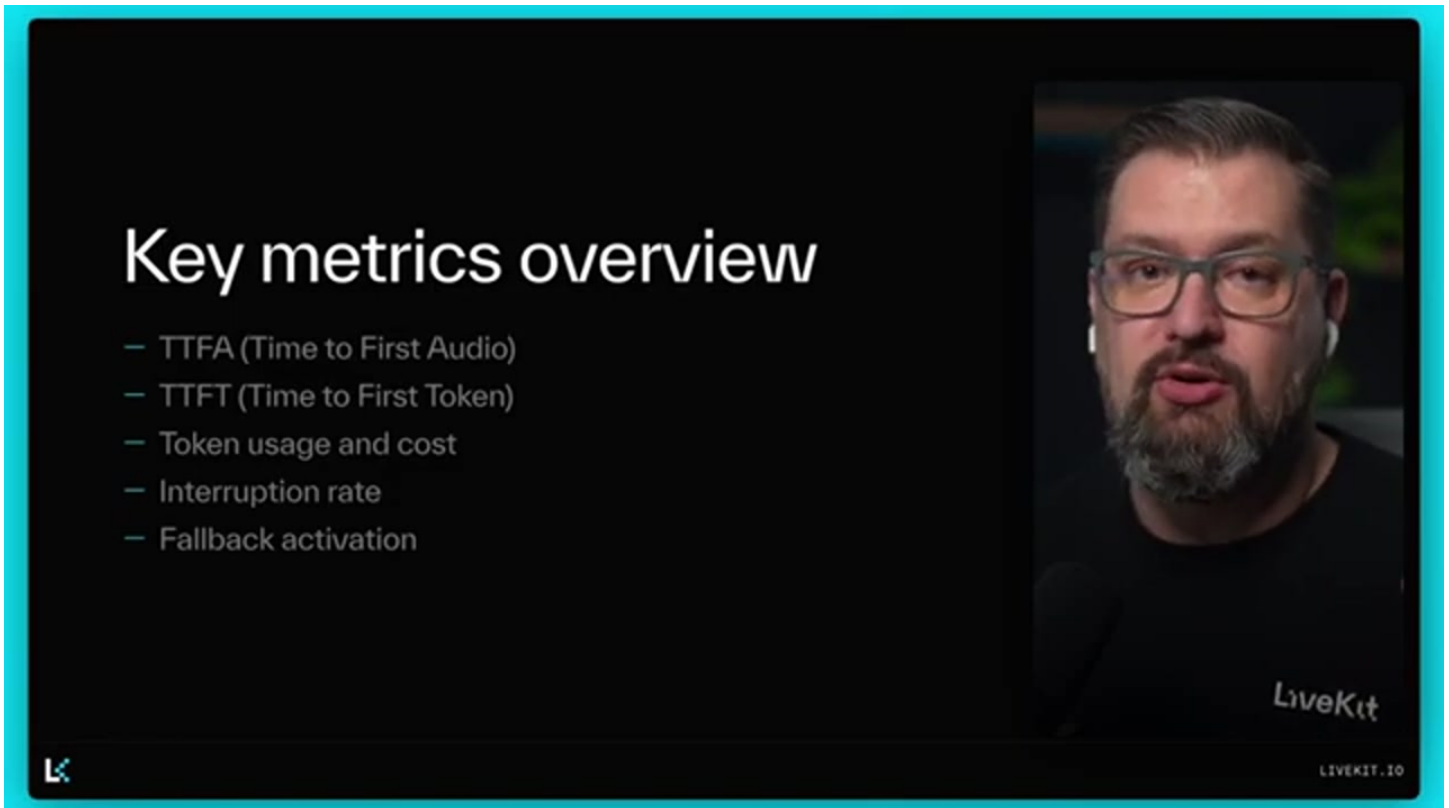
Key metrics overview

- TTFA (Time to First Audio)
- TTFT (Time to First Token)

LiveKit

LIVEKIT.IO

generating which helps isolate whether delays are coming from transcription the model or speech synthesis. You'll also track token usage for cost estimations, interruption rate to see how often users cut off the agent and fallback activations if your primary providers fail over. We're going to see these metrics show up in console logs during



The video frame shows a presentation slide with a black background and white text. The title 'Key metrics overview' is at the top left. Below it is a bulleted list of five metrics. On the right side of the slide is a portrait of a man with a beard and glasses, wearing a black shirt with the 'LiveKit' logo. The 'LiveKit' logo is also visible in the bottom right corner of the slide. A small 'L' logo is in the bottom left corner of the video frame.

Key metrics overview

- TTFA (Time to First Audio)
- TTFT (Time to First Token)
- Token usage and cost
- Interruption rate
- Fallback activation

LiveKit

LIVEKIT.IO

metrics show up in console logs during development and in LifeKit cloud's dashboard in production. The key takeaway here is going to be TTFA time to first audio. We want that to be less than 1 second for most responses. Let's start by collecting usage metrics. The first thing we're going to do is add some imports here. So, we're going to import agent state changed event metrics

■ [27:40 - 28:00]

agent.py •

livekit-voice-agent > agent.py

```
1  import logging
2
3  from dotenv import load_dotenv
4  from livekit import agents
5  from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6  from livekit.plugins import noise_cancellation, silero
7  from livekit.plugins.turn_detector.multilingual import MultilingualModel
8  from livekit.agents import llm, stt, tts, inference
9  from livekit.agents import AgentStateChangedEvent, MetricsCollectedEvent, metrics
10
11  logger = logging.getLogger(__name__)
12
13  load_dotenv()
14
15
16  class Assistant(Agent):
17      def __init__(self) -> None:
18          super().__init__(
19              instructions="You are an upbeat, slightly sarcastic voice AI for tech support. "
20              "Help the caller fix issues without rambling, and keep replies under 3 sentences.",
21          )
22
```



import agent state changed event metrics collected event metrics. Uh, all that comes from LifeKit agents. And then we're just going to define our logger here. And then we'll go down to our entry point function. So just before the session start, we've added this code block here. So let's break this down to see what this does. The usage collector aggregates data across all conversation

■ [28:00 - 28:20]

```
agent.py x
livekit-voice-agent > agent.py
28  async def entrypoint(ctx: JobContext):
52      )
53
54      usage_collector = metrics.UsageCollector()
55      last_eou_metrics: metrics.EOUMetrics | None = None
56
57      @session.on("metrics_collected")
58      def _on_metrics_collected(ev: MetricsCollectedEvent):
59          nonlocal last_eou_metrics
60          if ev.metrics.type == "eou_metrics":
61              last_eou_metrics = ev.metrics
62
63              metrics.log_metrics(ev.metrics)
64              usage_collector.collect(ev.metrics)
65
66
67      async def log_usage():
68          summary = usage_collector.get_summary()
69          logger.info("Usage summary: %s", summary)
70
71
72      ctx.add_shutdown_callback(log_usage)
```



aggregates data across all conversation turns. It tracks token counts for the LLM, audio durations for ST and TTS and cost estimates when available. The metrics collected event fires after each component finishes processing. You're capturing uh EOU metrics specifically, which stands for end of utterance. This

■ [28:20 - 28:40]

```
agent.py x
livekit-voice-agent > agent.py
28  async def entrypoint(ctx: JobContext):
52      )
53
54      usage_collector = metrics.UsageCollector()
55      last_eou_metrics: metrics.EOUMetrics | None = None
56
57      @session.on("metrics_collected")
58      def _on_metrics_collected(ev: MetricsCollectedEvent):
59          nonlocal last_eou_metrics
60          if ev.metrics.type == "eou_metrics":
61              last_eou_metrics = ev.metrics
62
63          metrics.log_metrics(ev.metrics)
64          usage_collector.collect(ev.metrics)
65
66
67      async def log_usage():
68          summary = usage_collector.get_summary()
69          logger.info("Usage summary: %s", summary)
70
71
72      ctx.add_shutdown_callback(log_usage)
```



which stands for end of utterance. This is the moment when the the turn detector decides that the user has finished speaking and then when the uh worker shuts down log usage prints a summary so that you can see uh per session resource consumption. So the agent cycles through several states during a conversation listening thinking and speaking the

■ [28:40 - 29:00]



listening thinking and speaking the agent state changed event. It fires each time the agent transitions between these states. When the agent enters the speaking state, you want to know how long the human waited. So, back in our code, below that code block that we just added, let's add another. And then be sure to uh import time up above here.

■ [29:00 - 29:20]



```
Agents Editor workshop-demo
agent.py X
livekit-voice-agent > agent.py
1 import logging
2
3 from dotenv import load_dotenv
4 from livekit import agents
5 from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6 from livekit.plugins import noise_cancellation, silero
7 from livekit.plugins.turn_detector.multilingual import MultilingualModel
8 from livekit.agents import llm, stt, tts, inference
9 from livekit.agents import AgentStateChangedEvent, MetricsCollectedEvent, metrics
10
11 logger = logging.getLogger(__name__)
12
13 load_dotenv()
14
15
16 class Assistant(Agent):
17     def __init__(self) -> None:
18         super().__init__(
19             instructions="You are an upbeat, slightly sarcastic voice AI for tech support. "
20             "Help the caller fix issues without rambling, and keep replies under 3 sentences."
21         )
22
23
24 server = AgentServer()
25
```

workshop-demo 0 0 0 Cursor Tab Ln 9, Col 82 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

sure to uh import time up above here. We're using that in this new code block. So, this new block here, this gives you a real number to optimize against. You can watch the console output and see exactly how long users wait for each response. So, we'll see this being printed in the console after each response. Now, preemptive generation

■ [29:20 - 29:40]



lets the LLM start forming a response before the user finishes speaking. The agent waits for a clear end of turn before actually speaking, but the thinking has already begun. We can enable this in a single line. If we go back up to our agent session, we're going to add another line here. And

■ [29:40 - 30:00]

```
30 session = AgentSession(  
37     llm=llm.FallbackAdapter(  
38         [  
40             inference.LLM(model="google/gemini-2.5-flash"),  
41         ],  
42     ),  
43     tts=tts.FallbackAdapter(  
44         [  
45             inference.TTS.from_model_string(  
46                 "cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc"  
47             ),  
48             inference.TTS.from_model_string("inworld/inworld-tts-1"),  
49         ],  
50     ),  
51     vad=silero.VAD.load(),  
52     turn_detection=MultilingualModel(),  
53     preemptive_generation=True,  
54 )  
55  
56 usage_collector = metrics.UsageCollector()  
57 last_eou_metrics: metrics.EOMetrics | None = None  
58  
59 @session.on("metrics_collected")  
60 def _on_metrics_collected(ev: MetricsCollectedEvent):  
61     pass
```



going to add another line here. And we're just going, it's just preemptive generation and equals true. That's it. And so this can save hundreds of milliseconds off of perceived latency, especially for longer uh user turns. The LLM gets a head start on generating the response. But that's not to say that preemptive generation is 100% good all of the time. Preemptive generation uh

Preemptive generation tradeoffs



LIVEKIT.IO

of the time. Preemptive generation uh trades accuracy for speed. There are a few scenarios where uh this trade-off might not work in your favor. For instance, mids sentence direction changes. If a user says, "Book me a flight to New York." No, actually make that Chicago. The LLM might have already committed to New York. Second, complex

Preemptive generation tradeoffs

- Mid-sentence direction changes



LIVEKIT.IO

committed to New York. Second, complex multi-part instructions, long requests with multiple steps can get partially answered before the full context is available. Third, high accuracy domains, medical, legal, or financial applications where getting it right matters more than getting it fast. But for many conversational use cases,

Preemptive generation tradeoffs

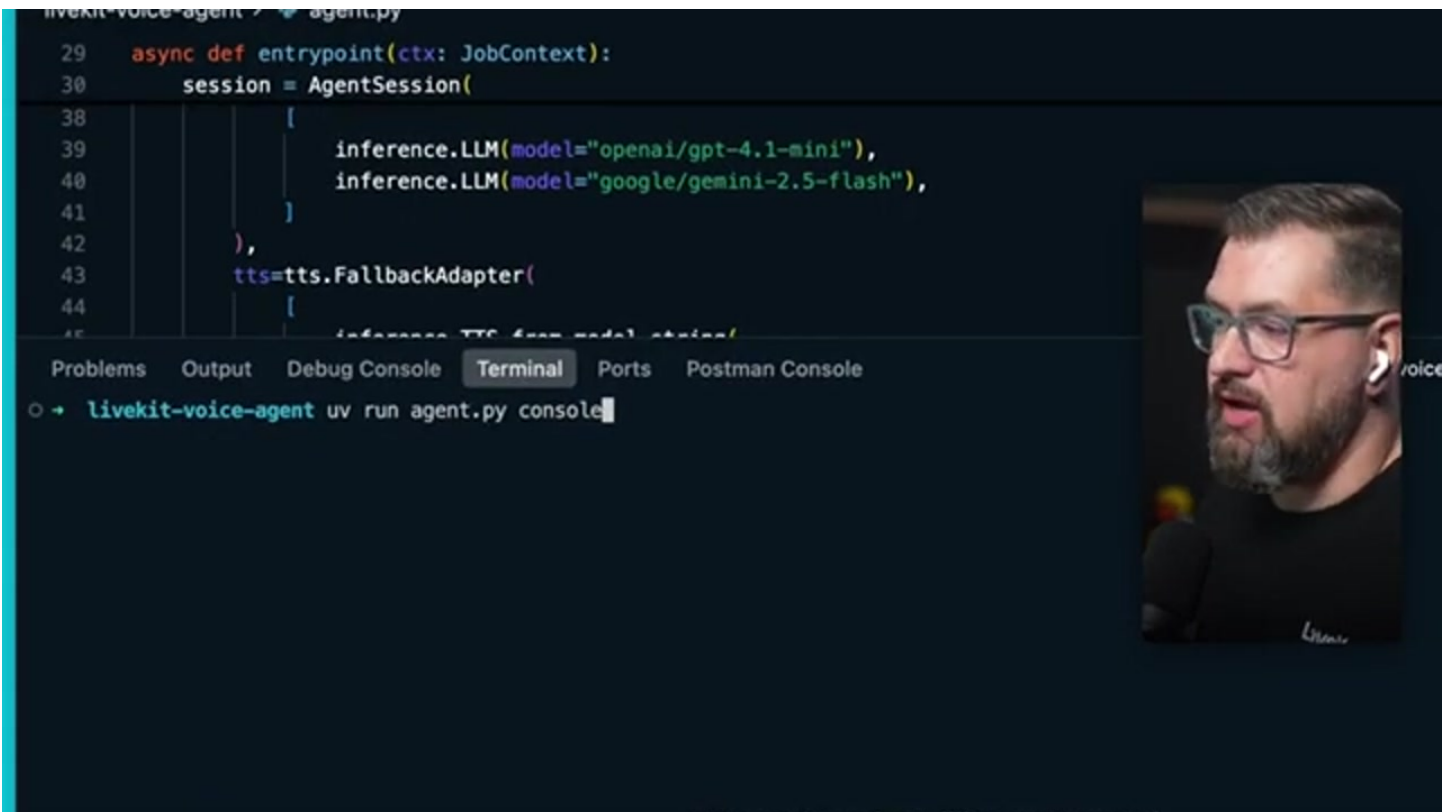
- Mid-sentence direction changes
- Complex multi-part instructions
- High-accuracy domains



LIVEKIT.IO

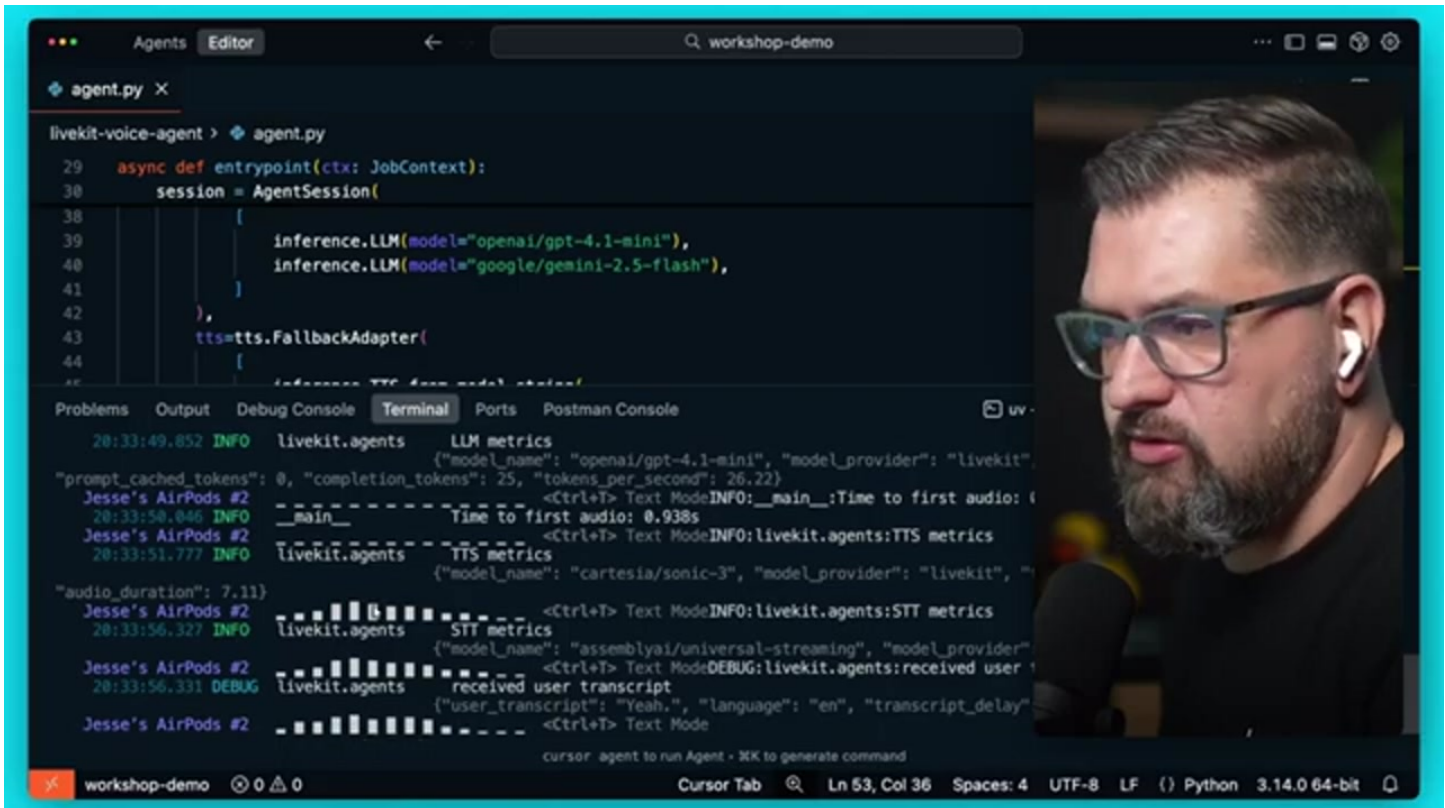
for many conversational use cases, preemptive generation works well. But if you notice that the agent is uh jumping to conclusions or misinterpreting intent, consider disabling it or tuning your turn detection thresholds. So let's test out these metrics and see them in action. So go ahead and open up your terminal and we'll run `uvun agent pi`

■ [31:00 - 31:20]



terminal and we'll run `uv run agent.py console`. Hello agent, how are you? Hey there. I'm great. Thanks for asking. Ready to tackle your tech troubles? >> Yeah, let's see. What troubles am I having? Yeah, my my iPods have been like making crackling noises. What do you

■ [31:20 - 31:40]



making crackling noises. What do you What's up with that? >> Crackling on iPods usually means dirty headphone jacks or a software glitch. >> Try clean. >> That sounds terrible. Okay, let's take a look at these metrics. So, let's go up to All right, so here we go. We've got some ST metrics here. We get the model name, the provider, and then the audio

■ [31:40 - 32:00]



```
ctx: JobContext):
  session(

  ence.LLM(model="openai/gpt-4.1-mini"),
  ence.LLM(model="google/gemini-2.5-flash"),

  backAdapter(

  -- STT from model stream

  sole  Terminal  Ports  Postman Console  zsh - livekit-voice-agent + v

  it.agents  STT metrics
  {"model_name": "assemblyai/universal-streaming", "model_provider": "livekit", "audio_duration": 7.4}
  it.agents  received user transcript
  {"user_transcript": "Yeah.", "language": "en", "transcript_delay": 0.02976512908935547}
  it.agents  STT metrics
  {"model_name": "assemblyai/universal-streaming", "model_provider": "livekit", "audio_duration": 7.2}
  it.agents  received user transcript
  {"user_transcript": "Let's see what troubles am I having yeah my ipods have been like making crackling",
  "language": "en", "transcript_delay": 0.45828700065612793}
  it._detector eou prediction
  {"eou_probability": 0.6739398837089539, "duration": 0.049, "input": "<|im_start|>user\nhello agent how"}

  cursor: agent to run Agent - MK to generate command
```

name, the provider, and then the audio duration. Uh we've got some received user transcript here. Let's move on. ST metrics again. Received user transcript. Here's some ELOU metrics here. Again, multilingual provider. Uh end of utterance delay 65. Uh time to first

■ [32:00 - 32:20]

```
session = AgentSession(  
    {  
        inference.LLM(model="openai/gpt-4.1-mini"),  
        inference.LLM(model="google/gemini-2.5-flash"),  
    },  
    tts=tts.FallbackAdapter(  
        {  
            inference.TTS(model="openai/tts-1"),  
            inference.TTS(model="google/text-to-speech"),  
        },  
    ),  
)
```

Output Debug Console Terminal Ports Postman Console zsh - livekit-

```
irPods #2 <Ctrl+T> Text ModeDEBUG:livekit.plugins.turn_detector:eou p  
.749 DEBUG livekit._detector eou prediction  
{"eou_probability": 0.6739398837089539, "duration": 0.049, "input": "the  
end|>\n<|im_start|>assistant\nhey there i'm great thanks for askingready to tackle your tech troubles what can i help yo  
d|>\n<|im_start|>user\nyeah let's see what troubles am i having yeah my ipods have been like making crackling noises wha  
irPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:using preemptive generation  
.751 DEBUG livekit.agents using preemptive generation {"preemptive_lead_time": 0.1899881362915039}  
irPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:EOU metrics  
.752 INFO livekit.agents EOU metrics  
{"model_name": "multilingual", "model_provider": "livekit", "end_of_utterance_delay": 0.65,  
on_delay": 0.46}  
irPods #2 <Ctrl+T> Text ModeINFO:__main__:Time to first audio: 0.889s  
.641 INFO __main__ Time to first audio: 0.889s  
irPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics  
.857 INFO livekit.agents LLM metrics  
cursor: agent to run Agent - ⌘K to generate command  
demo 0 0 Cursor Tab Ln 53, Col 36 Spaces: 4 UTF-8 LF Python 3.
```

utterance delay 65. Uh time to first audio. 8889 seconds. So, a lot of great information here that we can uh use to tune our agent. Now, when we press C to stop our agent, uh we then get this summary here, this nice little usage summary. This shows us the total tokens used, the total audio duration

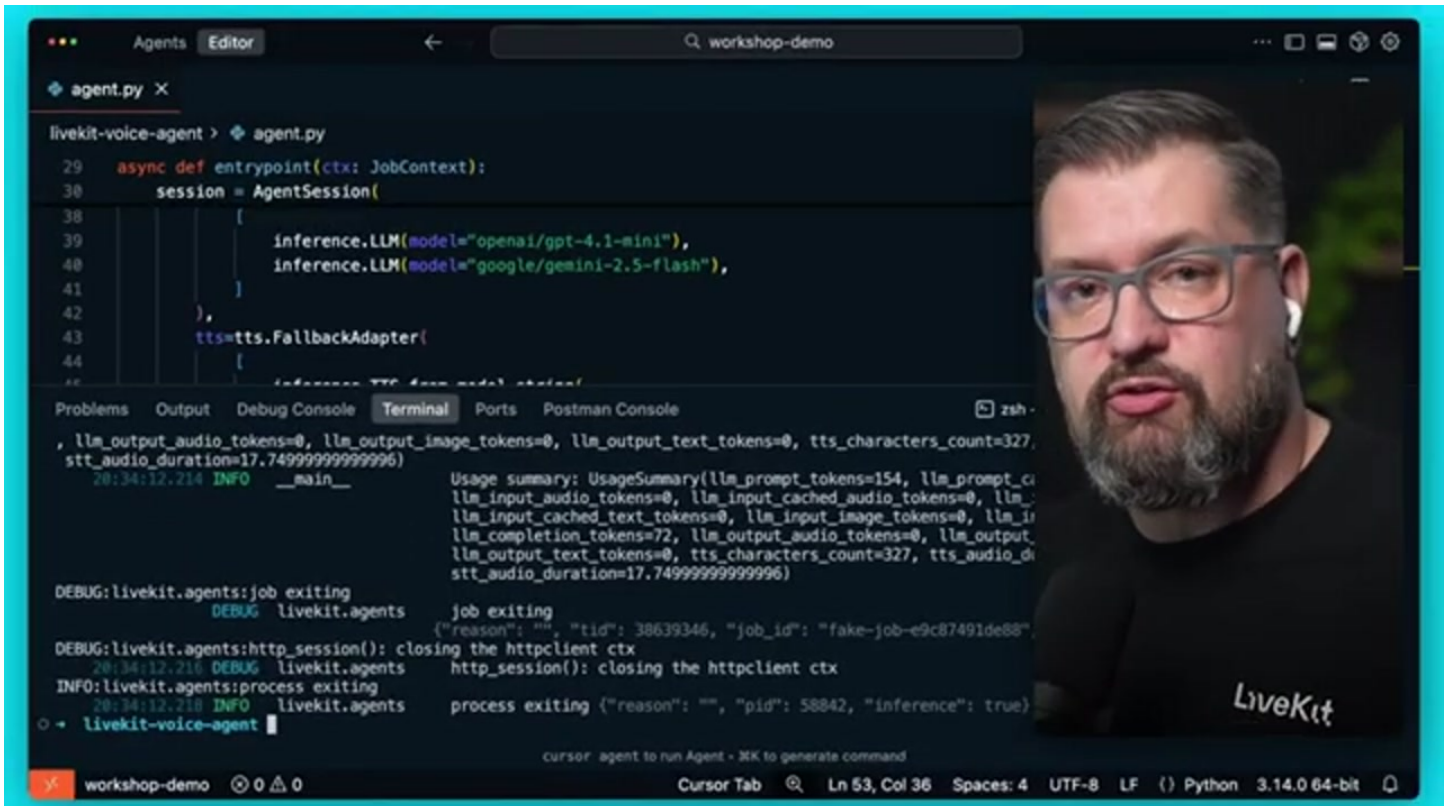
■ [32:20 - 32:40]

```
session = AgentSession(  
    {  
        inference.LLM(model="openai/gpt-4.1-mini"),  
        inference.LLM(model="google/gemini-2.5-flash"),  
    },  
    tts=tts.FallbackAdapter(  
        {  
            inference.TTS(model="openai/tts-1"),  
            inference.TTS(model="google/gemini-2.5-flash"),  
        },  
    ),  
)
```

output Debug Console Terminal Ports Postman Console zsh - livekit-
.213 DEBUG livekit.agents shutting down job task {"reason": "", "user_initiated": false}
Usage summary: UsageSummary(llm_prompt_tokens=154, llm_prompt_cached_tokens=0, llm_input_audio_tokens=0, llm_input_cached_audio_tokens=0, llm_input_text_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tokens=0, llm_output_audio_tokens=0, llm_output_image_tokens=0, llm_output_text_tokens=0, tts_characters_count=327, tts_audio_duration=19.086708333333333, stt_audio_duration=17.749999999999996)
.214 INFO __main__ Usage summary: UsageSummary(llm_prompt_tokens=154, llm_prompt_cached_tokens=0, llm_input_audio_tokens=0, llm_input_cached_audio_tokens=0, llm_input_text_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tokens=0, llm_output_audio_tokens=0, llm_output_image_tokens=0, llm_output_text_tokens=0, tts_characters_count=327, tts_audio_duration=19.086708333333333, stt_audio_duration=17.749999999999996)
t.agents:job exiting
DEBUG livekit.agents job exiting {"reason": "", "tid": 38639346, "job_id": "fake-job-e9c87491de88", "room_id": "FAKE_RM_ebcd"}
t.agents:http_session(): closing the httpclient ctx
.216 DEBUG livekit.agents http_session(): closing the httpclient ctx
cursor: agent to run Agent - KK to generate command
demo 0 0 Cursor Tab Ln 53, Col 36 Spaces: 4 UTF-8 LF Python 3.

used, the total audio duration processed, and estimated costs if they're available. So we can see uh completion tokens there. Uh character counts, audio duration, ST audio duration versus TTS audio duration, LLM prompt tokens 154. So you now have visibility into your agents

■ [32:40 - 33:00]



```
livekit-voice-agent > agent.py
29 async def entrypoint(ctx: JobContext):
30     session = AgentSession(
38         [
39             inference.LLM(model="openai/gpt-4.1-mini"),
40             inference.LLM(model="google/gemini-2.5-flash"),
41         ],
42     ),
43     tts=tts.FallbackAdapter(
44         [
45             inference.TTS(model="openai/tts-1"),
46             inference.TTS(model="google/text-to-speech"),
47         ],
48     ),
49 )
50 session.run()
51
```

```
Usage summary: UsageSummary(llm_prompt_tokens=154, llm_prompt_completion_tokens=72, llm_input_audio_tokens=0, llm_input_cached_audio_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_image_audio_tokens=0, llm_completion_tokens=72, llm_output_audio_tokens=0, llm_output_text_tokens=0, tts_characters_count=327, tts_audio_duration=17.749999999999996)
DEBUG:livekit.agents:job exiting
DEBUG:livekit.agents:job exiting {"reason": "", "tid": 38639346, "job_id": "fake-job-e9c87491de88"}
DEBUG:livekit.agents:http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting
INFO:livekit.agents:http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting {"reason": "", "pid": 58842, "inference": true}
livekit-voice-agent
```

now have visibility into your agents performance. The metrics show where time is spent, how many tokens are consumed, and how much audio is processed. So use this data to identify bottlenecks and optimize your pipeline. Remember this table from lesson one. With a best case total around 400 milliseconds, but a typical latency around 1 or two seconds,

Average latencies

Component	Best case	Typical
VAD	15-20ms	20-30ms
STT	200-300ms	400-600ms
LLM	100-200ms	500-1000ms
TTS	100-150ms	200-300ms
Total	~415ms	1.1s-2s



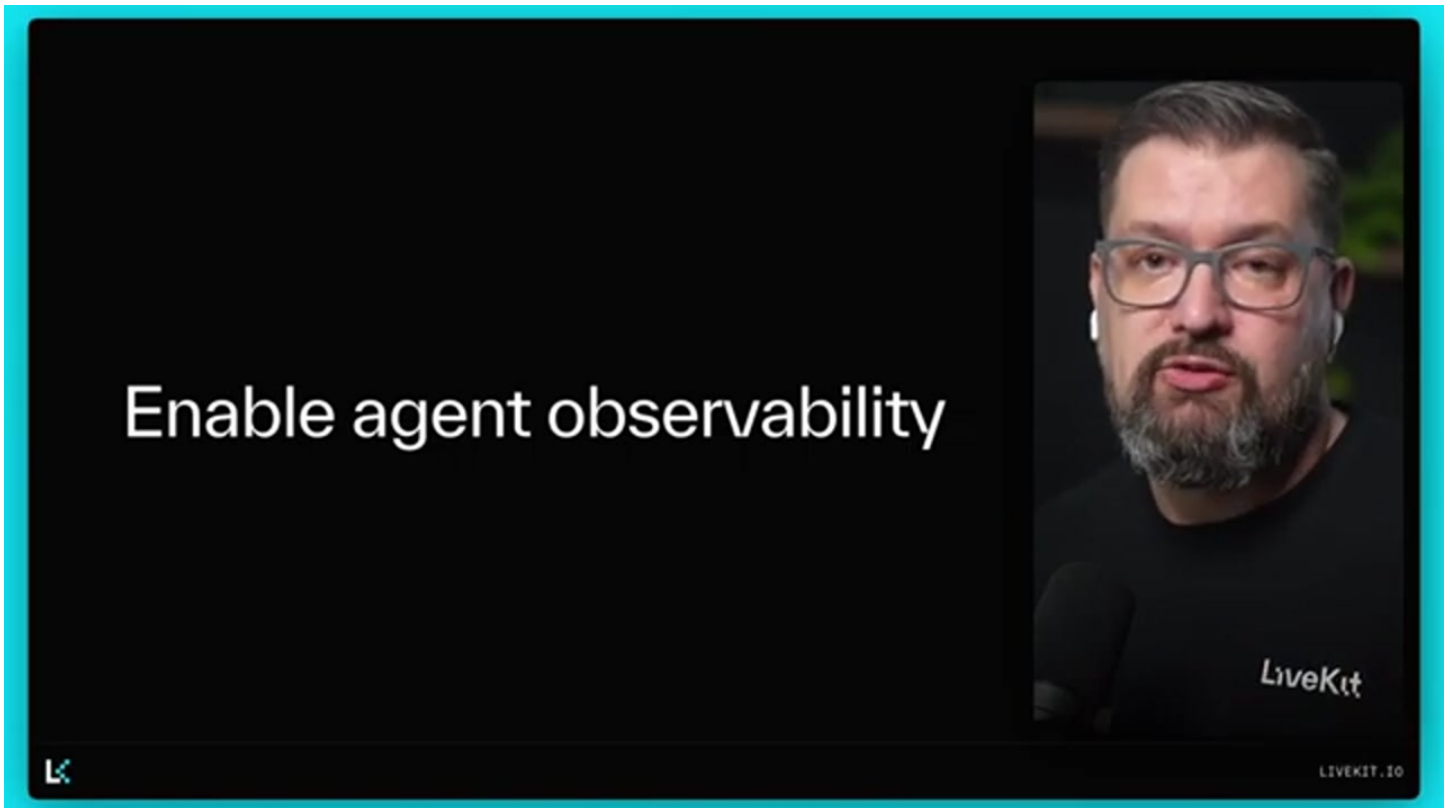
typical latency around 1 or two seconds, any TTFA under 1 second is solid. So, you're seeing about 800 milliseconds here, which puts this response in a good territory. Now, local testing and metrics is fine at first. Uh, but you'll want something much more robust in production. And so, you can use external tracing tools like Langfuse if you want.

Enable agent observability



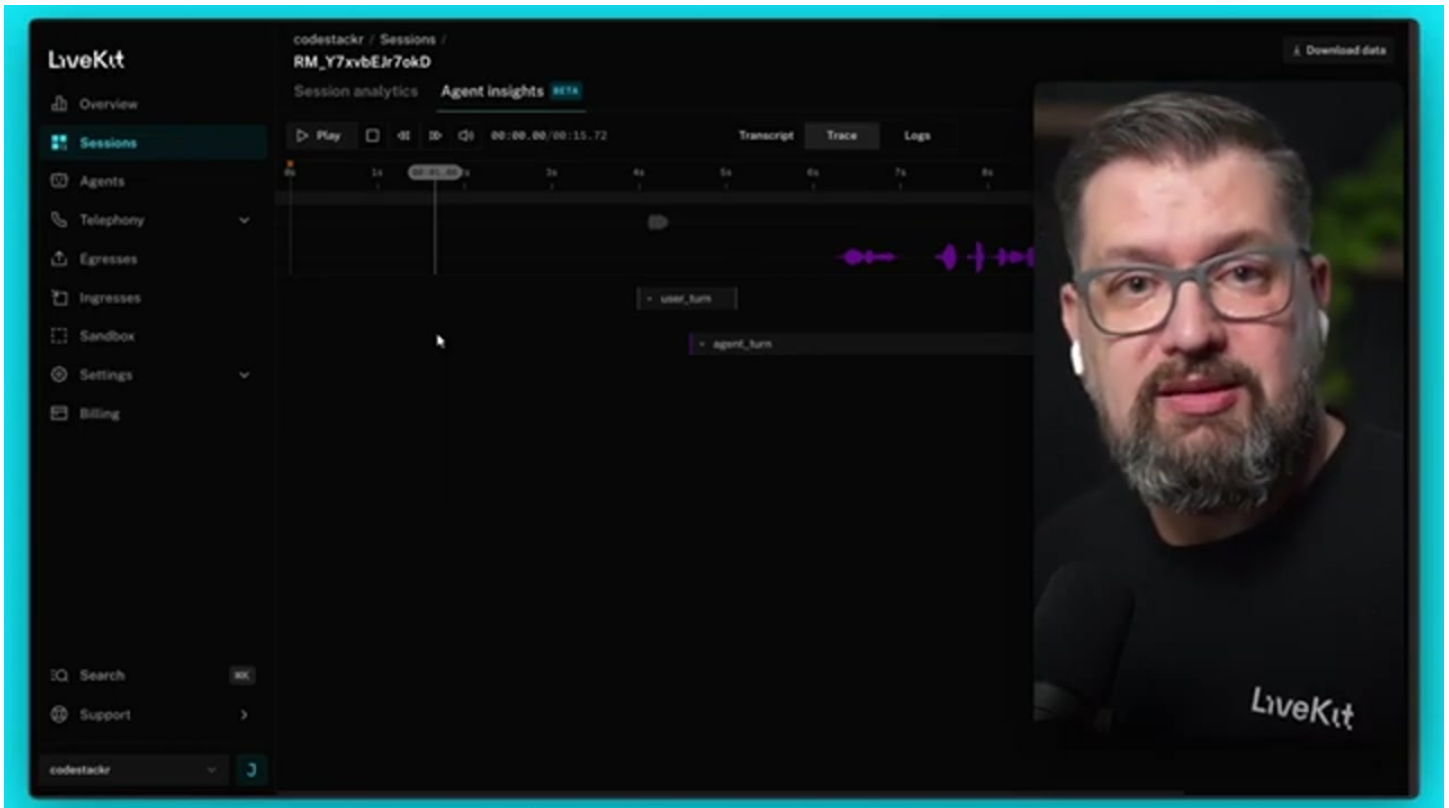
LIVEKIT.IO

tracing tools like Langfuse if you want. Uh, but LifeKit Cloud includes built-in observability for agent sessions. So, you can view transcripts, traces, logs, and even audio recordings directly in the LifeKit cloud dashboard without any uh additional steps. LiveKit's observability is also open telemetry compatible. So, you can also export



compatible. So, you can also export traces to any open telemetry compatible backend if you prefer. Our dashboard here shows three synchronized views. The the transcripts with audio playback let you scrub through the conversation. You can hear exactly what happened. The traces tab breaks down each turn into spans for uh ST, LLM, TTS, and tool

■ [34:00 - 34:20]



spans for uh ST, LLM, TTS, and tool calls with detailed timestamps and durations. And then logs shows real-time messages from your agent server in chronological order. All of this data is synchronized to a single timeline which makes it so easy and so simple to see exactly what is happening under the hood. Now observability needs to be

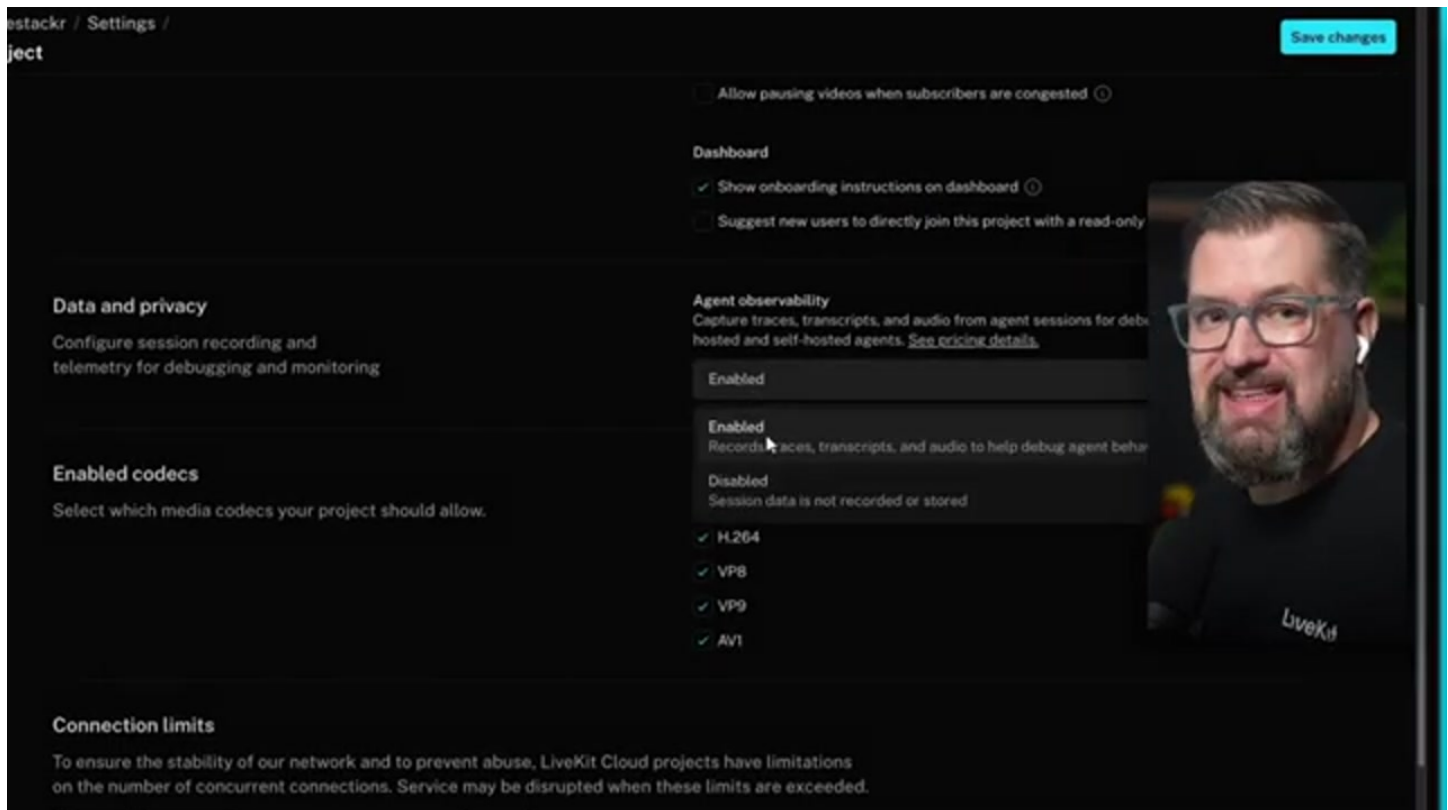
■ [34:20 - 34:40]

The screenshot displays the LiveKit dashboard interface. On the left is a sidebar with navigation links: Overview, Sessions (selected), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. The main content area is titled 'codestackr / Sessions / RM_Y7xvbEjr7okD'. It features a 'Session analytics' section with a 'Play' button and a timeline. Below this is a table of logs with columns for Timestamp, Level, Logger, and Message. The logs show various metrics and session events. On the right side of the dashboard, there is a video feed of a man with a beard and glasses, wearing a LiveKit-branded shirt, speaking into a microphone.

Timestamp	Level	Logger	Message
Dec 26, 00:52:52.837 PM	INFO	livekit.agents	STT metrics
Dec 26, 00:52:53.369 PM	INFO	livekit.agents	EOU metrics
Dec 26, 00:52:53.632 PM	INFO	livekit.agents	LLM metrics
Dec 26, 00:52:53.971 PM	INFO	...mp_main...	Time to first audio: 0.602s
Dec 26, 00:52:54.889 PM	INFO	livekit.agents	TTS metrics
Dec 26, 00:53:00.312 PM	INFO	livekit.agents	closing agent session due to participant disconnect id
Dec 26, 00:53:04.151 PM	INFO	...mp_main...	Usage summary: UsageSummary(lm_prompt_tokens=

hood. Now observability needs to be enabled in order to use it and we can do this at the project level. So go to your LiveKit dashboard and then under settings we'll go to project and then we can scroll down to data and privacy and then agent observability and just make sure that

■ [34:40 - 35:00]



that is enabled and then go ahead and save your changes. Now once it's enabled all agent sessions will automatically record data. The agents SDK version uh for Python needs to be at least 1.3.0 or higher or 1.0.18. 0.18 or higher for NodeJS and recording happens automatically. You don't need to change

codestackr / Settings / Project

LiveKit

Overview
Sessions
Agents
Telephony
Egresses
Ingresses
Sandbox
Settings
Project
Team members
API keys
Webhooks
Billing
Search
Support

Data and privacy
Configure session recording and telemetry for debugging and monitoring.

Enabled codecs
Select which media codecs your project should allow.

Connection limits
To ensure the stability of our network and to prevent abuse, LiveKit Cloud projects have limitations on the number of concurrent connections. Service may be disrupted when these limits are exceeded. If you anticipate needing more concurrent connections and/or inference credits for your project, [upgrade to the Scale plan](#).

Type	Limit	Peak usage (past 7 da
Concurrent participants	100	0
Concurrent Excess requests	2	0

Save changes

Allow pausing videos when sub...

Dashboard
☒ Show onboarding instructions on...
☐ Suggest new users to directly jo...

Agent observability
Capture traces, transcripts, and aud... hosted and self-hosted agents. [See...](#)
☒ Enabled

Opus
Audio RED (REDundant encoding)
H264
VP8
VP9
AV1

LiveKit

automatically. You don't need to change your agent code at all. Audio is collected locally and then uploaded after the session ends along with transcripts and traces. Now, to view observability data in the dashboard, you will need to deploy your agent to LifeKit Cloud. Running locally is again fine for development, but we're at a place now where we want to start observing the performance of our agent.

■ [35:20 - 35:40]

codestackr / Settings / Project

Save changes

Data and privacy
Configure session recording and telemetry for debugging and monitoring

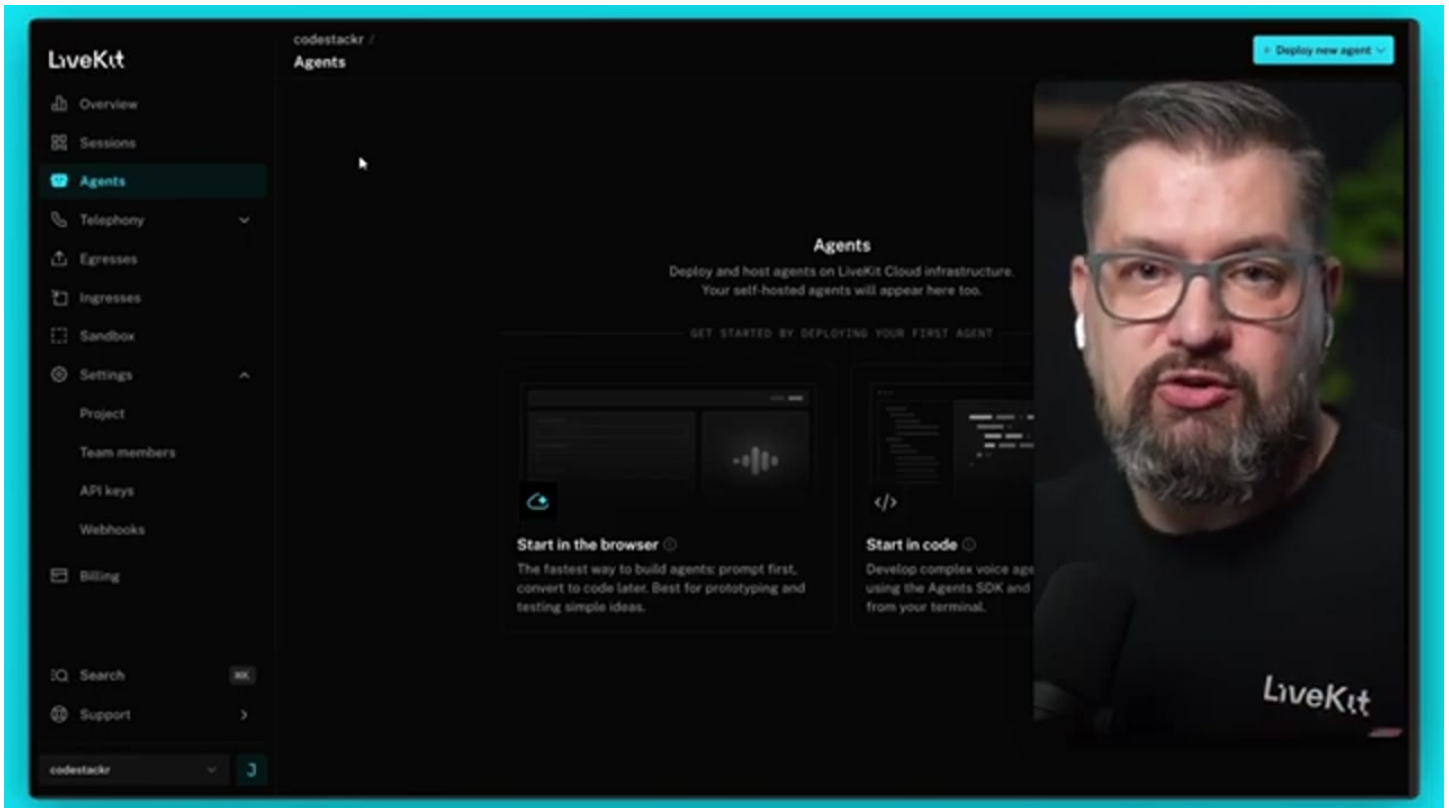
Enabled codecs
Select which media codecs your project should allow.

Connection limits
To ensure the stability of our network and to prevent abuse, LiveKit Cloud projects have limitations on the number of concurrent connections. Service may be disrupted when these limits are exceeded. If you anticipate needing more concurrent connections and/or inference credits for your project, [upgrade to the Scale plan](#).

Type	Limit	Peak usage (past 7 days)
Concurrent participants	100	0
Concurrent Egress requests	2	0

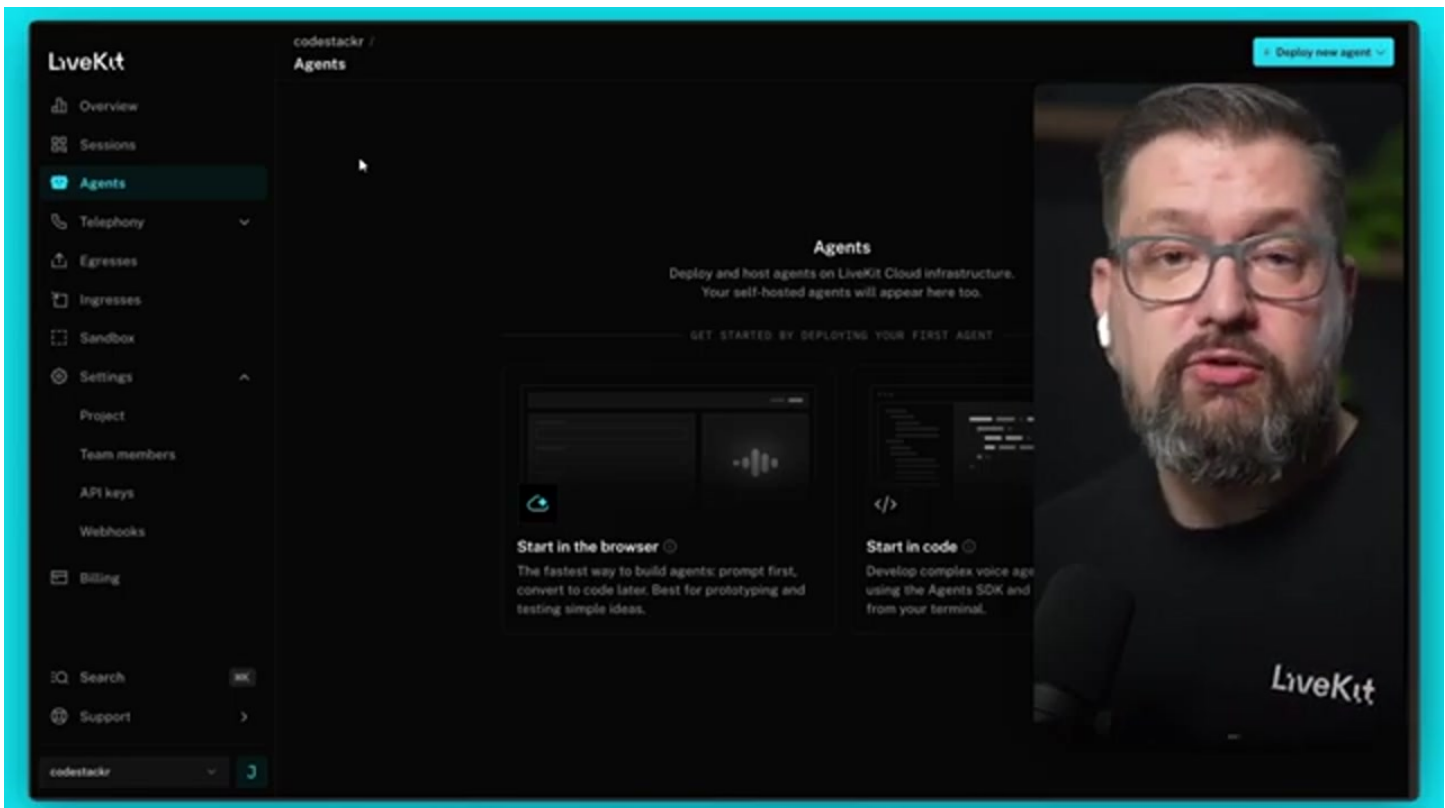
observing the performance of our agent. The decisions that we make from now on are going to have impacts on our agent and we want to know if a tool that we're adding or some feature that we're adding is introducing some poor performance. Now deploying to LiveKit Cloud gives you automatic scaling and load balancing. So your agent can handle multiple sessions

■ [35:40 - 36:00]



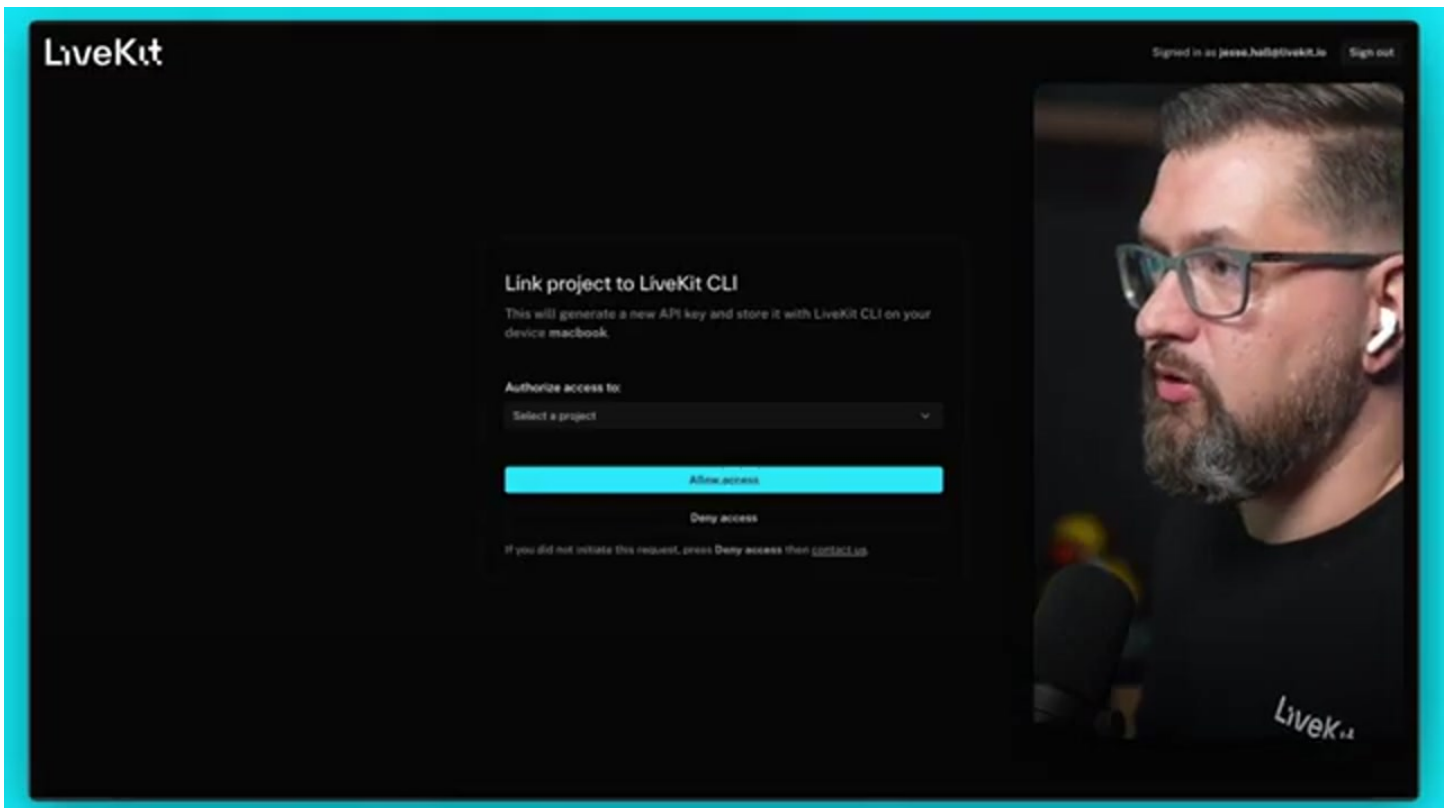
your agent can handle multiple sessions without manual configuration. And the built-in observability integration means that transcripts, traces, and audio recordings upload automatically. Your agent runs on LiveKit's global network infrastructure, reducing latency for users worldwide, and you don't need to manage servers or containers yourself. So, it's a win-win. First, you'll need

■ [36:00 - 36:20]



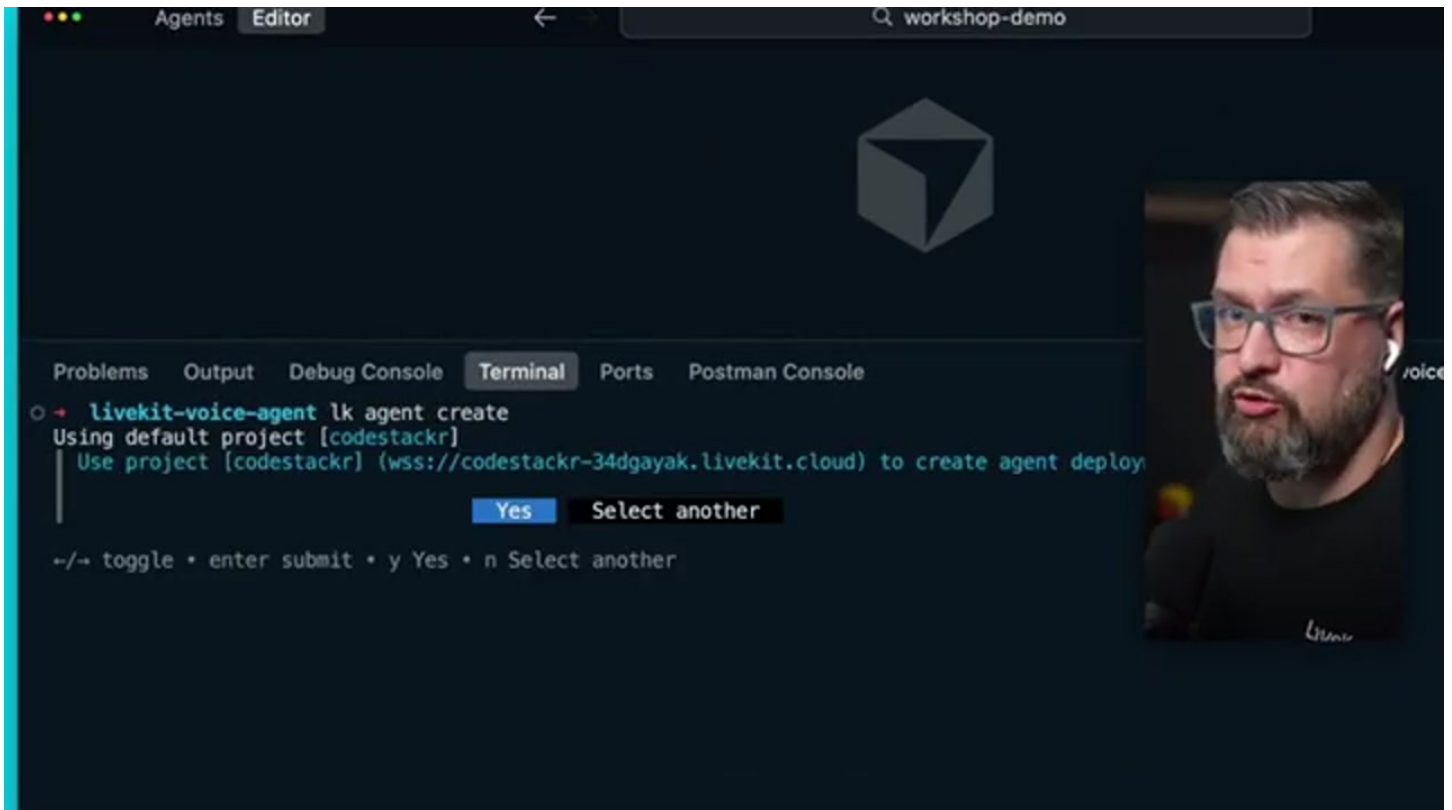
So, it's a win-win. First, you'll need to make sure that you have the LifeKit CLI installed. If you don't have it uh installed yet, you can install it with Homebrew on Mac OS, or you can download the binary from the LifeKit documentation. There's a link in the video description. So once you have that installed, uh we'll do LifeKit Cloud O. So that's going to pop us over to

■ [36:20 - 36:40]



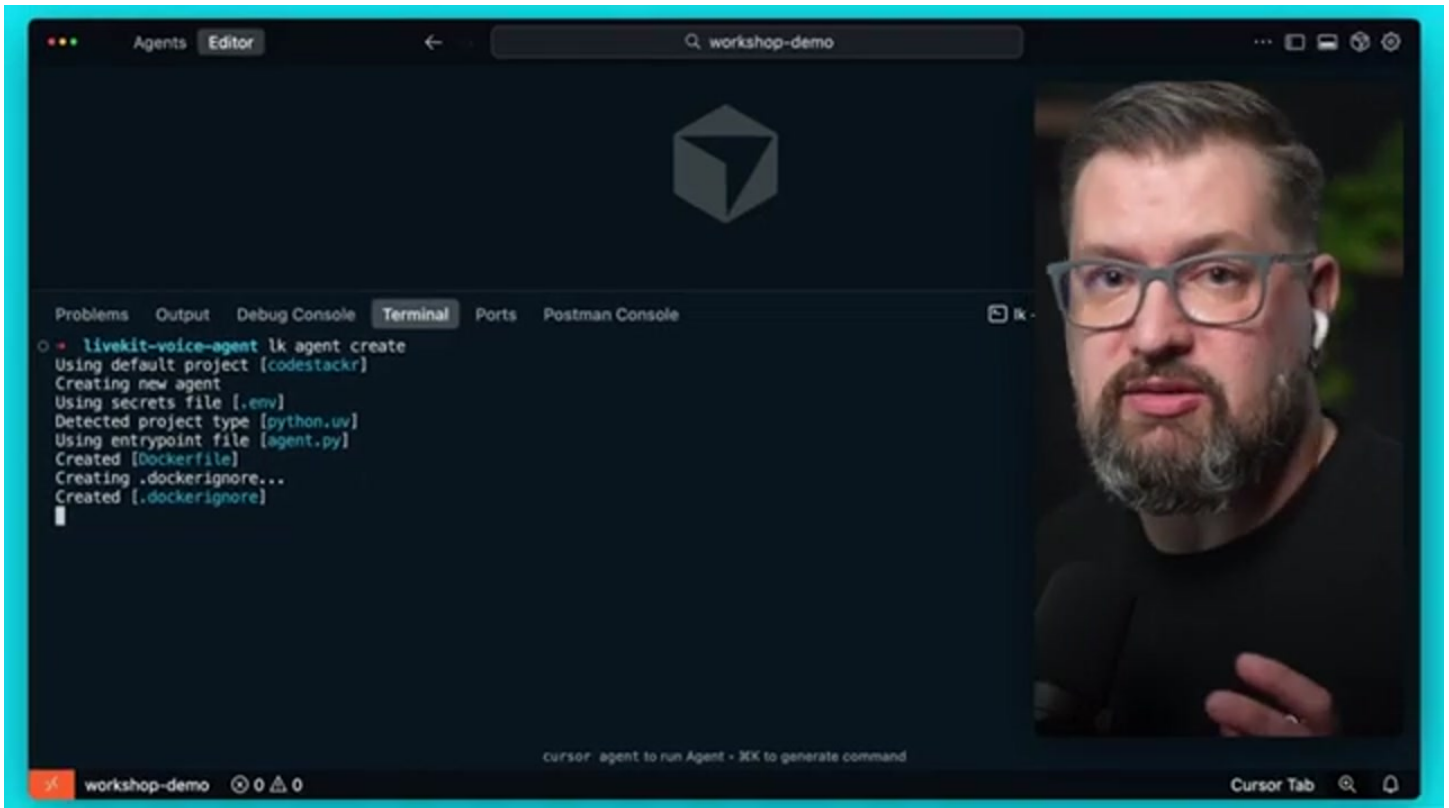
So that's going to pop us over to LifeKit Cloud and ask us to confirm and we'll go ahead and select a project that we want to give it access to and we're going to hit allow access. Cool. Now we can go back to our uh code editor. So now deploying our agent is as simple as a single command. Okay. Agent create. You can specify which project if you

■ [36:40 - 37:00]



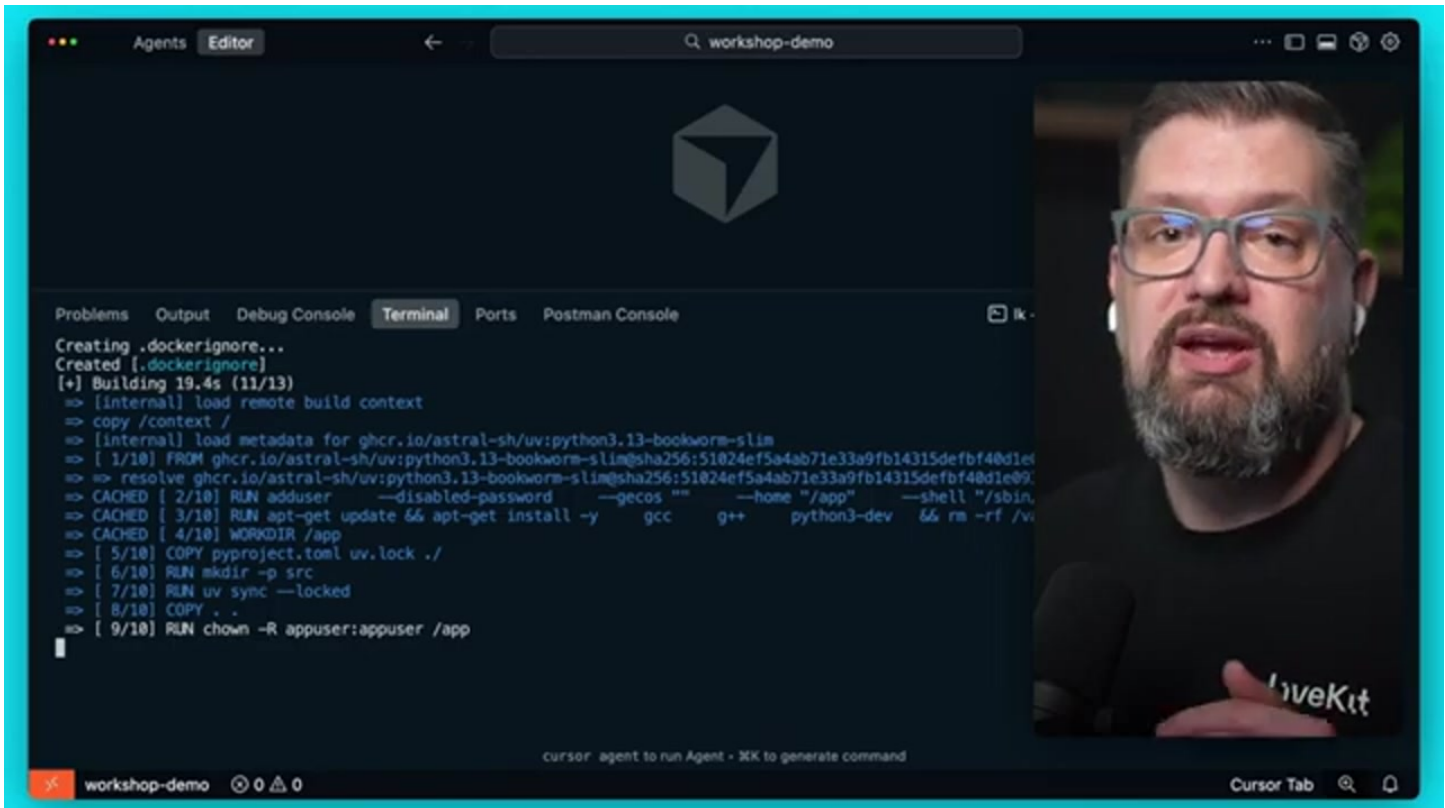
You can specify which project if you have multiple, but I'll just go ahead and select this project. Yes, select the secrets. I already have my environment variables set up. Uh if you didn't already have them, then uh it would create them for you. So, we'll go ahead and select the existing environment variables. Uh we can select a region, US East or EU Central. So, I'll do US East. So, this command it it does several

■ [37:00 - 37:20]



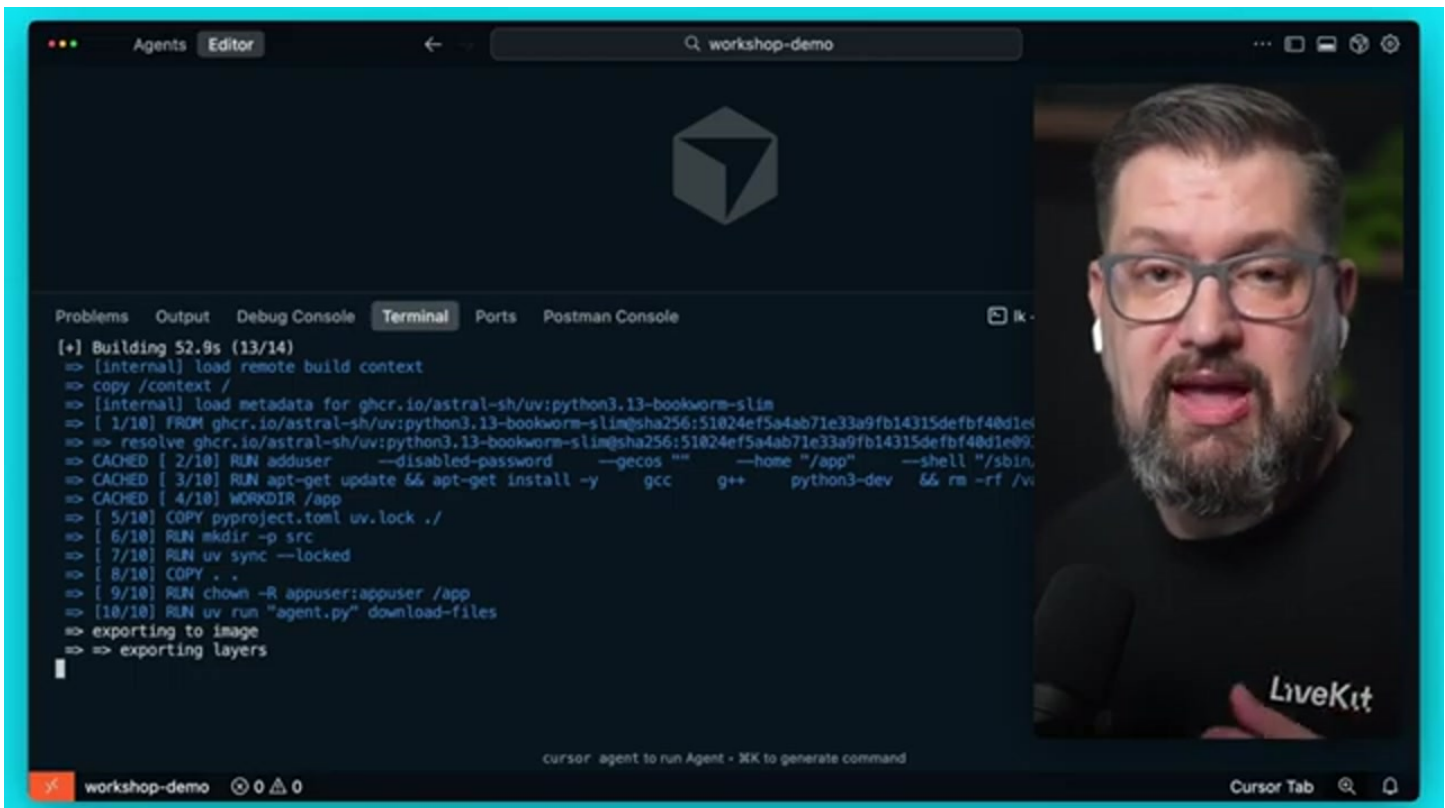
So, this command it it does several things. It registers your agent with LifeKit cloud and assigns a unique ID. The ID and the project configuration are written to a new LifeKit toml file. And if you don't already have a Docker file, the CLI is going to create that for you as well. And then it uploads your code to LifeKit Cloud's build service, builds a container image, and then deploys it

■ [37:20 - 37:40]



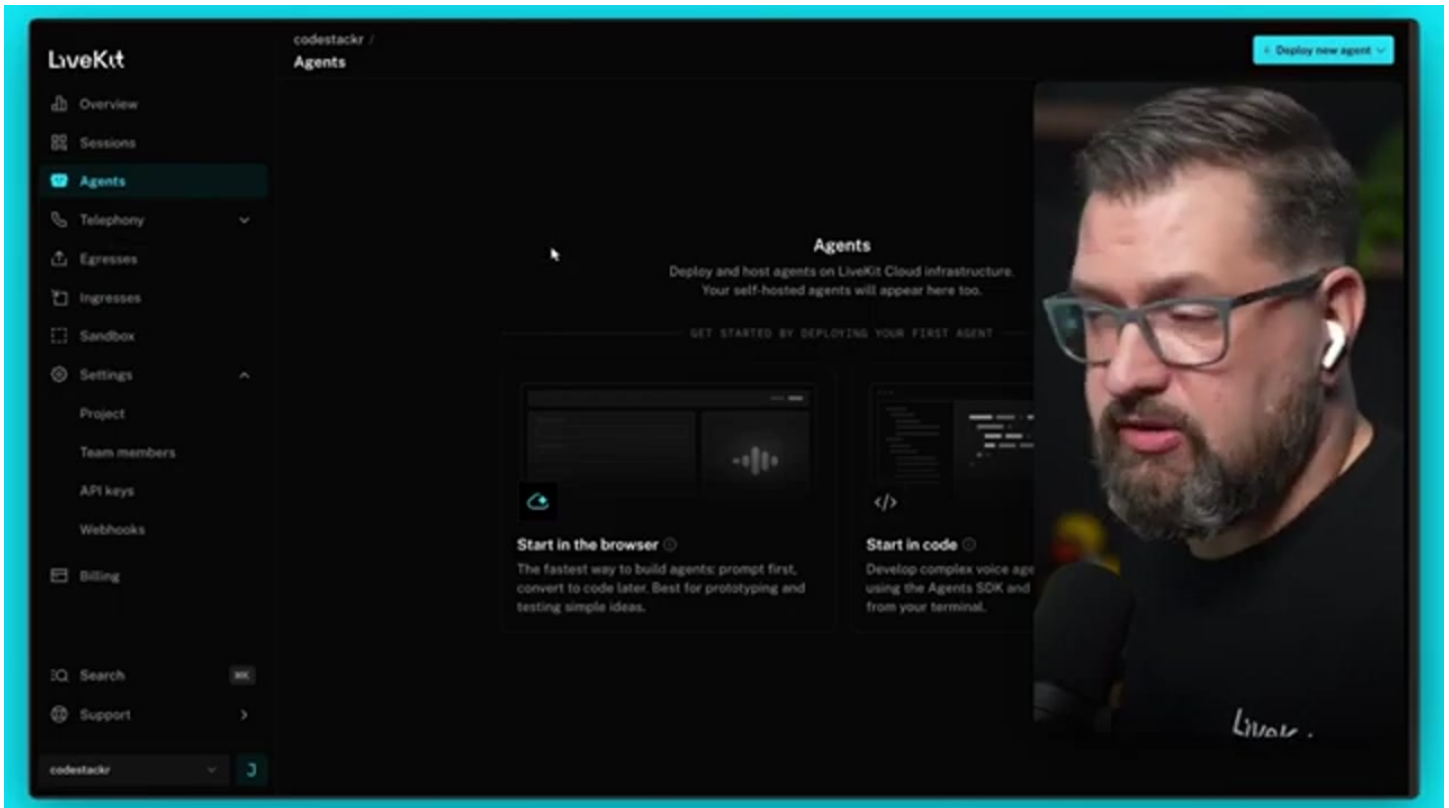
a container image, and then deploys it to your project. So you can see the build logs here streaming to our terminal uh as the the CA container image is creating. And this usually just takes a minute or two depending on your dependencies. Now, once the deployment is complete, your agent is going to be live on LifeKit Cloud and ready to handle sessions. The agent automatically

■ [37:40 - 38:00]



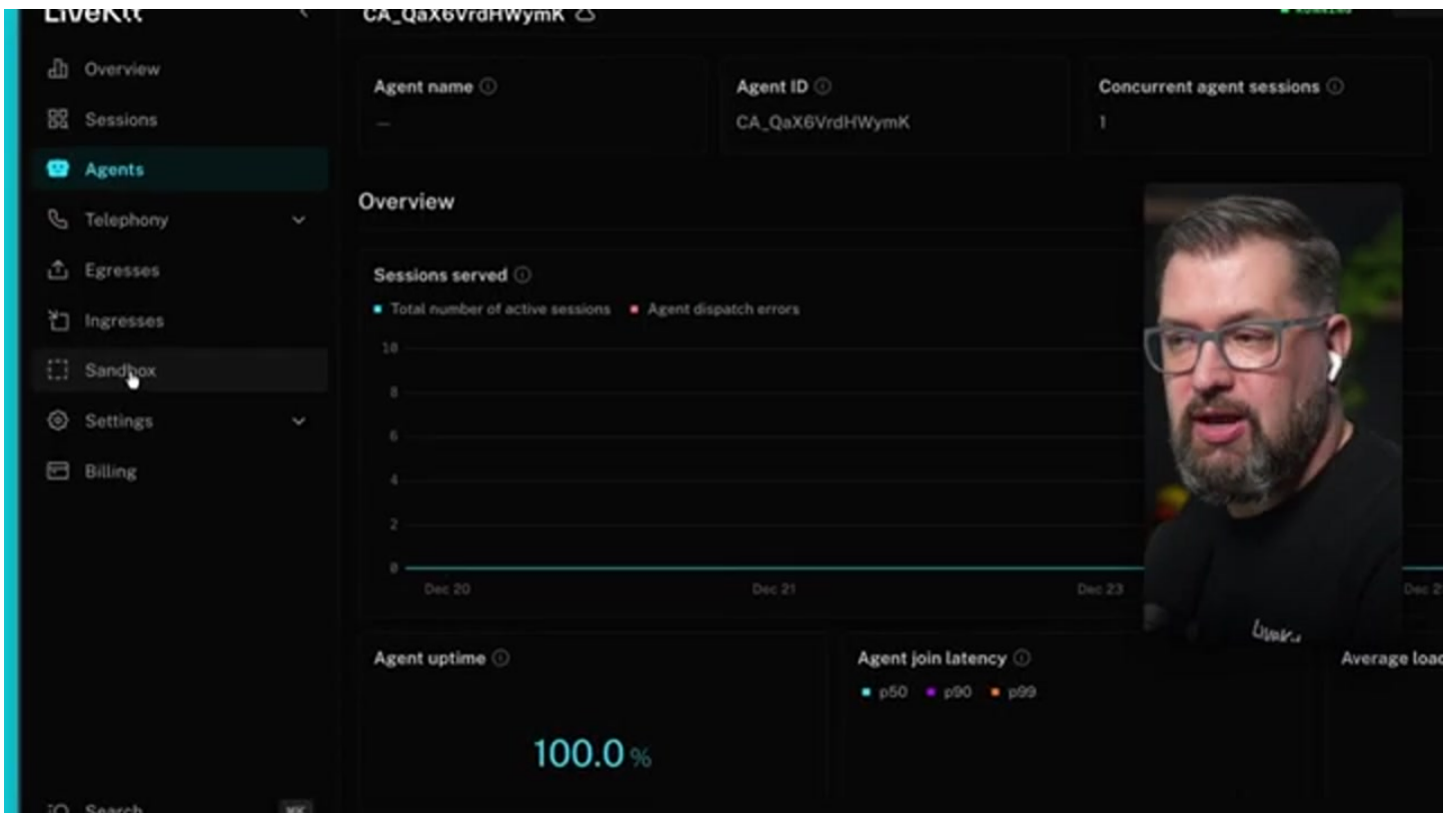
handle sessions. The agent automatically scales up and down based on demand. And all of the observability data will upload to the dashboard after each session. All right. So now it's asking me if I want to view logs. No, I don't care about logs. And we're done. So now let's test the deployed agent to generate some session data. So notice here we didn't have anything under

■ [38:00 - 38:20]



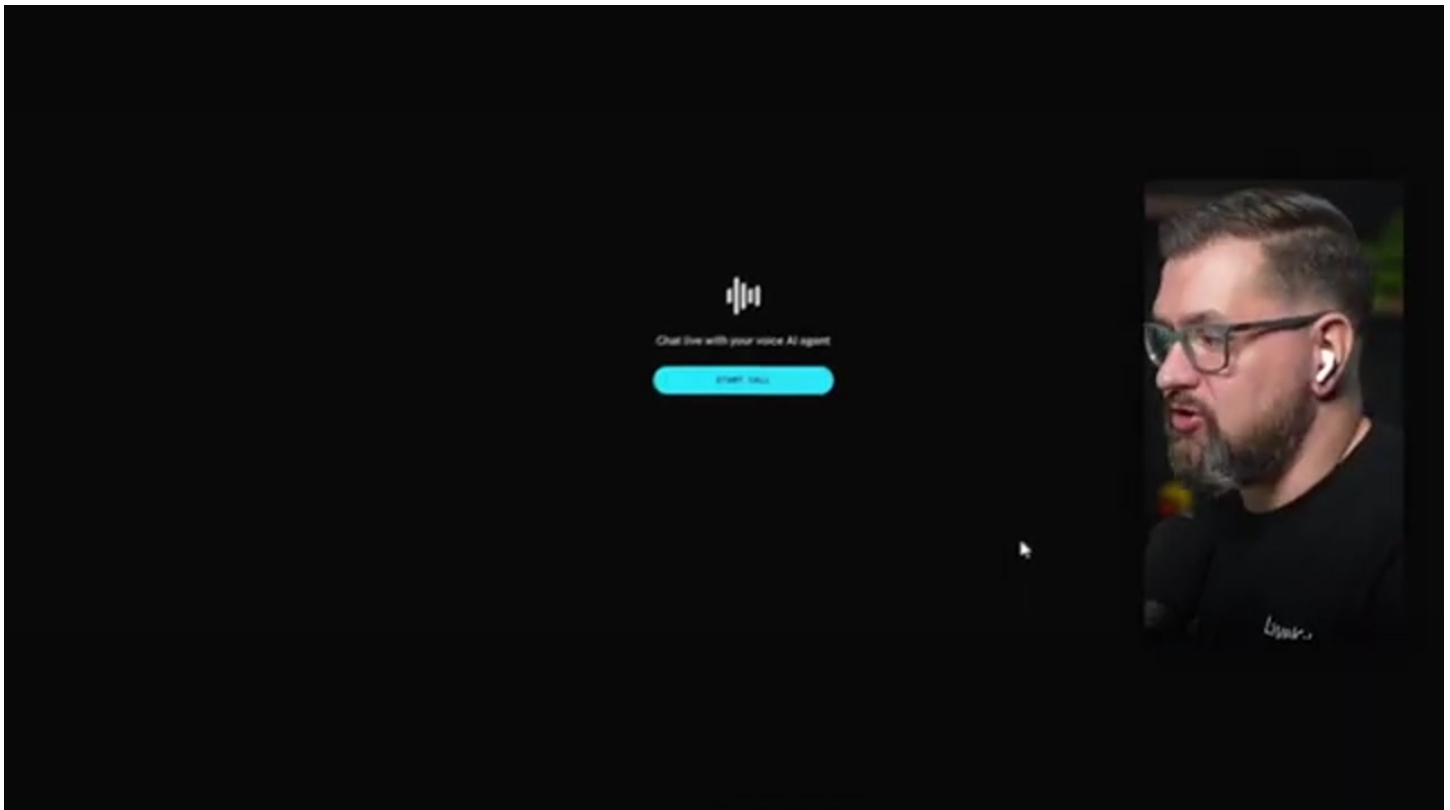
here we didn't have anything under agents. Let me refresh the page. And now we can see our new agent. It's here. So we can get an overview of our agents here. If we click on our agent, uh we can then see some more detailed information about our agent. But in order to test it, let's go over to sandbox and we're going to create a new

■ [38:20 - 38:40]



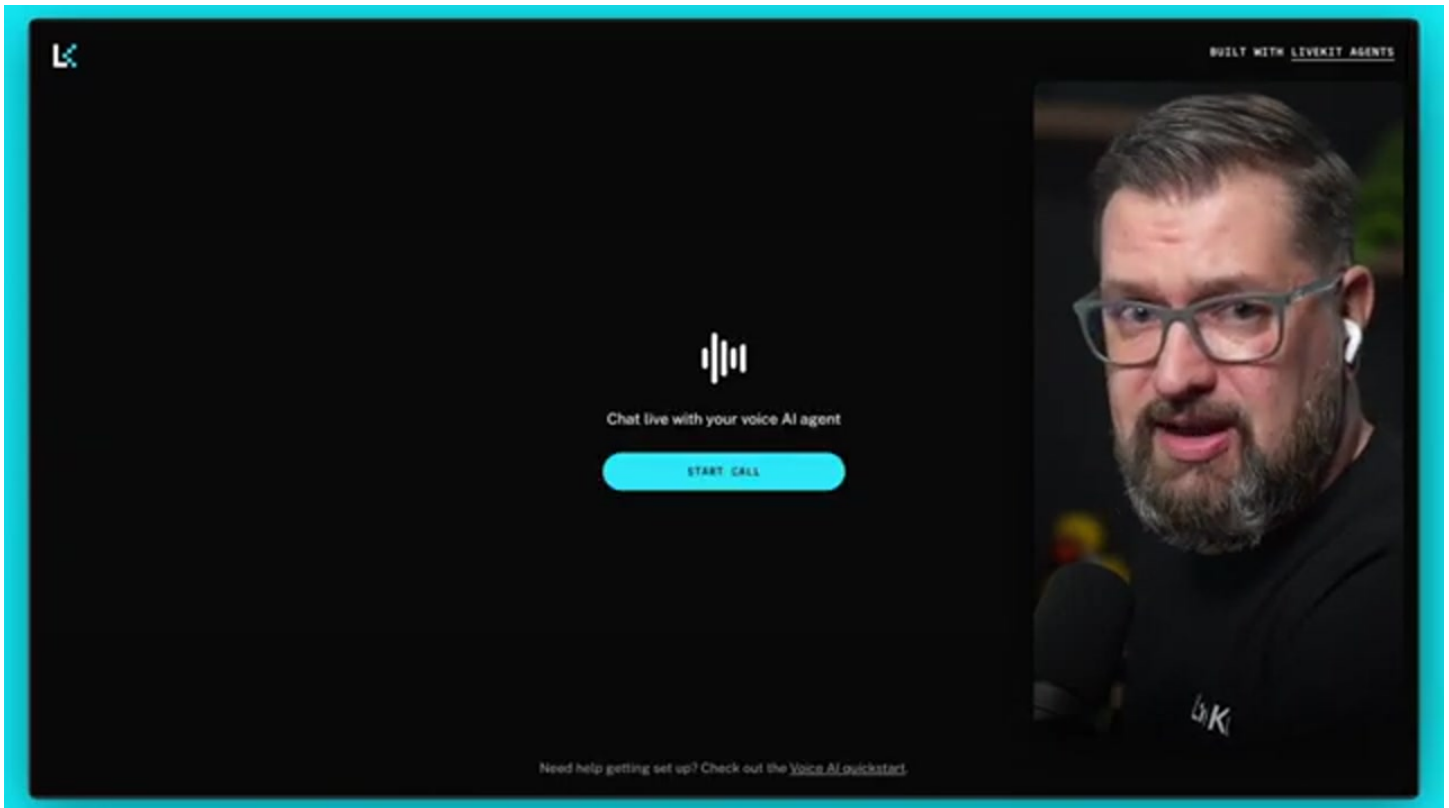
sandbox and we're going to create a new sandbox. So we'll create a new web voice agent uh sandbox. And let's go ahead and create create this uh sandbox here. You can give it a name or just let it randomly generate a name. Let's go ahead and create it. And then we don't need these instructions. Let's go ahead and launch this. It's going to use our agent. So, let's go ahead and click

■ [38:40 - 39:00]



agent. So, let's go ahead and click start call and we'll need to enable the microphone. Hello. >> Hey there. What tech trouble can I zap for you today? >> Uh, more tech trouble. Um, I'm actually not having much tech trouble. Uh, so thank you. We'll go ahead and end the call. And now we should have some observability data on that really brief

■ [39:00 - 39:20]



observability data on that really brief call there. Uh, but you can see it was still the same agent that we created. uh the same voice asking about tech trouble. Let's go ahead and go back over to our dashboard and this time let's go ahead and click on sessions and we can see our latest session here. We can click on it. We have this agent insights tab and this is where we can see our

■ [39:20 - 39:40]

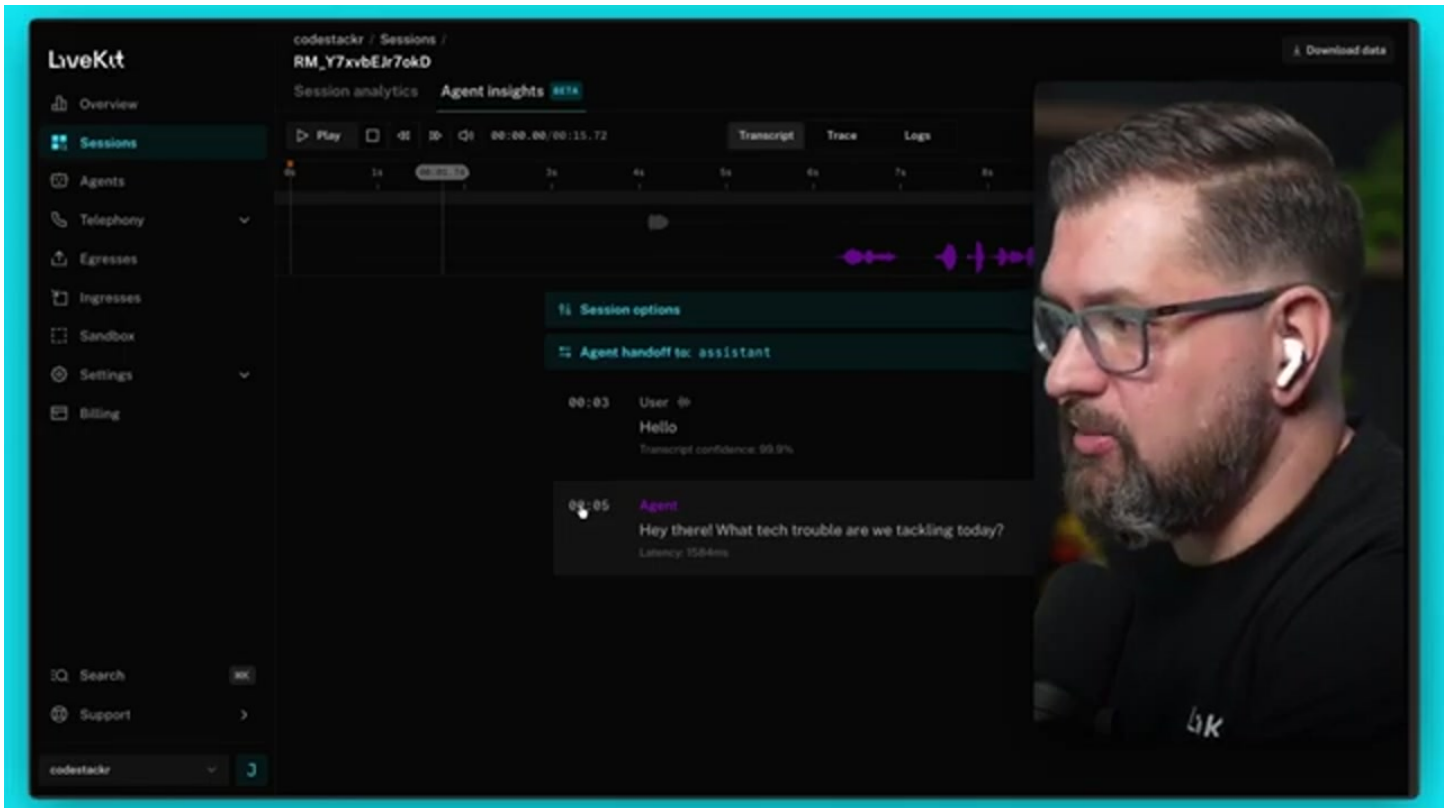
The screenshot displays the LiveKit dashboard interface. On the left is a sidebar with navigation links: Overview, Sessions (highlighted), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. The main content area is titled 'codestackr / Sessions / RM_Y7xvbEjr7okD'. It features two tabs: 'Session analytics' and 'Agent insights' (which is active and marked with a 'BETA' badge). A notification banner at the top of the main area states: 'Still processing — you're seeing partial data for now.' Below this, there is a table with session data:

Room name ⓘ	Room session ID ⓘ	Uniqu...
sbx-24hmb9-5Sx7yGpC9NpW5nQ8...	RM_Y7xvbEjr7okD	—
End time ⓘ	Duration ⓘ	Total
Dec 26, 2025, 8:53:03 PM	15 sec	—

Below the table is a section titled 'Session timeline' with a sub-section 'Participants ⓘ'. A message at the bottom of the timeline area reads: 'No data for the selected time range.' On the right side of the dashboard, there is a video feed showing a man with glasses and a beard speaking into a microphone.

tab and this is where we can see our observability data and here is that timeline. So we can go ahead and click play here. Hello. >> Hey there. What tech trouble are we tackling today? >> Looks like I ended it before my last response um actually made its way up.

■ [39:40 - 40:00]



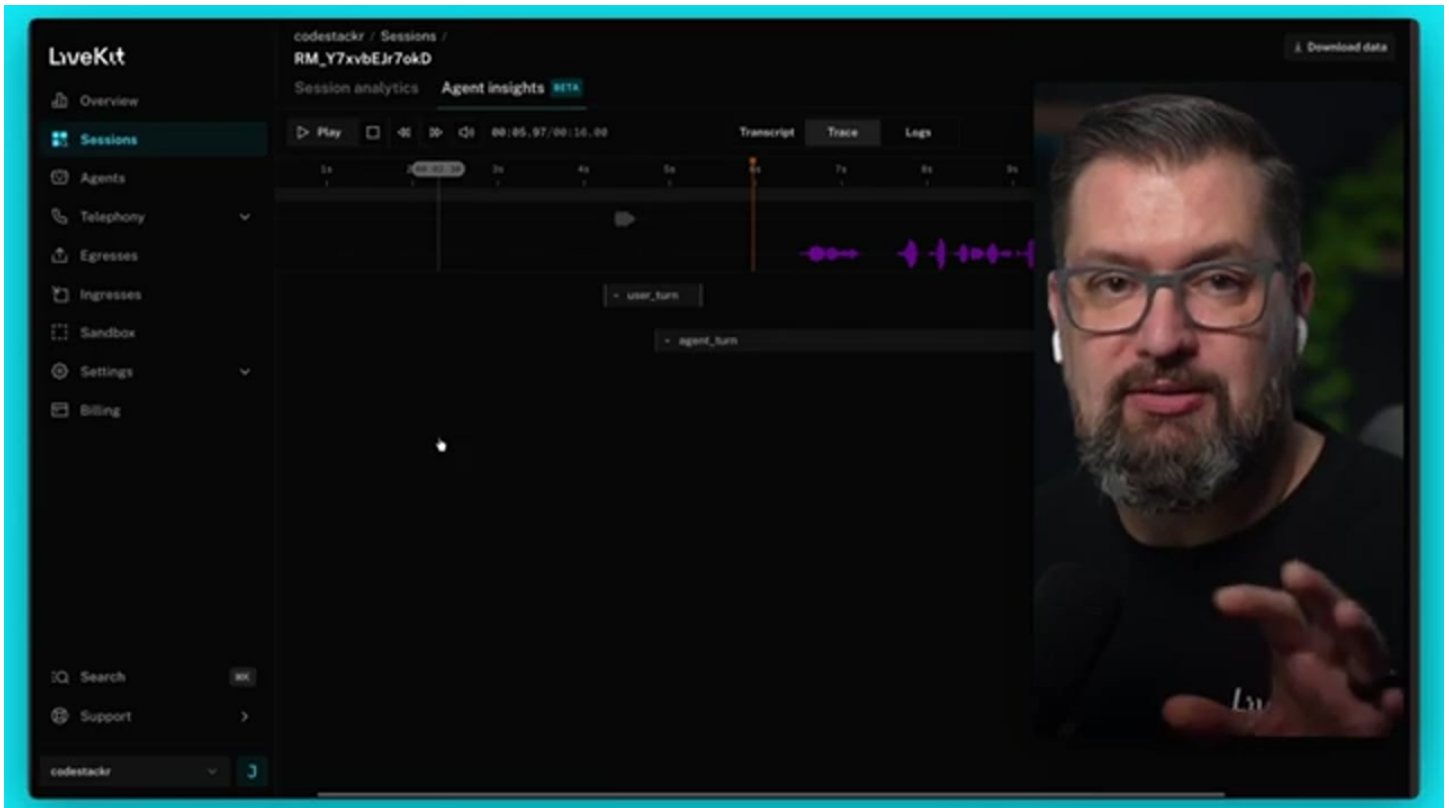
response um actually made its way up. So, um I probably should have talked a little bit longer, but you can see it recorded my audio, the agent's audio. So, the transcript view shows the conversation timeline with audio playback. You can scrub to any point and hear exactly what the user uh and agent said. In inline alerts, highlight key events like tool calls. And this view is

■ [40:00 - 40:20]

The screenshot displays the LiveKit web interface. On the left is a dark sidebar with navigation links: Overview, Sessions (active), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. Below these are Search and Support links. The main content area is titled 'codestackr / Sessions / RM_Y7xvbEjr7okD'. It features tabs for 'Session analytics' and 'Agent insights' (which is selected and marked 'BETA'). A timeline at the top shows a play button, a stop button, and a progress bar from 00:00.07 to 00:16.00. Below the timeline, there are two expandable sections: 'Session options' and 'Agent handoff to: assistant'. The 'Agent handoff to: assistant' section is expanded, showing a conversation log. The log entry for 00:04 shows a 'User' saying 'Hello' with a 'Transcript confidence: 99.9%'. The entry for 00:05 shows an 'Agent' responding 'Hey there! What tech trouble are we tackling today?' with a 'Latency: 1584ms'. On the right side of the interface, there is a video feed of a man with a beard and glasses, wearing a black shirt, who appears to be speaking into a microphone.

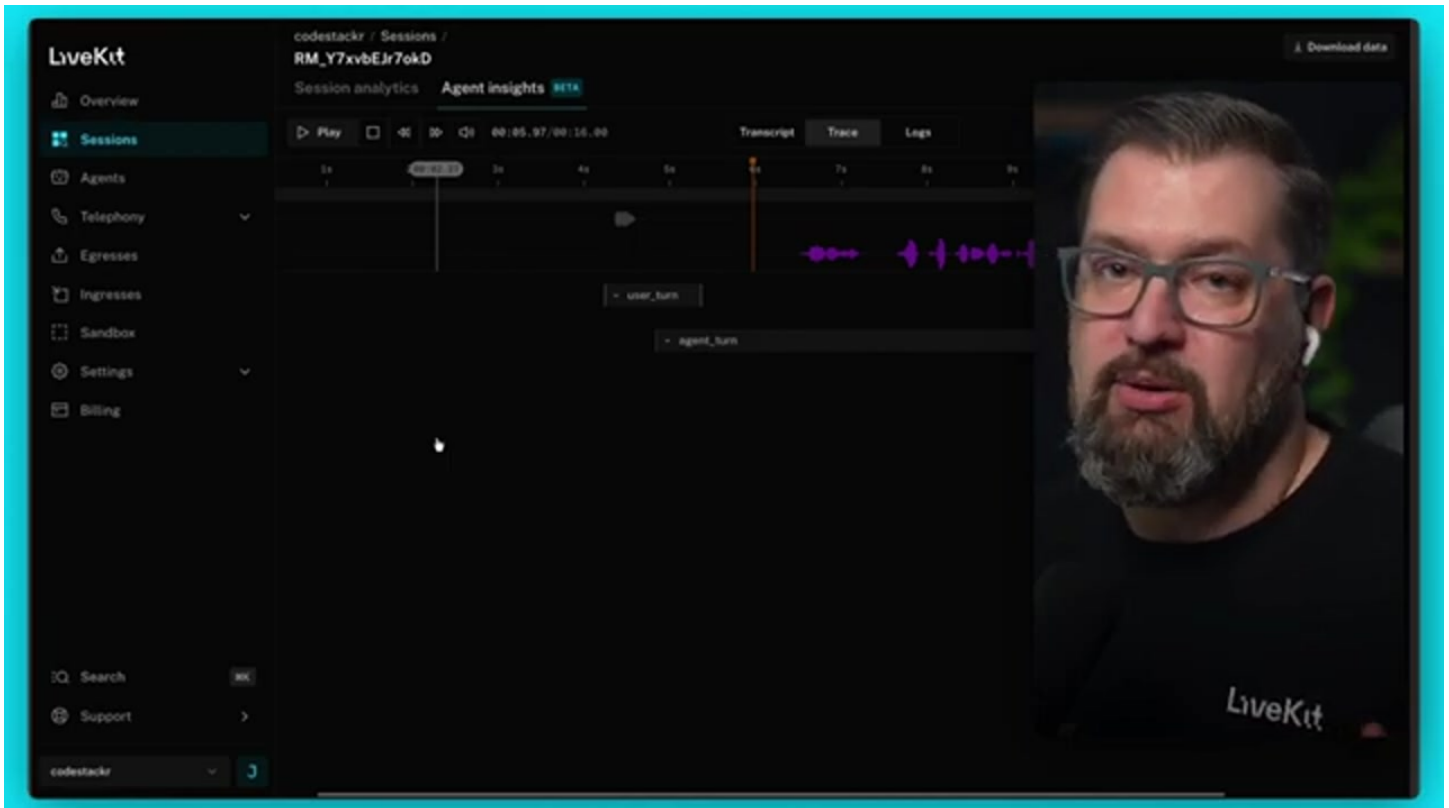
events like tool calls. And this view is great for spotting interruptions and understanding the conversation flow. So we can see here that the agent latency was 1.5 seconds. If we click on it, we can see some more stats. Now the trace view is where you'll spend most of your optimization time. Each agent response breaks down into spans for ST, LLM, and

■ [40:20 - 40:40]



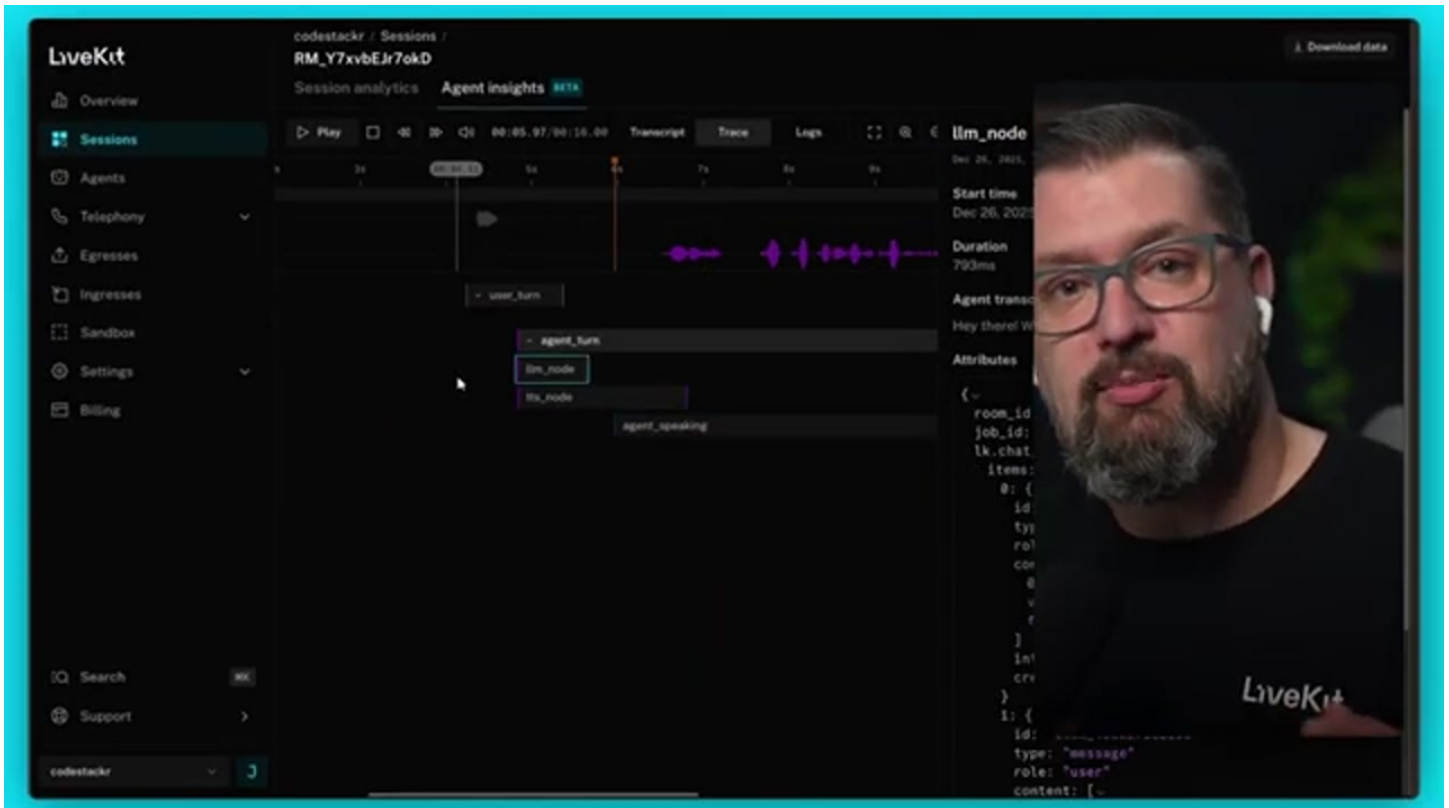
breaks down into spans for ST, LLM, and TTS. You can click on any of these and see extra details, timestamps, and durations. And this is where you can find and identify bottlenecks. If your LLM span is consistently over 500 milliseconds, you might consider a faster model or enabling preemptive

■ [40:40 - 41:00]



faster model or enabling preemptive generation if you haven't already. Uh if TTS is the bottleneck, try a different voice provider. So here again, we can click on the agent turn, we can see the LLM node and we can get all this extra information. Same with the TTS node and when the agent speaks, we get all of this uh extra details that can help us. So we can see exactly where time is

■ [41:00 - 41:20]



So we can see exactly where time is spent in each turn. So in this example, we can see the LLM node is taking 793 milliseconds. The TTS node is taking almost 2 seconds. So that's right there. That's where we can start considering optimization. So going over to the logs view, uh this is where it will show information, warnings, errors, and debug

■ [41:20 - 41:40]

The screenshot displays the LiveKit dashboard interface. On the left is a sidebar with navigation links: Overview, Sessions (selected), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. The main content area is titled 'codestackr / Sessions / RM_Y7xvbEjr7okD'. It features tabs for 'Session analytics' and 'Agent insights' (which is active). Below the tabs is a timeline with a play button and a 'Download data' link. A table of logs is visible, with columns for Timestamp, Level, Logger, and Message. The logs show various metrics and session events. On the right side of the dashboard, there is a video feed of a man with a beard and glasses, wearing a black shirt, who appears to be speaking into a microphone.

Timestamp	Level	Logger	Message
Dec 26, 00:52:52.837 PM	INFO	livekit.agents	STT metrics
Dec 26, 00:52:53.369 PM	INFO	livekit.agents	EOU metrics
Dec 26, 00:52:53.632 PM	INFO	livekit.agents	LLM metrics
Dec 26, 00:52:53.971 PM	INFO	...mp_main...	Time to first audio: 0.602s
Dec 26, 00:52:54.889 PM	INFO	livekit.agents	TTS metrics
Dec 26, 00:53:00.312 PM	INFO	livekit.agents	closing agent session due to participant disconnect id
Dec 26, 00:53:04.151 PM	INFO	...mp_main...	Usage summary: UsageSummary(tim_prompt_tokens=

information, warnings, errors, and debug messages from your agent code, the media server, and client connections. So if something fails or you see unexpected behavior in the transcript, click here for the technical details. Again, we can click on each of these to show the extra information. Now, the really cool thing about observability here is that

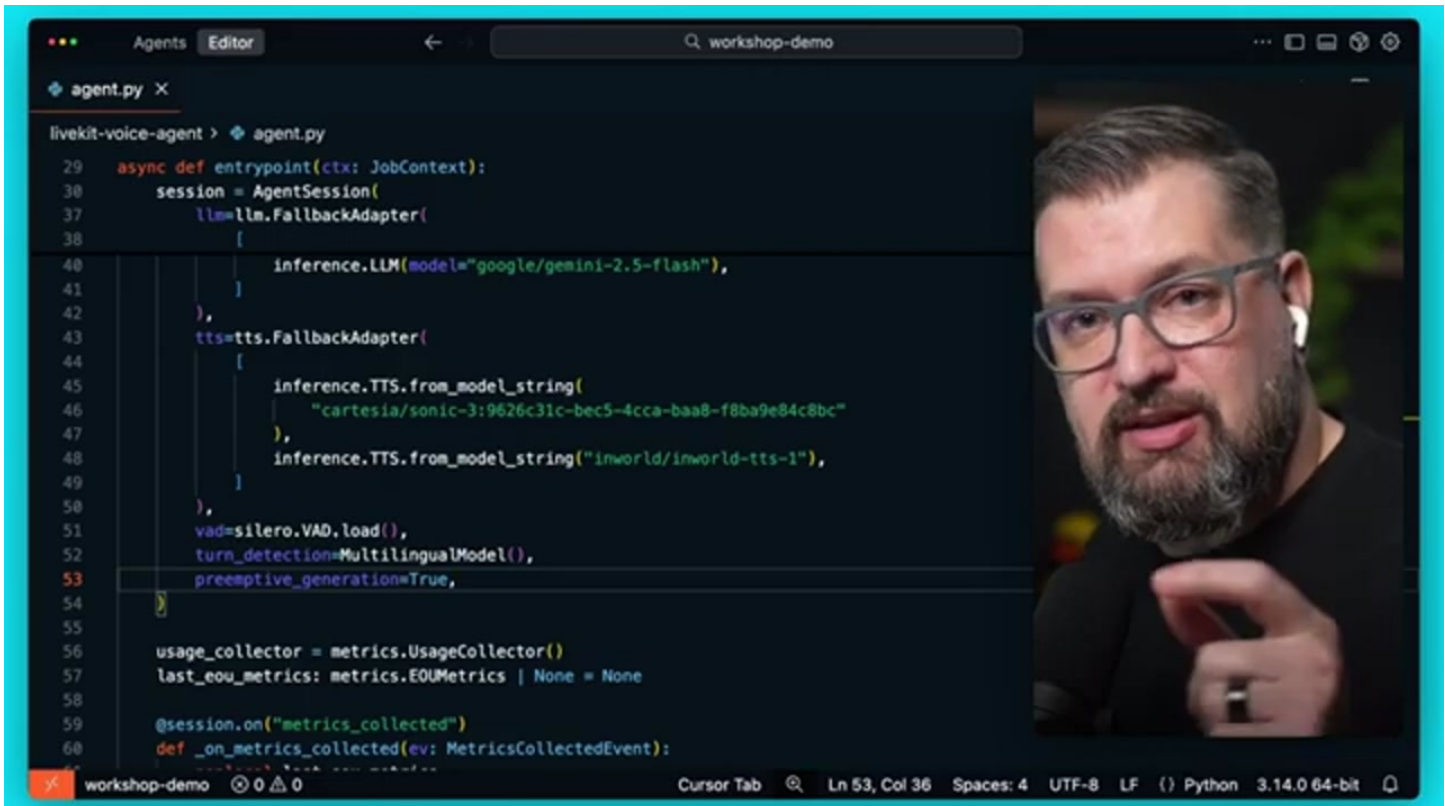
■ [41:40 - 42:00]

The screenshot displays the LiveKit web interface. On the left is a sidebar with navigation links: Overview, Sessions (active), Agents, Telephony, Egresses, Ingresses, Sandbox, Settings, and Billing. The main content area is titled 'codestackr / Sessions / RM_Y7xvEjr7okD'. It features a 'Session analytics' section with a timeline and a table of logs. The 'Agent insights' tab is selected, showing a table with columns for Timestamp, Level, Logger, and Message. The logs table contains several entries, including STT metrics, EOU metrics, LLM metrics, and TTS metrics. On the right, there is a video feed of a man with glasses and a beard, wearing a black t-shirt with the LiveKit logo. Below the video feed, there is a section for 'Inspect log' showing the details of a selected log entry, including its severity, message, and attributes.

Timestamp	Level	Logger	Message
Dec 26, 00:52:52.837 PM	INFO	livekit.agents	STT metrics
Dec 26, 00:52:53.369 PM	INFO	livekit.agents	EOU metrics
Dec 26, 00:52:53.632 PM	INFO	livekit.agents	LLM metrics
Dec 26, 00:52:53.971 PM	INFO	...mp_main...	Time to first audio: 0.602s
Dec 26, 00:52:54.809 PM	INFO	livekit.agents	TTS metrics
Dec 26, 00:53:00.312 PM	INFO	livekit.agents	closing agent session due to participant disconnect L...
Dec 26, 00:53:04.151 PM	INFO	...mp_main...	Usage summary: UsageSummary{on_prompt_tokens...

about observability here is that everything is all aligned to the same audio track. And so as we're scrubbing through the audio, we are in the same timeline looking at the same things in all of these uh all three of these different views. So they're all in sync. So if you have observability enabled at the project level, but you want to disable it for a specific session or for

■ [42:00 - 42:20]

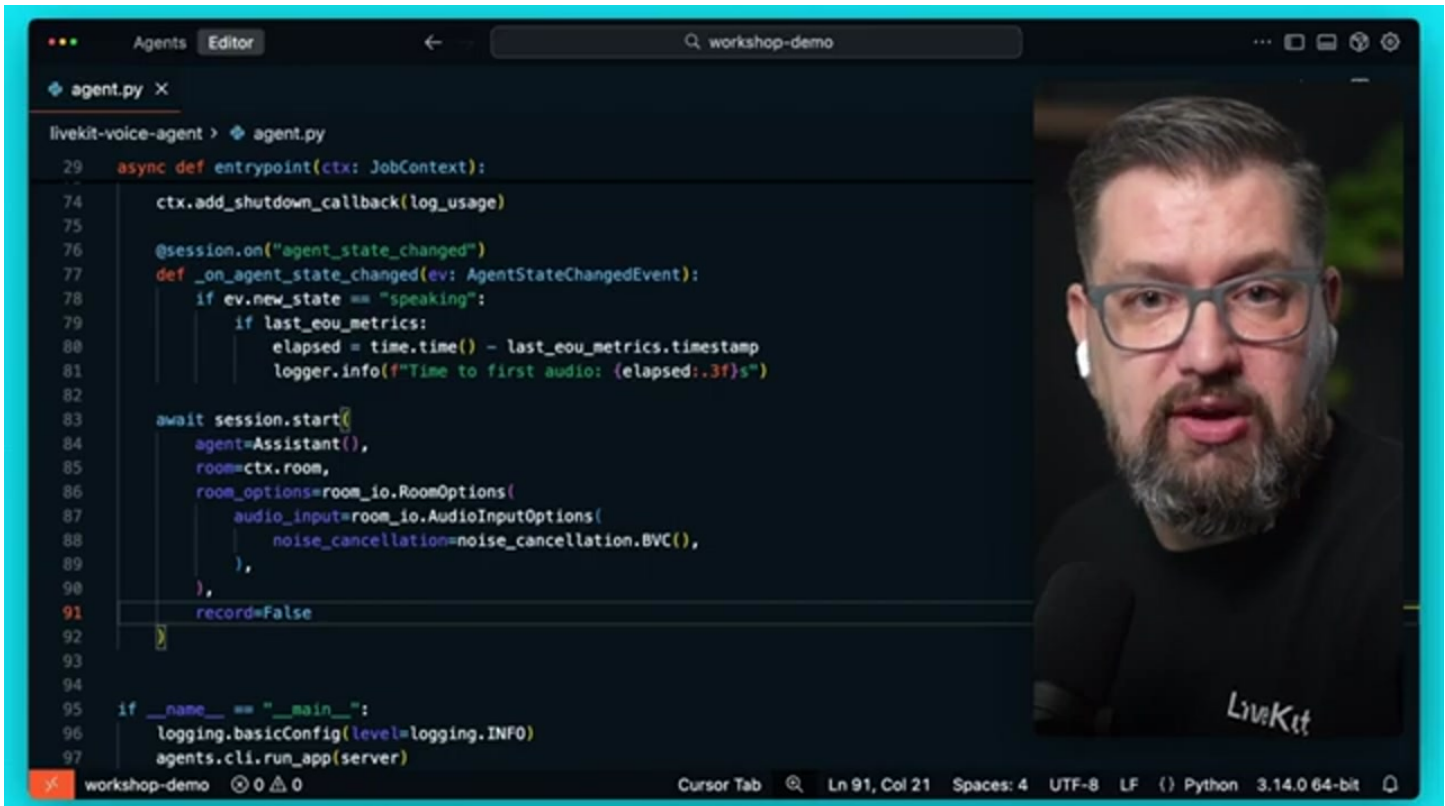


```
agent.py x
livekit-voice-agent > agent.py
29 async def entrypoint(ctx: JobContext):
30     session = AgentSession(
37         llm=llm.FallbackAdapter(
38             [
40                 inference.LLM(model="google/gemini-2.5-flash"),
41             ],
42         ),
43         tts=tts.FallbackAdapter(
44             [
45                 inference.TTS.from_model_string(
46                     "cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc"
47                 ),
48                 inference.TTS.from_model_string("inworld/inworld-tts-1"),
49             ],
50         ),
51         vad=silero.VAD.load(),
52         turn_detection=MultilingualModel(),
53         preemptive_generation=True,
54     )
55
56     usage_collector = metrics.UsageCollector()
57     last_eou_metrics: metrics.EOUMetrics | None = None
58
59     @session.on("metrics_collected")
60     def _on_metrics_collected(ev: MetricsCollectedEvent):
```

workshop-demo 0 0 0 Cursor Tab Ln 53, Col 36 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

disable it for a specific session or for a specific agent, you can do that. So we'll just go down here to session start and then we can add an option here. We can add `record_eou=False`. Pretty simple. And now this agent will no longer collect observability data. So this disables the upload of audio transcripts, traces and logs for the

■ [42:20 - 42:40]

A screenshot of a video call interface. On the right side, there is a video feed of a man with a beard and glasses, wearing a black shirt with the 'LiveKit' logo. He is speaking into a microphone. On the left side, there is a code editor window titled 'agent.py'. The code is written in Python and shows the configuration of a LiveKit agent. The code includes imports, class definitions, and a main function. The status bar at the bottom of the code editor shows 'workshop-demo', 'Cursor Tab', 'Ln 91, Col 21', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.14.0 64-bit'.

```
29 async def entrypoint(ctx: JobContext):
74     ctx.add_shutdown_callback(log_usage)
75
76     @session.on("agent_state_changed")
77     def _on_agent_state_changed(ev: AgentStateChangedEvent):
78         if ev.new_state == "speaking":
79             if last_eou_metrics:
80                 elapsed = time.time() - last_eou_metrics.timestamp
81                 logger.info(f"Time to first audio: {elapsed:.3f}s")
82
83     await session.start(
84         agent=Assistant(),
85         room=ctx.room,
86         room_options=room_io.RoomOptions(
87             audio_input=room_io.AudioInputOptions(
88                 noise_cancellation=noise_cancellation.BVC(),
89             ),
90         ),
91         record=False
92     )
93
94
95 if __name__ == "__main__":
96     logging.basicConfig(level=logging.INFO)
97     agents.cli.run_app(server)
```

transcripts, traces and logs for the entire session even if the project level setting is enabled. So use this when you need to handle sensitive data or you want to reduce storage costs for specific sessions. Now all observability data is stored in the US and is retained for 30 days and data older than 30 days is automatically deleted. Now you can

Data retention

- 30-day retention window
- Automatic deletion after 30 days
- Data stored in the US



LIVEKIT.IO

is automatically deleted. Now you can download the audio transcripts and logs directly from the session if you need a copy. Now for projects on the free build plan, some uh anonymized session data may be retained longer for model improvement purposes. Uh paid plans including ship scale and enterprise have full data deletion after that 30-day window. A low latency voice requires

Latency optimization



LIVEKIT.IO

window. A low latency voice requires parallel work across the stack. So here's some best practices. First, stream everything. ST LLM TTS should operate incrementally, not in big batches. Second, parallelize. Start uh TTS as soon as the LLM emits the first words. Don't wait for the full sentence.

Latency optimization

- Stream everything
- Parallelize STT → LLM → TTS



LIVEKIT.IO

words. Don't wait for the full sentence. Third, avoid blocking IO. Keep tool calls and storage async. Bound your timeouts and retries. And fourth, keep prompts tight. Fewer tokens means faster first audio and lower cost. Now, the metrics you collect at the application level only tell you part of the story. Network conditions also affect perceived

Latency optimization

- Stream everything
- Parallelize STT → LLM → TTS
- Avoid blocking I/O
- Keep prompts tight



LIVEKIT.IO

Network conditions also affect perceived latency. WebRTC handles this with Opus audio compression, jitter buffering, and adaptive bit rate. If you see unexpectedly high time to first audio that doesn't correlate with your pipeline metrics, network conditions may be the culprit. Use agent observability to validate and and fine-tune agent

Latency optimization

- Stream everything
- Parallelize STT → LLM → TTS
- Avoid blocking I/O
- Keep prompts tight



LIVEKIT.IO

to validate and and fine-tune agent performance. The trace view shows exactly where time is spent in each turn. So, this makes it easier to identify bottlenecks. So to recap, these are the metrics worth tracking as you continue building. You've seen them in the console output during development and in the trace view in production. And

Key metrics

- TTFT (Time to First Token)
- TTFA (Time to First Audio)
- Interruption rate
- Tool latency
- Fallback activation
- Target: <1000ms



LIVEKIT.IO

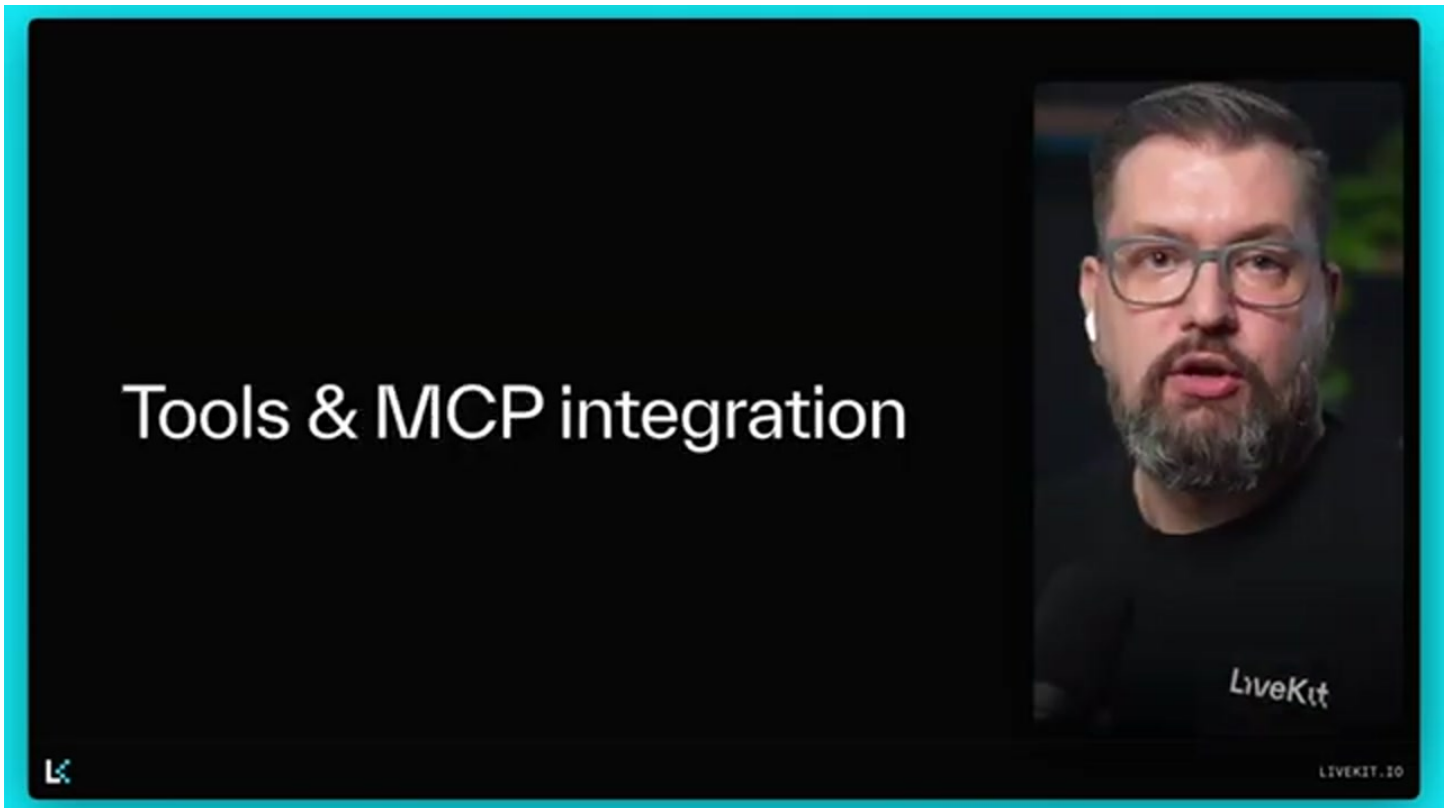
and in the trace view in production. And so aim for a time to first audio under 1 second in most cases. And use the trace breakdown to identify which component to optimize when you're over budget. So you've learned which metrics matter for voice agents and how to collect them locally. You've improved latency with preemptive generation. You've deployed

- ✓ Key metrics
- ✓ Usage collection
- ✓ Preemptive generation
- ✓ LiveKit Cloud deployment



LIVEKIT.IO

preemptive generation. You've deployed your agent to LifeKit Cloud, and you've seen how agent observability uh gives you transcripts, traces, and log views to debug and optimize each session. With metrics and observability in place, you can make datadriven decisions as you add tools and workflows in the next lesson.



tools and workflows in the next lesson. So, your agent can speak naturally, and you can measure its performance. Now, it's time to give it capabilities beyond conversation tools. Let your agent call external APIs, look up data, and take actions on behalf of users. And without tools, your agent can only respond based on what's in its training data and

Why tools matter



LIVEKIT.IO

on what's in its training data and conversation context, and that's pretty limiting. Tools let you fetch real-time information like weather, stock prices, or order status. They let you take actions in external systems like creating tickets, sending emails, or updating records. You can access your private data from customer databases or

Why tools matter

- Fetch real-time info
- Take actions in external systems
- Access private data



LIVEKIT.IO

private data from customer databases or internal docs. And you can trigger UI updates on the front end via RPC tools. Turn your agent from a chatbot into something that can actually get things done. So Lifegate uses the function tool decorator to expose Python functions to the LLM. The LLM decides when to call

@function_tool decorator



LIVEKIT.IO

the LLM. The LLM decides when to call them based on the conversation context. So let's add a weather lookup tool. This example uses the open Matteo API which is free and doesn't require an API key. Okay, first thing at the top, let's import uh HTTP ex so that we can make HTTP requests

■ [46:20 - 46:40]

```
agent.py x
livekit-voice-agent > agent.py
1  import logging
2
3  from dotenv import load_dotenv
4  from livekit import agents
5  from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6  from livekit.plugins import noise_cancellation, silero
7  from livekit.plugins.turn_detector.multilingual import MultilingualModel
8  from livekit.agents import llm, stt, tts, inference
9  from livekit.agents import AgentStateChangedEvent, MetricsCollectedEvent, metric
10 import time
11 import httpx
12
13 logger = logging.getLogger(__name__)
14
15 load_dotenv()
16
17
18 class Assistant(Agent):
19     def __init__(self) -> None:
20         super().__init__(
21             instructions="You are an upbeat, slightly sarcastic voice AI for tech support. "
22             "Help the caller fix issues without rambling, and keep replies under 3 sentences.",
23         )
```



ex so that we can make HTTP requests and let's not forget to install that package as well. So, u add httpex. Then we also need to import some more things from lifekit agents. We need the function tool, the run context and the tool error. Let's go down to our instructions and let's update these as well. So, you are an upbeat, slightly

■ [46:40 - 47:00]

```
from agents import function_tool, RunContext, ToolError
...
logging.getLogger(__name__)
...

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions=[
                "You are an upbeat, slightly sarcastic voice AI for tech support. "
                "Help the caller fix issues without rambling, and keep replies under 3 sentences. "
                "You can also look up the weather if asked."
            ],
        )

def agent_server():
    ...

def main_session():
    ...
```



well. So, you are an upbeat, slightly sarcastic tech support, all that same stuff. Help the caller fix issues without rambling. Uh, but then we've added you can also look up the weather if asked. So, we're going to add this uh this function right here. And so, let's go ahead and walk through this. So, the the function tool decorator registers this method as a callable tool for the LLM.

■ [47:00 - 47:20]

```
agent.py x
livekit-voice-agent > agent.py

19 class Assistant(Agent):
20     def __init__(self) -> None:
21         super().__init__(
22             instructions=(
23
24                 "You can also look up the weather if asked."
25             ),
26         )
27
28
29 @function_tool()
30 async def lookup_weather(
31     self,
32     context: RunContext,
33     location: str,
34 ) -> dict:
35     """Look up current weather for a location.
36
37     Args:
38         location: City name or location to get weather for.
39     """
40     async with httpx.AsyncClient() as client:
41         # First, geocode the location to get coordinates
42         geo_response = await client.get(
43             f"https://geocode-api.open-meteo.com/v1/search?"
```



method as a callable tool for the LLM. Then we have the context parameter which is a run context and this provides access to the current session, the speech handle for the interruption detection and any custom user data that you've stored. We've got our location. Now the dock string is critical here.

■ [47:20 - 47:40]

```
agent.py x
livekit-voice-agent > agent.py
19 class Assistant(Agent):
30     async def lookup_weather(
32         context: RunContext,
33         location: str,
34     ) -> dict:
35         """Look up current weather for a location.
36
37         Args:
38             location: City name or location to get weather for.
39         """
40         async with httpx.AsyncClient() as client:
41             # First, geocode the location to get coordinates
42             geo_response = await client.get(
43                 "https://geocoding-api.open-meteo.com/v1/search",
44                 params={"name": location, "count": 1},
45             )
46             geo_data = geo_response.json()
47
48             if not geo_data.get("results"):
49                 raise ToolError(f"Could not find location: {location}")
50
51             lat = geo_data["results"][0]["latitude"]
52             lon = geo_data["results"][0]["longitude"]
```



Now the dock string is critical here. The LLM reads the text to understand what the tool does and when it should be called. When a user says, "What is the weather in Paris?" The LLM scans through all of the available tools, reads each dock string, and decides which one matches the user's intent. A vague dock

■ [47:40 - 48:00]

```
agent.py x
livekit-voice-agent > agent.py
19 class Assistant(Agent):
30     async def lookup_weather(
32         context: RunContext,
33         location: str,
34     ) -> dict:
35         """Look up current weather for a location"""
36
37         Args:
38             location: City name or location to get weather for.
39         """
40         async with httpx.AsyncClient() as client:
41             # First, geocode the location to get coordinates
42             geo_response = await client.get(
43                 "https://geocoding-api.open-meteo.com/v1/search",
44                 params={"name": location, "count": 1},
45             )
46             geo_data = geo_response.json()
47
48             if not geo_data.get("results"):
49                 raise ToolError(f"Could not find location: {location}")
50
51             lat = geo_data["results"][0]["latitude"]
52             lon = geo_data["results"][0]["longitude"]
```



matches the user's intent. A vague dock string means that the LLM might not call your tool when it should, or it might call it incorrectly. And the args section in the dock string is just as important. It tells the LLM what each parameter means and what values are appropriate. For location, here we are specify city name or location to get weather for. So the LLM knows to extract

■ [48:00 - 48:20]

```
agent.py x
livekit-voice-agent > agent.py

19 class Assistant(Agent):
30     async def lookup_weather(
32         context: RunContext,
33         location: str,
34     ) -> dict:
35         """Look up current weather for a location.
36
37         Args:
38             location: City name or location to get weather for.
39         """
40         async with httpx.AsyncClient() as client:
41             # First, geocode the location to get coordinates
42             geo_response = await client.get(
43                 "https://geocoding-api.open-meteo.com/v1/search",
44                 params={"name": location, "count": 1},
45             )
46             geo_data = geo_response.json()
47
48             if not geo_data.get("results"):
49                 raise ToolError(f"Could not find location: {location}")
50
51             lat = geo_data["results"][0]["latitude"]
52             lon = geo_data["results"][0]["longitude"]
```



weather for. So the LLM knows to extract the city from the user's request. Without this description, the LLM might pass the wrong data or misunderstand what the parameter expects. So then here is our HTTP request where we are calling this uh geocod geocoding API from open mateo

■ [48:20 - 48:40]

```
exit voice-agent > agent.py
19 class Assistant(Agent):
30     async def lookup_weather(
34         ) -> dict:
35         """Look up current weather for a location.
36
37         Args:
38             location: City name or location to get weather for.
39         """
40         async with httpx.AsyncClient() as client:
41             # First, geocode the location to get coordinates
42             geo_response = await client.get(
43                 "https://geocoding-api.open-meteo.com/v1/search",
44                 params={"name": location, "count": 1},
45             )
46             geo_data = geo_response.json()
47
48             if not geo_data.get("results"):
49                 raise ToolError(f"Could not find location: {location}")
50
51             lat = geo_data["results"][0]["latitude"]
52             lon = geo_data["results"][0]["longitude"]
53             place_name = geo_data["results"][0]["name"]
54
55             # Get current weather for those coordinates
56             weather_response = await client.get(
```



mateo and we're going to pass in that location there and then we will basically just return those results. So if we go back down to the bottom here, we're returning the location, the temperature, and the condition. So then the LLM will have that information to then answer our question. Now notice we do have uh the tool error here. So if no location is

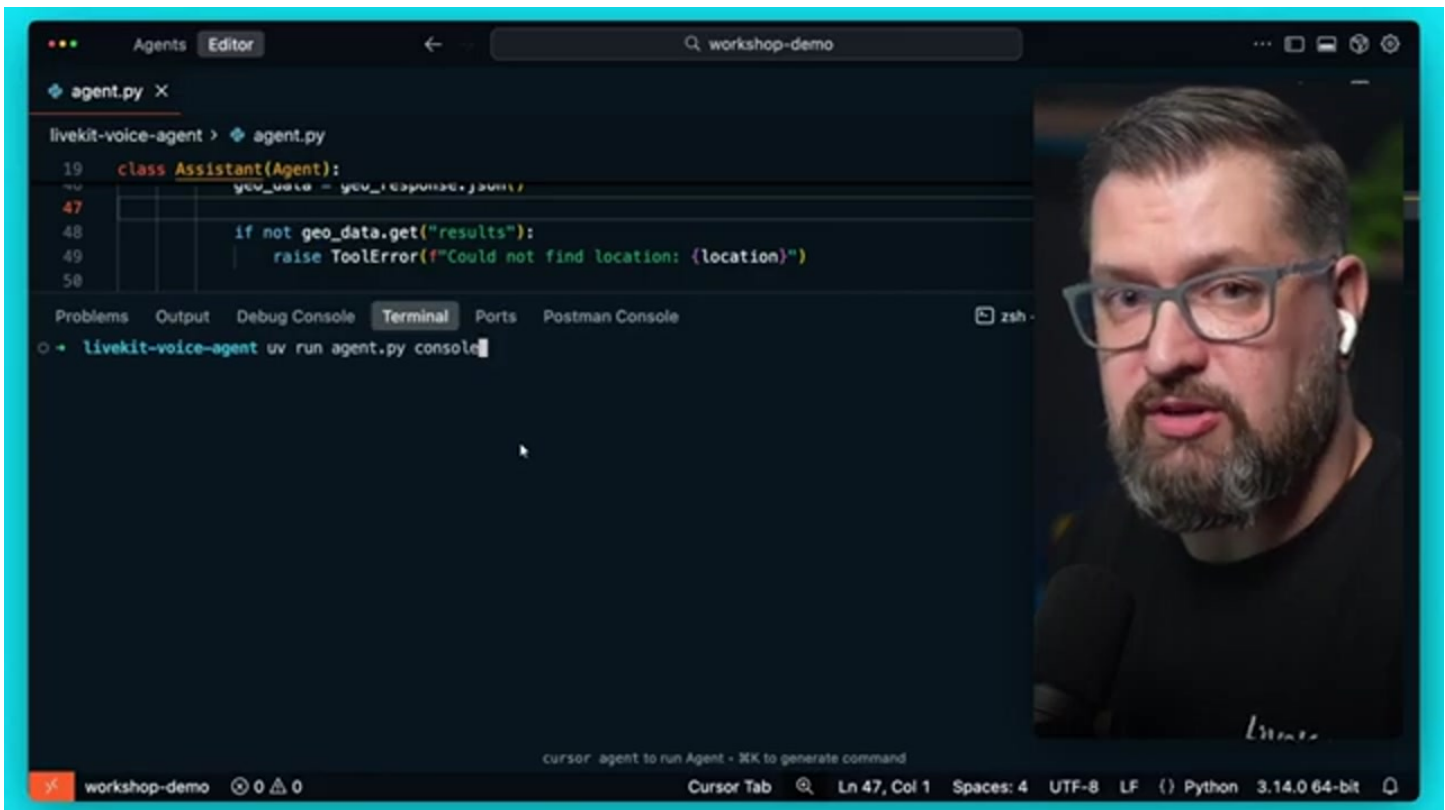
■ [48:40 - 49:00]

```
agent.py x
vekit-voice-agent > agent.py
19 class Assistant(Agent):
34     ) -> dict:
40     async with httpx.AsyncClient() as client:
42         geo_response = await client.get(
44             params={"name": location, "count": 1},
45         )
46         geo_data = geo_response.json()
47
48         if not geo_data.get("results"):
49             raise ToolError(f"Could not find location: {location}")
50
51         lat = geo_data["results"][0]["latitude"]
52         lon = geo_data["results"][0]["longitude"]
53         place_name = geo_data["results"][0]["name"]
54
55         # Get current weather for those coordinates
56         weather_response = await client.get(
57             "https://api.open-meteo.com/v1/forecast",
58             params={
59                 "latitude": lat,
60                 "longitude": lon,
61                 "current": "temperature_2m,weather_code",
62                 "temperature_unit": "fahrenheit",
```



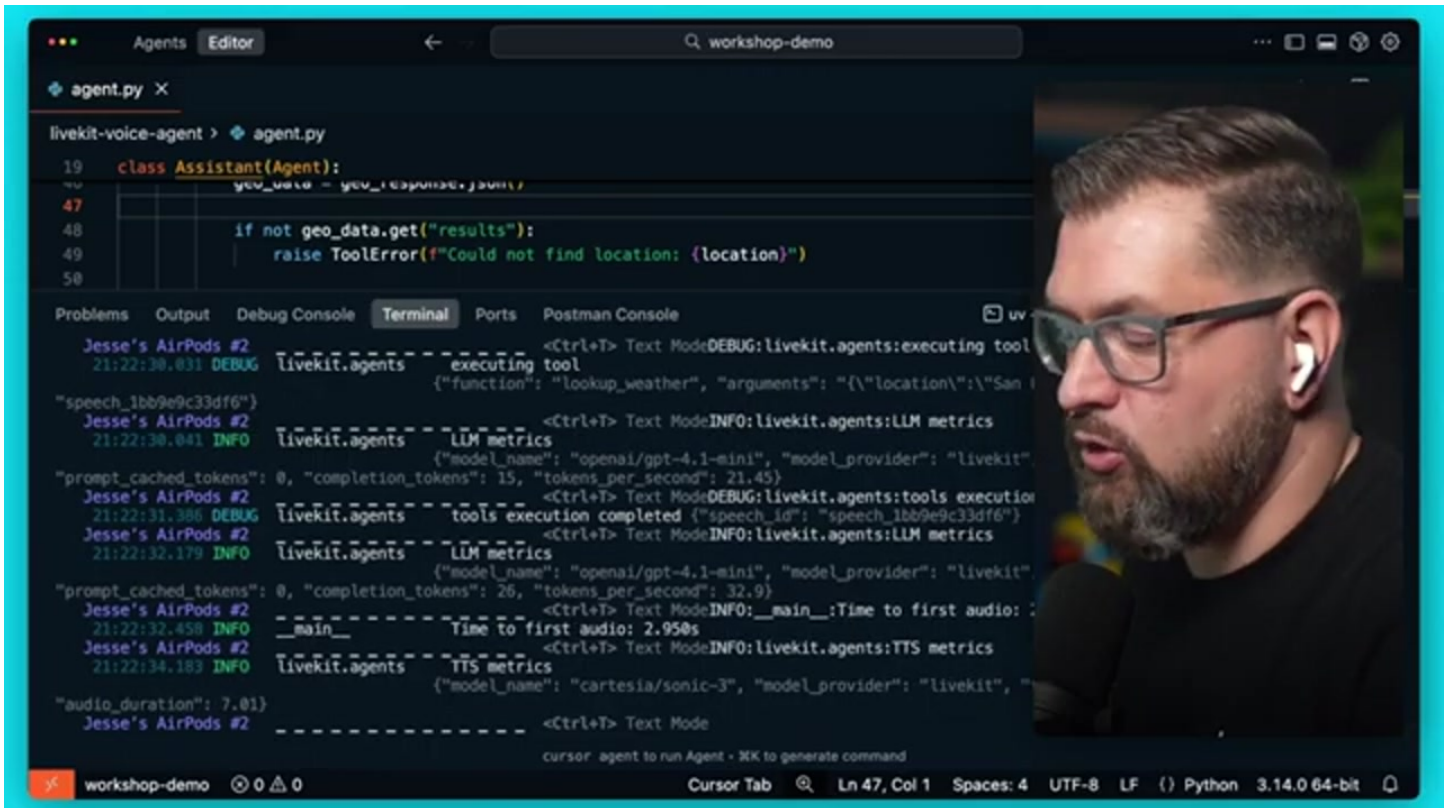
tool error here. So if no location is found, it will return that. And so let's go ahead and test this. Let's see if it works. So make sure that you save this. Let's open up our terminal. And we'll do UV run agent. And then in the console. Now we could uh sync this with our our agent in the cloud which we deployed in

■ [49:00 - 49:20]



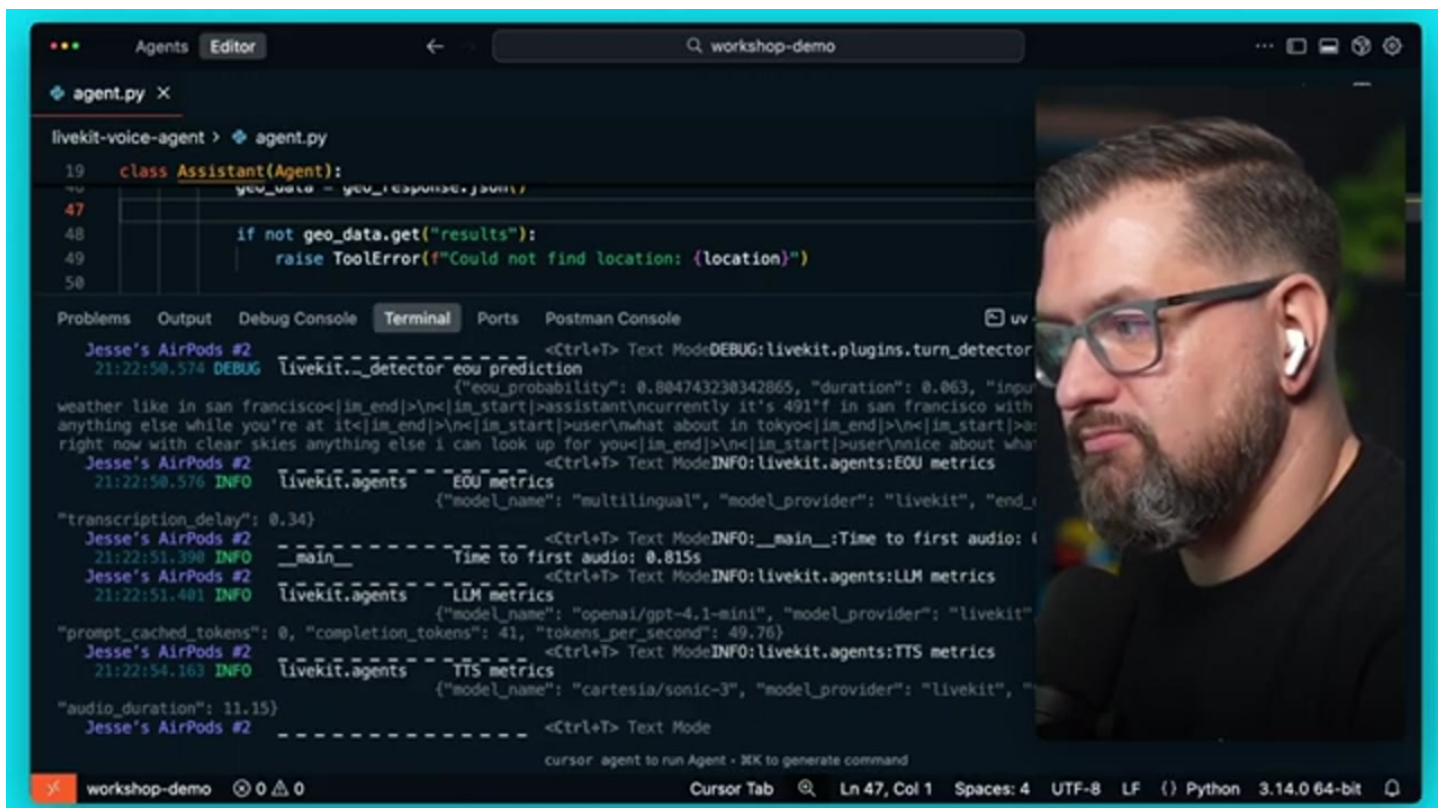
agent in the cloud which we deployed in the last lesson. Uh but for simple testing and development we can stay here in the console. Hey uh so what's what's the weather like in San Francisco? >> Currently it's 49.1 degrees Fahrenheit in San Francisco with clear conditions. >> Cool. What about in Tokyo?

■ [49:20 - 49:40]



>> Cool. What about in Tokyo? Tokyo is chilling at 42.6 degrees Fahrenheit right now with clear skies. >> Nice. >> Anything else I can about um What's the weather like on Mars? >> Mars? I wish I had that weather report for Earthly. I'm your go-to AI, but Martian weather

■ [49:40 - 50:00]



```
agent.py x
livekit-voice-agent > agent.py
19 class Assistant(Agent):
20     you_said = you_response.janus()
47
48     if not geo_data.get("results"):
49         raise ToolError(f"Could not find location: {location}")
50

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 21:22:50.574 DEBUG livekit._detector eou prediction
{"eou_probability": 0.804743230342865, "duration": 0.063, "input": "weather like in san francisco<|im_end>>\n<|im_start>assistant\ncurrently it's 491'f in san francisco with anything else while you're at it<|im_end>>\n<|im_start>user\nwhat about in tokyo<|im_end>>\n<|im_start>assistant\nright now with clear skies anything else i can look up for you<|im_end>>\n<|im_start>user\nnice about what"}
Jesse's AirPods #2 21:22:50.576 INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_token": 41, "tokens_per_second": 49.76, "transcription_delay": 0.34}
Jesse's AirPods #2 21:22:51.390 INFO __main__ Time to first audio: 0.815s
Jesse's AirPods #2 21:22:51.401 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit", "prompt_cached_tokens": 0, "completion_tokens": 41, "tokens_per_second": 49.76}
Jesse's AirPods #2 21:22:54.163 INFO livekit.agents TTS metrics
{"model_name": "cartesia/sonic-3", "model_provider": "livekit", "audio_duration": 11.15}
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode
```

workshop-demo 0 0 0 Cursor Tab Ln 47, Col 1 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

I'm your go-to AI, but Martian weather might be. >> All right, it looks like it works. So, if we go down here, we can see that the uh the tool uh did get executed. So, look up weather. it passed the the location there with San Francisco and then it provided us the the the response there. Another really cool tool that we

Model Context Protocol (MCP)



LiveKit

LIVEKIT.IO

there. Another really cool tool that we can use is MCP or model context protocol. This lets you connect to external tool servers. Now this is useful when you want to share tools across multiple agents. Tools are managed maybe they're managed by separate teams or services or maybe you want to integrate with thirdparty uh MCP

Model Context Protocol (MCP)



LIVEKIT.IO

want to integrate with thirdparty uh MCP compatible systems. So let's connect to the LiveKit documentation MCP server. And so this turns our agent into a voice interface for searching LiveKit docs. So in our terminal, we'll uh UV add LiveKit agents MCP. And let's add another import here from

■ [50:40 - 51:00]

```
voice-agent > agent.py

import logging

from dotenv import load_dotenv
from livekit import agents
from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
from livekit.plugins import noise_cancellation, silero
from livekit.plugins.turn_detector.multilingual import MultilingualModel
from livekit.agents import llm, stt, tts, inference
from livekit.agents import AgentStateChangedEvent, MetricsCollectedEvent, metrics
from livekit.agents import function_tool, RunContext, ToolError
from livekit.agents import mcp

import time
import httpx

logger = logging.getLogger(__name__)

load_dotenv()

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions=(
                "You are an upbeat, slightly sarcastic voice AI for tech support. "
```



And let's add another import here from LifeKit agents and that is MCP. And then we'll connect our MCP server to our session using MCP servers. that is going to be under agent session. So let's go find that agent session. We'll add another option here under agent session MCP servers and then we'll

■ [51:00 - 51:20]

```
79 async def entrypoint(ctx: SubContext):
80     session = AgentSession(
81         tts=tts.FallbackAdapter(
82             [
83                 inference.TTS.from_model_string(
84                     "cartesia/sonic-3:9626c31c-bec5-4cca-baa8-f8ba9e84c8bc"
85                 ),
86                 inference.TTS.from_model_string("inworld/inworld-tts-1"),
87             ]
88         ),
89         vad=silero.VAD.load(),
90         turn_detection=MultilingualModel(),
91         preemptive_generation=True,
92         mcp_servers=[
93             mcp.MCPServerHTTP(url="https://docs.livekit.io/mcp"),
94         ],
95     )
96
97     usage_collector = metrics.UsageCollector()
98     last_eou_metrics: metrics.EOUMetrics | None = None
99
100     @session.on("metrics_collected")
101     def _on_metrics_collected(ev: MetricsCollectedEvent):
102         nonlocal last_eou_metrics
103         if ev.metrics.type == "eou_metrics":
```



agent session MCP servers and then we'll set the MCP server http to docs.livekit.io/mcp. And then lastly, let's go back up to our instructions and let's update these instructions to include everything. So, not only are we a a sarcastic um tech

■ [51:20 - 51:40]

```
voice-agent > agent.py
import httpx

logger = logging.getLogger(__name__)

load_dotenv()

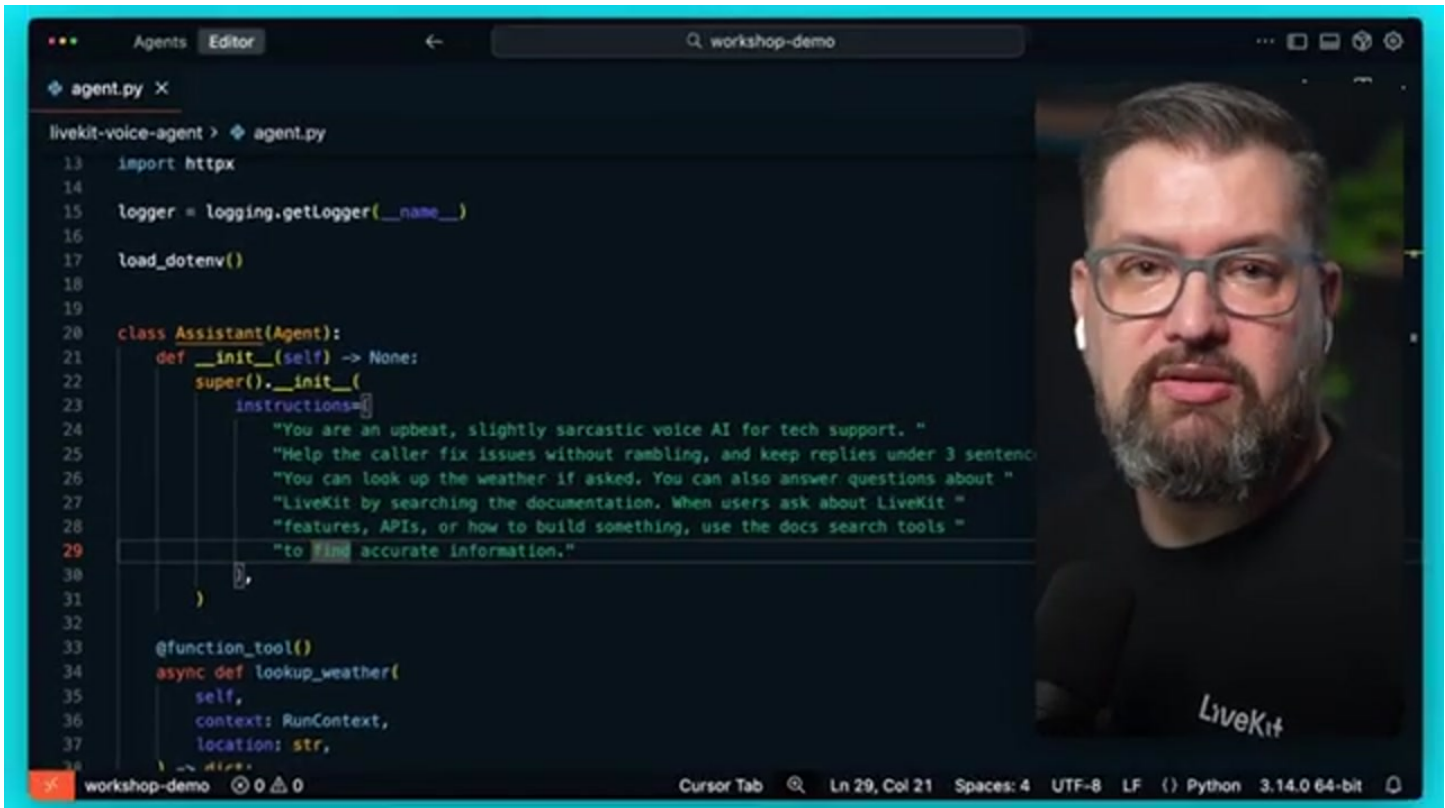
class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions=[
                "You are an upbeat, slightly sarcastic voice AI for tech support. "
                "Help the caller fix issues without rambling, and keep replies under 3 sen
                "You can look up the weather if asked. You can also answer questions about
                "LiveKit by searching the documentation. When users ask about LiveKit "
                "features, APIs, or how to build something, use the docs search tools "
                "to find accurate information."
            ],
        )

    @function_tool()
    async def lookup_weather(
        self,
        context: RunContext,
```



not only are we a a sarcastic um tech support person and we we try not to ramble, but we can also look up the weather and we can also answer questions about LifeKit by searching the documentation. When a user asks about LifeKit features, APIs, or how to build something, use the docs search tools to find accurate information. Amazing. So, when the session starts, it connects to

■ [51:40 - 52:00]

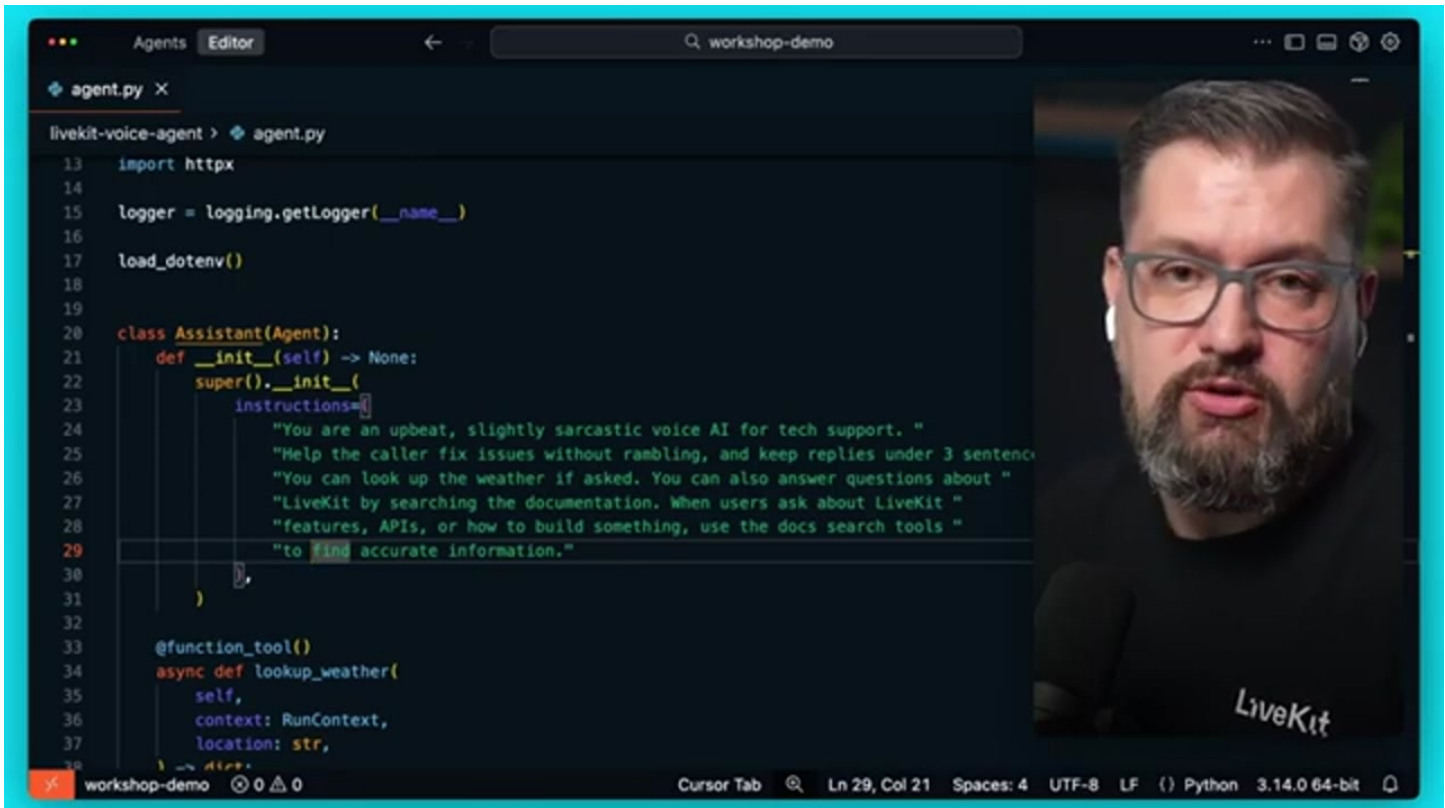


```
agent.py X
livekit-voice-agent > agent.py
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18
19
20 class Assistant(Agent):
21     def __init__(self) -> None:
22         super().__init__(
23             instructions=[
24                 "You are an upbeat, slightly sarcastic voice AI for tech support. ",
25                 "Help the caller fix issues without rambling, and keep replies under 3 sentences.",
26                 "You can look up the weather if asked. You can also answer questions about "
27                 "LiveKit by searching the documentation. When users ask about LiveKit "
28                 "features, APIs, or how to build something, use the docs search tools "
29                 "to find accurate information."
30             ],
31         )
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
37         location: str,
38     ) -> str:
39         pass
```

workshop-demo 0 0 0 Cursor Tab Ln 29, Col 21 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

when the session starts, it connects to the MCP server and fetches a list of the available tools. Each MCP tool then has a name and a description just like the the local function tool methods. So these MCP tools are automatically registered alongside your local tools and the LLM sees all of them together and it knows when to use them I guess

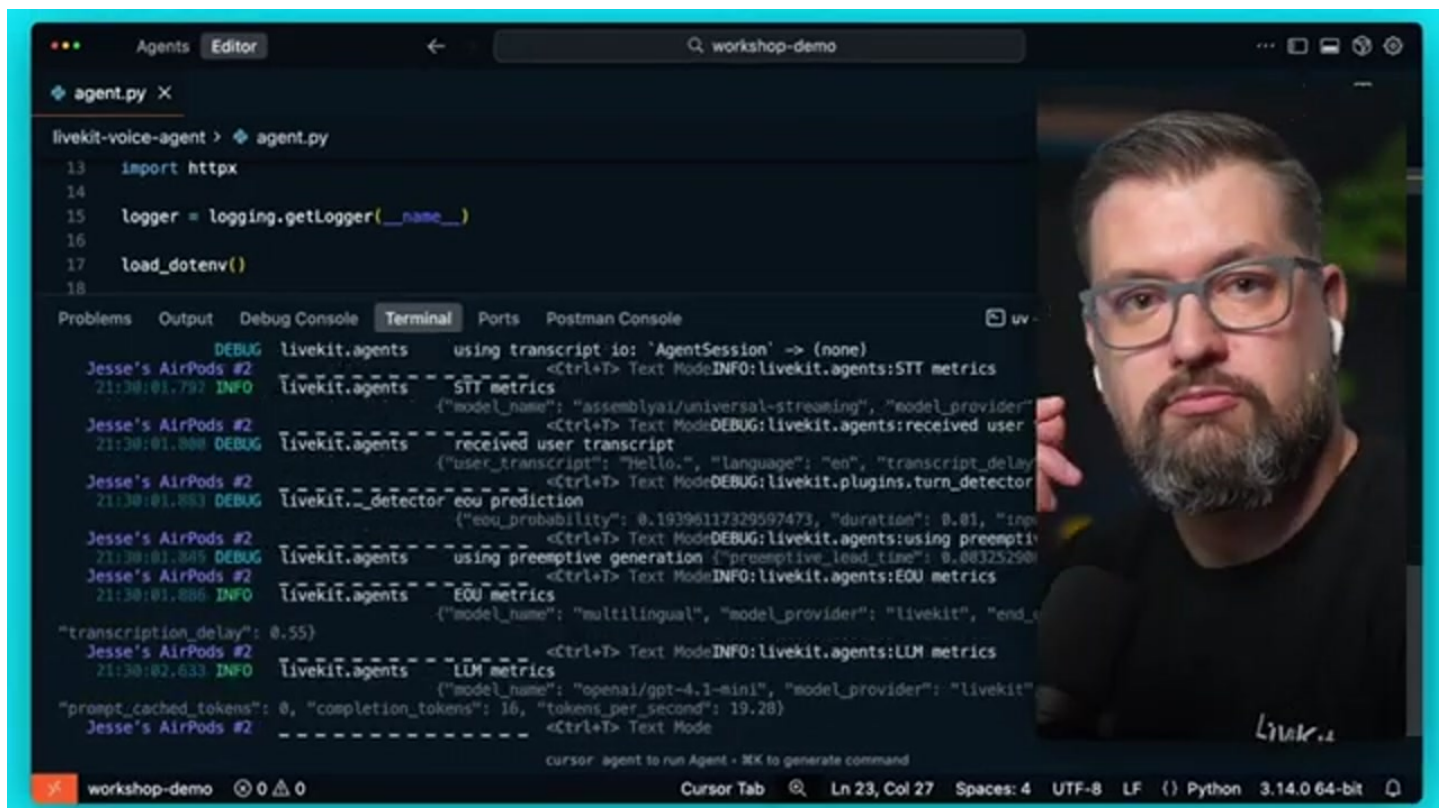
■ [52:00 - 52:20]

A screenshot of a video call interface. On the right side, there is a video feed of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. On the left side, there is a code editor window titled 'workshop-demo' showing the contents of 'agent.py'. The code defines an 'Assistant' class that inherits from 'Agent'. It includes an 'instructions' attribute with a multi-line string describing the AI's role as a tech support voice AI. It also features a '@function_tool()' decorated asynchronous function 'lookup_weather' that takes 'self', 'context', and 'location' as arguments. The status bar at the bottom of the editor shows 'Ln 29, Col 21', 'Spaces: 4', 'UTF-8', 'LF', and 'Python 3.14.0 64-bit'.

```
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18
19
20 class Assistant(Agent):
21     def __init__(self) -> None:
22         super().__init__(
23             instructions="""
24             "You are an upbeat, slightly sarcastic voice AI for tech support. "
25             "Help the caller fix issues without rambling, and keep replies under 3 sentences. "
26             "You can look up the weather if asked. You can also answer questions about "
27             "LiveKit by searching the documentation. When users ask about LiveKit "
28             "features, APIs, or how to build something, use the docs search tools "
29             "to find accurate information."
30         """
31     )
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
37         location: str,
38     ) -> dict:
```

and it knows when to use them I guess somehow. I don't know. No, no, no. It uses the the tool descriptions from the MCP server uh to guide the LLM on when to use each tool. So let's go ahead and test this out. Again, make sure that you have saved this and then let's go ahead and UV run uh agents in the console again. Hello.

■ [52:20 - 52:40]



The screenshot shows a Cursor IDE window with a file named `agent.py` open. The code in the editor is as follows:

```
livekit-voice-agent > agent.py
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18
```

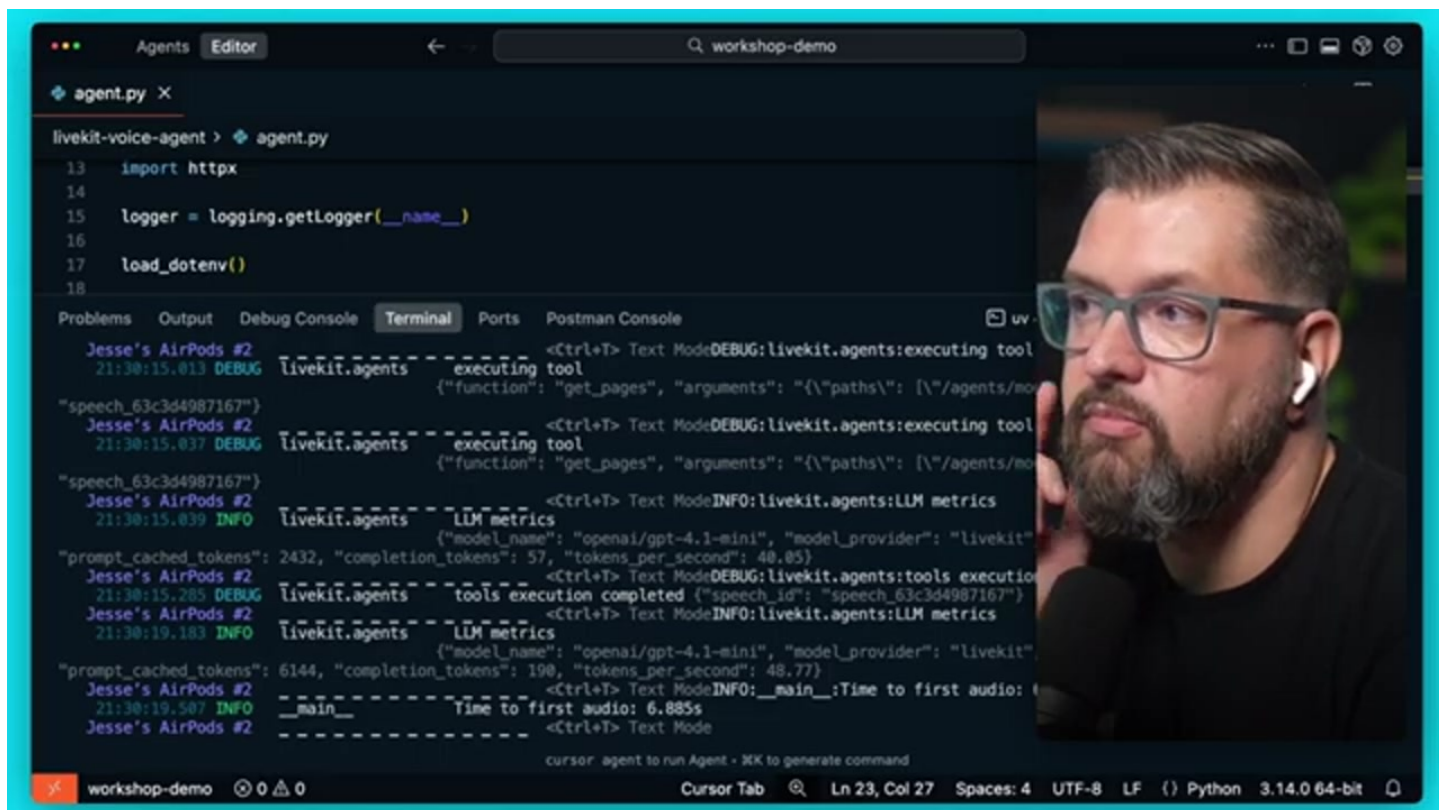
The terminal window displays the following logs:

```
DEBUG livekit.agents using transcript io: 'AgentSession' -> (none)
Jesse's AirPods #2 21:30:01.797 INFO livekit.agents STT metrics
{"model_name": "assemblyai/universal-streaming", "model_provider": "livekit"}
Jesse's AirPods #2 21:30:01.800 DEBUG livekit.agents received user transcript
{"user_transcript": "Hello.", "language": "en", "transcript_delay": 0.55}
Jesse's AirPods #2 21:30:01.803 DEBUG livekit._detector eou prediction
{"eou_probability": 0.19396117329597473, "duration": 0.81, "input_tokens": 16, "completion_tokens": 0, "tokens_per_second": 19.28}
Jesse's AirPods #2 21:30:01.849 DEBUG livekit.agents using preemptive generation {"preemptive_lead_time": 0.083252901}
Jesse's AirPods #2 21:30:01.886 INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_token": 16, "completion_tokens": 0, "tokens_per_second": 19.28}
Jesse's AirPods #2 21:30:02.633 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit", "end_token": 16, "completion_tokens": 0, "tokens_per_second": 19.28}
Jesse's AirPods #2 21:30:02.633 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit", "end_token": 16, "completion_tokens": 0, "tokens_per_second": 19.28}
```

The video inset shows a man with glasses and a beard, wearing a black t-shirt with the word "Link" on it, holding a microphone.

Hello. >> Hey there. How can I help you with tech or live kit today? >> Yeah. So, um what uh what ST providers does live kit support?

■ [52:40 - 53:00]

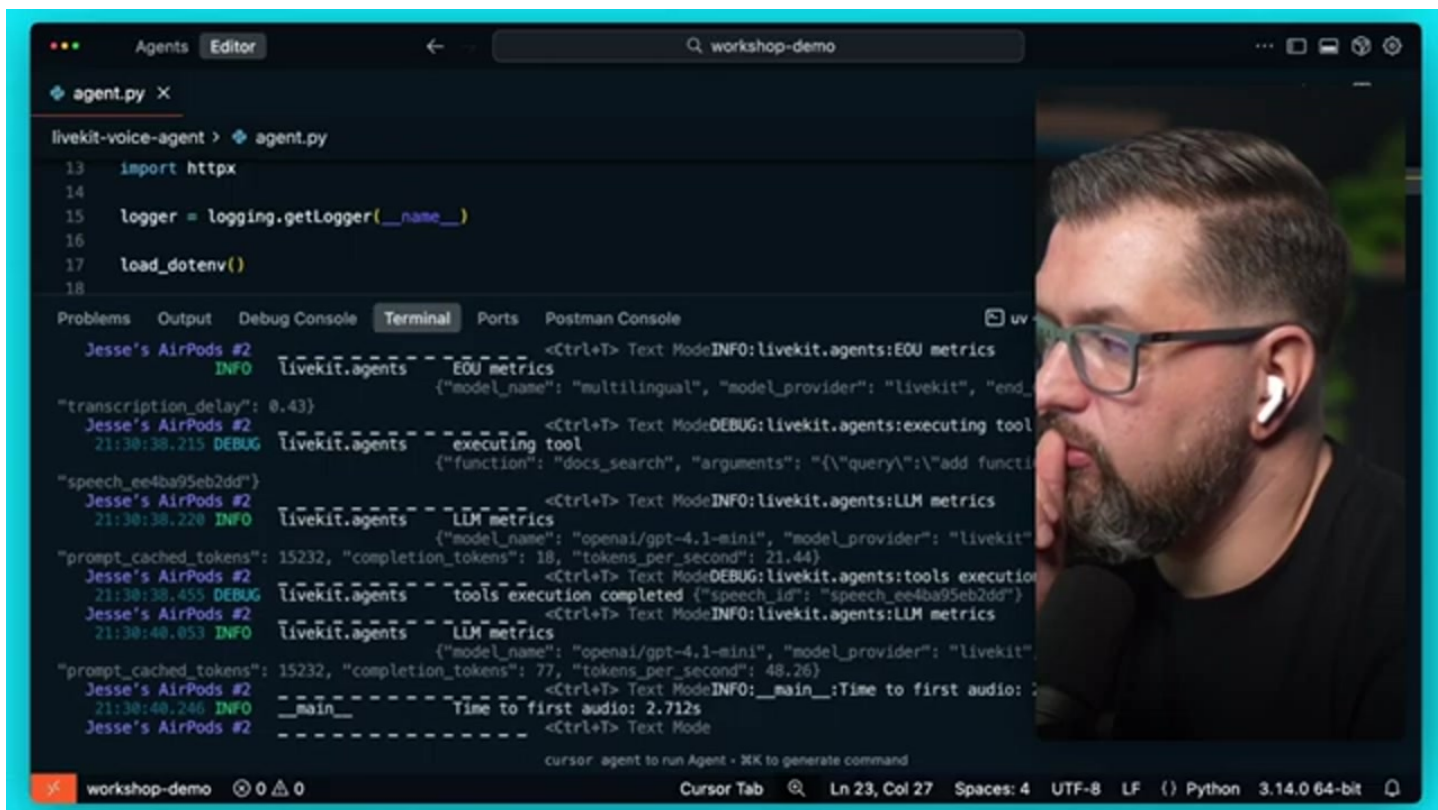


```
agent.py X
livekit-voice-agent > agent.py
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 21:30:15.013 DEBUG livekit.agents executing tool
{"function": "get_pages", "arguments": "{\\"paths\\": [\\"/agents/mo
speech_63c3d4987167"}
Jesse's AirPods #2 21:30:15.037 DEBUG livekit.agents executing tool
{"function": "get_pages", "arguments": "{\\"paths\\": [\\"/agents/mo
speech_63c3d4987167"}
Jesse's AirPods #2 21:30:15.039 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 2432, "completion_tokens": 57, "tokens_per_second": 48.05}
Jesse's AirPods #2 21:30:15.205 DEBUG livekit.agents tools execution completed {"speech_id": "speech_63c3d4987167"}
Jesse's AirPods #2 21:30:19.183 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 6144, "completion_tokens": 190, "tokens_per_second": 48.77}
Jesse's AirPods #2 21:30:19.507 INFO __main__ Time to first audio: 6.885s
Jesse's AirPods #2 Time to first audio: 6.885s
cursor: agent to run Agent - KK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 23, Col 27 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

LiveKit supports these speech-to-text SDT providers via LiveKit inference cloud service assembly AI English and six multilingual languages Cartisia Inc. Whisper 98 languages deepgram gram several English and multilingual models 11. >> How do I add a function tool to my agent? To add a function tool to your agent in LiveKit, you use the `@function_tool`

■ [53:00 - 53:20]



```
agent.py X
livekit-voice-agent > agent.py
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:EOU metrics
INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_
"transcription_delay": 0.43}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
21:30:38.215 DEBUG livekit.agents executing tool
{"function": "docs_search", "arguments": "{\\"query\\":\\"add functi
"speech_ee4ba95eb2dd"}
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
21:30:38.220 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit"
"prompt_cached_tokens": 15232, "completion_tokens": 18, "tokens_per_second": 21.44}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:tools execution
21:30:38.455 DEBUG livekit.agents tools execution completed {"speech_id": "speech_ee4ba95eb2dd"}
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
21:30:40.053 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit"
"prompt_cached_tokens": 15232, "completion_tokens": 77, "tokens_per_second": 48.26}
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:Time to first audio:
21:30:40.246 INFO __main__ Time to first audio: 2.712s
Jesse's AirPods #2 <Ctrl+T> Text Mode
cursor: agent to run Agent - XK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 23, Col 27 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

To add a function tool to your agent in LiveKit, you use the `@function_tool` decorator on a method in your agent class. This lets the LLM access. >> We can see here in the terminal executing tool, the tool name is docs search and uh speech to text providers is what it passed in and so it used the tool and then uh here's tool execution completed. Uh it executed another tool

■ [53:20 - 53:40]

```
13 import httpx
14
15 logger = logging.getLogger(__name__)
16
17 load_dotenv()
18
```

Problems Output Debug Console **Terminal** Ports Postman Console

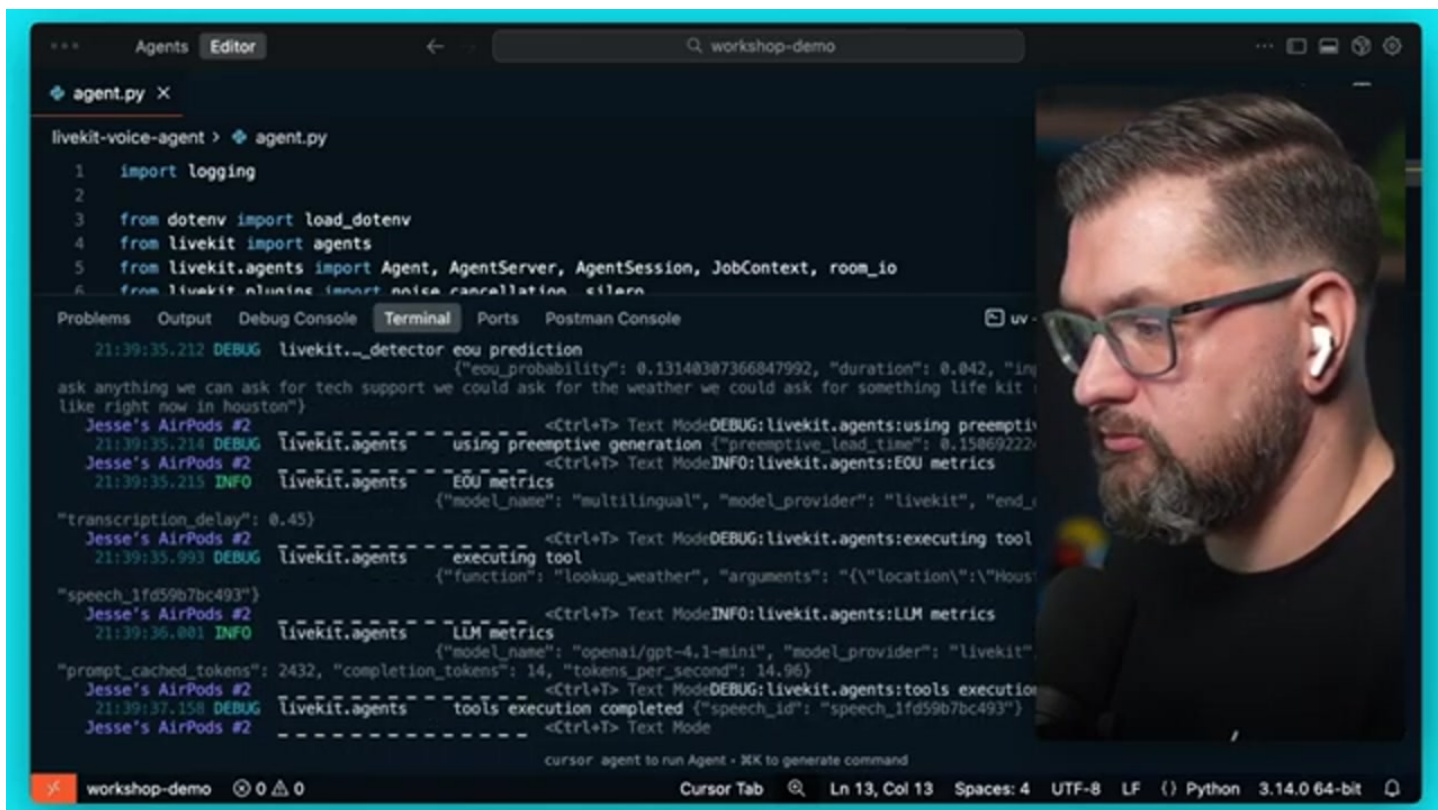
```
INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit"}
"transcription_delay": 0.63}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:exec
21:30:13.310 DEBUG livekit.agents executing tool
{"function": "docs_search", "arguments": "{\"query\": \"\
"speech_63c3d4987167"}
Jesse's AirPods #2 <Ctrl+T> Text ModeINFO:livekit.agents:LLM s
21:30:13.315 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider":
"prompt_cached_tokens": 0, "completion_tokens": 17, "tokens_per_second": 23.41}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:tool
21:30:13.613 DEBUG livekit.agents tools execution completed {"speech_id": "speech_63c3d
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:exec
21:30:15.013 DEBUG livekit.agents executing tool
{"function": "get_pages", "arguments": "{\"paths\": [\"
"speech_63c3d4987167"}
Jesse's AirPods #2 <Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
21:30:15.037 DEBUG livekit.agents executing tool
{"function": "get_pages", "arguments": "{\"paths\": [\"/agents/models/stt/#plug
"speech_63c3d4987167"}

cursor agent to run Agent • ⌘K to generate command
```



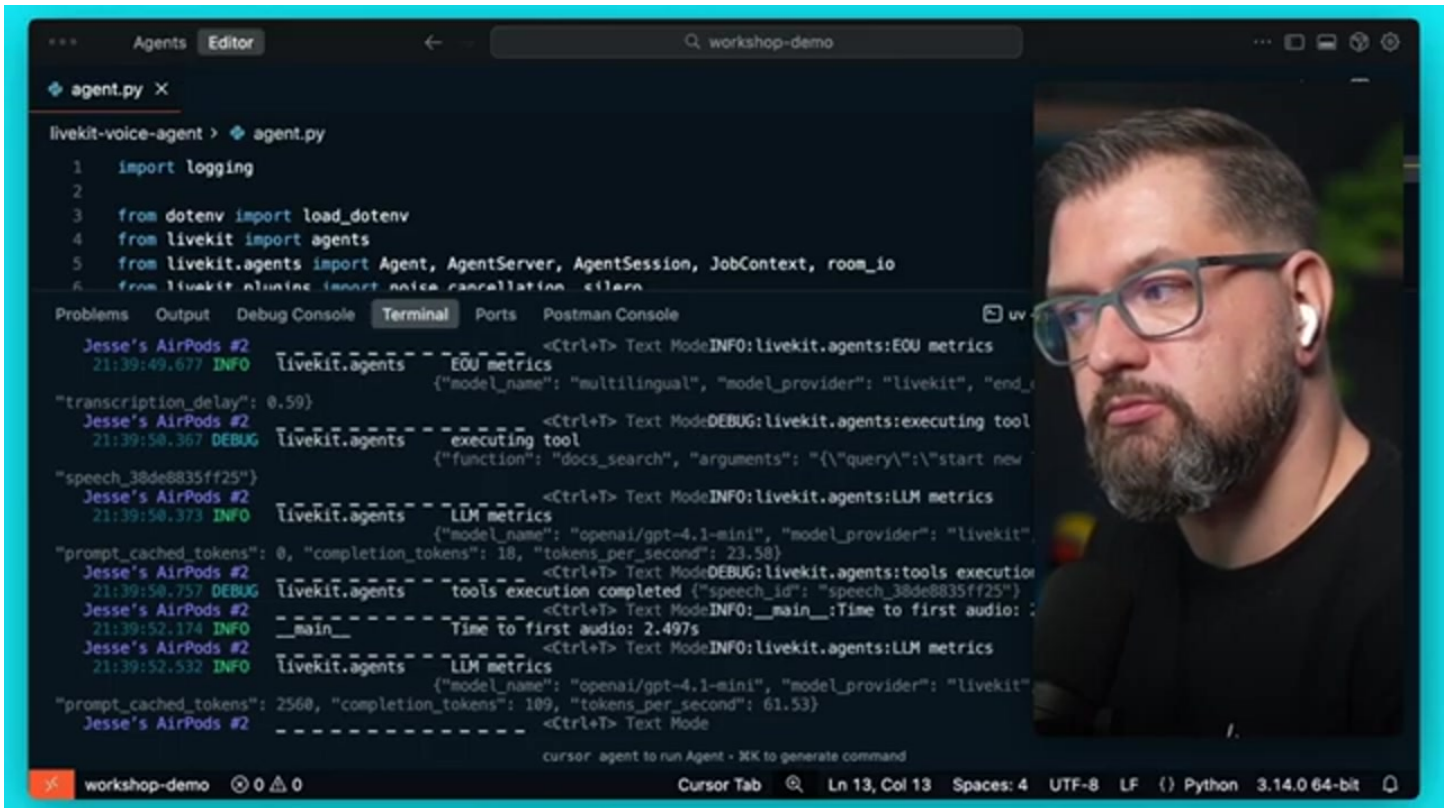
completed. Uh it executed another tool get pages and so we can see it is using our MCP server perfectly. So now we could ask um anything. We could ask for tech support. We could ask for the weather. We could ask for something LifeKit related. So let's say um what is the weather like right now in Houston?

■ [53:40 - 54:00]



Houston? >> The weather in Houston right now is around 70.4 degrees F and conditions are clear. Anything else you'd like to know? >> Yeah. Yeah. How can I quickly start up a new LiveKit agent? To quickly start a new LiveKit agent, your agent code first registers as an agent server process with the LiveKit

■ [54:00 - 54:20]

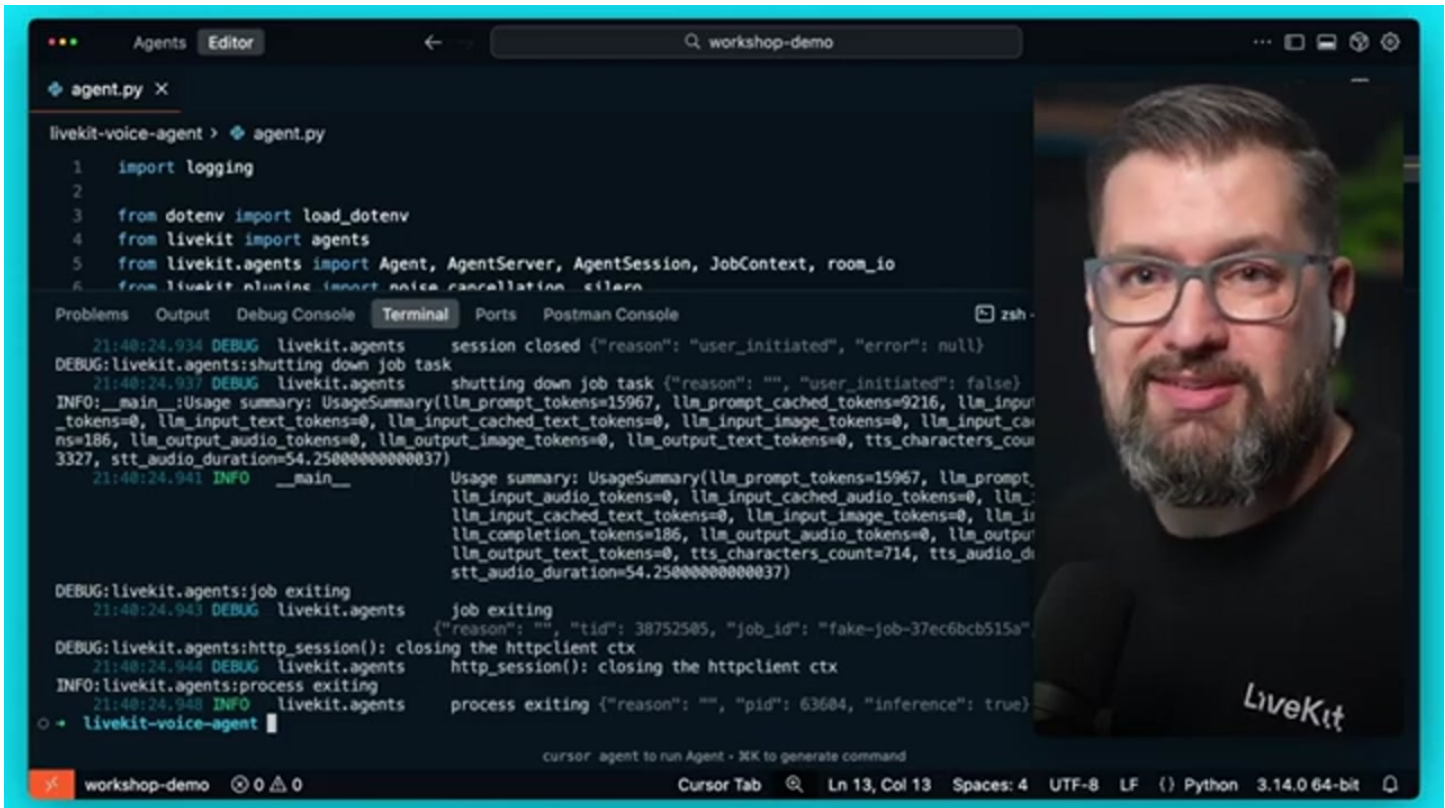


```
*** Agents Editor *** workshop-demo
agent.py X
livekit-voice-agent > agent.py
1 import logging
2
3 from dotenv import load_dotenv
4 from livekit import agents
5 from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6 from livekit.plugins import noise_cancellation, silence

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 21:39:49.677 INFO livekit.agents <Ctrl+T> Text ModeINFO:livekit.agents:EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_
"transcription_delay": 0.59}
Jesse's AirPods #2 21:39:50.367 DEBUG livekit.agents <Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
{"function": "docs_search", "arguments": "{\\"query\\":\\"start new
"speech_38de8835ff25"}
Jesse's AirPods #2 21:39:50.373 INFO livekit.agents <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 0, "completion_tokens": 18, "tokens_per_second": 23.58}
Jesse's AirPods #2 21:39:50.757 DEBUG livekit.agents <Ctrl+T> Text ModeDEBUG:livekit.agents:tools execution
tools execution completed {"speech_id": "speech_38de8835ff25"}
Jesse's AirPods #2 21:39:52.174 INFO __main__ <Ctrl+T> Text ModeINFO: __main__:Time to first audio: :
Jesse's AirPods #2 21:39:52.532 INFO livekit.agents <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 2560, "completion_tokens": 109, "tokens_per_second": 61.53}
Jesse's AirPods #2 <Ctrl+T> Text Mode
cursor: agent to run Agent - RK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 13, Col 13 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

agent server process with the LiveKit server and waits for requests. When a user connects to a room, LiveKit dispatches a job request to an available agent server, which starts a new agent session process to handle the job. >> Regular explanation. Thank you so much for that. We'll talk to you later. Cool. So now the agent selects the right tool for each question. And the console logs, you can search through those and see

■ [54:20 - 54:40]



The screenshot shows a VS Code editor window with a file named `agent.py` open. The code is a Python script for a LiveKit voice agent. It imports `logging`, `dotenv`, and `livekit` modules. The script defines a `main` function that sets up logging, loads environment variables, and initializes a `livekit` agent. The terminal output shows the agent's lifecycle, including session closure, job task shutdown, and process exit. The output also includes a usage summary for the agent's LLM and TTS components.

```
agent.py X
livekit-voice-agent > agent.py
1 import logging
2
3 from dotenv import load_dotenv
4 from livekit import agents
5 from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6 from livekit.plugins import noise_cancellation, silence

21:40:24.934 DEBUG livekit.agents session closed {"reason": "user_initiated", "error": null}
DEBUG:livekit.agents:shutting down job task
21:40:24.937 DEBUG livekit.agents shutting down job task {"reason": "", "user_initiated": false}
INFO:__main__:Usage summary: UsageSummary(llm_prompt_tokens=15967, llm_prompt_cached_tokens=9216, llm_input_tokens=0, llm_input_text_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tokens=0, llm_output_audio_tokens=0, llm_output_image_tokens=0, llm_output_text_tokens=0, tts_characters_count=186, llm_output_audio_duration=54.250000000000037)
21:40:24.941 INFO __main__ Usage summary: UsageSummary(llm_prompt_tokens=15967, llm_prompt_cached_tokens=9216, llm_input_audio_tokens=0, llm_input_cached_audio_tokens=0, llm_input_cached_text_tokens=0, llm_input_image_tokens=0, llm_input_cached_image_tokens=0, llm_completion_tokens=186, llm_output_audio_tokens=0, llm_output_text_tokens=0, tts_characters_count=714, tts_audio_duration=54.250000000000037)
DEBUG:livekit.agents:job exiting
21:40:24.943 DEBUG livekit.agents job exiting {"reason": "", "tid": 38752505, "job_id": "fake-job-37ec6bcb515a"}
DEBUG:livekit.agents:http_session(): closing the httpclient ctx
21:40:24.944 DEBUG livekit.agents http_session(): closing the httpclient ctx
INFO:livekit.agents:process exiting
21:40:24.948 INFO livekit.agents process exiting {"reason": "", "pid": 63604, "inference": true}
○ • livekit-voice-agent
```

workshop-demo 0 0 0 Cursor Tab Ln 13, Col 13 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

you can search through those and see which tool is being called and with which parameters. So let's take a look at a few best practices for tools. So be specific about what the tool does and when to use it. Uh keep tools fast under two seconds ideally or provide user feedback during long operations and then handle errors gracefully with the tool

Tool best practices

- Be specific in descriptions
- Keep tools fast (<2 seconds)
- Handle errors with ToolError



LIVEKIT.IO

handle errors gracefully with the tool error exception and return meaningful data that helps the LLM form a good response. Now, some tools take a long time and and for those you'll want to give the user feedback and handle potential interruptions. So, to do that, um what we could do in

■ [55:00 - 55:20]

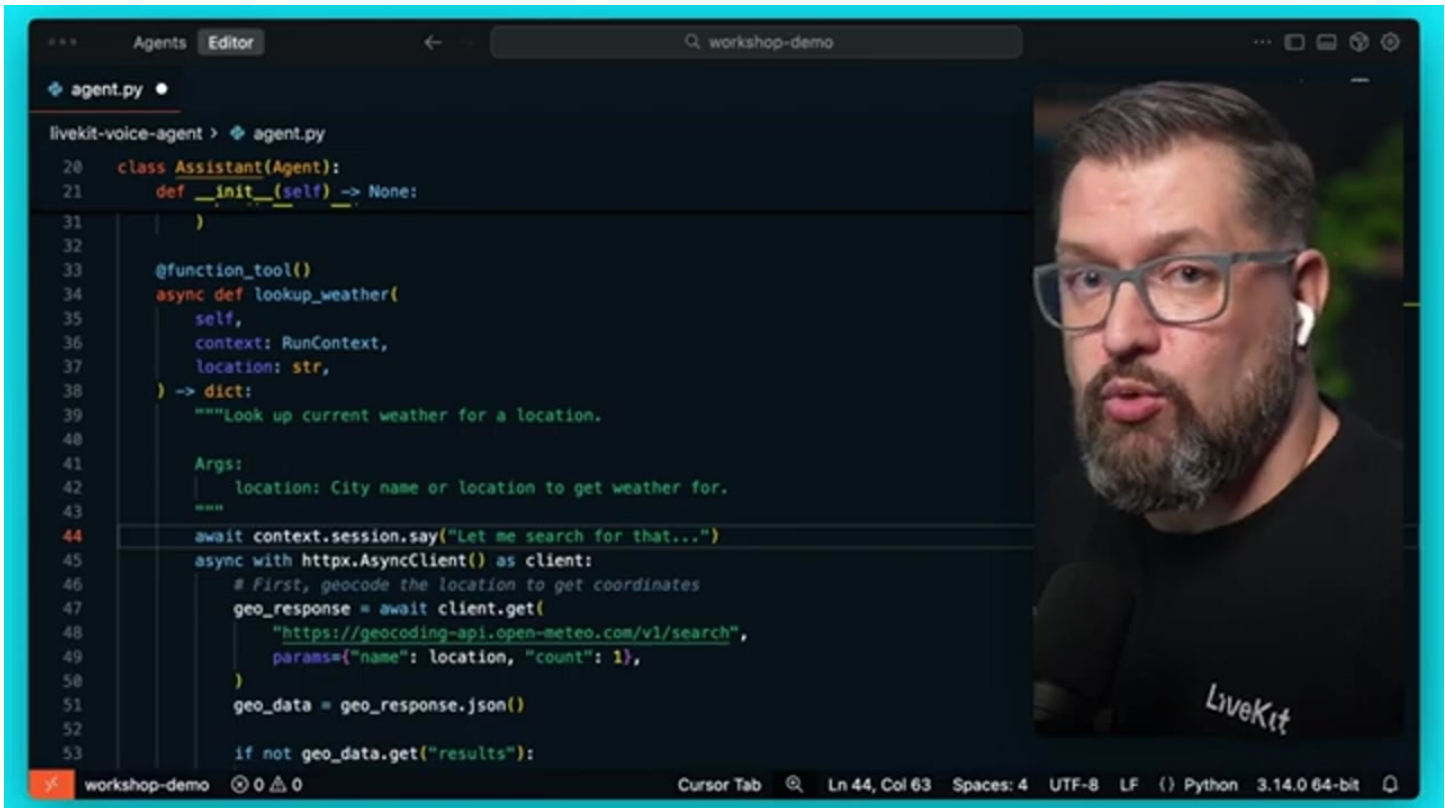


```
Agents Editor workshop-demo
agent.py x
livekit-voice-agent > agent.py
20 class Assistant(Agent):
21     def __init__(self) -> None:
31
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
37         location: str,
38     ) -> dict:
39         """Look up current weather for a location.
40
41         Args:
42             location: City name or location to get weather for.
43         """
44         async with httpx.AsyncClient() as client:
45             # First, geocode the location to get coordinates
46             geo_response = await client.get(
47                 "https://geocoding-api.open-meteo.com/v1/search",
48                 params={"name": location, "count": 1},
49             )
50             geo_data = geo_response.json()
51
52             if not geo_data.get("results"):
53                 raise ToolError(f"Could not find location: {location}")
```

workshop-demo 0 0 0 Cursor Tab Ln 13, Col 13 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

So, to do that, um what we could do in any function tool, um we could add to that await context session say, let me search for that. So, we're adding that extra context. We're giving the user um confirmation that we're doing something. Now if your tool takes external actions that can't be rolled back uh then we can

■ [55:20 - 55:40]



```
... Agents Editor workshop-demo
agent.py
livekit-voice-agent > agent.py
20 class Assistant(Agent):
21     def __init__(self) -> None:
31         )
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
37         location: str,
38     ) -> dict:
39         """Look up current weather for a location.
40
41         Args:
42             location: City name or location to get weather for.
43         """
44         await context.session.say("Let me search for that...")
45         async with httpx.AsyncClient() as client:
46             # First, geocode the location to get coordinates
47             geo_response = await client.get(
48                 "https://geocoding-api.open-meteo.com/v1/search",
49                 params={"name": location, "count": 1},
50             )
51             geo_data = geo_response.json()
52
53             if not geo_data.get("results"):
```

workshop-demo 0 0 0 Cursor Tab Ln 44, Col 63 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

that can't be rolled back uh then we can disable interruptions at the start. For instance, if um your tool creates an order uh is processing a payment, well that can't be undone. We don't want the user to interrupt that part of the agent session. And so for that we could go into any tool again uh just like we did

■ [55:40 - 56:00]

```
vekit-voice-agent > agent.py
20 class Assistant(Agent):
21     def __init__(self) -> None:
31         )
32
33     @function_tool()
34     async def lookup_weather(
35         self,
36         context: RunContext,
37         location: str,
38     ) -> dict:
39         """Look up current weather for a location.
40
41         Args:
42             location: City name or location to get weather for.
43         """
44         await context.session.say("Let me search for that...")
45         async with httpx.AsyncClient() as client:
46             # First, geocode the location to get coordinates
47             geo_response = await client.get(
48                 "https://geocoding-api.open-meteo.com/v1/search",
49                 params={"name": location, "count": 1},
50             )
51             geo_data = geo_response.json()
```



into any tool again uh just like we did here with this await context say we can do context.allow disallow interruptions. So that will prevent any interruptions once this tool is called. Uh there could be no interruptions until the tool returns. You can also add or remove tools at runtime based on conversation state. Here's what that could look like. We

■ [56:00 - 56:20]



```
32
33
34
35
36
37
38
39  # Add a tool dynamically
40  await agent.update_tools(agent.tools + [new_tool])
41
42  # Remove a tool
43  await agent.update_tools(agent.tools - [old_tool])
44
45
46
47
48
49
50
51
52
53
54
55
```

workshop-demo 0 0 0 Cursor Tab Ln 39, Col 5 Spaces: 4 UTF-8

Here's what that could look like. We could add a tool dynamically. We could remove a tool. So, this is useful for progressive disclosure where you're only showing certain tools after a user has been authenticated or reached a certain point in a workflow. Your agent can now call external APIs with function tools, connect to MCP servers for shared

- ✓ Function tools
- ✓ MCP integration



LIVEKIT.IO

tools, connect to MCP servers for shared capabilities, and handle errors gracefully. And the LLMs automatically select the right tool based on what the user asks. In the final lesson, you're going to build production workflows for collecting consent and escalating to human agents. Your agent has tools. It handles failures gracefully, and you can

Consent & escalations



LIVEKIT.IO

handles failures gracefully, and you can measure its performance. For production deployments, you'll also need to handle operational workflows like collecting consent and escalating to human agents when needed. Realworld voice agents need to do more than just answer questions. You need to collect recording consent before capturing calls, which is often

Production workflows

- Collect recording consent



LIVEKIT.IO

before capturing calls, which is often legally required. You need to escalate to humans when the AI can't help or uh the user requests it. You need to handoff between specialized agents based on conversational context. And you need to preserve context across these transitions. So let's build these capabilities step by step. Tasks are

Production workflows

- Collect recording consent
- Escalate to humans
- Handoff between agents
- Preserve context



LIVEKIT.IO

capabilities step by step. Tasks are focused units of work that complete and return a result. They're perfect for structured interactions like consent collection where you need a clear yes or no answer before proceeding. So let's build a task that collects recording consent. We're going to add another uh import here from live kit agents uh live

■ [57:40 - 58:00]

```
livekit-voice-agent > agent.py
1  import logging
2
3  from dotenv import load_dotenv
4  from livekit import agents
5  from livekit.agents import Agent, AgentServer, AgentSession, JobContext, room_io
6  from livekit.plugins import noise_cancellation, silero
7  from livekit.plugins.turn_detector.multilingual import MultilingualModel
8  from livekit.agents import llm, stt, tts, inference
9  from livekit.agents import AgentStateChangedEvent, MetricsCollectedEvent, metric
10 from livekit.agents import function_tool, RunContext, ToolError
11 from livekit.agents import mcp
12 from livekit.agents import AgentTask
13 import time
14 import httpx
15
16 logger = logging.getLogger(__name__)
17
18 load_dotenv()
19
20
21 class Assistant(Agent):
22     def __init__(self) -> None:
23         super().__init__(
```



import here from live kit agents uh live agent task. So next we're going to define the consent task. So we have a new class here, collect consent, which extends the agent task. And so this is going to return a boolean true or false. The instructions ask for recording consent and get a clear yes or no answer. Be polite and professional.

■ [58:00 - 58:20]

```
10 from livekit.agents import function_tool, RunContext, ToolError
11 from livekit.agents import mcp
12 from livekit.agents import AgentTask
13 import time
14 import httpx
15
16 logger = logging.getLogger(__name__)
17
18 load_dotenv()
19
20 class CollectConsent(AgentTask[bool]):
21     def __init__(self, chat_ctx=None):
22         super().__init__()
23         instructions="""
24         Ask for recording consent and get a clear yes or no answer.
25         Be polite and professional.
26         """
27         chat_ctx=chat_ctx,
28     )
29
30     async def on_enter(self) -> None:
31         await self.session.generate_reply(
32             instructions="""
33             Briefly introduce yourself, then ask for permission to record
34             the call for quality assurance and training purposes.
35             """
```



answer. Be polite and professional. That's important. On enter, this method starts when the task starts. And so there's instructions here. Briefly introduce yourself and then ask for permission to record the call for quality assurance and training purposes. Make it clear that they can decline. And then we have two function tools. We have consent given and consent declined. And

■ [58:20 - 58:40]

```
20 class CollectConsent(AgentTask[bool]):
21     def __init__(self, chat_ctx=None):
22         super().__init__(
23             chat_ctx=chat_ctx,
24         )
25
26     async def on_enter(self) -> None:
27         await self.session.generate_reply(
28             instructions="""
29             Briefly introduce yourself, then ask for permission to record
30             the call for quality assurance and training purposes.
31             Make it clear that they can decline.
32             """
33         )
34
35     @function_tool()
36     async def consent_given(self) -> None:
37         """Use this when the user gives consent to record."""
38         self.complete(True)
39
40     @function_tool()
41     async def consent_denied(self) -> None:
42         """Use this when the user denies consent to record."""
43         self.complete(False)
44
45
46
47
48
```



consent given and consent declined. And it basically just returns true or false. So now we can use this task in our main agent. So now if we go into our assistant first, let's go ahead and update our instructions. We're going to change these from our last agent. Uh you are a friendly customer service representative. Very simple. And then we don't need any of these other tools uh

■ [58:40 - 59:00]

The image shows a code editor window with a dark theme. The title bar at the top says "workshop-demo". The editor has a file explorer on the left showing "agent.py" and "livekit-voice-agent". The main editor area displays Python code for an Assistant class. The code uses the OpenWeatherMap API to fetch weather data. The status bar at the bottom indicates the current position is Ln 67, Col 24 (302 selected), with 4 spaces, UTF-8 encoding, LF line endings, Python 3.14.0 64-bit. A video inset on the right shows a man with glasses and a beard speaking.

don't need any of these other tools uh that we used in our last lesson. So we'll just go ahead and replace all of this. So here is our new uh assistant class. So you're a friendly customer service representative on enter collect consent. If we have consent then thank them and offer them your assistance. Uh if they declined consent let them know

■ [59:00 - 59:20]

it-voice-agent > agent.py

```
class CollectConsent(AgentTask[bool]):
    async def consent_denied(self) -> None:

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions="You are a friendly customer service representative."
        )

    async def on_enter(self) -> None:
        consent = await CollectConsent(chat_ctx=self.chat_ctx)

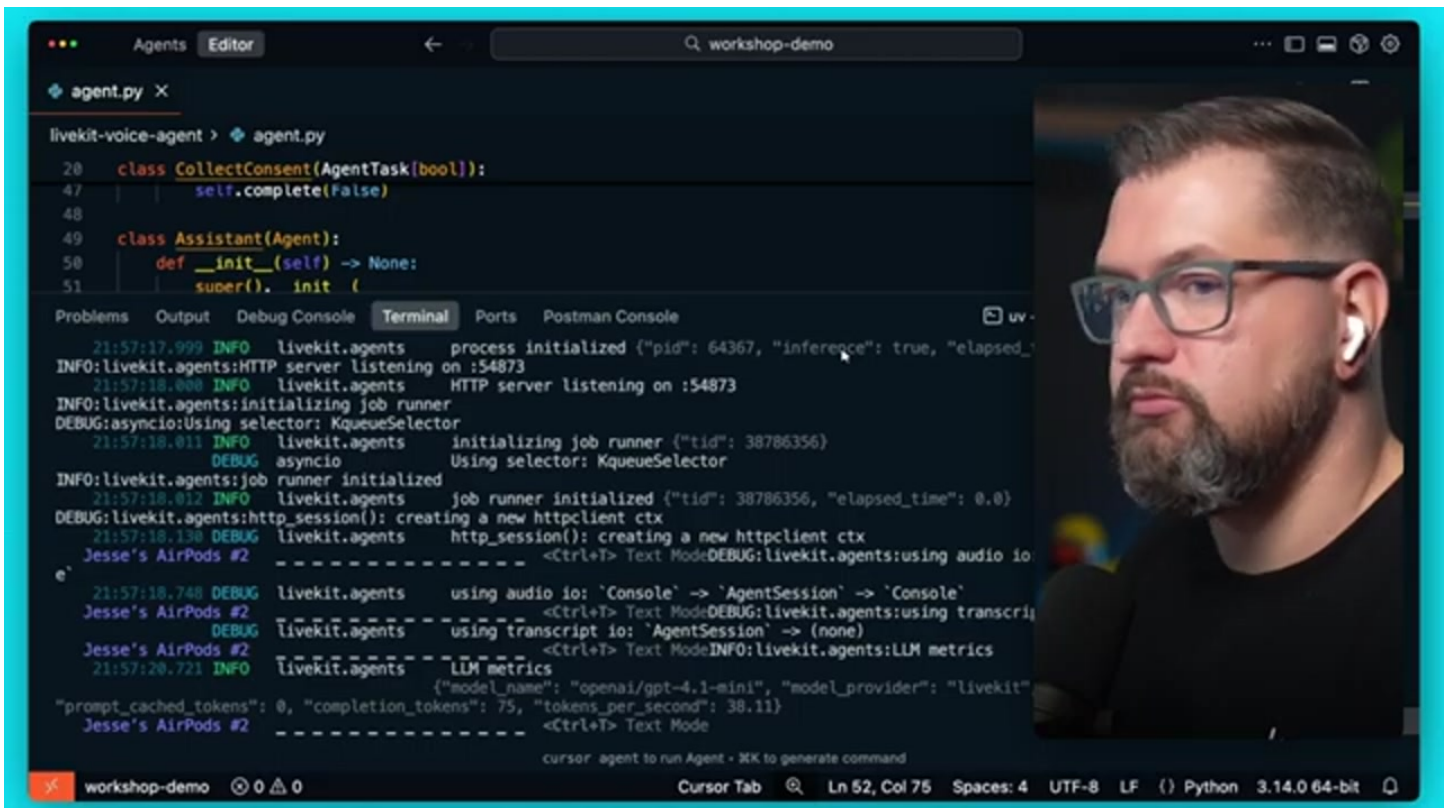
        if consent:
            await self.session.generate_reply(
                instructions="Thank them and offer your assistance."
            )
        else:
            await self.session.generate_reply(
                instructions="Let them know you understand and will proceed without recording."
            )

server = AgentServer()
```



if they declined consent let them know that you understand and you will proceed without recording. So this pattern keeps consent collection clean and separate from your main agent logic. So let's go ahead and try this out. Make sure you save it. Open up your terminal and let us run the agent in the console. >> Hello. Before we proceed, I'd like to introduce myself. I'm your assistant

■ [59:20 - 59:40]

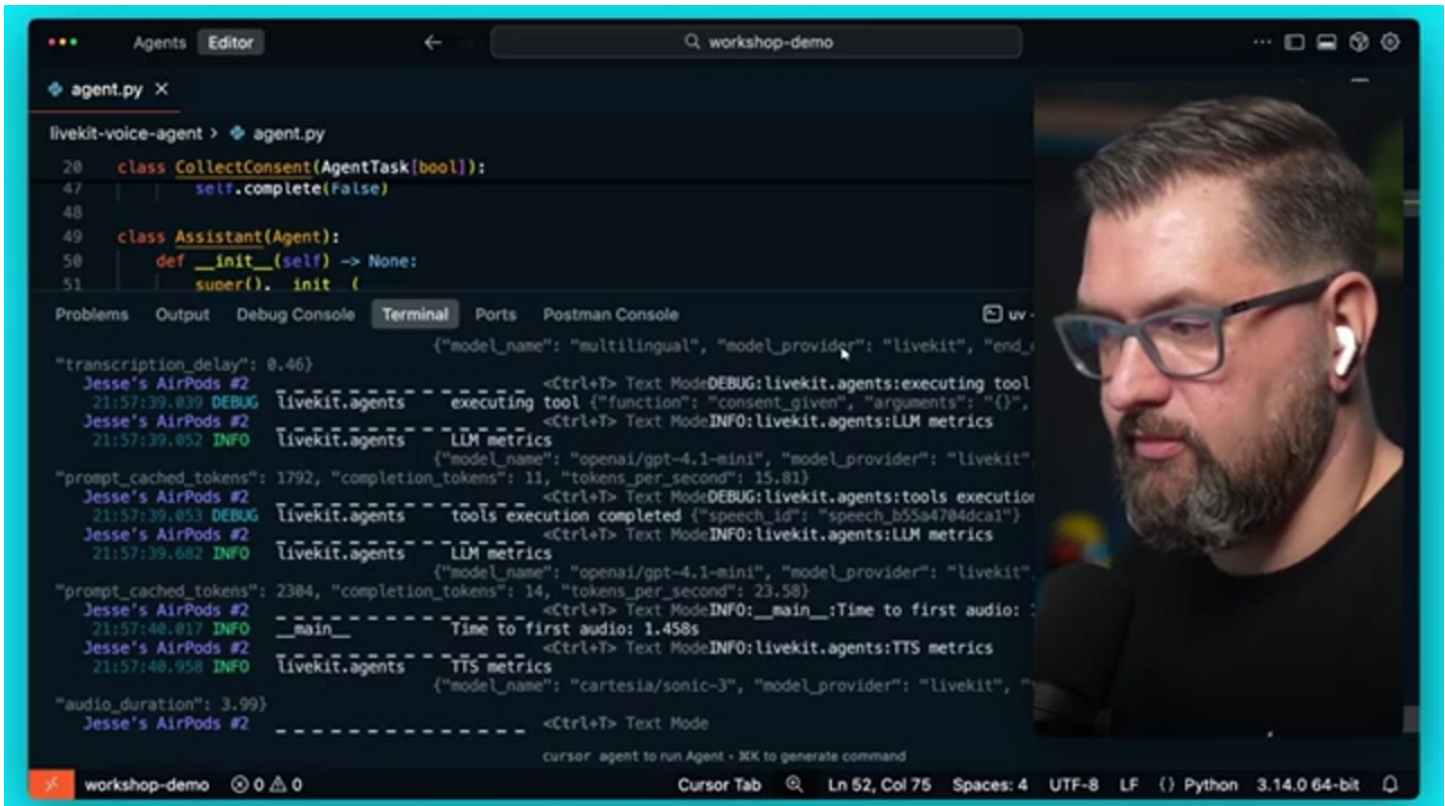


```
agent.py
livekit-voice-agent > agent.py
20 class CollectConsent(AgentTask[bool]):
47     self.complete(False)
48
49 class Assistant(Agent):
50     def __init__(self) -> None:
51         super().__init__()

Problems Output Debug Console Terminal Ports Postman Console
21:57:17.999 INFO livekit.agents process initialized {"pid": 64367, "inference": true, "elapsed_
INFO:livekit.agents:HTTP server listening on :54873
21:57:18.000 INFO livekit.agents HTTP server listening on :54873
INFO:livekit.agents:initializing job runner
DEBUG:asyncio:Using selector: KqueueSelector
21:57:18.011 INFO livekit.agents initializing job runner {"tid": 38786356}
DEBUG:asyncio Using selector: KqueueSelector
INFO:livekit.agents:job runner initialized
21:57:18.012 INFO livekit.agents job runner initialized {"tid": 38786356, "elapsed_time": 0.0}
DEBUG:livekit.agents:http_session(): creating a new httpclient ctx
21:57:18.130 DEBUG livekit.agents http_session(): creating a new httpclient ctx
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:using audio io
e'
21:57:18.748 DEBUG livekit.agents using audio io: 'Console' -> 'AgentSession' -> 'Console'
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:using transcrip
DEBUG livekit.agents using transcript io: 'AgentSession' -> (none)
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
21:57:20.721 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit"
"prompt_cached_tokens": 0, "completion_tokens": 75, "tokens_per_second": 38.11}
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode
cursor: agent to run Agent - RK to generate command
workshop-demo 0 0 0 Cursor Tab Ln 52, Col 75 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit
```

introduce myself. I'm your assistant here to help with any questions or support you need. May I have your permission to record this call for quality assurance and training purposes? You are free to decline if you prefer. Could you please let me know if that is okay with you? A clear yes or no would be great. >> Yes. >> Thank you for your consent. How can I assist you today?

■ [59:40 - 01:00:00]

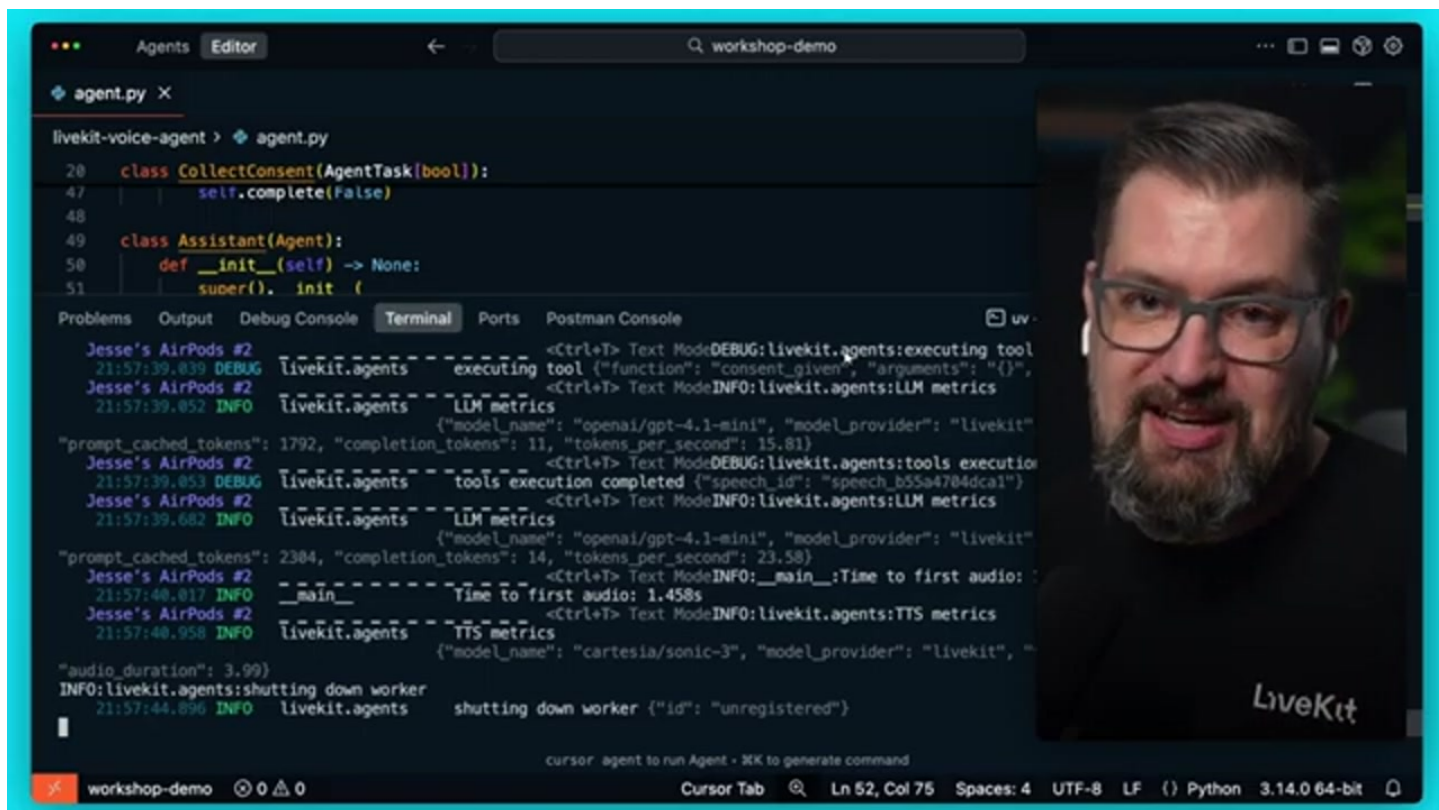


```
agent.py x
livekit-agent > agent.py
20 class CollectConsent(AgentTask[bool]):
47     self.complete(False)
48
49 class Assistant(Agent):
50     def __init__(self) -> None:
51         super().__init__(
            {"model_name": "multilingual", "model_provider": "livekit", "end_

"transcription_delay": 0.46)
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:executing tool
21:57:39.839 DEBUG livekit.agents executing tool {"function": "consent_given", "arguments": "{}",
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
21:57:39.852 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 1792, "completion_tokens": 11, "tokens_per_second": 15.81)
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeDEBUG:livekit.agents:tools execution
21:57:39.853 DEBUG livekit.agents tools execution completed {"speech_id": "speech_b55a4704dca1"}
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeINFO:livekit.agents:LLM metrics
21:57:39.882 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 2304, "completion_tokens": 14, "tokens_per_second": 23.58)
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeINFO:livekit.agents:TTS metrics
21:57:40.017 INFO __main__ Time to first audio: 1.458s
Jesse's AirPods #2 ----- <Ctrl+T> Text ModeINFO:livekit.agents:TTS metrics
21:57:40.958 INFO livekit.agents TTS metrics
{"model_name": "cartesia/sonic-3", "model_provider": "livekit",
"audio_duration": 3.99)
Jesse's AirPods #2 ----- <Ctrl+T> Text Mode
cursor: agent to run Agent - KK to generate command
```

assist you today? >> Very good. So, it works. Now, if you're using LiveKit's agent observability features and you want session recordings to respect the user's consent choice, then you could start a new agent session with the record parameter set to false when uh consent is denied. This gives you fine grain control over which sessions get recorded. So, check the

■ [01:00:00 - 01:00:20]



The screenshot shows a code editor window titled "Agents Editor" with a file named "agent.py". The code defines a `CollectConsent` class and an `Assistant` class. The `Assistant` class has an `__init__` method that calls `super().init()`. Below the code, the "Terminal" tab displays the output of the agent's execution. The output shows the agent executing a tool, logging LLM metrics, and completing the tool execution. It also shows the agent's TTS metrics and the time to first audio. The output is formatted with timestamps and log levels (DEBUG, INFO). A video inset in the top right corner shows a man with glasses and a beard, wearing a black t-shirt with the "LiveKit" logo, speaking.

```
livekit-voice-agent > agent.py
20 class CollectConsent(AgentTask[bool]):
47     self.complete(False)
48
49 class Assistant(Agent):
50     def __init__(self) -> None:
51         super().init (

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 21:57:39.839 DEBUG livekit.agents executing tool
Jesse's AirPods #2 21:57:39.852 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 1792, "completion_tokens": 11, "tokens_per_second": 15.81}
Jesse's AirPods #2 21:57:39.853 DEBUG livekit.agents tools execution
Jesse's AirPods #2 21:57:39.882 INFO livekit.agents LLM metrics
{"model_name": "openai/gpt-4.1-mini", "model_provider": "livekit",
"prompt_cached_tokens": 2384, "completion_tokens": 14, "tokens_per_second": 23.58}
Jesse's AirPods #2 21:57:40.017 INFO __main__ Time to first audio: 1.458s
Jesse's AirPods #2 21:57:40.958 INFO livekit.agents TTS metrics
{"model_name": "cartesia/sonic-3", "model_provider": "livekit",
"audio_duration": 3.99)
INFO:livekit.agents:shutting down worker
21:57:44.896 INFO livekit.agents shutting down worker {"id": "unregistered"}
```

sessions get recorded. So, check the agent observability docs for uh more details on session level recording settings. Now, what happens when a user says they want to talk to a manager or escalate in some way? Let's take a look at that. Let's go ahead and add another class. So, this is going to be our

■ [01:00:20 - 01:00:40]

```
livekit-voice-agent > agent.py
20 class CollectConsent(AgentTask[bool]):
44     @function_tool()
45     async def consent_denied(self) -> None:
46         """Use this when the user denies consent to record."""
47         self.complete(False)
48
49 class ManagerAgent(Agent):
50     def __init__(self, chat_ctx=None):
51         super().__init__(
52             instructions="""
53             You are a customer service manager. You handle escalated issues
54             that frontline agents couldn't resolve. Be empathetic and
55             solution-focused. You have authority to offer refunds, credits,
56             or other accommodations.
57             """,
58             chat_ctx=chat_ctx,
59             tts="cartesia/sonic-3:6f84f4b8-58a2-430c-8c79-688dad597532",
60         )
61
62     async def on_enter(self) -> None:
63         await self.session.generate_reply(
64             instructions="""
65             Introduce yourself as a manager. Acknowledge that the customer
66             asked to speak with someone senior. Let her you can help resolve
```



class. So, this is going to be our manager agent. And here's the instructions. You are a customer service uh manager. You handle escalated escalated issues that frontline agents could not resolve. Be empathetic and solution focused. You have authority to offer refunds, credits, or other accommodations. Very nice. And we can also change uh the the voice here. Of

■ [01:00:40 - 01:01:00]

```
class CollectConsent(AgentTask[bool]):
    @function_tool()
    async def consent_denied(self) -> None:
        """Use this when the user denies consent to record."""
        self.complete(False)

class ManagerAgent(Agent):
    def __init__(self, chat_ctx=None):
        super().__init__(
            instructions="""
            You are a customer service manager. You handle escalated issues
            that frontline agents couldn't resolve. Be empathetic and
            solution-focused. You have authority to offer refunds, credits,
            or other accommodations.
            """,
            chat_ctx=chat_ctx,
            tts="cartesia/sonic-3:6f84f4b8-58a2-430c-8c79-688dad597532",
        )

    async def on_enter(self) -> None:
        await self.session.generate_reply(
            instructions="""
            Introduce yourself as a manager. Acknowledge that the customer
            asked to speak with someone senior. Ask how you can help resolve
            their concern.
            """
```



also change uh the the voice here. Of course, we want to make sure that the voice is different because the manager obviously is a different person. So, let's go back into our main assistant agent and let's update the instructions. All right. So, you're a friendly customer service representative. Help customers with general inquiries. If they ask for a manager or you can't resolve their issue, use the escalate to

■ [01:01:00 - 01:01:20]

```
py ●
ice-agent > agent.py

class ManagerAgent(Agent):
    async def on_enter(self) -> None:

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions="""
            You are a friendly customer service representative. Help customers
            with general inquiries. If they ask for a manager or you can't
            resolve their issue, use the escalate_to_manager tool.
            """
        )

    async def on_enter(self) -> None:
        consent = await CollectConsent(chat_ctx=self.chat_ctx)

        if consent:
            await self.session.generate_reply(
                instructions="Thank them and offer your assistance."
            )
        else:
            await self.session.generate_reply(
```



resolve their issue, use the escalate to manager tool. And so, let's go ahead and add that tool now. So escalate to manager uh is going to be transfer the customer to a manager when requested or you can't resolve their in their issue and return manager agent. So it's going to call that manager agent and say

■ [01:01:20 - 01:01:40]

```
class ManagerAgent(Agent):
    async def on_enter(self) -> None:

class Assistant(Agent):
    def __init__(self) -> None:
        super().__init__(
            instructions="""
            You are a friendly customer service representative. Help customers
            with general inquiries. If they ask for a manager or you can't
            resolve their issue, use the escalate_to_manager tool.
            """
        )

    @function_tool()
    async def escalate_to_manager(self, context: RunContext) -> ManagerAgent:
        """Transfer the customer to a manager when requested or when you cannot resolve their
        return ManagerAgent(chat_ctx=self.chat_ctx), "Transferring you to a manager now."

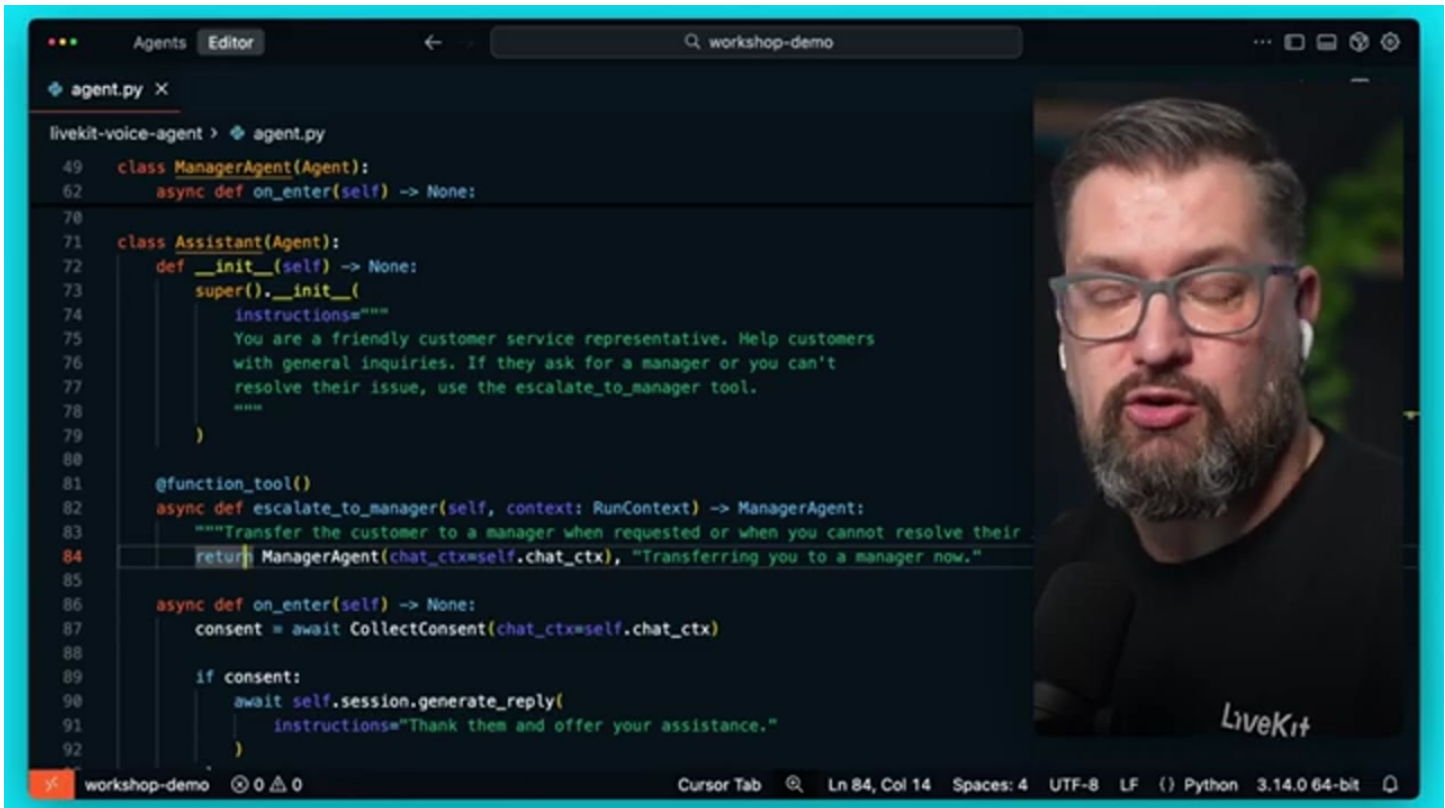
    async def on_enter(self) -> None:
        consent = await CollectConsent(chat_ctx=self.chat_ctx)

        if consent:
            await self.session.generate_reply(
                instructions="Thank them and offer your assistance."
            )
```



to call that manager agent and say transferring you to a manager now. And so when a tool uh returns a new agent as the first value in the tuple, the session automatically hands off control to that agent. And the second value there is the message that is going to speak before that handoff completes. Now, by default, each agent starts

■ [01:01:40 - 01:02:00]

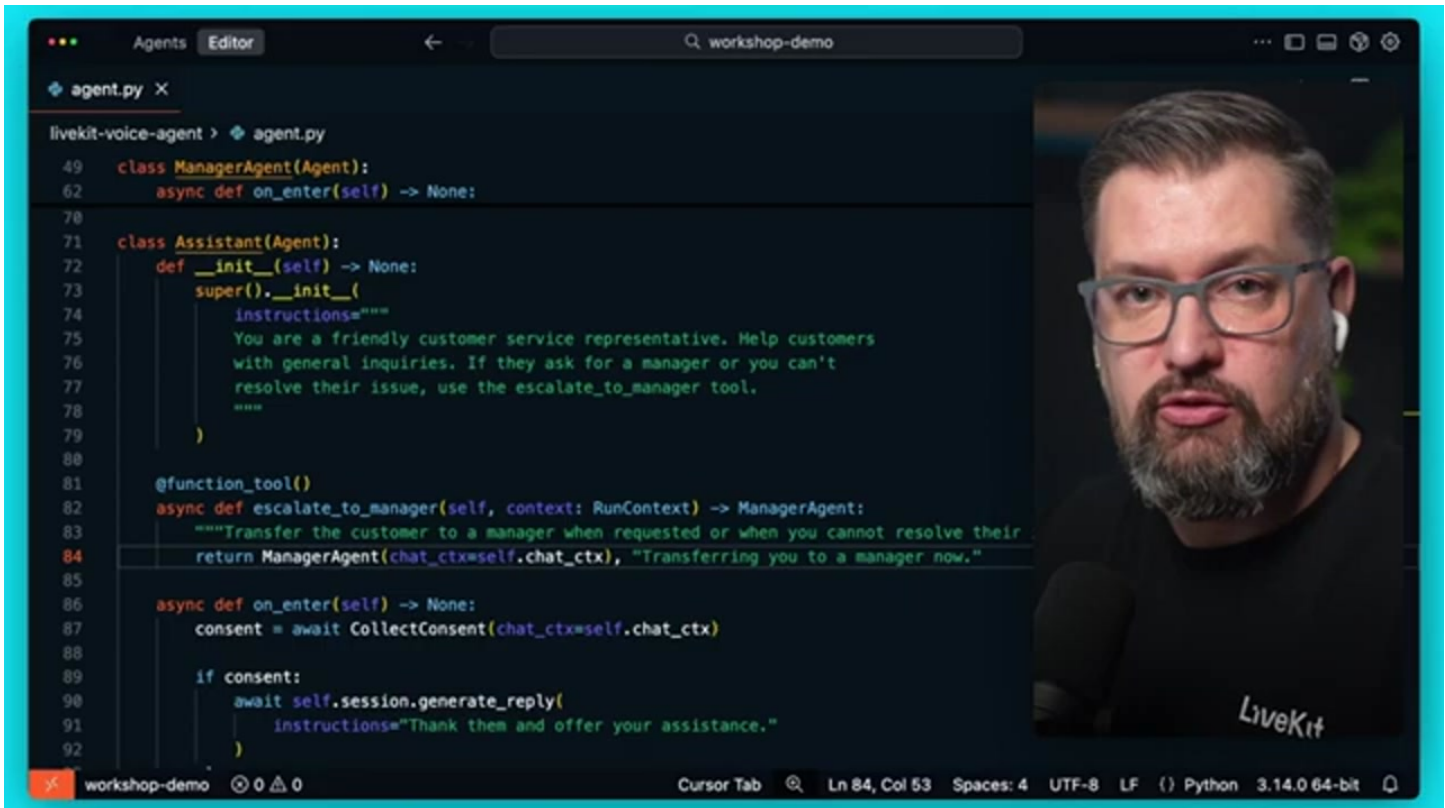


```
livekit-voice-agent > agent.py
49 class ManagerAgent(Agent):
62     async def on_enter(self) -> None:
70
71 class Assistant(Agent):
72     def __init__(self) -> None:
73         super().__init__(
74             instructions="""
75             You are a friendly customer service representative. Help customers
76             with general inquiries. If they ask for a manager or you can't
77             resolve their issue, use the escalate_to_manager tool.
78             """
79         )
80
81     @function_tool()
82     async def escalate_to_manager(self, context: RunContext) -> ManagerAgent:
83         """Transfer the customer to a manager when requested or when you cannot resolve their
84         return ManagerAgent(chat_ctx=self.chat_ctx), "Transferring you to a manager now."
85
86     async def on_enter(self) -> None:
87         consent = await CollectConsent(chat_ctx=self.chat_ctx)
88
89         if consent:
90             await self.session.generate_reply(
91                 instructions="Thank them and offer your assistance."
92             )
```

workshop-demo 0 0 0 Cursor Tab Ln 84, Col 14 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

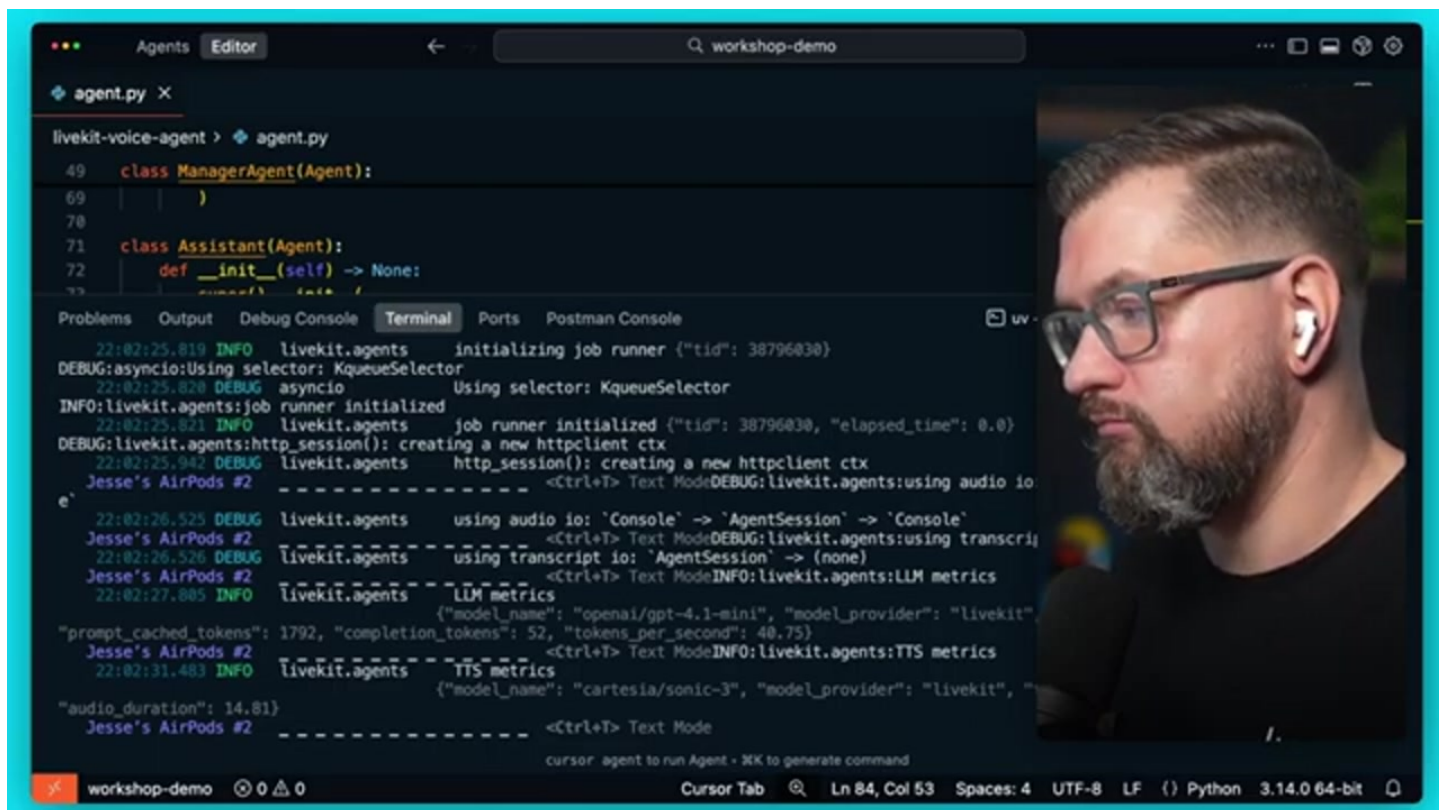
Now, by default, each agent starts fresh, but you usually want the manager to know what's already been discussed. So, we want to make sure that that conversation history persists. So, notice uh in the escalation tool here, uh in the manager agent, we're actually passing in that context here to the new agent. This carries the full conversation history to the manager. so

■ [01:02:00 - 01:02:20]

The image is a screenshot of a video call interface. On the right side, there is a video feed of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. He is looking towards the camera. On the left side, there is a code editor window titled 'agent.py'. The code is written in Python and defines two classes: 'ManagerAgent' and 'AssistantAgent'. The 'AssistantAgent' class has an 'on_enter' method that calls 'CollectConsent' and 'generate_reply'. The video call interface has a dark blue header with 'Agents Editor' and a search bar containing 'workshop-demo'. The code editor has a dark theme and shows line numbers from 49 to 92. The status bar at the bottom indicates 'workshop-demo', 'Cursor Tab', 'Ln 84, Col 53', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '3.14.0 64-bit'.

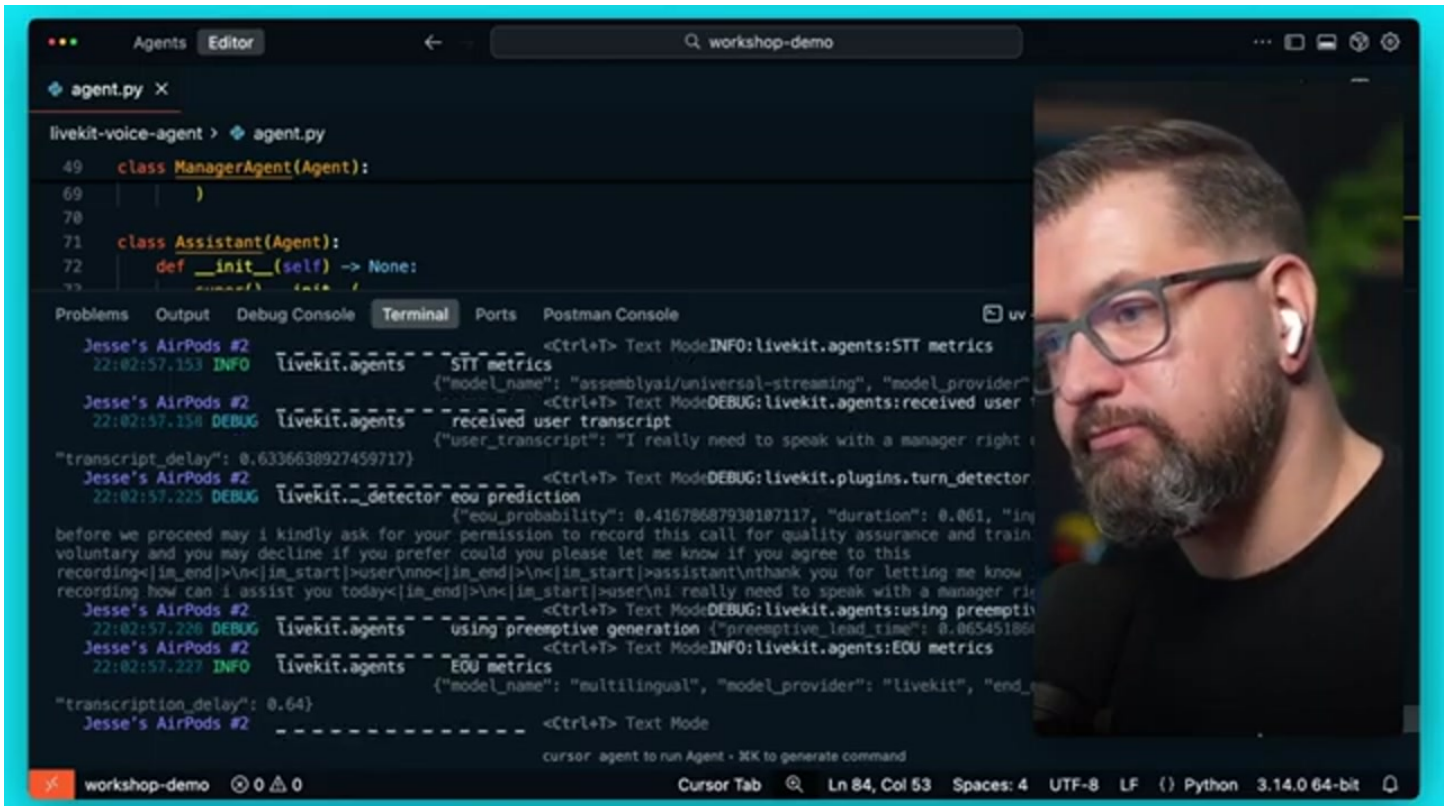
conversation history to the manager. so they can pick up right where the previous agent left off. Let's give this one a whirl. >> Hello. Before we proceed, may I kindly ask for your permission to record this call for quality assurance and training purposes. Your consent is entirely voluntary and you may decline if you

■ [01:02:20 - 01:02:40]



voluntary and you may decline if you prefer. Could you please let me know if you agree to this recording? >> No. Thank you for letting me know. I understand and we'll proceed without recording. How can I assist you today? I really need to speak with a manager right now, please.

■ [01:02:40 - 01:03:00]

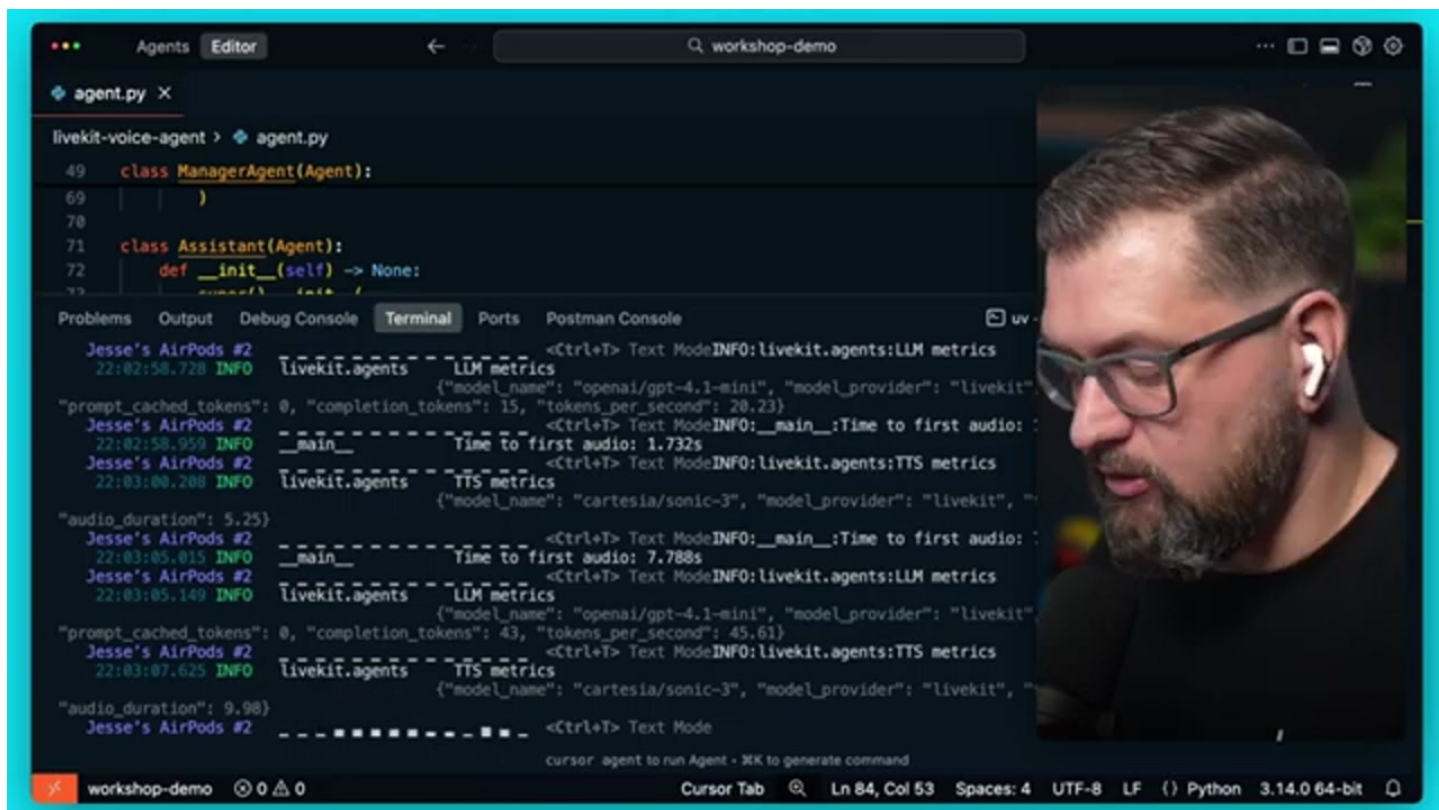


```
agent.py X
livekit-voice-agent > agent.py
49 class ManagerAgent(Agent):
69     )
70
71 class AssistantAgent(Agent):
72     def __init__(self) -> None:
73         super().__init__()

Problems Output Debug Console Terminal Ports Postman Console
Jesse's AirPods #2 22:02:57.153 INFO livekit.agents STT metrics
{"model_name": "assemblyai/universal-streaming", "model_provider": "assemblyai"}
Jesse's AirPods #2 22:02:57.158 DEBUG livekit.agents received user transcript
{"user_transcript": "I really need to speak with a manager right now"}
"transcript_delay": 0.6336638927459717}
Jesse's AirPods #2 22:02:57.225 DEBUG livekit.plugins.turn_detector eou prediction
{"eou_probability": 0.41678687938187117, "duration": 0.861, "info": "before we proceed may i kindly ask for your permission to record this call for quality assurance and train our model"}
Jesse's AirPods #2 22:02:57.226 DEBUG livekit.agents using preemptive generation ("preemptive_lead_time": 0.065451864)
Jesse's AirPods #2 22:02:57.227 INFO livekit.agents EOU metrics
{"model_name": "multilingual", "model_provider": "livekit", "end_time": 22:02:57.227}
"transcription_delay": 0.64}
Jesse's AirPods #2
```

right now, please. >> I am transferring you to a manager now. They will assist you shortly. >> Hello, I'm the customer service manager. I understand you asked to speak with someone senior. I'm here to help. Could you please share the details of your concern so I can assist in resolving it? No concerns. I just wanted to speak to

■ [01:03:00 - 01:03:20]



The screenshot shows a video call interface. On the right, a man with glasses and a beard is visible. The main area is a code editor with a dark theme. The top bar shows 'Agents Editor' and a search bar with 'workshop-demo'. The code editor displays the file 'agent.py' with the following Python code:

```
livekit-voice-agent > agent.py
49 class ManagerAgent(Agent):
69     )
70
71 class Assistant(Agent):
72     def __init__(self) -> None:
73         pass
```

Below the code editor is a terminal window showing logs for 'livekit-voice-agent'. The logs include timestamps, log levels, and messages such as 'livekit.agents LLM metrics', 'livekit.agents TTS metrics', and 'livekit.agents LLM metrics'. The logs also show 'prompt_cached_tokens', 'completion_tokens', 'tokens_per_second', and 'audio_duration'.

workshop-demo 0 0 0 Cursor Tab Ln 84, Col 53 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

you. Thanks. Bye. How easy was that to get to a manager? Now, for complex workflows with multiple steps like collecting shipping information, uh you can use a task group. A task group lets you define multiple tasks that run in sequence. Each task has an ID and a description. And when the group completes, you can then access the

■ [01:03:20 - 01:03:40]

Multi-step workflows



LiveKit

LIVEKIT.IO

completes, you can then access the results by that ID. And here is an example of a task group that collects both an email and shipping address. So first we're going to import our task group from agents beta workflows and then our agent task function tool and run context like we have in the past as well as this data class to define our uh email result and address result. And

■ [01:03:40 - 01:04:00]

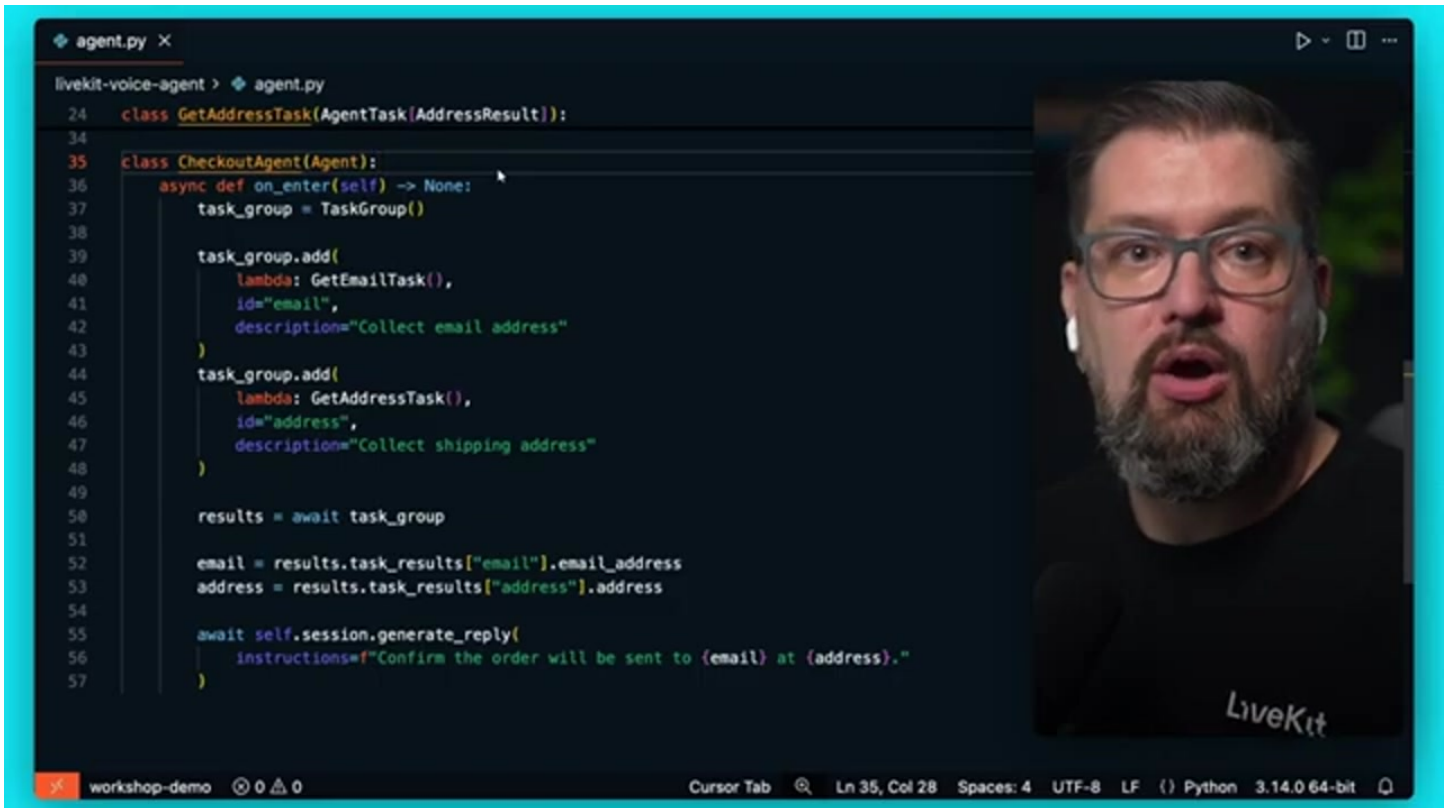


```
agent.py x
livekit-voice-agent > agent.py
1 from livekit.agents.beta.workflows import TaskGroup
2 from livekit.agents import AgentTask, function_tool, RunContext
3 from dataclasses import dataclass
4
5 @dataclass
6 class EmailResult:
7     email_address: str
8
9 @dataclass
10 class AddressResult:
11     address: str
12
13 class GetEmailTask(AgentTask[EmailResult]):
14     def __init__(self) -> None:
15         super().__init__(
16             instructions="Collect the user's email address."
17         )
18
19     @function_tool()
20     async def record_email(self, context: RunContext, email: str) -> None:
21         """Record the user's email address"""
22         self.complete(EmailResult(email_address=email))
23
24 class GetAddressTask(AgentTask[AddressResult]):
25     def __init__(self) -> None:
26         super().__init__(
27             instructions="Collect the user's shipping address."
28         )
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

workshop-demo 0 0 0 Cursor Tab Ln 6, Col 19 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

email result and address result. And then we're just going to define two simple tasks. We have a get email task and a get address task. The get email task collects an email address and stores it in a data class. And then the um get address task does the same thing for the shipping address. And then in the checkout agent, uh we're going to

■ [01:04:00 - 01:04:20]



```
agent.py x
livekit-voice-agent > agent.py
24 class GetAddressTask(AgentTask[AddressResult]):
34
35 class CheckoutAgent(Agent):
36     async def on_enter(self) -> None:
37         task_group = TaskGroup()
38
39         task_group.add(
40             lambda: GetEmailTask(),
41             id="email",
42             description="Collect email address"
43         )
44         task_group.add(
45             lambda: GetAddressTask(),
46             id="address",
47             description="Collect shipping address"
48         )
49
50         results = await task_group
51
52         email = results.task_results["email"].email_address
53         address = results.task_results["address"].address
54
55         await self.session.generate_reply(
56             instructions=f"Confirm the order will be sent to {email} at {address}."
57         )
```

workshop-demo 0 0 0 Cursor Tab Ln 35, Col 28 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

the checkout agent, uh we're going to create a task group. And then we're going to add both of those to the task group. Along with that, we will add an ID and a description for each. And each task is wrapped in a lambda so it can be uh reinitialized if the user needs to go back and make a correction. And then we just await that task group. And then it runs each task in sequence. And once

■ [01:04:20 - 01:04:40]



```
agent.py X
livekit-voice-agent > agent.py
24 class GetAddressTask(AgentTask[AddressResult]):
34
35 class CheckoutAgent(Agent):
36     async def on_enter(self) -> None:
37         task_group = TaskGroup()
38
39         task_group.add(
40             lambda: GetEmailTask(),
41             id="email",
42             description="Collect email address"
43         )
44         task_group.add(
45             lambda: GetAddressTask(),
46             id="address",
47             description="Collect shipping address"
48         )
49
50     results = await task_group
51
52     email = results.task_results["email"].email_address
53     address = results.task_results["address"].address
54
55     await self.session.generate_reply(
56         instructions=f"Confirm the order will be sent to {email} at {address}."
57     )
```

workshop-demo 0 0 0 Cursor Tab Ln 50, Col 35 Spaces: 4 UTF-8 LF () Python 3.14.0 64-bit

runs each task in sequence. And once complete, we access the results by their task ID and confirm the order details with the user. So one of the great things about task groups is they support going back to previous steps if the user needs to make corrections. So this makes multi-step data collection feel natural. And then for the ultimate escalation, uh

■ [01:04:40 - 01:05:00]

Warm handoff to a human



LIVEKIT.IO

And then for the ultimate escalation, uh you might need to transfer to an actual human agent. And so this typically involves SIP or telefan. So, first you'll want to disable interruptions so that the transfer message plays fully and then use the SIP integration to dial out to your phone system and bridge to a human agent. Now, SIP and telefan configuration is its own topic uh with

Warm handoff to a human



LiveKit

LIVEKIT.IO

configuration is its own topic uh with more setup involved. And so, we'll have a separate videos that go deeper into telefan that's not going to be covered in this workshop. And so, don't worry if this part seems a little complex. For now, I'll show you the patterns for how human handoff works. And that would look like this. So we're going to import uh

■ [01:05:20 - 01:05:40]

```
agent.py •
livekit-voice-agent > agent.py
15
16 from livekit.agents import function_tool, RunContext, get_job_context
17
18 @function_tool()
19 async def transfer_to_human(self, context: RunContext) -> None:
20     """Transfer the call to a human agent."""
21     context.disallow_interruptions()
22
23     await context.session.say(
24         "I'm transferring you to a human agent now. Please hold."
25     )
26
27     room = get_job_context().room
28     await room.local_participant.publish_sip_participant(
29         sip_trunk_id="your-trunk-id",
30         dial_to="sip:support@your-pbx.com",
31     )
32
33
34
35
36
37
```



like this. So we're going to import uh function tool run context uh and get a job context and then define this tool transfer to human and then transfer the call to a human agent. So we're going to disallow the interruption and then we're going to say I'm transferring you to a human agent now. Please hold on. So then we get the room context and then we await this uh publish SIP participant

■ [01:05:40 - 01:06:00]

```
livekit-voice-agent > agent.py
15
16 from livekit.agents import function_tool, RunContext, get_job_context
17
18 @function_tool()
19 async def transfer_to_human(self, context: RunContext) -> None:
20     """Transfer the call to a human agent."""
21     context.disallow_interruptions()
22
23     await context.session.say(
24         "I'm transferring you to a human agent now. Please hold."
25     )
26
27     room = get_job_context().room
28     await room.local_participant.publish_sip_participant(
29         sip_trunk_id="your-trunk-id",
30         dial_to="sip:support@your-pbx.com",
31     )
32
33
34
35
36
37
38
39
```



await this uh publish SIP participant including the the SIP trunk ID and the dial to uh information that is specific to your SIP configuration. So then the SIP integration dials out to your phone system and bridges in the human agent. So a complete production uh agent flow might look like this. First, the consent

■ [01:06:00 - 01:06:20]

Production flow



LIVEKIT.IO

might look like this. First, the consent task collects uh recording permission and then a triage agent determines the user's needs and then a specialized agent handles the specific request whether it's billing, support or sales and then escalations to a manager occurs uh as needed and then a human handoff is

Production flow

- Consent task
- Triage agent
- Specialized agent
- Manager escalation



uh as needed and then a human handoff is the last resort. Each transition preserves relevant context while allowing agents to have uh different personalities, tools, and permissions. Congratulations. You built a production quality voice agent from scratch. You covered ST, LLM, TTS, and why Web RTC

■ [01:06:40 - 01:07:00]

What you built

— Voice agent foundations



LIVEKIT.IO

covered ST, LLM, TTS, and why Web RTC matters for real-time audio. You you learned semantic turn detection to prevent awkward interruptions. You crafted system prompts, selected voices, and built fallback adapters for resilience. You measured performance and enabled preemptive generation. You connected your agent to external APIs

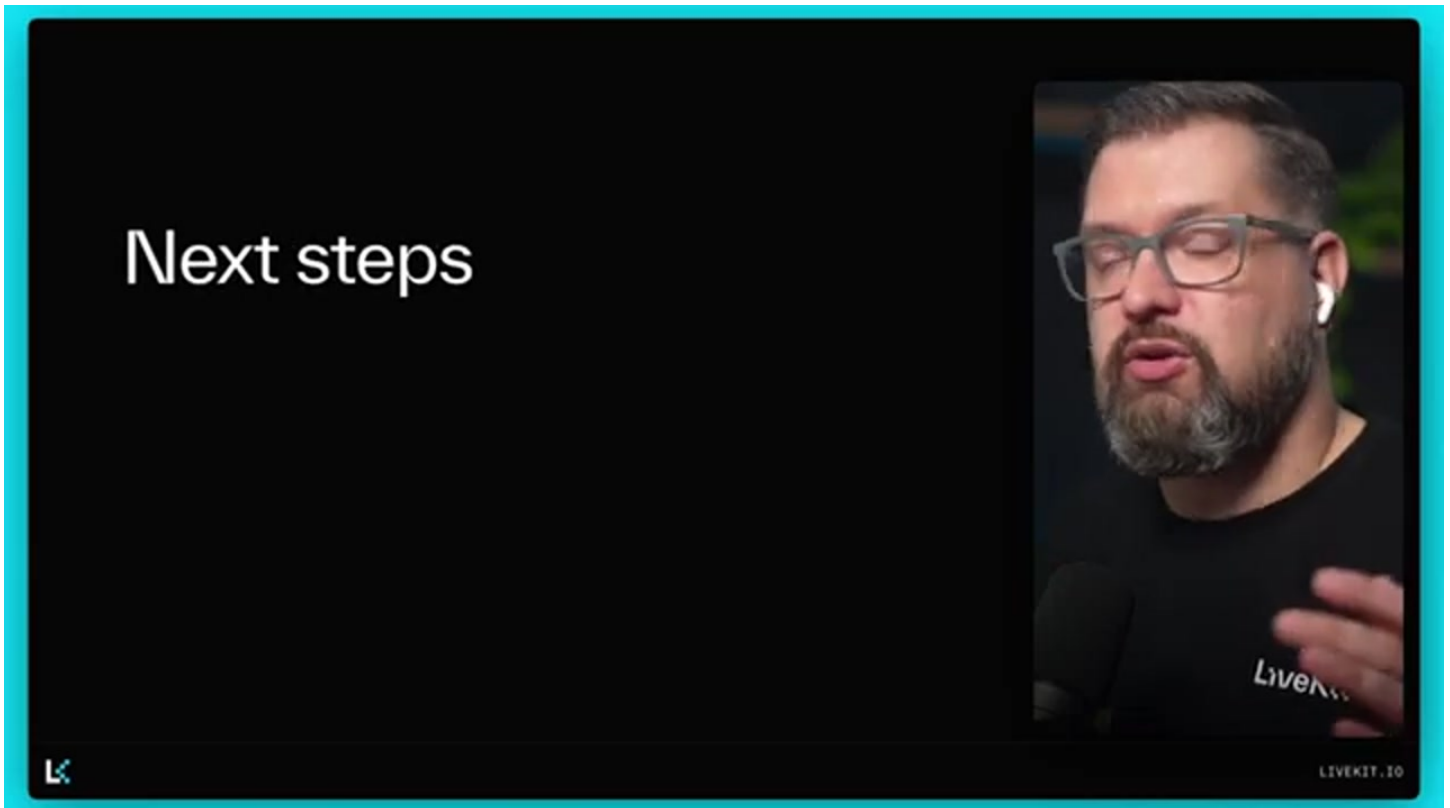
What you built

- Voice agent foundations
- Semantic turn detection
- Personality & fallbacks
- Metrics & preemptive generation
- Tools & MCP integration



LIVEKIT.IO

connected your agent to external APIs and MCP servers. And you built production workflows with tasks and agent handoffs. Your agent can now listen in real time, respond naturally, call external services, collect structured information, and hand off to specialized agents, or even humans when needed. So, here is where you go next.



needed. So, here is where you go next. Browse the Lifegate agents documentation for advanced topics like telefan, video processing, and real-time models. Explore the Python agents example repository for more complex workflows. Join the LifeKit community Slack to ask questions and share what you're building. And be sure to check out

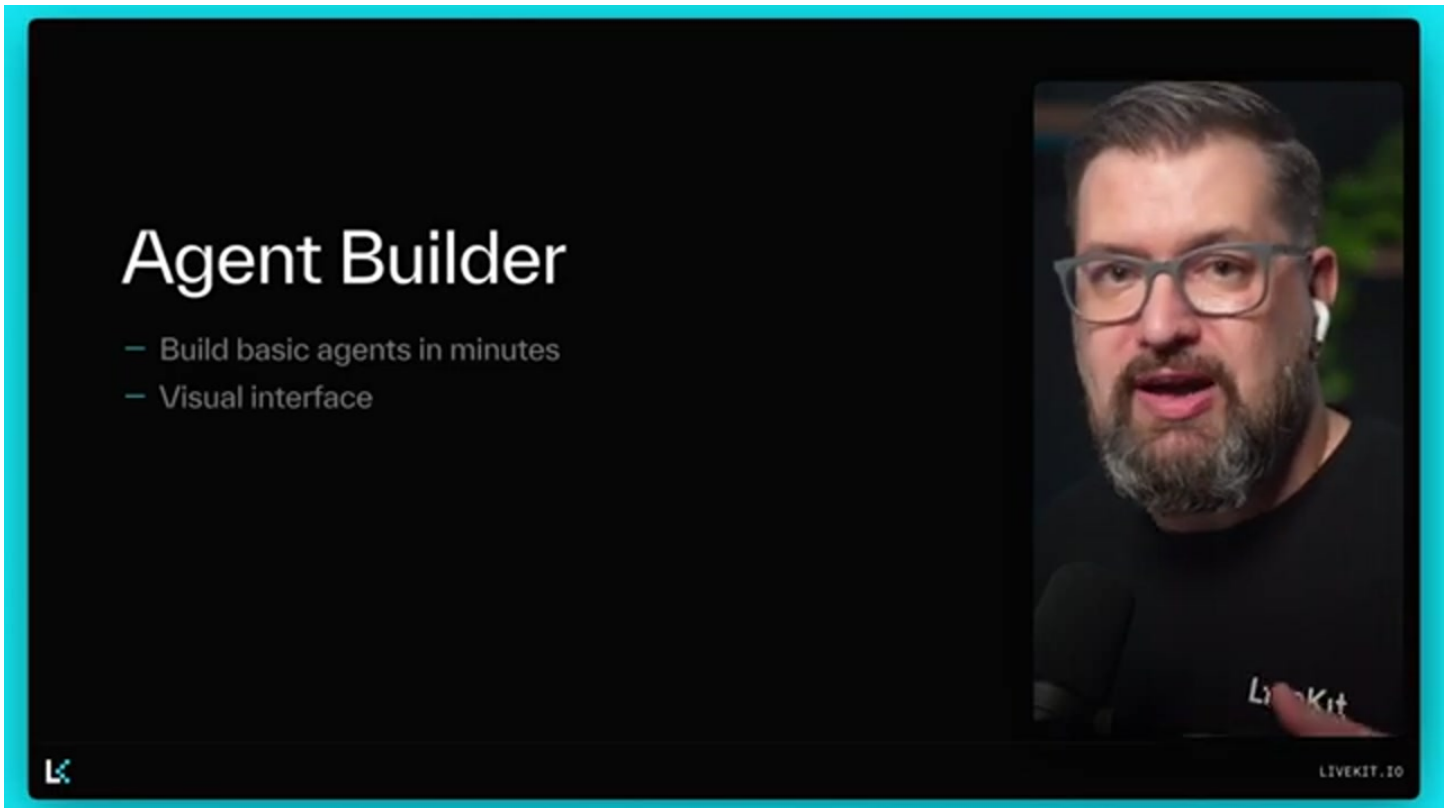
Next steps

- LiveKit Agent docs
- Python examples
- Community Slack
- LiveKit Cloud



LIVEKIT.IO

building. And be sure to check out LiveKit Cloud for production hosting with built-in observability like we looked at. Links to all of these things are in the video description. And one more thing that I want to introduce you to is the LiveKit Cloud Agent Builder. Agent Builder is a visual tool that generates a basic voice agent in minutes. You configure the providers



The slide features a dark background with a cyan border. On the left, the title "Agent Builder" is displayed in a large, white, sans-serif font. Below the title, two bullet points are listed in a smaller, white, sans-serif font: "– Build basic agents in minutes" and "– Visual interface". On the right side of the slide, there is a video inset showing a man with a beard and glasses, wearing a black t-shirt with the "LiveKit" logo, speaking into a microphone. In the bottom left corner of the slide, there is a small "LK" logo. In the bottom right corner, the text "LIVEKIT.IO" is visible.

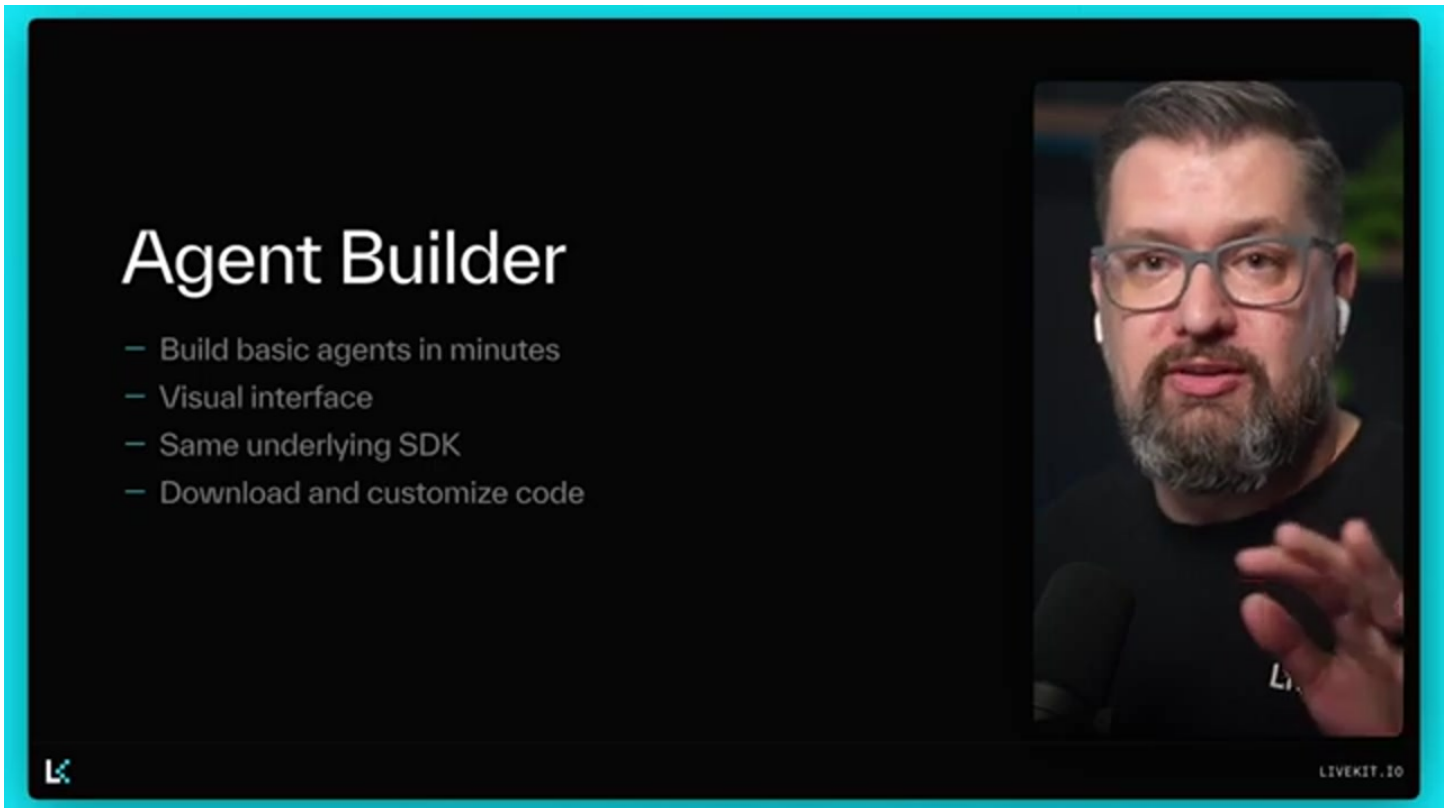
Agent Builder

- Build basic agents in minutes
- Visual interface

LK

LIVEKIT.IO

minutes. You configure the providers through the web interface. You set the system prompts and it generates the Python code for you that you can then download and even deploy directly from the interface. The code that agent builder generates is yours. You can download it, open it up in your own editor, customize it, add the tools, the workflows, and the custom logic that you learned in this workshop. Agent Builder



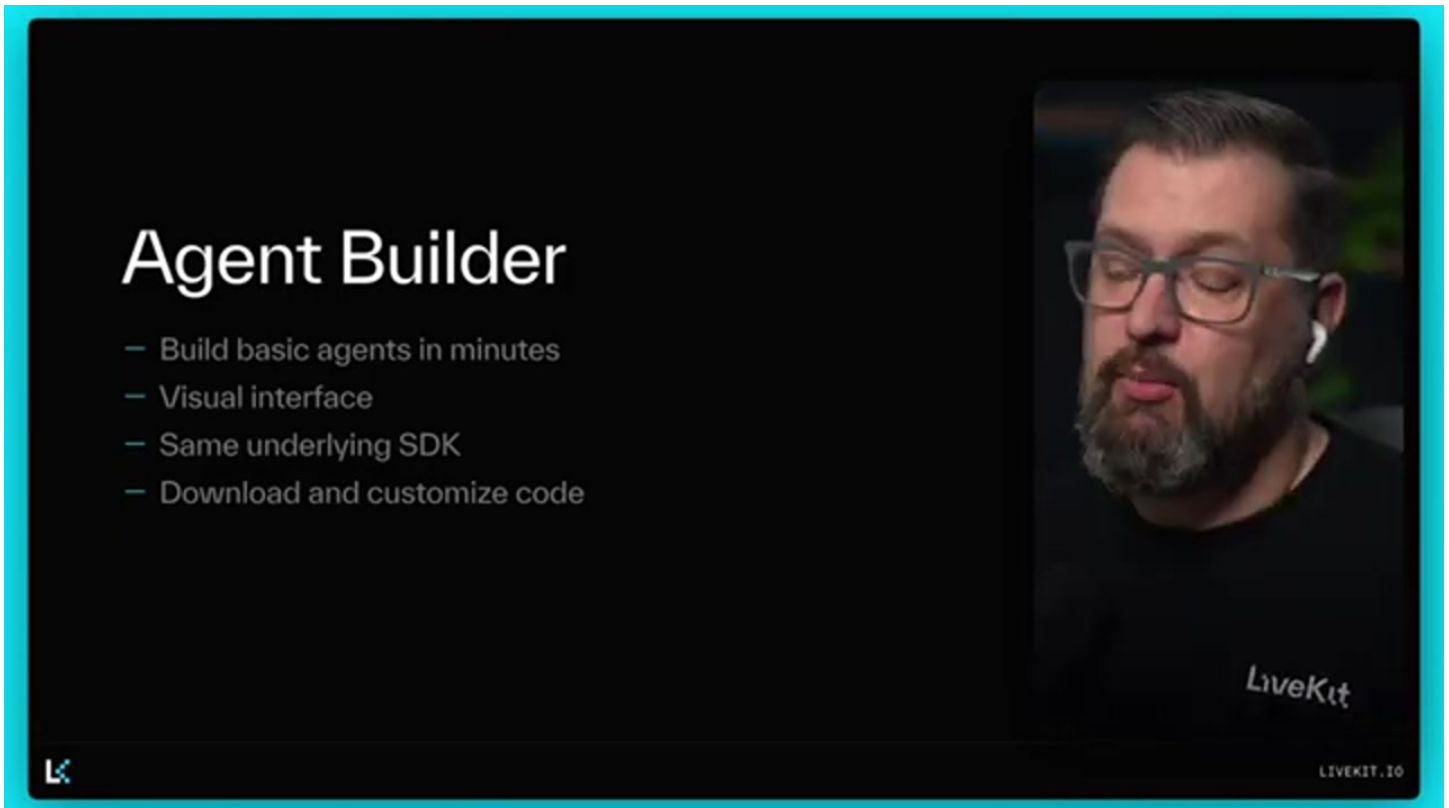
The slide features a dark background with a cyan border. On the left, the title 'Agent Builder' is displayed in a large, white, sans-serif font. Below the title, a bulleted list of four features is shown in a smaller, white, sans-serif font. On the right side of the slide, there is a video inset showing a man with a beard and glasses speaking into a microphone. In the bottom left corner of the slide, there is a small logo consisting of a stylized 'L' and 'K'. In the bottom right corner, the text 'LIVEKIT.IO' is visible in a small, white, sans-serif font.

Agent Builder

- Build basic agents in minutes
- Visual interface
- Same underlying SDK
- Download and customize code

LK LIVEKIT.IO

learned in this workshop. Agent Builder gives you a great starting point. Everything that you learned here applies uh to extending that code. Now, why didn't we just start with Agent Builder? Well, it would have been much easier, yes, but uh I wanted you to see exactly how to start from scratch. Agent Builder can get you up and running quickly. Now,

A presentation slide for 'Agent Builder' with a black background and a cyan border. On the right is a video of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. The slide text includes a title and a bulleted list of features.

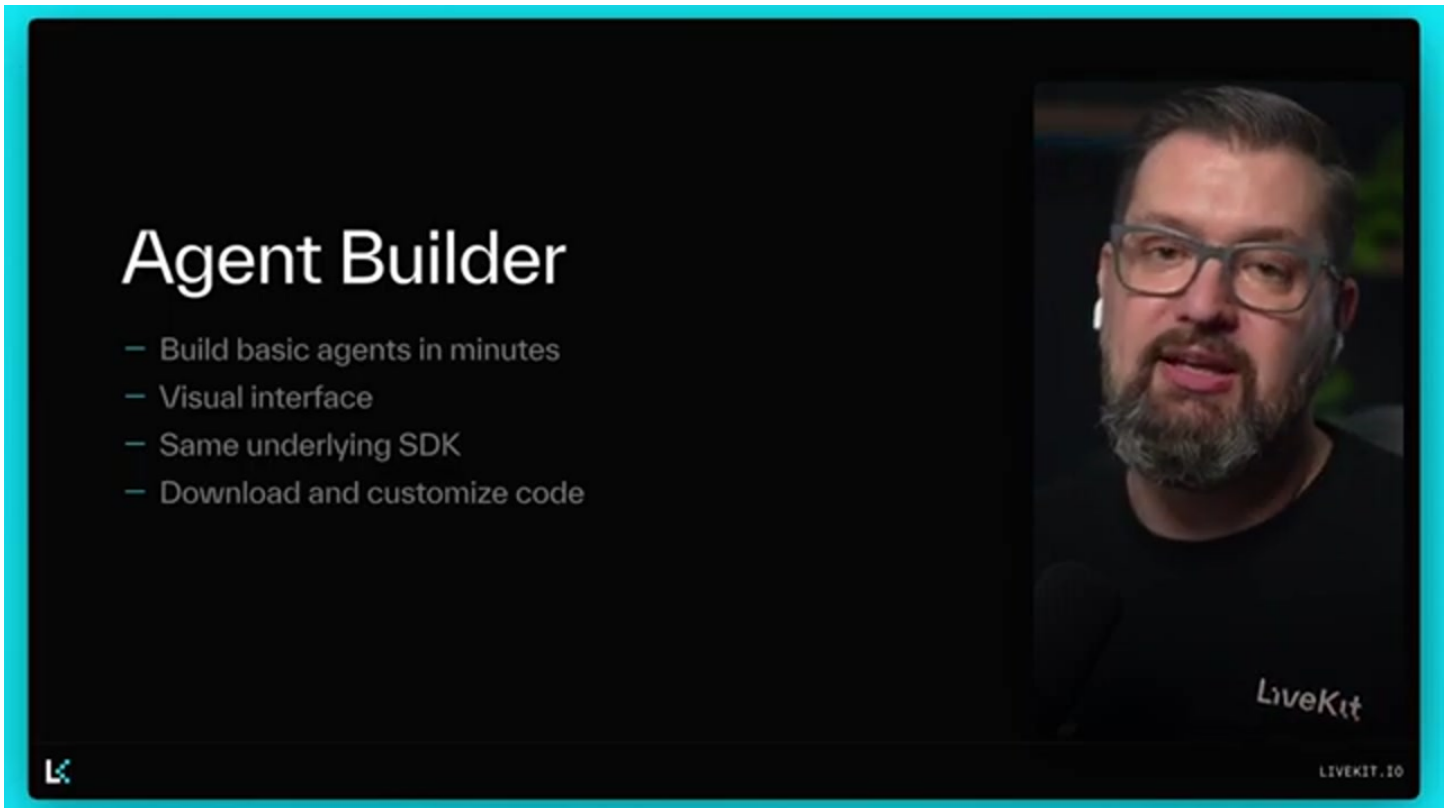
Agent Builder

- Build basic agents in minutes
- Visual interface
- Same underlying SDK
- Download and customize code

LiveKit

LIVEKIT.IO

beyond the basics, you can also configure tools directly in the builder interface and test your agent with a built-in test interface before deploying. And when you're ready, oneclick deploy pushes your agent to LiveKit Cloud. If you want a full walkthrough of agent builder specifically, go check out our other video tutorial on that. I'll leave a



The video frame shows a presentation slide with a black background and white text. The title 'Agent Builder' is prominently displayed at the top left. Below it, a bulleted list outlines four key features: building basic agents in minutes, a visual interface, a shared underlying SDK, and the ability to download and customize code. In the top right corner, there is a small inset video of a man with a beard and glasses, wearing a black t-shirt with the 'LiveKit' logo. The bottom left corner of the slide features a small 'LK' logo, and the bottom right corner displays the website 'LIVEKIT.IO'.

Agent Builder

- Build basic agents in minutes
- Visual interface
- Same underlying SDK
- Download and customize code

LK

LIVEKIT.IO

link in the video description. You can find agent builder in your LifeKit cloud dashboard under agents. So go build a basic agent there, download the code, and you'll recognize those same patterns and components that you built from scratch here in this workshop. Now, I hope this workshop was helpful. Uh thank you for following along. Now go build something